CS 7295 — Georgia Tech OMSCS

GPU Hardware and Software

Lecture Notes

kinginu

Spring 2027

Contents

1 Introduction of GPU 1
- 1.1 CPUs and GPUs . . . 1
- 1.2 Modern processor paradigms . . . 2
- 1.3 How to write parallel programs . . . 3
- 1.4 Introduction to GPU architecture . . . 5
- 1.5 History trend of GPU architectures . . . 7

2 Parallel Programming 9
- 2.1 Parallel programming patterns . . . 9
- 2.2 Shared-memory programming with OpenMP . . . 10
- 2.3 Scheduling, data sharing and synchronisation . . . 13
- 2.4 Programming with MPI . . . 15

3 GPU Programming Introduction 18
- 3.1 Introduction of CUDA programming . . . 18
- 3.2 Occupancy . . . 22
- 3.3 Host code . . . 24
- 3.4 Stencil operation with CUDA . . . 26

4 GPU Architecture 29
- 4.1 Multi-threaded architecture . . . 29
- 4.2 Bank conflicts . . . 31
- 4.3 GPU architecture pipeline . . . 35
- 4.4 Global memory coalescing . . . 37

5 Advanced GPU Programming 40
- 5.1 What suits a GPU, and where the time goes . . . 40
- 5.2 Data-transfer optimisations . . . 41
- 5.3 Memory-access optimisations . . . 44
- 5.4 Reducing computation . . . 45
- 5.5 Newer CUDA features . . . 46
- 5.6 Libraries and other platforms . . . 48
- 5.7 Sorting networks . . . 49

6 GPU Architecture Optimizations - I 51
- 6.1 Handling divergent branches . . . 51
- 6.2 Register optimizations . . . 55
- 6.3 Unified register file and shared memory . . . 57

7 GPU Architecture Optimizations - II 60
- 7.1 GPU virtual memory . . . 60
- 7.2 Unified virtual address space and on-demand paging . . . 63
- 7.3 Warp scheduling . . . 66

8 GPU Simulation 70
- 8.1 Cycle-level simulation . . . 70
- 8.2 Divergence and memory in GPU simulators . . . 73
- 8.3 Analytical models of GPUs . . . 75
- 8.4 Accelerating GPU simulation . . . 79

9 Multi GPU 82
- 9.1 Multi-GPU hardware . . . 82
- 9.2 Concurrency on GPUs . . . 86

10 Compiler Background - I 90
- 10.1 GPU compiler flow . . . 90
- 10.2 PTX . . . 93
- 10.3 IR and basic blocks . . . 94
- 10.4 Introduction to data-flow analysis . . . 95
- 10.5 An example of reaching definitions . . . 98

11 Compiler Background - II **101**
11.1 Live-variable analysis . . . 101
11.2 Static single assignment . . . 104
11.3 Examples of compiler optimizations . . . 106
11.4 Divergence analysis . . . 108

12 ML Accelerations On GPUs **111**
12.1 GPU and ML . . . 111
12.2 Floating-point formats . . . 115
12.3 Tensor core . . . 118

Bibliography **121**

Index **126**

Lesson 1

Introduction of GPU

Lecture	M1, lessons 1–5 (Instructor and Course Introduction; Modern Processor Paradigms; How to Write Parallel Programs; Introduction to GPU Architecture; History Trend of GPU Architectures)
Sources	NVIDIA A100 [1] and H100 [2] whitepapers (generation figures)
Textbook	K12 [3] ch. 1; A18 [4] ch. 1; CPG [5] §1 Introduction, §2 Programming Model, §4 Hardware Implementation; PMPP [6] ch. 1; PPP [7] ch. 2 (optional); BPG [8] §4.1.3 Understanding Scaling
Keywords	throughput, latency, virtual ISA, SIMD, SPMD, SIMT, multithreading, Amdahl's law, reduction, Flynn's taxonomy, streaming multiprocessor, warp, unified shader, tensor core

This lesson sets the frame for the whole book. It contrasts the design goals of CPUs and GPUs, reviews the techniques by which a CPU core extracts parallelism (superscalar issue, out-of-order execution, multithreading, multiprocessing), states Amdahl's law and the two ways of decomposing a program into parallel work, and then introduces the vocabulary of GPU hardware: the streaming multiprocessor, the SIMT lane and the warp. It closes with the evolution of the GPU from a fixed-function graphics pipeline to the programmable, tensor-core-equipped accelerator of today. Lesson 2 builds on the decomposition ideas with OpenMP and MPI, and Lesson 3 turns the warp and the SPMD style into concrete CUDA programs.

1.1 CPUs and GPUs

A *GPU* (graphics processing unit) was born as a processor for 3D graphics and has grown into a general parallel processor: today it is the hardware most associated with machine-learning acceleration and high-performance computing. A *CPU* (central processing unit) is built for the opposite point of the design space, and the contrast between the two, summarised in Table 1.1, organises the rest of this book.

Source M1, CPU vs GPU; K12 §1.2; CPG §1.1 The Benefits of Using GPUs.

Table 1.1: CPU and GPU design points.

	CPU	GPU
Target applications	latency-sensitive	throughput-sensitive
Precise exceptions	yes	no
Role	host	accelerator
ISA	public (open)	open or closed; PTX as a public virtual ISA
Programming model	SISD, SIMD	SPMD

Target applications. A CPU is optimised for *latency*, the time to finish one task, so a single thread runs as fast as possible. A GPU is optimised for *throughput*, the amount of work finished per unit time, and reaches it by running thousands of threads at once; any single thread may be slow.

Latency and throughput are independent goals: a design that doubles throughput by running two threads side by side leaves each thread's latency unchanged.

Precise exceptions. A *precise exception* is an exception after which the architectural state is exactly that of the sequential program at the faulting instruction: every earlier instruction has completed and no later one has changed state. CPUs guarantee it, because the operating system and debuggers rely on it (page faults, breakpoints, signals). GPUs do not: a faulting kernel is normally reported as a whole, and the machine state inside the kernel is not recoverable instruction by instruction.

Host and accelerator. The CPU runs the operating system and the I/O stack; the GPU does neither, so it depends on the CPU to be given work. The CPU is therefore the *host* and the GPU the *accelerator*, a co-processor attached to the host over an I/O link.

K12 §1.2 (2012) notes that the PCIe link "provides 4 Gbit/s (Gen 2) or 8 Gbit/s (Gen 3)" per lane and "often becomes one of the biggest bottlenecks" [3]. Lesson 5 treats host–device transfer.

ISA. The *instruction set architecture (ISA)* of a CPU is public and stable, because compiled software must keep running on every later implementation. A GPU runs in accelerator mode behind a driver, which translates the program into whatever the current hardware generation executes; the native ISA may be closed and may change from one generation to the next. To give programmers a stable target NVIDIA published PTX, a *virtual ISA*: an instruction set that no hardware executes directly, but that the driver compiles to the native ISA of the installed GPU.

PTX itself, its registers and its place in the compiler flow, is the subject of Lesson 10.

Programming model. CPUs are programmed as *SISD* (single instruction, single data: one instruction stream operating on one data item at a time), with *SIMD* (single instruction, multiple data) vector extensions where one instruction applies the same operation to a short vector of elements. GPUs are programmed as *SPMD* (single program, multiple data): every thread runs the same program, but on its own data, and may take its own branches.

Caution SPMD describes the *program*, SIMT the *hardware* that runs it: a GPU runs SPMD code on SIMT hardware (Section 1.3); the two acronyms are not alternatives.

SIMD is efficient for regular data-parallel work (image processing, vector arithmetic) because one instruction fetch drives many operations, but all lanes execute in lockstep, so data-dependent branches waste lanes. SPMD is more flexible, since each thread follows its own control flow, at the cost of managing many threads and their communication. Lesson 6 shows how the hardware pays for that flexibility when threads of one group diverge.

Note 1.1. The course's own objectives are to explain how a GPU works and to optimise programs for it, to prepare for research in GPU architecture and compilers, and to explain the recent hardware accelerators; it builds a foundation for CUDA rather than a survey of its latest features. It assumes the basic five-stage CPU pipeline and C++ programming, and no prior GPU programming.

1.2 Modern processor paradigms

Source M1, CPU pipeline and ILP; PMPP ch. 1; A18 ch. 1.

A GPU reuses most of the machinery of a CPU core, so this section reviews that machinery and the ways in which CPU designers have raised performance.

1.2.1 The pipeline

The CPU model used throughout this book has the stages of Figure 1.1: a *front end* fetches instructions, usually from an instruction cache; a *decode* stage interprets them (a complex x86 instruction is cracked here into several micro-operations); registers are read after decode; a *scheduler* selects the instructions to execute; the *execute* stage performs the computation or the memory access; and a *write-back* stage stores the result. The scheduler is where the two basic CPU organisations differ.

Many CPU pipelines also *rename* registers between decode and scheduling, so that false dependences through reused register names do not serialise instructions.

In order. A core that uses *in-order execution* issues instructions in program order: an instruction waits until every earlier one has issued, even when its own operands are ready.

Out of order. A core that uses *out-of-order execution* issues any instruction whose operands are ready, regardless of program order, so that an instruction stalled on a cache miss does not block the independent ones behind it.

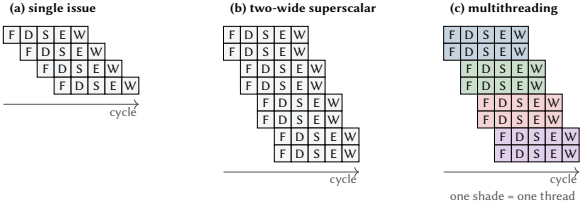


Figure 1.1: A five-stage pipeline (fetch, decode, schedule, execute, write-back) fed with one instruction per cycle (a), with two instructions per cycle in a two-wide superscalar (b), and with two instructions per cycle drawn from four threads (c).

1.2.2 Instruction-level parallelism

A single-issue pipeline starts at most one instruction per cycle. A *superscalar* processor fetches, decodes and executes several instructions per cycle (Figure 1.1b), so its *instructions per cycle (IPC)* exceeds one. How many instructions can actually run together is bounded by the program's *instruction-level parallelism (ILP)*: the number of instructions that are independent of each other and could execute at the same time.

Width in practice: Intel's Golden Cove core (2021) decodes up to six x86 instructions per cycle [9, §2.4.1].

Example 1.2. A machine has two execution units. In the pair

(a) independent	(b) dependent
`R1 = R2 * R3`	`R1 = R2 * R3`
`R4 = R5 - R6`	`R4 = R1 - R6`

the instructions of (a) read and write disjoint registers, so both issue in the same cycle. In (b) the second instruction reads `R1`, which the first one writes, so it cannot start before the first has produced its result: the pair takes two issue slots in sequence and the second unit stays idle.

Out-of-order execution exists to find more independent instructions than the next one in program order, and raising ILP has been one of the main themes of CPU design. Two further levers raise single-thread performance without more parallelism: a deeper pipeline allows a higher clock frequency, but needs better *branch prediction*, guessing the direction and target of a branch before it executes so that the longer pipeline stays full; and larger caches cut the number of memory accesses that miss and reach DRAM.

Tip Two instructions are independent when neither reads or writes a register the other writes. Out-of-order hardware looks further down the stream for an instruction that passes this test.

The Pentium 4 took the first lever to 20 pipeline stages (2000) [10] and 31 (2004) [11].

1.2.3 Multithreading and multiprocessors

The next source of parallelism is *multithreading*: a core holds the state of several threads at once and, when the current thread has no ready instruction, issues from another thread (Figure 1.1c). Independent threads supply ready instructions without any search for ILP, and the hardware cost is modest, chiefly the extra program counters and registers. Multithreading is one of the key ingredients of GPU performance: a GPU core keeps tens of thread groups in flight and switches among them every cycle.

The last lever is to replicate whole cores: a *multiprocessor* contains several cores, each with its own pipeline and usually its own first-level cache, and raises throughput in proportion to the number of cores for work that can be divided among them. Both CPUs and GPUs are multiprocessors; they differ in how they spend their transistors (Figure 1.2). A CPU has a few large cores, each built for ILP and single-thread latency, over a large shared cache. A GPU has many small cores, each multithreaded and SIMD-wide, with a comparatively small cache between them.

On the GPU this is fine-grained multithreading with a context switch every cycle; Lesson 4 shows how it hides memory latency.

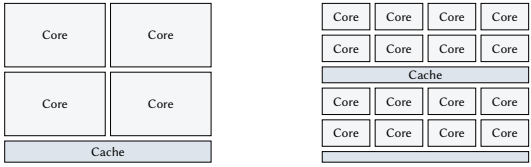


Figure 1.2: A CPU spends its area on a few large cores and a large cache; a GPU on many small cores and a small cache.

Scale today: AMD's EPYC 9654 CPU has 96 cores (2022) [12]; NVIDIA's H100 has 132 streaming multiprocessors [2], each with 128 lanes.

Key point
CPU performance comes from ILP (superscalar, out-of-order), clock frequency (deep pipelines plus branch prediction), caches, multithreading and multiple cores. A GPU keeps multithreading and multiple cores, drops most of the ILP machinery, and multiplies the thread count instead.

1.3 How to write parallel programs

Hardware with many cores only pays off for programs that expose parallel work. This section gives the law that bounds the gain, the two ways of splitting a program into parallel parts, and the taxonomy in which GPUs are placed.

Source M1, Amdahl's law and decomposition; PPP ch. 2 [7]; BPG §4.1.3 Understanding Scaling [8].

1.3.1 Amdahl's law

Definition 1.3 (Amdahl's law). Let a program spend a fraction S of its sequential execution time in work that cannot be parallelised, the *serial fraction*, and the remaining fraction $P = 1 - S$

in work that N processors share evenly. *Amdahl's law* gives the speedup over the sequential execution as

$$\text{speedup} = \frac{1}{S + P/N}. \tag{1.1}$$

When the serial fraction is small and the parallel part is large, the speedup grows almost linearly with N. When the serial fraction is significant, scalability collapses: as $N \to \infty$ the term P/N vanishes and the speedup approaches $1/S$, however much hardware is added.

Notation: S and P are fractions of the *sequential* run time, $S + P = 1$; N is the number of processors; the parallel part is assumed to divide perfectly.

Tip The ceiling is $1/S$. A serial fraction of 10 % caps the speedup at 10×; 1 % at 100×.

Example 1.4. A program has a serial fraction $S = 0.1$. With $N = 8$ processors Equation (1.1) gives $1/(0.1 + 0.9/8) = 1/0.2125 \approx 4.7$; with $N = 64$ it gives $1/(0.1 + 0.9/64) \approx 8.8$; with $N = 1024$, $1/(0.1 + 0.9/1024) \approx 9.9$. Going from 8 to 1024 processors, a 128× increase in hardware, barely doubles the speedup, because the serial 10 % alone already costs as much as the whole parallel part on 8 processors.

Figure 1.3 plots Equation (1.1) against the serial fraction for several N. The curves stay bunched together until S falls below about 0.3 and only separate for small serial fractions, which is why the first task of a parallel programmer is to shrink the serial part, not to add processors.

Caution Amdahl's law assumes a fixed problem size (strong scaling). Gustafson's law, BPG §4.1.3.2 Weak Scaling and Gustafson's Law [8], grows the problem with N and gives $N + (1 - N)\,s$, where s is the serial fraction of the *parallel* run: the serial part then matters less. The BPG names meshes, grids and some Monte Carlo simulations as workloads that grow with N.

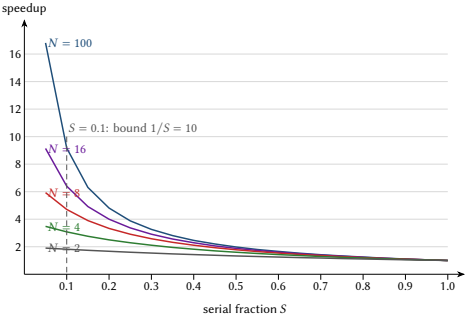


Figure 1.3: Speedup under Amdahl's law as a function of the serial fraction S for $N = 2$, 4, 8, 16 and 100 processors. At $S = 0.1$ even $N = 100$ gives only 9.2×.

1.3.2 Task and data decomposition

Given a program with enough parallel work, there are two ways of distributing it. The running example is an array A over which both the sum and the minimum and maximum are wanted, on two cores (Figure 1.4).

By task. The split called *task decomposition* gives different *operations* to different cores: core 1 computes the sum, core 2 the minimum and maximum, and the whole of A must be sent to both.

By data. The split called *data decomposition* gives different *data* to the cores, each of which performs every operation on its part: core 1 sums and scans the first half of A, core 2 the second half. The partial results then have to be combined; the combining step is a *reduction*, an operation that folds many values into one with an associative operator (sum, minimum, maximum).

Data decomposition scales naturally: with more cores each simply takes a smaller part of A, while task decomposition is limited by the number of distinct operations. Its price is the reduction, which is serial work in the sense of Amdahl's law unless it is itself parallelised as a tree.

Lesson 2 shows the reduction as an OpenMP clause and as an MPI collective; Lesson 5 touches the warp-level primitives that implement it on a GPU.

1.3.3 Flynn's taxonomy

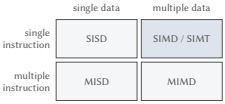


Figure 1.5: Flynn's taxonomy. GPUs sit in the single-instruction, multiple-data quadrant as SIMT.

Flynn's taxonomy classifies a machine by the number of instruction streams and the number of data streams it processes at a time (Figure 1.5): SISD, SIMD, MISD (multiple instruction, single data, a quadrant with almost no commercial examples) and *MIMD* (multiple instruction, multiple data), in which independent processors run independent instruction streams on independent data, as in a multicore CPU. A CPU can be SISD (one core, scalar code), SIMD (its

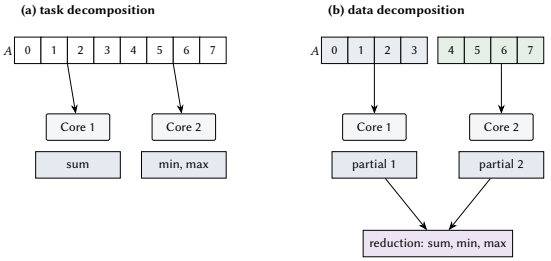


Figure 1.4: Task decomposition sends the whole array to each core and gives each a different operation; data decomposition gives each core part of the array and combines the partial results with a reduction.

vector units) or MIMD (its many cores). A GPU is placed in the SIMD quadrant under the name *SIMT* (single instruction, multiple threads): like SIMD, one fetched instruction drives many operations, but the operations belong to distinct threads, each with its own registers, executed on many ALUs rather than on a vector unit; the group shares one program counter and an active mask (Section 1.4, Lesson 6). The distinction is developed in Section 1.4 and in Lesson 4.

Caution CPG §4.1 SIMT Architecture draws the line as: SIMD exposes the vector width to software, SIMT does not; a SIMT thread is written as an ordinary scalar thread. Since Volta the hardware also keeps a program counter per thread (Independent Thread Scheduling) [5]; Lesson 5 returns to it.

1.3.4 SPMD on SIMT hardware

In the SPMD style all threads run the same program and differ only in the data they touch; data decomposition is its natural pattern. SPMD is the typical way of programming a GPU, and SIMT is the execution model of the hardware that runs it: the programmer writes one scalar program for one thread, the hardware fetches each of its instructions once for a whole group of threads.

A CUDA program consists of host code and a kernel, the function that runs on the GPU. Every thread executes the same kernel and uses its thread index to select its own data. The kernel below squares one element per thread; the index arithmetic and the launch syntax are explained in Lesson 3.

```
__global__ void square(const float *A, float *B, int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) B[i] = A[i] * A[i];
}
```

Summing the array in the same style is not as simple: a kernel in which every thread executes `sum += A[i]` on one shared variable is a race, since thousands of threads read, add and write the same location concurrently. The sum needs a reduction, built from per-thread partial sums and a combining step (Lesson 2 for OpenMP, Lesson 5 for the GPU primitives).

Caution `sum += A[i]` from every thread of a kernel is incorrect, not merely slow: the result depends on the interleaving. Treat any shared accumulator as a reduction.

1.4 Introduction to GPU architecture

A GPU is a multiprocessor of many cores, each of which runs many threads. This section names the parts of such a core, sketches its pipeline and defines the unit in which it executes threads.

Source M1, GPU architecture overview and warps; K12 §1.2.1; A18 ch. 1; CPG §4.1 SIMT Architecture.

1.4.1 Streaming multiprocessors and lanes

NVIDIA calls the core of a GPU a *streaming multiprocessor (SM)*, also a SIMT multiprocessor. Inside an SM (Figure 1.6) sit an instruction cache, a decoder, a block of shared memory, a set of program counters and register files for the threads it holds, and many execution units. Each execution unit is a *SIMT lane*: one ALU that executes one thread's share of each instruction. NVIDIA markets a lane as a *stream processor (CUDA core)*, which invites confusion with a CPU core. This book therefore fixes the vocabulary: a *core* is an SM, and a SIMT lane is never called a core.

K12 §1.2.1: NVIDIA calls the core an SM and AMD a compute unit (CU); each holds a number of SIMD units, also called stream processors [3].

Caution In this course *core* ≠ stream processor. A core is the SM; a stream processor is one lane of it. A "16 896-core" H100 is a 132-SM GPU with 128 lanes per SM.

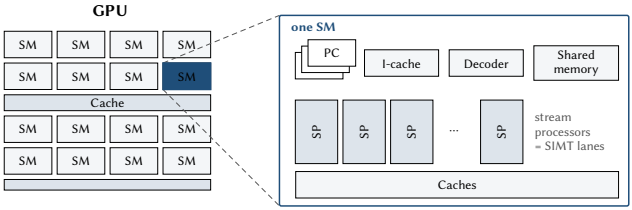


Figure 1.6: A GPU as a grid of streaming multiprocessors sharing a cache, and the contents of one SM: program counters, instruction cache, decoder, shared memory, stream processors (SIMT lanes) and caches.

1.4.2 The GPU pipeline

The word *warp* comes from weaving, "the first parallel thread technology": CPG §4.1 SIMT Architecture [5].

The pipeline of an SM (Figure 1.7) has the stages of the CPU pipeline of Section 1.2, arranged for many threads. The fetch stage keeps one program counter per warp, the thread group defined below, and fetches for whichever warp a scheduler selects, by a round-robin policy or by a greedy policy informed by cache misses and branch outcomes. Decode and register read follow. A second scheduler with a scoreboard then picks a group whose source operands are ready, the lanes execute the instruction, a memory-access unit handles loads and stores, and the write-back stage returns results to the registers.

Lesson 4 traces this pipeline cycle by cycle with its scoreboard and register banks; Lesson 7 compares the scheduling policies.

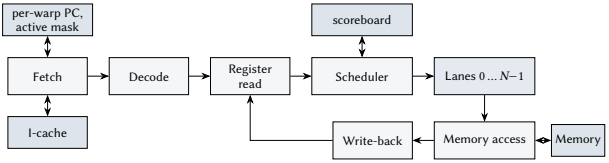


Figure 1.7: The GPU pipeline: fetch with one program counter per warp, decode, register read, a scheduler with a scoreboard that selects a ready warp, the SIMT lanes, memory access and write-back.

1.4.3 Warps

Definition 1.5 (Warp). A *warp* is a group of threads that the hardware executes as a unit: one instruction is fetched per warp and executed for all of its threads, one thread per lane. The number of threads in the group is the *warp size*. AMD calls the same unit a *wavefront*; this book uses the two words interchangeably.

The warp is the fundamental unit of execution on a GPU. Its size has stayed at 32 on NVIDIA hardware for a long time, but the architecture does not fix it, and future hardware may change it. A kernel launch names a number of threads, grouped into thread blocks (Lesson 3); the hardware cuts each block into warps of the warp size. When a warp's source operands are ready, the whole warp executes the instruction; then another ready warp is selected, and so on. The warp need not be as wide as the SM: a 32-thread warp on an SM with 8 lanes issues over four cycles.

Warp size 32 for every compute capability from 5.0 to 9.0: CPG Table 21, Technical Specifications per Compute Capability [5].

G80 (2006) had 8 lanes per SM, so a 32-thread warp took four cycles per instruction: K12 §1.4.1 [3]. Current SMs have 128 FP32 lanes split into four partitions [2].

Tip Threads per block ÷ warp size, rounded up, is the number of warps per block; a block of 100 threads with warp size 32 makes four warps, the last one only 4/32 full.

Example 1.6. A kernel of three instructions is launched with 12 threads (one thread block) on an SM whose warp size is 4 and which has 4 lanes (Figure 1.8). The threads form warps W_0, W_1, W_2. With a round-robin scheduler, one warp instruction per cycle and no stalls, the execution order is

cycle	1	2	3	4	5	6	7	8	9
warp	W_0	W_1	W_2	W_0	W_1	W_2	W_0	W_1	W_2
instruction	1	1	1	2	2	2	3	3	3

Nine instruction fetches execute $12 \times 3 = 36$ thread instructions. If instruction 2 of W_0 had to wait for the product of instruction 1 (a multiplier latency longer than the three cycles between them), the scheduler would skip W_0 in cycle 4 and issue W_1 and W_2 first: this is the multithreading of Section 1.2 at the granularity of warps.

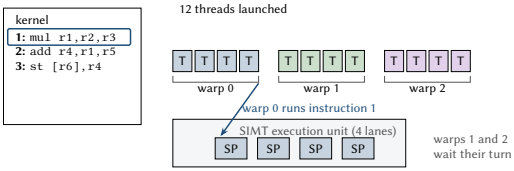


Figure 1.8: Twelve launched threads form three warps of four; warp 0 occupies the four lanes of the SIMT execution unit for instruction 1 while the other warps wait.

Key point
A GPU is a multiprocessor of SMs; an SM is a multithreaded SIMT core of many lanes; a warp is the group of threads (32 on NVIDIA GPUs) that shares one instruction fetch and is scheduled as a unit. All later performance discussion, from occupancy to divergence and coalescing, is in terms of warps.

1.5 History trend of GPU architectures

Source M1, graphics pipeline and GPU evolution; K12 §1.1; A18 ch. 1; HWP [2].

GPU architecture has passed through four stages.

1. *Traditional pipelined GPUs*, with separate fixed-function units for vertex processing and for pixel processing;
2. *programmable GPUs*, in which the vertex and pixel units became programmable shaders;
3. the progression of the shaders into unified, CUDA-programmable cores, which opened general-purpose computing (GPGPU);
4. *modern GPUs*, which add machine-learning accelerator components such as tensor cores alongside the programmable cores.

1.5.1 The traditional 3D graphics pipeline

The *graphics pipeline* of the early GPU (Figure 1.9) takes 3D primitives, typically triangles given by their vertices, as input. A geometry stage transforms and lights the vertices and projects them into 2D primitives on the screen plane; a rasterisation stage converts each 2D primitive into the pixels it covers and computes their colours. The program, or originally the fixed function, applied to each vertex is the *vertex shader*; the one applied to each pixel (or fragment) is the *pixel shader*.

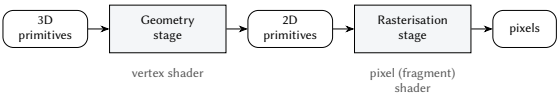


Figure 1.9: The traditional 3D graphics pipeline: geometry processing turns 3D primitives into 2D primitives, rasterisation turns them into pixels.

Both stages apply the same operation to a large number of independent vertices or pixels: the data parallelism that GPGPU later exploited was built into the graphics workload from the start.

The shaders were at first fixed-function units, then became programmable, and finally the separate vertex and pixel units were replaced by one pool of *unified shaders*, processors that run either kind of shader program. The unified shader is the hallmark of NVIDIA's Tesla architecture, and with it the GPU became usable for computation other than graphics. The term *GPGPU* (general-purpose computing on GPUs) was coined to mark that non-graphics use; now that most GPU cycles in data centres go to non-graphics work the distinction has lost its purpose, although the term survives in titles.[1]

K12 §1.1 places the inflection point at the G80 family (GeForce 8800, 2006), a programmable unified graphics-and-compute processor, together with CUDA [3].

[1] Before CUDA, GPGPU meant expressing a computation as a graphics operation: data stored in textures, computation written as a pixel shader in a shading language through DirectX or OpenGL, with stream extensions such as Brook adding a thin layer on top (K12 §1.1 [3]). The unified shader and CUDA removed that indirection. FP64 at half the FP32 rate is a data-centre feature: the A100 delivers 9.7 TFLOP/s FP64 against 19.5 TFLOP/s FP32 [1].

1.5.2 Evolution of the programmable GPU

Once programmable, the GPU acquired, generation by generation, the features that general-purpose and then machine-learning workloads needed:

- cache hierarchies (L1 and L2 caches), where the graphics pipeline had needed only streaming access to textures;
- *double precision (FP64)* arithmetic in addition to *single precision (FP32)*, for high-performance computing;

- atomic operations and fast integer arithmetic, for more diverse workloads;
- *high bandwidth memory (HBM)*, DRAM dies stacked beside the GPU on an interposer and connected through a very wide interface, which raised memory bandwidth far beyond what a board-level memory bus provides;
- *half precision (FP16)*, a 16-bit floating-point format that doubles arithmetic throughput and halves memory traffic for machine-learning workloads that tolerate the reduced precision;
- the *tensor core*, an execution unit that performs a small matrix multiply-add (for instance a $4 \times 4 \times 4$ product in one step) rather than a scalar operation per lane, for the dense linear algebra of neural networks;
- the *Transformer Engine*, Hopper's combination of FP8 tensor-core arithmetic with software and hardware that choose the number format and rescale layer by layer [2], for transformer models.

Caution The lecture says "transformer cores". Hopper has no separate core for transformers: the Transformer Engine is the tensor cores' FP8 path plus the per-layer format management [2].

1.5.3 A modern GPU: NVIDIA H100

The H100 (Hopper, 2022) illustrates the current state of the list. It supports FP64, FP32, FP16, INT8 and FP8 arithmetic; its tensor cores and Transformer Engine serve machine learning; its register files are larger; a *Tensor Memory Accelerator (TMA)* moves multidimensional blocks of data between global and shared memory asynchronously, relieving the threads of address generation; the number of SMs and of floating-point units per SM has grown again; and an NVLink Switch System connects many GPUs into one machine. Table 1.2 follows four data-centre generations through the figures that this evolution changed.

Caution Lecture vs whitepaper: the register file per SM is 256 KiB on A100 and H100 alike; only the total grows with the SM count, from 27 MiB to 33 MiB (HWP Table 3 [2]).

Lesson 12 treats FP8 and the tensor-core formats; Lesson 9 the NVLink interconnect and multi-GPU systems.

Caution The H100 column holds the preliminary figures of the V1.02 whitepaper [2]; shipped products differ in detail (the PCIe H100 has 114 SMs).

Table 1.2: Four NVIDIA data-centre generations (A100 whitepaper [1]; H100 whitepaper V1.02, SXM5, preliminary [2]).

	P100	V100	A100	H100
Year	2016	2017	2020	2022
SMs	56	80	108	132
FP32 lanes per SM	64	64	64	128
Tensor cores per SM	—	8	4	4
Register file per SM	256 KiB	256 KiB	256 KiB	256 KiB
L2 cache	4 MiB	6 MiB	40 MiB	50 MiB
HBM bandwidth	720 GB/s	900 GB/s	1555 GB/s	3000 GB/s
Transistors	15.3×10^9	21.1×10^9	54.2×10^9	80×10^9
Process	16 nm	12 nm	7 nm	4N

Key point — Lesson summary

A CPU minimises the latency of one thread with ILP, deep pipelines and large caches; a GPU maximises throughput with many small multithreaded cores, acts as an accelerator under a host CPU, and is programmed in the SPMD style on SIMT hardware. Amdahl's law caps the speedup at $1/S$, so a parallel program must shrink its serial fraction; data decomposition with a reduction is the pattern that scales. The GPU's core is the streaming multiprocessor, its lanes are stream processors, and the warp (32 threads on NVIDIA GPUs) is the unit of fetch and scheduling. The GPU grew from a fixed-function graphics pipeline through programmable and unified shaders to a general processor with caches, FP64, HBM, FP16, tensor cores and a Transformer Engine.

Parallel Programming

Lecture	M2, lessons 1–4 (Parallel Programming Patterns; OpenMP vs. MPI, parts 1–2; Programming with MPI)
Sources	[13] (required); [14]; [15]
Textbook	PPP [7] ch. 3–5, App. A–B (optional); PMPP [6] ch. 20
Keywords	parallel programming pattern, master/worker, loop parallelism, fork/join, pipeline, shared memory, distributed memory, OpenMP, thread, reduction, data race, critical section, scheduling, barrier, MPI, stencil, halo

Lesson 1 introduced the GPU as a throughput machine and the two ways of splitting a program into parallel work, task decomposition and data decomposition. This lesson shows how that work is expressed in code on a conventional multiprocessor, before the GPU-specific model of Lesson 3. It first collects the recurring *patterns* of parallel programs, then contrasts the two memory models a parallel program can be written against: shared memory, where threads communicate through one address space and are programmed with OpenMP, and distributed memory, where processes exchange messages with MPI. Both matter for CUDA, which combines them: some memory is shared by every thread of a block, while other data must be moved explicitly. The chapter closes with a stencil computation whose boundary exchange between processes returns, on the GPU, as the halo loading of Lesson 3.

2.1 Parallel programming patterns

A parallel application is built from a serial one in three moves: the work is divided into pieces, the data structures are adjusted so that the pieces can be worked on independently, and the pieces are distributed to the processors (Figure 2.1). A *parallel programming pattern* is a reusable way of organising this division and distribution; the handful of patterns below cover most parallel programs, including the GPU kernels of the later lessons.

Source M2, parallel programming patterns; PPP ch. 3–5.

2.1.1 Four steps

Designing a parallel program proceeds in four steps.[1]

1. **Discover concurrency.** The *concurrency* of a computation is the extent to which its parts may proceed independently of one another; the first step finds where such independence exists (which tasks, which data).
2. **Structure the algorithm** so that the concurrency can be exploited: decide what the units of parallel work are and how they communicate.
3. **Implement** it in a language and toolset suited to the target: OpenMP, MPI, CUDA.
4. **Execute and optimise** on the parallel system, measuring and tuning until the performance goal is met.

[1] The steps follow the four *design spaces* of [7]: Finding Concurrency, Algorithm Structure, Supporting Structures and Implementation Mechanisms. The book places the patterns below in different spaces (Pipeline in Algorithm Structure; SPMD, Master/Worker, Loop Parallelism and Fork/Join in Supporting Structures); the course lists them side by side.

2.1.2 Five patterns

Master/worker In the *master/worker pattern* one master thread or process sets up the problem, places the pieces of work in a *task queue* (a "bag of tasks"), launches a pool of workers, waits, collects their results and terminates the computation (Figure 2.2). Each worker

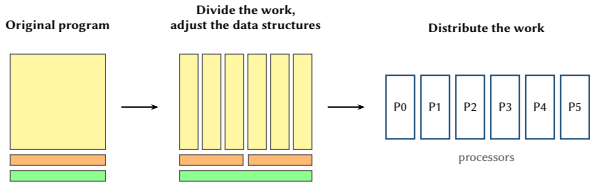


Figure 2.1: Creating a parallel application: divide the work, adjust the data structures, distribute the pieces. The orange and green bars are parts of the program that split differently from the main body.

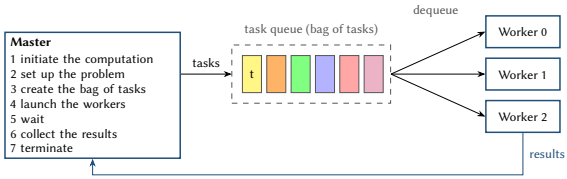


Figure 2.2: The master/worker pattern. The master fills the task queue and waits; workers dequeue tasks until the queue is empty and return their results.

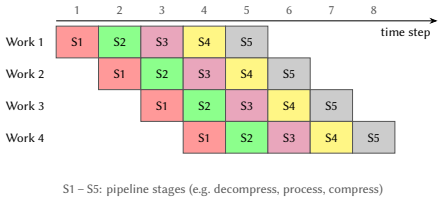


Figure 2.4: The pipeline pattern. Four work items pass through five stages; at steps 4 and 5 four of the five stages are busy with different items (with five or more items in flight every stage would be). With s stages and n items the pipeline takes $s + n - 1$ steps instead of sn: here 8 instead of 20.

repeatedly dequeues a task and runs it; since workers take the next task as soon as they finish the previous one, the load balances itself. The pattern suits work that splits into independent pieces of unequal or unknown size.

The OpenMP `dynamic` schedule of Section 2.3 is a task queue whose tasks are chunks of loop iterations.

Loop parallelism In *loop parallelism* the iterations of a loop run concurrently when they are independent, as in

```
for (int i = 0; i < 16; i++) c[i] = a[i] + b[i];
```

Unlike master/worker tasks, the iterations are usually identical operations on different data, which makes this the pattern of *data decomposition* (Lesson 1).

The same vector addition is the first CUDA kernel of Lesson 3: one thread per iteration.

SPMD Every processing element runs the same program on its own portion of the data; *SPMD* (Lesson 1) is the usual choice for GPU programming, where thousands of threads run one kernel and pick their data by their thread index.

Fork/join In the *fork/join pattern* a parent task forks child tasks, continues or waits, and joins them (waits for their completion) before going on (Figure 2.3). It mixes serial and parallel phases from a single entry point; an OpenMP parallel region is a fork/join.

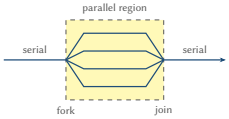


Figure 2.3: The fork/join pattern: one flow of control forks into four parallel tasks and joins again.

Pipeline The *pipeline pattern* assigns each processor one stage of a multi-stage computation and streams the data items through the stages, exactly as a CPU pipeline streams instructions (Figure 2.4). It suits data streams: signal processing, the graphics pipeline of Lesson 1, or a compression workflow (decompress, process, compress).

> **Key point**
> Patterns are not exclusive: a GPU program is SPMD at the level of the kernel, loop parallelism inside it (one thread per iteration), and fork/join at the level of the host that launches the kernel and waits for it.

2.2 Shared-memory programming with OpenMP

Source M2, OpenMP vs. MPI (part 1); PPP App. A; [13], memory architectures and programming models.

How the pieces of a parallel program communicate depends on whether they share a memory.

2.2.1 Shared versus distributed memory

Definition 2.1 (Shared-memory programming). In *shared-memory programming* all processors read and write one memory: a value that processor 1 writes to an address is seen by processor 2 when it reads that address (Figure 2.5, left). Communication is implicit in ordinary loads and stores.

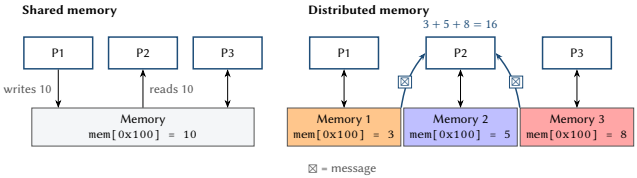


Figure 2.5: Shared memory (left) and distributed memory (right). The same address `0x100` names one location on the left and three unrelated locations on the right.

Definition 2.2 (Distributed-memory programming). In *distributed-memory programming* each processor has its own memory, invisible to the others. Data that lives in another processor's memory must be requested and sent in a *message* (Figure 2.5, right): processor 2 asks processors 1 and 3 for the contents of an address in their memories, adds the three values and writes the result back into its own memory.

Shared memory is the easier model to program, because data sharing needs no explicit transfer, but it does not scale: every processor contends for the same memory, and the programmer is responsible for synchronising the accesses. Distributed memory scales with the number of processors and keeps local accesses fast, at the price of making the programmer specify every transfer.

Caution "Shared memory" in this lesson is the CPU sense, one address space for all threads. CUDA's `__shared__` memory of Lesson 3 is a small on-chip scratchpad shared only by the threads of one block.

The largest machines are hybrids: shared memory inside a node, messages between nodes; the common pairings are MPI with OpenMP, or MPI with CUDA. [13].

2.2.2 OpenMP

Definition 2.3 (OpenMP). *OpenMP* is an open standard for shared-memory parallel programming in C, C++ and Fortran. It is a collection of *compiler directives* (`#pragma omp ...`), library routines (`omp_...()`) and *environment variables* (`OMP_...`) with which a serial program is annotated, rather than rewritten, to run in parallel.

Whatever the tool, a shared-memory program has to settle five things: the *parallelisation* itself (which loop or region runs in parallel), the *number of threads*, the *scheduling* of work onto those threads, the *data sharing* policy (which variables are shared, which are private), and the *synchronisation* that keeps shared data consistent. OpenMP has a directive or clause for each; this section covers the first and Section 2.3 the other four.

OpenMP 1.0 appeared in 1997 (Fortran) and 1998 (C/C++); the specification cited here is 5.2 (2021), the current one 6.0 (2024). `openmp.org`.

2.2.3 Threads

Definition 2.4 (Thread). A *thread* is an entity that runs a sequence of instructions. It has its own registers and its own stack, and it shares the rest of the address space (code, global data, heap) with the other threads of the same process.

A thread is a software concept, not a core: one core may run several threads, time-sliced by the operating system or interleaved by the hardware. The word therefore means different things on a CPU and on a GPU: a CPU thread is a heavyweight, independently scheduled flow of control, whereas a GPU thread is one lane of a warp that executes in lockstep with 31 others (Lesson 1). This lesson assumes, for simplicity, one thread per core.

On Linux a new thread's stack defaults to the `ulimit -s` soft limit at program start, 8 MiB on most distributions; `pthread_attr_setstacksize` overrides it per thread. [16].

Multithreading gives a CPU core two or more hardware thread contexts (Lesson 1); a GPU SM holds up to 64 resident warps on current data-centre GPUs ([5, Table 21]; Lesson 3), each a hardware-scheduled group of threads.

2.2.4 Parallelising a sum by hand

The running example is the sum of a ten-element array. Serially:

```
#include <iostream>
int main() {
  const int size = 10;
  int data[size];
  for (int i = 0; i < size; ++i) data[i] = i + 1;
  int sum = 0;
  for (int i = 0; i < size; ++i) sum += data[i];
  std::cout << "sum = " << sum << '\n';   // 55
  return 0;
}
```

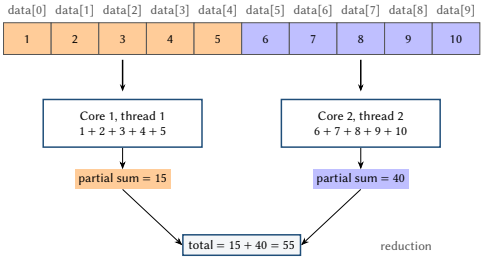


Figure 2.6: Data decomposition of the sum: each thread adds its half, then the two partial sums are reduced to the total.

Table 2.1: An interleaving of two unprotected updates of the shared `sum`; one partial sum is lost.

Step	Thread 1 (partial 15)	Thread 2 (partial 40)	`sum`
1	load `sum` → 0		0
2		load `sum` → 0	0
3	add: 0 + 15 = 15		0
4		add: 0 + 40 = 40	0
5	store 15		15
6		store 40	40

Data decomposition over two cores splits the array in half: `data[0..4]` goes to a thread on core 1 and `data[5..9]` to a thread on core 2 (Figure 2.6). Done by hand, the parallel version has three steps:

1. create two threads;
2. give each thread its half of the array, on which it computes a *partial sum* (15 and 40 for the values $1, \ldots, 10$);
3. merge the two partial sums into the total, $15 + 40 = 55$.

The merge is a *reduction* (Lesson 1): many values combined by an associative operation into one.

With thousands of GPU threads the reduction becomes a tree: partial sums are combined in pairs over $\log_2$ rounds, within a warp through the shuffle operations of Lesson 5.

2.2.5 Reduction in shared memory and the race

In shared memory the reduction is naturally written as each thread adding its partial sum into one shared variable `sum`, initially 0. The statement `sum += partial` is three machine operations: load `sum`, add, store `sum`. If the two threads interleave as in Table 2.1, both load the initial 0, and the second store overwrites the first: the result is 40 (or 15, if the stores happen in the other order) instead of 55.

Definition 2.5 (Data race, critical section). A *data race* occurs when two threads access the same variable without ordering and at least one access is a write; the outcome depends on the interleaving. A piece of code that accesses shared data and must therefore be executed by one thread at a time is a *critical section*.

Caution In C and C++ a data race is undefined behaviour, not merely a wrong number: the compiler may assume it does not happen and, for instance, keep `sum` in a register across the whole loop.

Definition 2.6 (Mutex). A *mutex* (from *mutual exclusion*) is a *lock* that at most one thread can hold. A thread *locks* (acquires) the mutex before entering the critical section and *unlocks* (releases) it on leaving; a thread that finds the mutex held waits. With the update of `sum` inside a mutex, the two additions happen one after the other and the result is 55.

Pthreads is IEEE POSIX 1003.1c (1995), a library with no compiler support; every thread and every lock is explicit. [13].

The low-level way to write this on a POSIX system is *POSIX threads (pthreads)*, a C library of calls:

- `pthread_create` starts a thread on a given function;
- `pthread_join` waits for a thread to finish;
- `pthread_mutex_lock` and `pthread_mutex_unlock` bracket the critical section.

Thread creation, data distribution, locking and joining are all the programmer's code, and each is a place for a bug.

2.2.6 The OpenMP version

OpenMP hides all of that behind one directive.

Tip Compiled without OpenMP support (no `-fopenmp`), the pragma is ignored and the program is the serial one; OpenMP can be added to a working serial program one loop at a time.

```
#include <iostream>
#include <omp.h>
int main() {
  const int size = 1000;
  int data[size];
  for (int i = 0; i < size; ++i) data[i] = i + 1;
  int sum = 0;
  #pragma omp parallel for reduction(+:sum)
  for (int i = 0; i < size; ++i) sum += data[i];
  std::cout << "sum = " << sum << '\n';   // 500500
  return 0;
}
```

`parallel for` invokes the loop-parallelism pattern: it creates a team of threads, divides the iterations among them and joins the threads at the end of the loop. `reduction(+:sum)` tells the compiler that `sum` is a reduction over `+`: every thread gets a private copy initialised to the identity 0, accumulates into it, and the copies are combined into the shared `sum` when the loop ends, with the combination protected (`*`, `max`, `min` and the bitwise and logical operators are allowed as well).

Definition 2.7 (Directive and clause). An OpenMP directive has the form

```
#pragma omp directive [clause [clause] ...]
```

The *directive* names the construct (`parallel`, `for`, `critical`, …); a *clause* modifies it (`reduction(+:sum)`, `num_threads(4)`, `schedule(dynamic)`). The compiler replaces each directive with calls into the OpenMP runtime library, which creates, schedules and joins the threads.

2.3 Scheduling, data sharing and synchronisation

The remaining four tasks of a shared-memory program, the thread count, the schedule, the sharing policy and the synchronisation, each have an OpenMP construct.

Source M2, OpenMP vs. MPI (part 2); PPP App. A; [14].

2.3.1 How many threads

The number of threads in a parallel region is set in one of three places:

- outside the program, by the environment variable `OMP_NUM_THREADS`;
- inside the program, by the library call `omp_set_num_threads()`;
- on one region, by the clause `num_threads(`n`)`.

If nothing is set, the default is the hardware parallelism of the machine, i.e. the number of logical processors (hardware threads) the operating system reports. More threads than cores is allowed: the operating system time-slices them, which is the situation the next example starts from.

The default is implementation-defined [14]; GCC's libgomp starts one thread per CPU available to the process [17], so a 16-core CPU with two hardware threads per core starts 32 threads.

2.3.2 Loop scheduling

A `parallel for` must decide which thread executes which iterations. The decision matters when the threads do not progress at the same rate.

Example 2.8 (Threads that progress unevenly). A loop of 600 000 iterations of equal cost runs with 6 threads on a machine with 4 cores. The operating system places two threads on each of cores 0 and 1 and one thread on each of cores 2 and 3, so the four threads on the doubled-up cores each receive half a core. Let t be the time a whole core needs for 100 000 iterations.

- *Static split*, 100 000 iterations per thread: the threads on cores 2 and 3 finish at time t, the four others at $2t$. The loop ends at $2t$, with two cores idle for half of it.
- *Ideal*: 600 000 iterations spread over 4 cores is 150 000 per core, i.e. $1.5t$.
- *Dynamic, chunks of* 1000: each thread takes the next chunk when it finishes one. A thread with a whole core takes chunks twice as often as a thread with half a core, so all six finish within about one chunk of $1.5t$, after 600 chunk requests.

The static schedule takes 33 % longer than the dynamic one here, purely from the uneven placement of the threads.

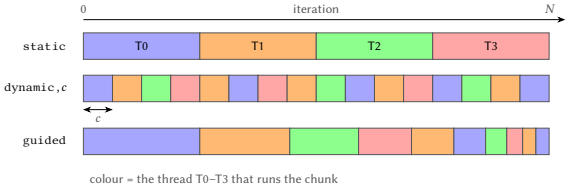


Figure 2.7: Three loop schedules for four threads. Static assigns one block per thread in advance; dynamic hands out fixed chunks in the order the threads ask for them; guided hands out shrinking chunks.

OpenMP's `schedule(`*kind*`,` *chunk*`)` clause selects the kind of schedule; the three used in practice are (Figure 2.7):

Two further kinds: `auto` leaves the choice to the compiler or runtime, `runtime` takes it from the environment variable `OMP_SCHEDULE`. [14].

static *static scheduling* divides the iterations into chunks before the loop starts and assigns them to the threads in a fixed round-robin order; with no chunk size, each thread gets one contiguous block of about $N/\text{threads}$ iterations. No run-time cost, no adaptation.

dynamic *dynamic scheduling* hands out chunks of the given *chunk size* (default 1) on request: a thread that finishes its chunk asks for the next. Chunk size 1 adapts best but pays the cost of a request, an atomic update of a shared counter, for every iteration; 1000 amortises that cost over a thousand iterations.

guided *guided scheduling* starts with large chunks and shrinks them as the loop proceeds (for instance from 1000 down to 100): the early large chunks keep the request cost low, the late small ones even out the finish times.

The guided chunk is proportional to the remaining iterations divided by the number of threads, never below the *chunk* argument except for the final chunk. [14].

Dynamic and guided scheduling adapt to anything that makes threads unequal at run time: uneven placement as in Example 2.8, a slower machine in a heterogeneous cluster, iterations of varying cost, or interference from other programs. Keeping all threads busy until the end is *load balancing*; the time of a parallel loop is the time of its slowest thread.

Tip Regular loops with equal iterations: `static`. Irregular or unknown cost: `dynamic` with a chunk large enough that a request costs far less than a chunk; `guided` when the iteration count is large.

2.3.3 Data sharing

In the sum, each thread's partial sum must be its own copy, while the total is one location. OpenMP calls the first a *private variable* (one instance per thread, as with the `reduction` copies) and the second a *shared variable* (one instance seen by all). The programmer states the policy with the `private(...)` and `shared(...)` clauses; by default, variables declared outside the parallel region are shared and the loop index is private.

Caution A variable left shared by the default and written in the loop is a data race. `default(none)` forces every variable to be listed explicitly and turns the omission into a compile error.

2.3.4 Synchronisation

Synchronisation is what makes the shared-memory program correct: it orders the accesses of different threads to shared data. OpenMP provides three constructs.

Barrier A *barrier* is a point that every thread of the team must reach before any of them continues (Figure 2.8): the work after the barrier (green) does not start until all the work before it (blue) has finished. It is written `#pragma omp barrier`; the end of a `parallel` region is always an implicit barrier, and so is the end of a `for`, `sections` or `single` construct unless a `nowait` clause removes it.

CUDA's `__syncthreads()` (Lesson 3) is a barrier over the threads of one block; a barrier over the whole grid exists only through cooperative groups with a cooperative launch (Lesson 5).

Critical `#pragma omp critical [(name)]` makes the following block a critical section: one thread at a time executes it, the others wait at its entry (Figure 2.9). With four threads in the region, the four executions of the block are serialised back to back. Critical sections with different names may run concurrently; unnamed ones all share one lock.

```
#pragma omp parallel num_threads(4)
{
  // parallel work ...
  #pragma omp critical
  {
    shared_total += local_value;   // one thread at a time
  }
}
```

Atomic `#pragma omp atomic` makes the single update that follows it an *atomic operation*: its load, add and store happen as a whole or not at all, so no other thread's update can

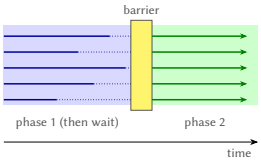


Figure 2.8: A barrier: no thread starts phase 2 before all five have finished phase 1.

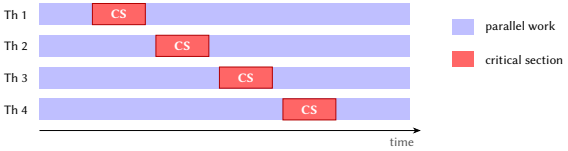


Figure 2.9: Four threads and one critical section: the red executions never overlap, the rest of the work does.

slip between them. It is implemented with a hardware read-modify-write instruction where one exists and with a lock otherwise, and is cheaper than a critical section when it applies.

CUDA's `atomicAdd()` plays the same role on the GPU; Lesson 5 lists it among the operations to use sparingly.

```
#pragma omp parallel for
for (int i = 0; i < n; ++i) {
  #pragma omp atomic
  counter++;          // load, add, store as one unit
}
```

Caution `atomic` covers one scalar update of the forms `x++`, `x += e` and the like. Two statements, or an update of two variables that must stay consistent, need `critical`.

2.3.5 Work outside loops

Not all parallel work is a loop: *parallel sections* runs independent blocks of code concurrently, one thread per section:

OpenMP 3.0 (2008) added `#pragma omp task`, which generalises `sections` to a dynamic set of tasks, the master/worker pattern of Section 2.1. [14].

```
#pragma omp parallel sections
{
  #pragma omp section
  { work_1(); }
  #pragma omp section
  { work_2(); }
}
```

Two further constructs restrict, rather than create, parallelism:

ordered Inside a loop declared `for ordered`, the block marked `#pragma omp ordered` executes in iteration order even though the rest of each iteration runs in parallel. A loop that prints "thread k is doing iteration i" from five threads prints its lines in an arbitrary order (1, 2, 4, 3, 5, say) without `ordered` and as 1, 2, 3, 4, 5 with it.

Caution Everything inside the `ordered` block is serialised; a loop whose body is mostly ordered gains nothing from being parallel.

single `#pragma omp single` runs the block on exactly one thread of the team, unspecified which, while the others skip it and wait at the implicit barrier after it; typical for a one-time initialisation or a message inside a parallel region.

Tip When the other threads need not wait for the `single` block, `single nowait` drops its barrier; the next `for` or `barrier` still orders everything.

> **Key point**
> OpenMP makes the five decisions of a shared-memory program explicit: `parallel for` (what), `OMP_NUM_THREADS` (how many), `schedule` (who does which iteration), `private`/`shared` (whose data), and `barrier`/`critical`/`atomic` (in what order). The same five decisions return in CUDA as the kernel, the launch configuration, the block scheduler, the memory spaces and `__syncthreads()`.

2.4 Programming with MPI

CUDA programming combines the two models of Section 2.2: some memory regions are shared among all the cores of a GPU, while others are invisible from outside and need explicit

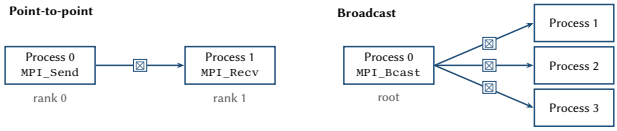


Figure 2.10: Point-to-point (left) and collective (right) communication in MPI.

communication. A GPU programmer therefore needs the distributed-memory habits as much as the shared-memory ones, which is why message passing is introduced here.

Source M2, programming with MPI; PPP App. B; [13], message passing model; PMPP ch. 20.

2.4.1 Processes and messages

Definition 2.9 (MPI). *MPI (Message Passing Interface)* is the standard library for *message passing*: a set of *processes*, each with its own address space, on the same node or on different nodes, exchange data only by sending and receiving messages. Each process is identified by its *rank*, an integer from 0 to the number of processes minus 1.

MPI Forum 1992; MPI-1 1994; version 4.1 (2023) is cited here, 5.0 (2025) is current. [13].

A message costs a latency (sending a 0-byte message, typically microseconds) plus the size divided by the bandwidth; many small messages are far worse than one large one. [13].

Two kinds of operation cover most of the library (Figure 2.10):

Point-to-point *point-to-point communication* involves exactly two processes: process 0 calls `MPI_Send()` with an integer and the destination rank 1; process 1 calls `MPI_Recv()` naming source rank 0 and obtains the value. The message is delivered only to a matching receive posted by the destination; both sides take part in every transfer.

Collective *collective communication* involves every process of a group at once. The simplest is the *broadcast*, `MPI_Bcast()`: the root process's value is delivered to all the other processes; all of them call the same function.

Other collectives: *scatter* (split one array among the ranks), *gather* (the reverse) and *reduce* (combine one value per rank, e.g. a global sum). [15].

A minimal program shows both calls. The same executable is started once per process (`mpirun -np 4 ./a.out` starts four), every copy runs it from the top, and the rank is the only thing that tells the copies apart and decides what each one does (SPMD again).

```
#include <mpi.h>
int main(int argc, char **argv) {
  int rank, value = 0;
  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  if (rank == 0) {
    value = 42;
    MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
  } else if (rank == 1) {
    MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
  }
  MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
  MPI_Finalize();
  return 0;
}
```

2.4.2 A stencil over distributed data

Definition 2.10 (Stencil). A *stencil* computation updates every element of a grid from a fixed pattern of its neighbours. The example here replaces each element C by the average of its four neighbours, north, west, east and south (Figure 2.11, left):

$$C' = \frac{N + W + E + S}{4}. \tag{2.1}$$

Stencils are named by their point count: this is the 5-point stencil (most variants read the centre too), and Equation (2.1) is one Jacobi sweep for the discrete Laplace equation, i.e. steady-state heat conduction. Image convolution with a 3×3 filter is a 9-point stencil, Lesson 3.

Stencils are common in high-performance computing (finite-difference solvers, image filters).

When the grid is distributed over four processes, each process owns one quadrant and can access only that (Figure 2.11, left). An element C at the edge of process 2's region needs neighbour 1, which belongs to process 0, and neighbour 3, which belongs to process 3. No load can fetch them; they must arrive by message.

Definition 2.11 (Halo and boundary exchange). The cells of a neighbouring region that a process reads but does not own form its *halo (ghost cells)*: for a stencil of radius 1, one row or column along each shared edge (Figure 2.11, right). Before each update step the processes

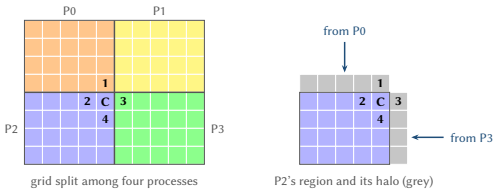


Figure 2.11: A stencil on a grid distributed over four processes. Left: the element C and its four neighbours straddle three regions. Right: process 2's region with the halo cells (grey) received from processes 0 and 3 before the update.

perform a *boundary exchange*: every process sends the edge cells its neighbours need and receives its halo from them, after which every stencil read is local.

Whenever a data dependence crosses the boundary between processes, this exchange is the price of distributed memory. The same structure returns in Lesson 3: a CUDA thread block owns a tile of the grid in its shared memory and must load the halo around the tile before computing, except that there the halo is read from global memory rather than received in a message.

> **Key point — Lesson summary**
> Parallel programs reuse a few patterns: master/worker with a task queue, loop parallelism, SPMD, fork/join and pipeline. In shared memory, threads communicate through one address space and OpenMP annotates a serial program with directives for parallel loops, thread count, scheduling (static, dynamic, guided), data sharing (private, shared) and synchronisation (barrier, critical, atomic); an unprotected update of a shared variable is a data race. In distributed memory, processes identified by rank exchange messages with MPI, point-to-point (`MPI_Send`/`MPI_Recv`) or collectively (`MPI_Bcast`). A distributed stencil needs a boundary exchange of halo cells before every step, the model for the halo loading of a CUDA tile in Lesson 3.

Each process sends and receives one message per neighbouring edge per step: with a 2×2 decomposition, two sends and two receives. Doubling the stencil radius doubles the halo width, not the message count.

Caution A process may not read a halo cell before the message carrying it has arrived, nor send its edge cells before it has updated them: a distributed stencil alternates communication and computation, and only the interior cells can be updated while the halo is still in flight.

Lesson 3

GPU Programming Introduction

Lecture	M3, lessons 1–4 (Introduction of CUDA Programming; Occupancy; Host Code; Stencil Operation with CUDA)
Sources	[18] (shared memory); NVIDIA Hopper whitepaper [2] (thread block clusters)
Textbook	K12 [3] §1.3; A18 [4] ch. 2; CPG [5] §2 Programming Model, §3.2 CUDA Runtime, §4.2 Hardware Multithreading; BPG [8] §11 Execution Configuration Optimizations; PMPP [6] ch. 2–4, 7–8
Keywords	kernel, host code, grid, thread block, thread index, execution configuration, shared memory, barrier synchronization, BSP, memory spaces, occupancy, compute capability, cudaMalloc, cudaMemcpy, stencil, tile, halo

Lesson 1 introduced the GPU as a throughput machine built from streaming multiprocessors that execute warps of threads, and Lesson 2 showed how parallel work is expressed with OpenMP in shared memory and with MPI across distributed memories. This lesson turns both into concrete CUDA programs. It introduces the kernel and the hierarchy of threads, blocks and grid in which it runs, the memory spaces those threads can see, and the barrier with which the threads of a block synchronise; it then asks how many blocks fit on one multiprocessor at once (occupancy), writes the host code that moves data to and from the device, and ends with the stencil of Lesson 2 rewritten for CUDA, where the boundary exchange of MPI becomes the loading of a halo into shared memory.

3.1 Introduction of CUDA programming

Source M3, CUDA programming introduction; K12 §1.3; CPG §2.1 Kernels, §2.2 Thread Hierarchy, §2.3 Memory Hierarchy; PMPP ch. 2–3; A18 ch. 2.

A CUDA program is split into two parts that run on two processors: code that runs on the CPU, and a function that the CPU hands to the GPU for execution by many threads at once.

3.1.1 Kernel and host code

Definition 3.1 (Kernel, host code, device code). A *kernel* is a function that runs on the GPU and is executed by many threads in parallel. It is declared with the `__global__` qualifier. The *host code* is the part of the program that runs on the CPU (the *host*, Lesson 1): it sets up the data, manages the GPU's memory and launches the kernels. The *device code* is the part that runs on the GPU, the *device*: the kernels and the functions they call.

The qualifiers: `__global__` marks a kernel (called from the host, runs on the device); `__device__` a function called from device code; `__host__` an ordinary CPU function. CPG §7.1 Function Execution Space Specifiers [5].

The first kernel adds two vectors. Each thread computes one element: it derives its own index `i` from built-in variables, checks that the index is inside the array, and performs one addition.

```
__global__ void vectorAdd(const float *a, const float *b,
                          float *c, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) c[i] = a[i] + b[i];
}
```

There is no loop over the elements: the loop of the serial program (the loop parallelism of Lesson 2) has become the set of threads, one per iteration. The host code launches the kernel with the triple-angle-bracket syntax, naming how many threads to create and how they are grouped:

```
int threads = 256;
int blocks = (n + threads - 1) / threads;  // ceil(n / 256)
vectorAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
```

Tip The number of blocks is the ceiling of n over the block size, written in integer arithmetic as `(n + threads - 1) / threads`; the last block then has idle threads, which the bounds check `if (i < n)` keeps from writing past the array.

Definition 3.2 (Kernel launch, execution configuration). A *kernel launch* is the statement `kernel<<<numBlocks, threadsPerBlock>>>(args)` by which the host starts a kernel on the device. The part between the angle brackets is the *execution configuration*: the number of thread blocks and the number of threads per block. The total number of threads created is their product.

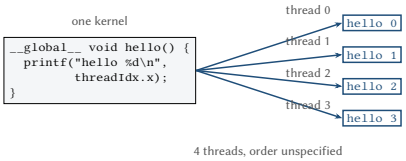


Figure 3.1: SPMD: one kernel, four threads. Every thread runs the same code; only `threadIdx.x` tells them apart, and nothing fixes the order of their output.

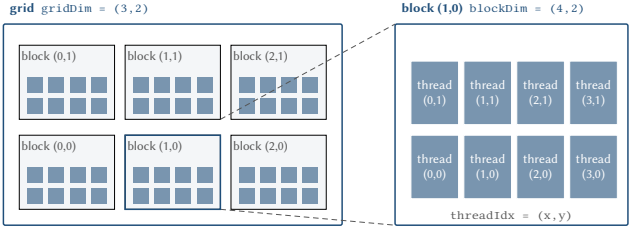


Figure 3.2: The thread hierarchy. A grid of 3 × 2 blocks, each of 4 × 2 threads; a thread is addressed by its `blockIdx` and its `threadIdx`.

3.1.2 SPMD and the thread index

The threads of a kernel all run the same code: GPU programming is *SPMD* (Lesson 1). A kernel that only prints a message, launched with four threads, prints it four times, in no particular order. To make the threads do different work, each must know which one it is.

```
__global__ void hello() {
  printf("hello %d\n", threadIdx.x);
}
// host:
hello<<<1, 4>>>();   // prints hello 0..3 in some order
```

Definition 3.3 (Thread index). The *thread index* `threadIdx` is a built-in variable, different in every thread, that gives the thread's position within its block. It has three components `threadIdx.x`, `.y` and `.z`, so that a block can be laid out as a line, a rectangle or a box of threads, matching the shape of the data (a vector, an image, a volume).

With the index, the simplest form of the vector addition is `c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x]`: thread 0 adds element 0, thread 1 element 1, and so on (Figure 3.1). This form works only for a single block, which is why the kernel above uses two more built-in variables.

3.1.3 The thread hierarchy

Definition 3.4 (Thread block, grid). Threads are grouped into blocks and blocks into a grid. A *thread block* is a group of threads that are executed together on one streaming multiprocessor and can cooperate through shared memory and barriers; PTX calls it a *cooperative thread array (CTA)*. The *grid* is the set of all blocks of one kernel launch. The *block index* `blockIdx` gives a block's position in the grid, and the built-in `blockDim` and `gridDim` hold the sizes of a block and of the grid in each dimension. This three-level structure, thread, block, grid, is the *thread hierarchy* (Figure 3.2).

The warp of Lesson 1 is absent from this hierarchy on purpose: a warp is a microarchitectural unit, the group of 32 consecutive threads of a block that the hardware issues together, and the programming model of early CUDA hid it. The block is the unit the programmer reasons about, and the grid is the unit of a launch.

Limits, CPG Table 21 [5]: 1024 threads per block; block $x, y \le 1024$, $z \le 64$; grid $x \le 2^{31}-1$, $y, z \le 65535$ (CC 5.0–9.0).

Later CUDA versions expose the warp through warp-level primitives (shuffle, vote, Lesson 5); the hardware still cuts every block into warps of `warpSize` = 32 consecutive threads, Lesson 4.

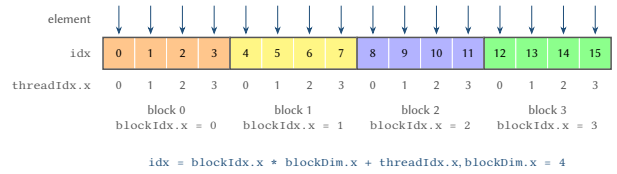


Figure 3.3: Global index of 16 elements over four blocks of four threads. The block index selects the group of four, the thread index the element within it.

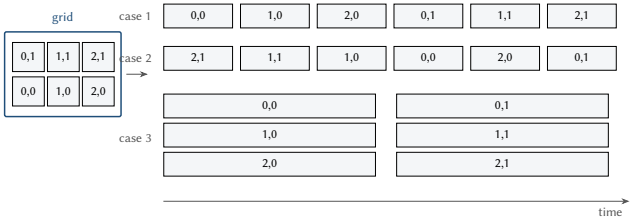


Figure 3.4: Three legal executions of a grid of six blocks: in index order, in an arbitrary order, and three at a time. The hardware chooses, and the choice may differ from one run to the next.

Global index. To give every thread of a multi-block grid a distinct element, the thread index is combined with the block index. For a one-dimensional grid and block,

$$\texttt{idx} = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x}. \tag{3.1}$$

Figure 3.3 shows 16 elements processed by four blocks of four threads: block 2 holds elements 8 to 11, and its thread 1 takes element 9.

Example 3.5 (Index arithmetic). An array of 1000 floats is processed with 128 threads per block.

- Number of blocks: $\lceil 1000/128 \rceil = 8$, creating $8 \times 128 = 1024$ threads; the last 24 threads fail the bounds check and do nothing.
- The thread with `blockIdx.x = 5` and `threadIdx.x = 77` handles element $5 \times 128 + 77 = 717$.
- Element 900 is handled by block $\lfloor 900/128 \rfloor = 7$, thread $900 - 7 \times 128 = 4$.

For a two-dimensional image of width W the same arithmetic is applied per dimension: `x = blockIdx.x * blockDim.x + threadIdx.x, y = blockIdx.y * blockDim.y + threadIdx.y`, and the element is `img[y * W + x]`.

Within a block, CPG §2.2 numbers the threads of a 2D block of size (D_x, D_y) as $x + yD_x$: consecutive `threadIdx.x` values are consecutive threads, and therefore in the same warp. [5].

3.1.4 Execution order

Nothing in the model orders the threads or the blocks. Four threads that print their index may print 0, 1, 2, 3 or 3, 2, 1, 0 or 0, 2, 3, 1; the six blocks of Figure 3.4 may run one after another in index order, one after another in any other order, or several at a time. A correct kernel produces the same result under every one of these orders.

CPG §2.2: thread blocks "are required to execute independently: It must be possible to execute them in any order, in parallel or in series." [5]. This independence is what lets a kernel run unchanged on a GPU with 2 or with 132 SMs.

The simplest mental model, used in this section, is that each block runs on one streaming multiprocessor and the blocks are independent of each other. Section 3.2 refines the first half: an SM usually holds several blocks at once.

3.1.5 Shared memory

Because the blocks of a grid are independent, data must be divided among them, and the threads of a block need a place in which to combine their work.

Latencies on Tesla and Fermi GPUs, K12 Table 1.4 [3]: shared memory 4–32 cycles, global memory hundreds of cycles. [18] gives roughly 100× lower latency for shared than for uncached global memory.

Definition 3.6 (Shared memory). The *shared memory* of a multiprocessor is a small, fast memory on the chip that is visible to all threads of one block and to no other block. It is a *scratchpad memory*: software-managed, so the kernel itself copies data into it and out of it, as opposed to a cache that the hardware fills. A variable is placed in shared memory with the `__shared__` qualifier, and lives as long as the block.

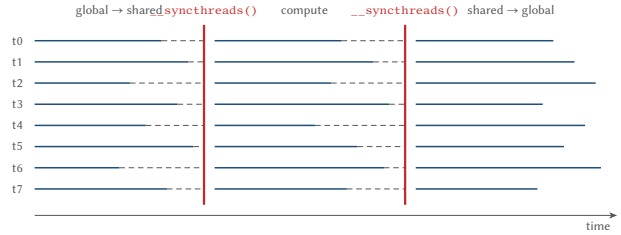


Figure 3.5: Bulk synchronous use of shared memory by eight threads of one block. Threads finish each phase at different times (solid) and wait at the barrier (dashed) until the slowest arrives.

In the kernel below each thread writes one sum into shared memory, and then reads the sum written by another thread:

```
__global__ void addReverse(const int *a, const int *b,
                           int *c) {
  __shared__ int s[256];          // one block of 256
  int i = threadIdx.x;
  s[i] = a[i] + b[i];
  __syncthreads();                // all writes done
  c[i] = s[255 - i];              // another thread's sum
}
```

The read `s[255 - i]` is correct only if thread $255-i$ has already stored its value; since threads run in no fixed order, the kernel must wait for all of them.

Caution Lesson 2 used "shared memory" in the CPU sense, one address space for all threads. CUDA shared memory is per block and on chip; a value written into it by one block is invisible to every other block, and it is gone when the block finishes.

3.1.6 Synchronisation

Definition 3.7 (Barrier synchronisation). `__syncthreads()` performs *barrier synchronization* over the threads of one block: every thread of the block waits at the call until all of them have reached it, after which all continue. It also guarantees that the shared and global memory writes made before the barrier are visible to the threads of the block after it.

A barrier spans one block only. Threads of different blocks cannot synchronise with `__syncthreads()`; when a computation needs all blocks to have finished a phase before any block begins the next, the phases are written as separate kernels, since a kernel launch completes only after all of its blocks have.

Caution Every thread of the block must reach the same `__syncthreads()`; a barrier inside an `if` that only some threads enter is undefined (in practice a hang). Put the barrier outside the condition. Since CUDA 9 a grid-wide barrier exists through cooperative groups and a cooperative launch, which needs every block resident at once (Lesson 5).

Definition 3.8 (Bulk synchronous parallel). The usual shape of a kernel that uses shared memory is three phases separated by barriers (Figure 3.5): load the block's data from global memory into shared memory; `__syncthreads()`; compute on the shared copy; `__syncthreads()`; store the results back to global memory. This alternation of a parallel phase and a barrier is the *bulk synchronous parallel (BSP)* pattern; the MPI stencil of Lesson 2, which alternates boundary exchange and computation, is the same pattern across processes.

The second barrier is needed when the compute phase writes its results into shared memory and the store phase has threads copy values produced by other threads, as in Figure 3.5, or when the buffer is reloaded for another tile (the next load would overwrite values that slow threads are still reading). A kernel whose compute phase writes each thread's result straight to global memory can omit it.

3.1.7 Several kernels

A program with several distinct tasks uses several kernels, each with its own execution configuration. A serial step is a kernel of one block and one thread; a data-parallel step is a kernel of many:

```
sortKernel<<<1, 1>>>(d_data, n);            // serial step
addKernel<<<blocks, threads>>>(d_data, d_out, n);
storeKernel<<<1, 1>>>(d_out, n);            // serial step
```

Kernels launched from the same host thread run one after another by default, so the second sees the results of the first.

Running kernels concurrently, or overlapping a kernel with a copy, needs CUDA streams, Lesson 5. A one-thread kernel uses one lane of one SM and leaves the rest of the GPU idle: serial steps belong on the host unless the data is already on the device.

Table 3.1: The CUDA memory spaces: who can see them, how long they live, where they are.

Space	Scope	Lifetime	Location, note
registers	one thread	thread	on chip, allocated by the compiler
local	one thread	thread	device memory (cached); spilled variables
shared	one block	block	on chip, `__shared__`
global	all blocks of all kernels	application	device memory; `cudaMalloc`
constant	all blocks	application	device memory, read-only, cached; `__constant__`
texture	all blocks	application	device memory, read-only, 2D cache

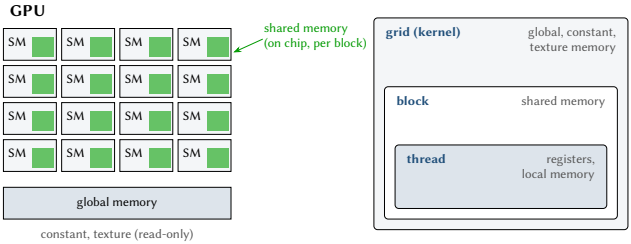


Figure 3.6: Left: each SM has its own shared memory; one global memory serves all of them. Right: the scope of each space follows the thread hierarchy.

3.1.8 Memory spaces

A thread can address five memory spaces, distinguished by who can see them and where they are (Table 3.1, Figure 3.6).

Definition 3.9 (Local, global, constant, texture memory). A thread's *local memory* is private to it; it holds the thread's variables that do not fit in registers. Shared memory is private to one block. The *global memory* is visible to every thread of every block of a kernel, and persists across kernel launches: it is where the host places the input and reads the output. The *constant memory* is a small read-only space, visible to all threads and cached, for values that do not change during a kernel, such as a physical constant or a filter. The *texture memory* is a read-only space with a cache optimised for two-dimensional locality. The last two are inherited from graphics, where they hold the constants of a scene and its texture images, and are rarely needed in general-purpose code.

Sizes (CPG Table 21 [5]): constant memory 64 KiB per device with an 8 KiB cache per SM; local memory up to 512 KiB per thread. Both live in device memory: local memory is as slow as global memory unless it hits in the L1 cache.

The hierarchy limits how information can be shared: data in a block's shared memory stays in that SM, and two blocks communicate only through global memory, in general across a kernel boundary. Since Hopper, a *thread block cluster* groups several blocks that are scheduled together and can read and write each other's shared memory, a new level between block and grid.[1]

[1] Thread block clusters (compute capability 9.0): the blocks of a cluster are guaranteed to be co-scheduled on one GPU processing cluster and access each other's shared memory as *distributed shared memory*. Up to 8 blocks per cluster is the portable size; H100 supports up to 16 (CPG §2.2.1 [5]; [2]). Lesson 5 returns to them.

> **Key point**
> A CUDA program is a host program plus kernels. A kernel runs as a grid of blocks of threads; each thread finds its data from `blockIdx`, `blockDim` and `threadIdx`, blocks run in no fixed order and must be independent, and threads of one block share an on-chip shared memory and synchronise with `__syncthreads()`.

3.2 Occupancy

A streaming multiprocessor usually runs more than one block at a time. How many, and why it matters, is the question of occupancy.

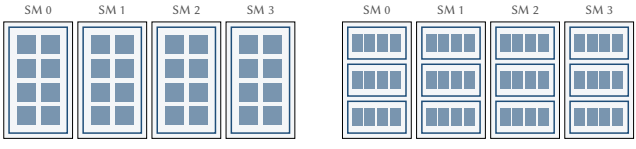
Source M3, occupancy; BPG §11.1 Occupancy, §11.3 Thread and Block Heuristics, §11.4 Effects of Shared Memory; CPG §4.2 Hardware Multithreading; PMPP ch. 4.

3.2.1 Three resources

Definition 3.10 (Resource constraint). Every block that is resident on an SM holds a share of three of the SM's resources for its whole lifetime: a number of thread slots, a number of registers, and an amount of shared memory. Each is a *resource constraint*: the number of blocks that can be resident at once is the smallest number allowed by any of the three (and by a fixed

Table 3.2: Who sets the per-block resource demand, and how to read it.

Quantity	Set by	Where it is visible
threads per block	host, at launch	the execution configuration
registers per thread	compiler	`nvcc --ptxas-options=-v`; capped by `__launch_bounds__` or `-maxrregcount`
shared memory per block	`__shared__` declarations (plus the optional third launch parameter)	same compiler report



(a) one block of 8 threads per SM
$4 \times 1 \times 8 = 32$ threads

(b) three blocks of 4 threads per SM
$4 \times 3 \times 4 = 48$ threads

Figure 3.7: Filling four SMs. (a) One block of 8 threads per SM; (b) three blocks of 4 threads per SM. The smaller blocks keep 48 threads resident instead of 32.

maximum count of blocks per SM). A *register* here is one 32-bit entry of the SM's register file; the registers a thread needs are fixed by the compiler, and a kernel that needs many is said to have high *register pressure*.

CC 7.0, 8.0 and 9.0 SMs: 65 536 registers (256 KiB), 2048 thread slots (64 warps), 32 blocks, 96 KiB to 228 KiB of shared memory; other CCs have fewer slots and blocks, Table 3.3.

Who decides each quantity is summarised in Table 3.2. Threads per block is the programmer's choice at launch. Registers per thread is the compiler's: unlike a CPU, whose ISA fixes the number of architectural registers (32 in most RISC ISAs), the GPU lets the compiler allocate as many as the kernel needs, up to a maximum, and the hardware then fits as many threads as the register file allows. Shared memory per block follows from the declarations: `__shared__ int s[1000]` is $4 \times 1000 = 4000$ bytes.

Example 3.11 (Blocks per SM). A multiprocessor offers 1024 thread slots, 32 768 registers and 48 KiB of shared memory. A kernel is launched with 64 threads per block, the compiler reports 40 registers per thread, and each block declares 6 KiB of shared memory.
threads $1024/64 = 16$ blocks;
registers $32\,768/(40 \times 64) = 12.8$, so 12 blocks;
shared memory $48/6 = 8$ blocks.
The SM can hold $\min(16, 12, 8) = 8$ blocks, limited by shared memory. Halving the shared memory per block would raise the limit to 12, after which registers bind; only then would the register count matter.

Caution The source's example quotes the register file as "64 kilobytes"; the number meant is 64 K = 65 536 registers, which at 4 bytes each is 256 KiB. Register counts in occupancy calculations are always counts of 32-bit registers, not bytes.

3.2.2 Why occupancy matters

More resident blocks mean more resident warps, and more warps give the SM more independent instructions to choose from while others wait for memory. Figure 3.7 compares two ways of filling four SMs: one block of 8 threads each (32 threads) or three blocks of 4 threads each (48 threads). The second keeps more threads in flight even though each block is smaller, and so, usually, achieves more of the SM's throughput.

Why more warps hide memory latency is the subject of Lesson 4 (multithreaded architecture); Lesson 7 shows the exception, where too many warps thrash the L1 cache.

Definition 3.12 (Occupancy, active warp). An *active warp* is a warp that is resident on a multiprocessor, with its registers and shared memory allocated, and therefore a candidate for issue whenever its next instruction is ready; a warp that is ready in the current cycle is called *eligible*. The *occupancy* of a kernel on a multiprocessor is the ratio of the number of active warps to the maximum number of warps the multiprocessor can hold [8, §11.1]. With T threads per block, R registers per thread and S bytes of shared memory per block, on an SM with the limits of Table 3.3,

$$\text{blocks/SM} = \min\left(\left\lfloor \frac{\text{Threads/SM}}{T} \right\rfloor, \text{Blocks/SM}, \left\lfloor \frac{\text{Regs/SM}}{RT} \right\rfloor, \left\lfloor \frac{\text{Smem/SM}}{S} \right\rfloor\right), \tag{3.2}$$

$$\text{occupancy} = \frac{\text{blocks/SM} \times \lceil T/32 \rceil}{\text{Warps/SM}}. \tag{3.3}$$

Equation (3.3) gives the *theoretical* occupancy; the *achieved* occupancy that Nsight Compute measures at run time is lower when blocks finish unevenly or the grid is too small to fill every SM.

Table 3.3: Per-SM limits by compute capability (CPG Table 21 [5]; K = 1024, KiB binary). The example GPUs are representative, not exhaustive.

Compute capability (example GPU)	7.0 V100	7.5 T4	8.0 A100	8.6 RTX 3090	8.9 L40	9.0 H100
max threads per SM	2048	1024	2048	1536	1536	2048
max warps per SM	64	32	64	48	48	64
max blocks per SM	32	16	32	16	24	32
32-bit registers per SM	64 K	64 K	64 K	64 K	64 K	64 K
max registers per thread	255	255	255	255	255	255
max shared memory per SM (KiB)	96	64	164	100	100	228
max shared memory per block (KiB)	96	64	163	99	99	227

Definition 3.13 (Compute capability). The *compute capability* is NVIDIA's version number for the features and limits of a GPU generation, written *major.minor*: 7.x Volta and Turing, 8.x Ampere and Ada, 9.0 Hopper. The limits of Table 3.3 are per compute capability, which is why an occupancy calculation must name the target: the x.0 parts (V100, A100, H100) hold 64 warps, CC 7.5, 8.6 and 8.9 (e.g. T4, RTX 3090, L40) only 32 or 48.

On CC 8.x and 9.0 the shared memory is carved out of a unified L1/shared array, and the split is configurable per kernel (CPG, Compute Capability 8.x [5]); the table gives the maximum. The warp scheduler that consumes the warps: Lesson 4.

Example 3.14 (Occupancy on an A100). A kernel is launched with $T = 256$ threads per block (8 warps), uses $R = 64$ registers per thread and $S = 48$ KiB of shared memory per block, on a compute capability 8.0 SM.
threads $2048/256 = 8$ blocks;
blocks 32;
registers $65\,536/(64 \times 256) = 4$ blocks;
shared memory $\lfloor 164/48 \rfloor = 3$ blocks.
Three blocks are resident, $3 \times 8 = 24$ warps, and the occupancy is $24/64 = 37.5\,\%$. Shared memory binds; cutting it to 32 KiB per block would allow 4 blocks (then registers bind) and $32/64 = 50\,\%$. Full occupancy (8 blocks) would need at most 32 registers per thread and at most about 19 KiB of shared memory per block ($164/8 = 20.5$ less the 1 KiB reservation).

Granularity, ignored in Equation (3.2): registers are allocated per warp in units of 256, and on CC 8.x and 9.0 CUDA reserves 1 KiB of shared memory per block ($\lfloor 164/49 \rfloor$ is still 3). BPG §11.1.1; CPG, Compute Capability 8.x [8, 5].

In practice the arithmetic is done by the occupancy calculator of Nsight Compute or by the runtime's occupancy API,

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &blocks, kernel, T, dynSmem);
```

where `dynSmem` is the dynamic shared memory per block (the static `__shared__` usage is taken from the kernel itself); both tools apply Equation (3.2) with the exact granularities of the target device (BPG §11.1.1).

Higher occupancy is not always faster. Beyond some point extra warps add nothing, and a kernel limited by instruction throughput or by its own memory bandwidth gains nothing from them; but low occupancy always weakens the SM's ability to hide memory latency [8, §11.1].

Tip Starting points from BPG §11.3: threads per block a multiple of the warp size; at least 64 per block, and then only with several blocks per SM; 128–256 as the first thing to try; several smaller blocks rather than one large block when latency matters.

> **Key point**
> Blocks per SM = the minimum over threads, registers, shared memory (and the block cap); occupancy = resident warps over the maximum. Threads per block is chosen at launch, registers per thread by the compiler, shared memory by the declarations, and the limits by the compute capability.

3.3 Host code

Shared memory is visible within a block and global memory to all blocks, but neither is visible to the CPU: global memory sits in the device's own memory, behind an I/O interface, and the host manages it through explicit calls.

Source M3, host code; CPG §3.2.2 Device Memory; K12 §1.3.6.

Definition 3.15 (Host memory, device memory). The *host memory* is the CPU's main memory, where the host program's variables live. The *device memory* is the GPU's own memory (HBM or GDDR), in which global, constant, texture and local memory are placed. The two are separate address spaces joined by an I/O interface (Figure 3.8): a pointer into one is meaningless on the other side, and data crosses only by an explicit copy.

The interface is the bottleneck. H100: HBM3 reported as about 3 TB/s (preliminary) [2] against 64 GB/s per direction of PCIe Gen 5 x16, about 47×. A100 40 GiB: HBM2 at 1555 GB/s [1] against about 32 GB/s per direction of Gen 4, about 49×.

Allocating device memory, moving data to and from it, and releasing it are three calls of the

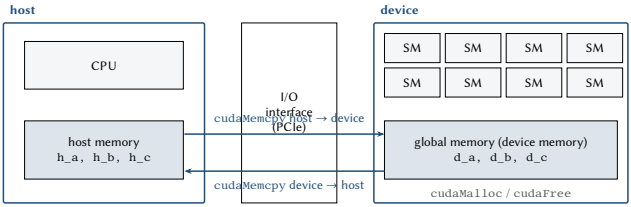


Figure 3.8: Host and device memory. `cudaMalloc` reserves global memory; `cudaMemcpy` moves data across the I/O interface in either direction.

CUDA runtime:

```
cudaMalloc((void **)&ptr, size);   // reserve size bytes
cudaMemcpy(dst, src, size, dir);   // copy size bytes
cudaFree(ptr);                     // release
```

Definition 3.16 (Memory allocation and transfer). On the device, *memory allocation* is done by `cudaMalloc`, which reserves `size` bytes of global memory and returns a device pointer in `ptr`; `cudaFree` releases it. `cudaMemcpy` copies `size` bytes in the direction `dir`, host to device (`cudaMemcpyHostToDevice`), device to host (`cudaMemcpyDeviceToHost`) or device to device. Every runtime call returns a `cudaError_t`; `cudaSuccess` means it worked. `cudaMemcpy` is synchronous with respect to the host: a device-to-host copy returns when the data has arrived, and a copy issued after a kernel launch first waits for the kernel to finish.

Asynchronous copies from pinned host memory, overlapped with kernels: Lesson 5.

The complete vector addition, kernel and host code, is then as follows (the kernel is the one of Section 3.1). The three memory-management steps, allocate, copy in, copy out, surround the launch; the host also checks the launch for an error.

Caution A kernel launch returns to the host immediately and reports no error of its own. `cudaGetLastError()` catches configuration errors (too many threads per block, too many resources for the SM) at launch; an error inside the kernel surfaces at the next synchronising call, such as the `cudaMemcpy` that reads the result. Check both return values in real code.

```
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *a, const float *b,
                          float *c, int n) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) c[i] = a[i] + b[i];
}

int main() {
  const int n = 1 << 20;                 // 1,048,576
  const size_t size = n * sizeof(float);
  float *h_a = (float *)malloc(size);
  float *h_b = (float *)malloc(size);
  float *h_c = (float *)malloc(size);
  for (int i = 0; i < n; ++i) {
    h_a[i] = i; h_b[i] = 2.0f * i;
  }

  float *d_a, *d_b, *d_c;                // device pointers
  cudaMalloc((void **)&d_a, size);
  cudaMalloc((void **)&d_b, size);
  cudaMalloc((void **)&d_c, size);
  cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
  cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

  int threads = 256;
  int blocks = (n + threads - 1) / threads;   // 4096
  vectorAdd<<<blocks, threads>>>(d_a, d_b, d_c, n);
  cudaError_t err = cudaGetLastError();
  if (err != cudaSuccess)
    printf("launch: %s\n", cudaGetErrorString(err));

  cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
  printf("%f\n", h_c[1000]);             // 3000.000000
```

```
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    free(h_a); free(h_b); free(h_c);
    return 0;
}
```

Example 3.17 (Cost of the copies). With $n = 2^{20}$ floats, each array is 4 MiB: 8 MiB go to the device and 4 MiB come back, 12 MiB in all, for 2^{20} additions. At a sustained 25 GB/s over PCIe the copies take about 0.5 ms; the kernel itself, reading 8 MiB and writing 4 MiB from device memory at 1.5 TB/s, needs under 10 µs. The transfer dominates by a factor of about 60: a vector addition alone is not worth moving to the GPU, and data should stay on the device across as many kernels as possible.

Definition 3.18 (Unified memory). With *unified memory* (`cudaMallocManaged`) a single pointer is usable from both host and device: the runtime and the GPU's memory management move pages between host and device memory on demand, and the explicit `cudaMemcpy` calls disappear. The copies still happen, as page migrations triggered by page faults, and their cost is the subject of Lesson 7.

Lesson 7 gives the numbers: a GPU page fault costs tens of microseconds and the driver handles faults in batches, so unified memory simplifies the code but does not remove the transfer.

3.4 Stencil operation with CUDA

The stencil of Lesson 2 replaces each element C of a grid by the average of its four neighbours, $C' = (N + W + E + S)/4$. Written for CUDA it brings together everything above: a grid of blocks that divide the data, shared memory for the values each block reuses, a barrier between loading and computing, and a halo at the block edges.

Source M3, stencil with CUDA; PMPP ch. 7 (convolution) and ch. 8 (stencil); BPG §10.2.3 Shared Memory.

3.4.1 Decomposition and reuse

The data decomposition is the one of Figure 3.9, left: each block owns one region of the image and each of its threads computes one output element, so all outputs are computed in parallel. Every interior input element is read by four threads, those of its four neighbours (five, if the stencil includes the centre).

Definition 3.19 (Data reuse, tile, tiling). The *data reuse* of a computation is the property that one value loaded from memory serves several of its operations. A *tile* is the rectangular sub-array that one block works on, and *tiling* is the technique of loading a tile once from global memory into shared memory and serving all the reuse from there, so that each global element is read about once per block instead of once per use. The reuse pattern of a stencil is known in advance, which is what makes a software-managed scratchpad the right cache.

The same tiling turns a matrix multiplication from one global load per multiply into one per tile size multiplies; Lesson 5 treats it, along with the bank conflicts of Lesson 4 that a badly laid out tile can cause.

Each thread therefore follows the BSP shape of Figure 3.5: load one element from global memory into the block's shared tile; `__syncthreads()`; compute its stencil from the tile; write the result to global memory.

3.4.2 Boundaries and the halo

Shared memory is per block, but an element at the edge of a block's region has neighbours in the regions of other blocks: in Figure 3.9, C at the corner of block (0,1) needs neighbour 1 from block (0,0) and neighbour 3 from block (1,1). Those blocks' shared memories are unreachable, so each block loads, in addition to its own region, the one-element border around it, the *halo* of Lesson 2, from global memory into its tile (Figure 3.9, right). This is the boundary exchange of the MPI stencil without the messages: the halo is simply read from global memory, which every block can see, and no synchronisation between blocks is needed because the input array is read-only during the kernel and the outputs go to a separate array.

Definition 3.20 (Boundary condition, zero padding). A *boundary condition* states what a stencil reads where a neighbour lies outside the array. The simplest is *zero padding*: positions outside the array are treated as zero, and the load phase stores 0 into the tile instead of reading global memory. Other choices (clamping to the edge value, wrapping around, a one-sided formula) only change that branch of the load.

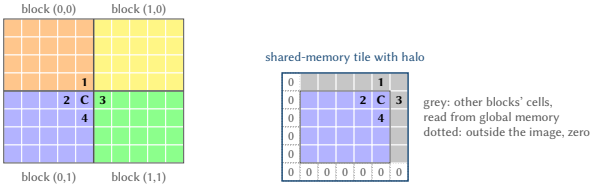


Figure 3.9: Left: a 10 × 8 image split among four blocks; C and its neighbours 1–4 lie in three of them. Right: the shared-memory tile of block (0,1): its own 5 × 4 region plus a one-cell halo. Halo cells inside the image (grey) are loaded from global memory; those outside it (dotted) receive zero.

3.4.3 From stencil to convolution

Rather than writing the average with four reads and an `if` for each boundary, the kernel below is written as a small convolution: a 3 × 3 filter is applied around every element, and the stencil is expressed as the filter's coefficients. This removes special cases and makes the same kernel serve any 3 × 3 stencil.

Definition 3.21 (Convolution, filter). A two-dimensional *convolution* computes each output as a weighted sum of the input elements around the same position; the array of weights is the *filter* (or kernel, a word avoided here because of its CUDA meaning). Convolution in the signal-processing sense flips the filter; with a symmetric filter the two agree, and the machine-learning usage followed here does not flip. With a filter f of radius r,

$$\mathrm{out}[y][x] = \sum_{i=-r}^{r} \sum_{j=-r}^{r} \mathrm{in}[y+i][x+j] \cdot f[i+r][j+r]. \tag{3.4}$$

The four-neighbour average is the 3×3 filter with 0.25 at the four edge-centres and 0 elsewhere; a 3 × 3 filter is a 9-point stencil, and the halo it needs is $r = 1$ wide.

3.4.4 The kernel

The design below launches a block of $(TILE + 2r)^2$ threads for a tile of $TILE^2$ outputs: every thread loads one element of the tile including the halo, and only the inner threads, those whose element is not a halo cell, compute and store an output (Figure 3.10). The filter lives in constant memory.

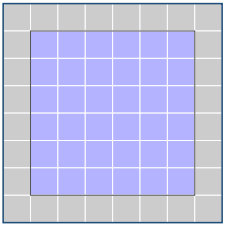


Figure 3.10: Thread roles in one block (drawn for 8 × 8 threads): the outer ring only loads halo cells; the inner threads load and compute.

```
#define TILE 14              // outputs per block side
#define RADIUS 1             // filter radius r
#define BLOCK (TILE + 2 * RADIUS) // 16 per side

__constant__ float filter[2 * RADIUS + 1][2 * RADIUS + 1];

__global__ void stencil2D(const float *in, float *out,
                          int width, int height) {
  __shared__ float tile[BLOCK][BLOCK];
  int tx = threadIdx.x, ty = threadIdx.y;
  int x = blockIdx.x * TILE + tx - RADIUS;  // image column
  int y = blockIdx.y * TILE + ty - RADIUS;  // image row

  // 1. every thread loads one cell, zero outside the image
  if (x >= 0 && x < width && y >= 0 && y < height)
    tile[ty][tx] = in[y * width + x];
  else
    tile[ty][tx] = 0.0f;
  __syncthreads();

  // 2. inner threads compute and 3. store one output each
  bool inner = tx >= RADIUS && tx < BLOCK - RADIUS &&
               ty >= RADIUS && ty < BLOCK - RADIUS;
  if (inner && x < width && y < height) {
    float acc = 0.0f;
    for (int i = -RADIUS; i <= RADIUS; ++i)
```

```
      for (int j = -RADIUS; j <= RADIUS; ++j)
        acc += tile[ty + i][tx + j]
             * filter[i + RADIUS][j + RADIUS];
    out[y * width + x] = acc;
  }
}
```

The host sets the filter once with `cudaMemcpyToSymbol` and launches with a two-dimensional configuration:

```
dim3 block(BLOCK, BLOCK);   // 16 x 16 threads
dim3 grid((width + TILE - 1) / TILE,
          (height + TILE - 1) / TILE);
stencil2D<<<grid, block>>>(d_in, d_out, width, height);
```

The index arithmetic is Equation (3.1) shifted by r: thread $(0,0)$ of block $(0,0)$ loads image position $(-1,-1)$, which is outside and becomes zero, and thread $(1,1)$ loads $(0,0)$, the first output. The offset of one block is `TILE`, not `BLOCK`, because neighbouring tiles overlap by the halo.

Caution One barrier suffices here: the tile is loaded once and only read afterwards. A kernel that reuses the shared buffer for several tiles needs a second `__syncthreads()` after the compute phase.

Example 3.22 (A 64×64 image). With $TILE = 14$ and $r = 1$ on a 64×64 image:

- grid $\lceil 64/14 \rceil \times \lceil 64/14 \rceil = 5 \times 5 = 25$ blocks of $16 \times 16 = 256$ threads, 6400 threads for 4096 outputs; in each of the 16 interior blocks 196 threads compute and 60 only load;
- shared memory per block $16 \times 16 \times 4 = 1$ KiB;
- tile loads: $25 \times 256 = 6400$, of which $72^2 = 5184$ read global memory (the rest are zero fills; per dimension the blocks read $15 + 16 + 16 + 16 + 9 = 72$ positions), against $9 \times 4096 = 36\,864$ if every thread read its nine inputs from global memory, a factor of about 7; the halo overhead is $256/196 = 1.31$ loads per output for an interior block ($5184/4096 \approx 1.27$ over the whole image).

The last block in each dimension covers columns 56–69 of a 64-wide image: its threads with $x \geq 64$ load zeros and skip the store.

Tip Halo fraction $1 - (TILE/BLOCK)^2$: 23 % of the threads here, 12 % for a 32×32 block. Larger tiles waste fewer threads but need more shared memory and leave fewer blocks per SM (Section 3.2).

All threads of a warp read the same filter coefficient at once; the constant cache serves one address to the whole warp as a broadcast, which is what constant memory is for.

The alternative design, in which the block has exactly $TILE^2$ threads and the halo is loaded by extra passes of the edge threads, launches no idle threads but needs more conditional code; PMPP ch. 8 shows both.

Key point — Lesson summary

A CUDA program is host code that allocates device memory (`cudaMalloc`), copies data in and out (`cudaMemcpy`) and launches kernels with an execution configuration `<<<blocks, threads>>>`. A kernel runs as a grid of independent thread blocks; each thread computes its global index from `blockIdx`, `blockDim` and `threadIdx`, blocks run in no fixed order, and the threads of one block share an on-chip shared memory and synchronise with `__syncthreads()` in the BSP pattern load, barrier, compute, store. The five memory spaces, local, shared, global, constant and texture, follow the thread hierarchy in scope. The number of blocks resident on an SM is the minimum allowed by its thread slots, registers and shared memory, and occupancy is the fraction of the SM's warp slots they fill; higher occupancy hides latency better but is not always faster. A stencil becomes a tiled kernel: each block loads its tile plus a halo into shared memory, with zero padding at the image edge, synchronises once, and computes one output per inner thread.

Lesson 4

GPU Architecture

Lecture	M4, lessons 1–4 (Multi-threaded Architecture; Bank Conflicts; GPU Architecture Pipeline; Global Memory Coalescing)
Sources	PTX ISA [19] (special registers); Harris, *Using Shared Memory in CUDA C/C++* [18]; NVIDIA A100 [1] and H100 [2] whitepapers (HBM figures)
Textbook	A18 [4] ch. 3 (assigned), ch. 4; K12 [3] §1.4.1 GPU Pipeline, §1.4.3 GPU Memory Systems; CPG [5] §4.2 Hardware Multithreading, §5.3.2 Device Memory Accesses; BPG [8] §10.2.1 Coalesced Access to Global Memory, §10.2.3 Shared Memory; PMPP [6] ch. 4–6
Keywords	latency hiding, fine-grained multithreading, context switch, program counter per warp, register file, port, bank, bank conflict, operand collector, scoreboard, active mask, special register, memory coalescing, memory transaction, SRAM, DRAM, HBM

Lesson 1 named the parts of a streaming multiprocessor and Lesson 3 showed how a kernel's threads are packed into blocks and warps and how many of them an SM can hold. This lesson opens the SM and follows a warp instruction through it. It begins with the reason the SM is multithreaded at all: a load that misses in the cache would otherwise stall the pipeline, and a GPU hides that latency by switching among warps whose program counters and registers are all kept on chip. The register file that makes this possible is so wide that it must be split into banks, which introduces bank conflicts, operand collectors and the scoreboard that selects a ready warp. A complete pipeline walk-through on a small example then shows fetch, decode, register read, execution and write-back for a warp, including the mask bits that switch lanes off. The lesson closes with the memory side: why a warp can generate 32 memory requests from one instruction, what SRAM, DRAM and HBM are, and how coalescing turns those requests into a few wide transactions. Lesson 5 turns these mechanisms into optimisation rules, and Lessons 6 and 7 return to the register file and the warp scheduler.

4.1 Multi-threaded architecture

A GPU consists of many SMs, each of which runs its warps by multithreading and holds a block of shared memory and hardware caches (the SM diagram of Lesson 1). This section explains what the multithreading buys, what state it requires, and how that state differs from a CPU's.

Source M4, multithreading; K12 §1.2.1; A18 ch. 3; CPG §4.2 Hardware Multithreading.

4.1.1 Three ways to survive a cache miss

Consider the four-instruction program of Figure 4.1 on the five-stage pipeline of Lesson 1 (front end, decode and rename, schedule, execute, write-back). The first instruction, `I1`, loads a word from memory; `I4` needs the loaded value, while `I2` and `I3` are independent of it. Suppose the load suffers a *cache miss*: the word is not in the cache and must come from DRAM, so the execute stage of `I1` cannot finish for many cycles. The pipeline *stalls*: cycles pass in which no instruction completes. Three designs react differently.

Latency scale: global-memory accesses cost "100s of cycles" on the Tesla and Fermi GPUs of K12 Table 1.4 [3]; an on-chip shared-memory access there costs 4–32 cycles.

In-order execution (Figure 4.1a) issues instructions in program order, so `I2`, `I3` and `I4` all wait behind the miss, although only `I4` depends on it.

Out-of-order execution (Figure 4.1b) lets `I2` and `I3` complete during the miss; only `I4` waits. The gain is limited by the independent instructions the hardware can find in one thread.

Multithreading (Figure 4.1c) issues `I1` of four different threads in four consecutive cycles. Instructions from different threads are independent by construction, so no search for independence is needed, and the four misses are outstanding at the same time: four memory requests overlap instead of being served one after another.

Definition 4.1 (Latency hiding by multithreading). Keeping the pipeline busy with useful instructions while a long-latency operation (a cache miss, a long ALU operation, a branch resolution) is in flight is called *latency hiding*. Holding the state of many threads on chip and switching among them at every cycle, so that a stalled thread simply is not selected, is *fine-grained multithreading*. The number of memory requests a processor keeps in flight at once is its *memory-level parallelism (MLP)*; the multithreaded pipeline of Figure 4.1c has an MLP of four.

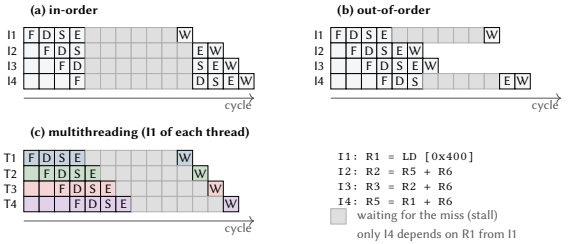


Figure 4.1: One program whose first instruction misses in the cache: (a) in-order, every later instruction waits; (b) out-of-order, only the dependent `I4` waits; (c) multithreading, four threads each issue their load and the four misses overlap.

Where a CPU spends its area on out-of-order logic, large caches and the search for ILP, a GPU spends it on threads: the stalls caused by cache misses, branches and long-latency arithmetic are covered by the other warps of the SM. The price is stated by one rule: the longer the memory latency, the more threads are needed to hide it.

Tip Threads needed ≈ latency ÷ cycles of independent work each thread supplies between two long-latency operations. Doubling the latency doubles the thread count; so does halving the independent work.

Example 4.2. An SM issues one warp instruction per cycle. Each warp, after issuing a load, can issue two more instructions before it needs the loaded value, and the load takes 400 cycles. To keep the issue slot busy for the whole 400 cycles the SM needs about $400/3 \approx 134$ warps that can each contribute three instructions; with a 800-cycle latency it needs about 267. No real SM holds that many warps (the limit is 64), which is why GPUs also rely on caches, on independent instructions within a warp and, as Section 4.4 shows, on cutting the number of memory requests.

Maximum resident warps per SM: 64 on compute capabilities 7.0, 8.0 and 9.0, 48 on 8.6 and 8.9, 32 on 7.5: CPG Table 21 [5]; the occupancy of Lesson 3 is measured against this limit.

4.1.2 Program counters and registers per thread

Multithreading changes the front end of the pipeline (Figure 4.2). A single-threaded core keeps one *program counter (PC)*, the address of the next instruction to fetch. An SM keeps one PC per warp: four resident warps mean four PC registers, and in every cycle the fetch stage reads the PC of the warp it has selected and fetches one instruction for all the threads of that warp. One static instruction thus serves a whole warp, which is the SPMD model of Lesson 1 realised in hardware. Behind the front end, the *register file* holds a private set of registers for every thread of every resident warp, so that switching from one warp to another needs no copying: the registers of the next warp are already there.

Per SM the register file holds 65 536 32-bit registers, 256 KiB, on every compute capability from 5.0 to 9.0: CPG Table 21 [5]. Fermi (2010) had 128 KiB per SM: K12 §1.4.1 [3].

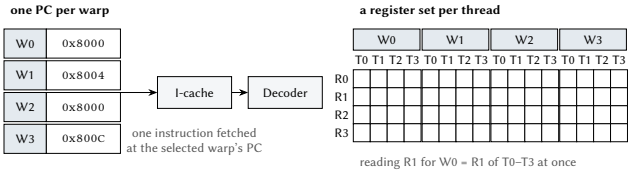


Figure 4.2: The front end of a multithreaded SM: one program counter per warp selects the instruction to fetch, and the register file keeps a register set for every thread, grouped by warp, so that reading `R1` for warp 0 reads `R1` of its four threads at once.

Definition 4.3 (Context switch). A *context switch* replaces the thread whose state (PC, registers) the pipeline is executing with another thread. On a CPU the pipeline, the PC and the registers belong to one thread at a time, so the switch stores the running thread's state to memory and loads the next thread's (Figure 4.3), a cost of many cycles. On a GPU every resident warp's PC and registers are already on chip; the switch only changes which PC and which register set the pipeline uses, and costs nothing.

CPG §4.2 Hardware Multithreading: the execution context of each warp "is maintained on-chip during the entire lifetime of the warp. Therefore, switching from one execution context to another has no cost" [5].

Caution The free GPU switch applies only to warps that are resident on the SM. Replacing a finished thread block by a new one does allocate registers and shared memory and is not free; Lesson 7 treats warp scheduling.

Note 4.4. The word *context switch* is used in two senses. An operating system's context switch saves a process or thread to memory and is expensive; the GPU's "context switch" between warps is a selection among states that are all resident and is free. A CPU with simultaneous

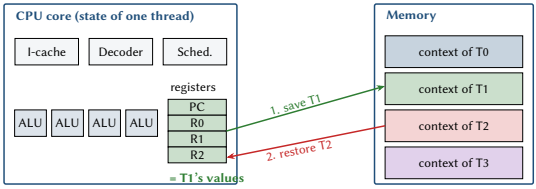


Figure 4.3: A CPU context switch: the core holds one thread's PC and registers, so switching from `T1` to `T2` saves `T1`'s state to memory and restores `T2`'s.

multithreading holds two thread contexts per core; an SM holds hundreds of threads.

K12 §1.2.1 contrasts the two hyperthreads of a CPU core with the hundreds of threads of an SM [3].

4.1.3 Resource requirements

The state that multithreading keeps on chip is counted in two quantities.

- *Program counters.* One per warp, so the number of PCs is the number of resident threads divided by the warp size. Later GPUs keep more front-end state than this; for the model of this book one PC per warp is assumed.
- *Registers.* K registers per thread times the number of resident threads, where K is chosen by the compiler for each kernel and varies from application to application. A larger K means fewer resident threads for a fixed register file, which is the register limit of the occupancy calculation of Lesson 3.

Since Volta, independent thread scheduling keeps a program counter and call stack per *thread*, so that diverged threads can be interleaved: CPG §4.1 SIMT Architecture [5]. Lesson 5 discusses it; Lesson 6 keeps the per-warp model.

Example 4.5. An SM holds 512 resident threads with a warp size of 32, so it keeps $512/32 = 16$ program counters. A kernel of 12 instructions executed by all of them costs $16 \times 12 = 192$ warp-instruction fetches, whereas the same work on single-threaded cores would need $512 \times 12 = 6144$ instruction fetches: one fetch serves 32 threads. If the compiler assigns $K = 40$ registers per thread, the resident threads occupy $40 \times 512 = 20\,480$ registers, 80 KiB at 4 B each.

Tip Per SM: PCs = threads ÷ warp size; registers = $K\times$ threads; a GPU repeats both in every SM. Both scale with the thread count, so the latency-hiding rule above is a statement about on-chip storage.

> **Key point**
> A GPU hides memory latency with fine-grained multithreading rather than with out-of-order execution: every resident warp's PC and registers stay on chip, so switching warps costs nothing, and the number of warps an SM can hold is a storage question (one PC per warp, K registers per thread).

4.2 Bank conflicts

The register file of the previous section must deliver operands for a whole warp at once, which makes its bandwidth, not its capacity, the design problem. This section introduces ports and banks, the conflicts banking creates in the register file and in shared memory, and the operand collector and scoreboard that absorb them.

Source M4, bank conflicts; K12 §1.4.1 (register read/write, scoreboarding); A18 ch. 3 (operand collector); BPG §10.2.3 Shared Memory; CPG Compute Capability 5.x, Shared Memory; [18].

4.2.1 Register file bandwidth

A typical arithmetic instruction reads two source registers and writes one destination. An SM executes an instruction for many threads at once: with an execution width of 16 lanes, one instruction moves $16 \times 3 = 48$ register values in a cycle, and a wider SM proportionally more. A memory array that can be read and written through that many paths at once is what the hardware must provide.

K12 §1.4.1 scales this up: a GPU of 1 TFLOP/s with two reads and one write per operation needs on the order of 64 TB/s of register read bandwidth [3], far beyond any single memory array.

Definition 4.6 (Port). A *port* is a hardware interface through which one value can be read from or written to a storage array in a cycle; a *read port* carries a value out, a *write port* carries one in. Each port is a set of wires and selection logic attached to every entry of the array, so its cost grows with the number of entries, and an array with many ports is large and slow.

Figure 4.4a shows the brute-force design for an execution width of 4: every register entry carries $2 \times 4 = 8$ read ports and 4 write ports, so that any 8 values can be read and any 4 written in one cycle. Figure 4.4b reaches the same throughput more cheaply by splitting the array.

Definition 4.7 (Bank). A *bank* is one of several independent storage arrays into which a register file or memory is divided, each with its own (few) ports. Banks are accessed simultaneously, so B banks with p ports each deliver up to Bp values per cycle as long as the values requested lie in different banks. In Figure 4.4b four banks with 2 read and 1 write port each give the 8 reads and 4 writes of the unbanked design with a quarter of the ports per entry.

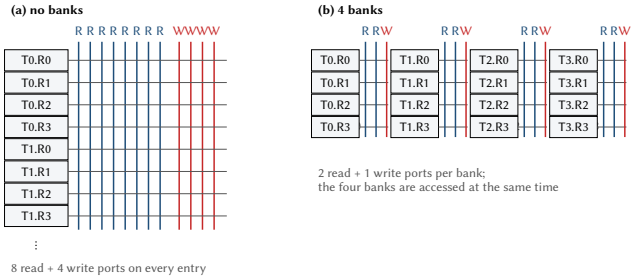


Figure 4.4: Ports versus banks for an execution width of 4. (a) Without banks every entry needs 8 read and 4 write ports. (b) Four banks with 2 read and 1 write ports each are accessed at the same time.

4.2.2 Register bank conflicts

The saving has a condition: the values a cycle needs must be spread over the banks.

Definition 4.8 (Bank conflict). A *bank conflict* occurs when one access (one warp instruction) needs more values from a single bank than that bank has ports. The hardware serialises the access over several cycles, one per group of values the bank can serve. In a register file this is a *register bank conflict*.

Figure 4.5 assigns the registers of thread k to bank k in a four-bank file with two read ports per bank.

- Reading `R0` of threads `T0`–`T3`, which is what a warp instruction does, touches one entry in each bank: four values in one cycle, as fast as the unbanked file.
- Reading `R0`–`R3` of thread `T0` touches four entries of bank 0, which has two read ports: two cycles.

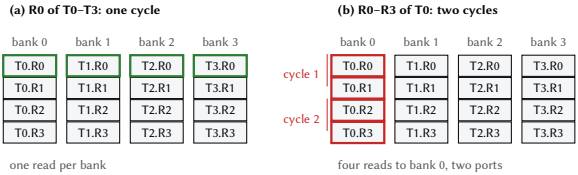


Figure 4.5: A four-bank register file with two read ports per bank. (a) `R0` of four threads lies in four banks and is read in one cycle. (b) Four registers of one thread lie in one bank and take two cycles.

Real designs spread a thread's registers over the banks (register r of a thread in bank r mod B, for instance), so that the conflicts come from the registers an instruction names, not from the thread count; the choice of layout is part of the compiler's and the hardware's contract.

The first case looks safe, since a warp always reads the same register number for all its threads. It stops being safe once the number of registers per thread varies, because the thread-to-bank layout then changes (Figure 4.6). With four registers per thread and one thread per bank (case 1), the instruction `R2 = R0 + R1` reads two values from every bank and completes in one cycle. With two registers per thread (case 2) two threads share a bank: for a warp of `T0`–`T3` the same instruction sends four reads to bank 0 (`R0` and `R1` of `T0` and `T1`) and four to bank 1, and takes two cycles while banks 2 and 3 idle.

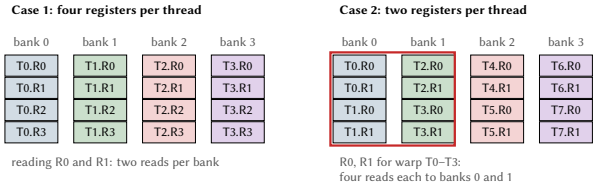


Figure 4.6: Thread-to-bank layout with a variable register count. Case 1: four registers per thread, one thread per bank. Case 2: two registers per thread, two threads per bank, so an instruction reading `R0` and `R1` for warp `T0–T3` sends four reads to each of banks 0 and 1.

4.2.3 Static and dynamic

The first remedy is the compiler. Register numbers are fixed in the program text, so the compiler can choose which registers an instruction uses and in what order instructions are emitted so that the operands of one instruction fall into different banks. It cannot remove every conflict, because the dependences of the program constrain the order and the number of registers constrains the choice. The distinction behind this remedy is used throughout the compiler lessons.

Register allocation, the compiler pass that assigns program values to registers, builds on the liveness analysis of Lesson 11; on a GPU the bank layout is one of its inputs.

Definition 4.9 (Static and dynamic). A property of a program is *static* when it is determined before the program runs and does not depend on its input; it is *dynamic* when it depends on the input and is known only during execution. Static analysis is compile-time analysis. The *static instruction count* is the number of instructions in the program text; the *dynamic instruction count* is the number of instructions executed in one run.

Example 4.10. The loop below sums 16 words and consists of five instructions.

```
LOOP: LD   R2, [R3]      // load a word
      ADD  R1, R1, R2    // accumulate
      ADD  R3, R3, #4    // next address
      SUB  R4, R4, #1    // count down from 16
      BNEZ R4, LOOP      // repeat while R4 != 0
```

Its static instruction count is 5. With `R4` initialised to 16 the loop body runs 16 times, so the dynamic instruction count is $16 \times 5 = 80$; with `R4` set from the input to 1000 it is 5000. The register numbers, and therefore the banks each instruction reads, are static; the trip count is dynamic.

4.2.4 Operand collectors and the scoreboard

The second remedy is in the pipeline. Because a register read may take several cycles, through a bank conflict or because a bank has a single read port, the SM does not read operands and execute in consecutive stages as the five-stage CPU does. Figure 4.7 inserts two things between register read and execution.

Definition 4.11 (Operand buffer and operand collector). An *operand buffer* is storage, one entry per waiting warp instruction and per lane, into which source operands are written as the register banks deliver them over several cycles. The *operand collector* is the unit that owns these buffers, issues the bank reads for the instructions it holds, and reports an instruction as ready when all of its operands have arrived.

Operand collector is the term of A18 ch. 3 [4]; K12 §1.4.1 describes the same structure as operand buffers and notes that it also serves as a result queue [3].

Definition 4.12 (Scoreboard). A *scoreboard* is a table that records, for each pending warp instruction, whether each of its source operands is ready. The scheduler consults it to select a warp whose operands are all ready and issues that warp's instruction to the lanes; among several ready warps it applies a policy such as oldest first.

Caution Lecture vs textbook: in A18 ch. 3 and K12 §1.4.1 the scoreboard records which registers have pending writes and blocks a warp's instruction on a RAW or WAW hazard before issue; operand collection follows issue, so the hardware order is decode, scoreboard check and issue, operand collection, execute [4, 3].

On a CPU the scoreboard was the mechanism of early out-of-order dynamic scheduling: it tracked the hazards among the instructions of one thread. On a GPU it answers a simpler question, whether a warp instruction has its operands, and the choice is among warps rather than among the instructions of one thread.

Example 4.13. Four warps `w0–w3` each have a two-operand instruction waiting in the operand

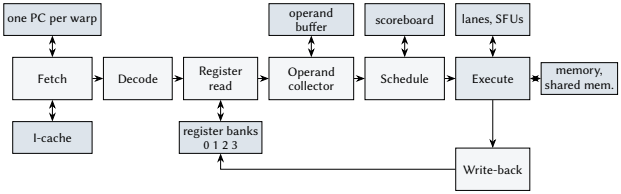


Figure 4.7: The SM pipeline of this lesson: fetch with one PC per warp, decode, register read from the banks, the operand collector with its buffers, the scheduler with its scoreboard, execution on the lanes and SFUs or in the memory units, and write-back to the banks.

collector, and the banks deliver one operand per cycle in the order shown. The scoreboard columns give the readiness of `Src1` and `Src2` after each cycle.

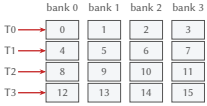
cycle	operand delivered	w0	w1	w2	w3
0	—	··	··	··	··
1	`w0.Src1`	•·	··	··	··
2	`w1.Src2`	•·	·•	··	··
3	`w2.Src1`	•·	·•	•·	··
4	`w0.Src2`	••	·•	•·	··

After cycle 4 `w0` is the only warp with both operands ready, so the scheduler selects it and its operands move from the buffers into the lanes; `w1` and `w2` keep waiting for their second operand. The order in which operands arrive, and hence which warp issues first, is dynamic: it depends on bank conflicts and on the memory system, not on the program text.[1]

[1] The scoreboard was introduced with the CDC 6600 (1964) for a single instruction stream. The GPU form is in-order per warp: K12 §1.4.1 observes that the scheduler is in order but "can select any warps that are ready", so that the execution of the SM as a whole "might look like an out-of-order execution" [3]. Lesson 7 compares the selection policies (round-robin, greedy-then-oldest) in detail.

4.2.5 Shared-memory bank conflicts

Shared memory, the per-block scratchpad of Lesson 3, is banked for the same reason as the register file: a warp's load must deliver 32 words in a cycle. Figure 4.8 shows a four-bank shared memory whose consecutive words are interleaved across the banks, word a in bank a mod 4. If thread k reads `sharedInput[threadIdx.x * 4]`, that is word $4k$, every thread of the warp reads bank 0 and the load is serialised over as many cycles as there are threads.



Tk reads word 4*k*: all in bank 0

Figure 4.8: Four shared-memory banks with interleaved words; a stride of four words sends every thread to bank 0.

Definition 4.14 (Shared-memory bank conflict). A *shared memory bank conflict* occurs when the threads of one warp access different words that lie in the same shared-memory bank in one instruction. The hardware splits the request into as many conflict-free requests as necessary, and the effective bandwidth falls by that factor. Threads that access the *same* word do not conflict: a read of one word by several threads is served by a broadcast, and a write of one word by several threads is performed by one of them (BPG §10.2.3.1 Shared Memory and Memory Banks [8]; CPG, Compute Capability 5.x, Shared Memory [5]).

On NVIDIA GPUs of compute capability 5.0 and later, shared memory has 32 banks, each 32 bit wide and delivering one word per clock, and successive 32-bit words are assigned to successive banks. With a warp of 32 threads a conflict can therefore arise between any two threads of the warp. From these rules the bank of a byte address a follows as

$$\text{bank}(a) = \lfloor a/4 \rfloor \bmod 32, \tag{4.1}$$

and for 32-bit elements accessed at a constant *stride* of s words, the distance between the elements read by consecutive threads (thread k reads element sk), the banks touched repeat after $32/\gcd(s, 32)$ threads, so the access is a $\gcd(s, 32)$-way conflict.

Shared-memory latency is "roughly 100x lower than uncached global memory latency" [18]; an n-way conflict divides the effective bandwidth by n (BPG §10.2.3.1).

Example 4.15. A warp reads `s[threadIdx.x * 8]` from an array of `float`. The word stride is $s = 8$, so thread k reads bank $8k$ mod 32: threads 0, 4, 8, ..., 28 all hit bank 0, threads 1, 5, ..., 29 bank 8, and only banks 0, 8, 16 and 24 are used. Eight threads share each bank, so the load is an 8-way conflict, $\gcd(8, 32) = 8$, and takes eight shared-memory cycles instead of one. With stride 7 the banks are $7k$ mod 32, all 32 distinct, and the load takes one cycle.

Caution Lecture vs guide: the four-bank model turns `threadIdx.x * 4` into a full conflict (every thread in one bank). On real 32-bank hardware the same access is a 4-way conflict, $\gcd(4, 32) = 4$, using eight banks.

The common case in practice is a tile stored row by row and read column by column, as in the tiled matrix multiplication of Lesson 5. With a 32×32 tile of 32-bit elements, the elements of one column are 32 words apart, a 32-way conflict. The standard fix is *padding*: declaring the

tile as 32×33 makes the column stride 33, which by the modulo arithmetic of Equation (4.1) behaves like a unit stride and is conflict-free, at the cost of one unused word per row.

BPG §10.2.3.3 Shared Memory in Matrix Multiplication ($C = AA^{T}$) is the source of the $[w][w+1]$ padding rule [8]; Harris discusses the same conflict for transposes [18].

```
__shared__ float tileA[32][32];  // column read: 32-way
__shared__ float tileB[32][33];  // column read: no conflict
// (tx, ty) reads tileA[tx][ty] / tileB[tx][ty]:
// stride 32 vs 33
```

Restructuring the data layout or the access pattern in this way is the software remedy for shared-memory conflicts, and the compiler cannot apply it automatically because the addresses are dynamic.

Tip Any odd word stride is conflict-free on 32 banks, since $\gcd(s, 32) = 1$; padding a row by one word makes an even row stride odd.

> **Key point**
> Banking gives a wide register file or shared memory the bandwidth of many ports at the cost of a few: values in different banks are read together, values in the same bank are serialised. Register bank conflicts are absorbed by the operand collector and the scoreboard, which lets the scheduler issue whichever warp has its operands; shared-memory conflicts are removed by the programmer through strides and padding.

4.3 GPU architecture pipeline

This section follows warp instructions through the pipeline of Figure 4.7 on a small configuration, stage by stage, and then adds the mask bits that let a warp instruction skip some of its threads.

Source M4, pipeline walk-through; K12 §1.4.1; A18 ch. 3; PTX ISA, special registers [19].

4.3.1 The stages

The stages of Figure 4.7 are, for one warp instruction:

Fetch. The *instruction fetch* stage reads the instruction at the selected warp's PC from the instruction cache, one instruction for the whole warp.

Decode. The *decode* stage identifies the operation, the source and destination registers and any immediate operand; an immediate is the same for every thread and is broadcast into every lane's operand buffer at once, already marked ready.

Register read. The register banks deliver the source values of each thread into the operand buffers, over one or more cycles.

Schedule and issue. When the scoreboard shows all operands ready the scheduler selects the warp; *issue* sends the instruction and its operands to the execution units.

Execute. The arithmetic lanes compute, one thread per lane. Transcendental functions (sine, reciprocal, square root) go to a *special function unit (SFU)*, of which an SM has a few, so that a warp's SFU instruction takes several passes; loads and stores go to the memory unit or to shared memory.[2]

Write-back. The *write-back* stage writes each thread's result into its register in the banks.

[2] K12 §1.4.1 gives the shape of the execution stage on G80: 8 lanes, so a 32-thread warp took four cycles per instruction; 2–4 SFUs per SM computing approximations of the transcendental functions; and no forwarding network as in a CPU pipeline, since a warp result is as wide as the whole warp (K12 puts it at 1 KiB), so that dependent back-to-back instructions rely on the other warps to fill the gap [3].

4.3.2 A walk-through

The configuration of the example is deliberately small: a warp size of 4, two thread blocks of 8 threads, hence four warps, and an SM with four lanes. Blocks and threads are numbered from 0 as in CUDA, so `%ctaid.x` is 0 or 1 and `%tid.x` runs from 0 to 7; registers are 32-bit and their initial values are arbitrary. The state before the first cycle is given by the three tables of Table 4.1.

The instructions use two *special registers* of PTX, read-only registers that the hardware fills for every thread: `%tid.x` holds the thread's index within its block and `%ctaid.x` the index of its block (CTA) within the grid. The scheduler is assumed to pick warps in the order `W0`, `W1`, `W3`, and each step below takes the instruction through all stages before the next warp is considered.

PTX ISA 9.4, special registers `%tid`, `%ntid`, `%ctaid`, `%nctaid` [19]; they are what `threadIdx`, `blockDim` and `blockIdx` of Lesson 3 compile to. Lesson 10 presents PTX as a whole.

1. `W0` *at* `0x8000`*:* `add r1, r1, 2`. Fetch reads one instruction for the four threads. Decode broadcasts the immediate 2 into `Src2` of all four lanes and marks it ready. Register read fills `Src1` with `R1` of `T0`–`T3`, that is 20, 21, 22, 23. Both operands are ready, the scoreboard lets the scheduler issue `W0`, the four lanes add, and write-back stores 22, 23, 24, 25 into `R1` of `T0`–`T3`. The PC of `W0` advances to `0x8004`.
2. `W1` *at* `0x8004`*:* `sub r2, r2, 1`. The same stages for threads `T4`–`T7`: the immediate 1 is broadcast, `R2` of `T4`–`T7` is read as 7, 7, 7, 7, and $7 - 1 = 6$ is written back to each. The

Table 4.1: Initial state of the walk-through: the PC of each warp, the instructions in the instruction cache, and the register file of block 0.

warp	block, threads	PC
W0	block 0, T0–T3	0x8000
W1	block 0, T4–T7	0x8004
W2	block 1, T0–T3	0x8000
W3	block 1, T4–T7	0x800C

address	instruction
0x8000	`add r1, r1, 2`
0x8004	`sub r2, r2, 1`
0x8008	`mov r1, %tid.x`
0x800C	`mov r2, %ctaid.x`
0x8010	`ld.shared r1, [A]`

block 0	T0	T1	T2	T3	T4	T5	T6	T7
R1	20	21	22	23	30	31	32	33
R2	5	5	5	5	7	7	7	7
R3	1	4	9	16	25	36	49	64
R4	8	6	7	5	3	0	9	2

hardware performs identical work for every thread of a warp; only the register values differ.

3. W3 *at* 0x800C: `mov r2, %ctaid.x`. W3 belongs to block 1, so the special register reads 1 in every lane and R2 of T4–T7 of block 1 becomes 1, 1, 1, 1. Had W3's PC been 0x8008 instead, `mov r1, %tid.x` would have written the thread indices 4, 5, 6, 7 into R1: the same instruction yields a different value per lane because the special register differs per thread.

Table 4.2 shows block 0's registers after the first two steps. Note that W2 has the same PC as W0 had: the two warps of different blocks execute the same static instruction on their own registers, and nothing in the pipeline ties them together.

Tip To trace a warp instruction by hand: fetch at the warp's PC; broadcast the immediate; read the named register for every thread of the warp; compute per lane; write back per lane; advance that warp's PC.

Table 4.2: Register file of block 0 after steps 1 and 2 (changed values in bold).

block 0	T0	T1	T2	T3	T4	T5	T6	T7
R1	**22**	**23**	**24**	**25**	30	31	32	33
R2	5	5	5	5	**6**	**6**	**6**	**6**
R3	1	4	9	16	25	36	49	64
R4	8	6	7	5	3	0	9	2

4.3.3 Mask bits

Not every instruction needs all the threads of its warp: a block of 6 threads with warp size 4 leaves two lanes of its second warp empty, and Lesson 6 shows that a branch taken by some threads only leaves the others idle until the paths reconverge. The pipeline handles both with one mechanism.

The masks of Lesson 6's SIMT stack are exactly these bits, pushed and popped as branches diverge and reconverge; Lesson 5's warp vote and shuffle functions expose the active mask to the program.

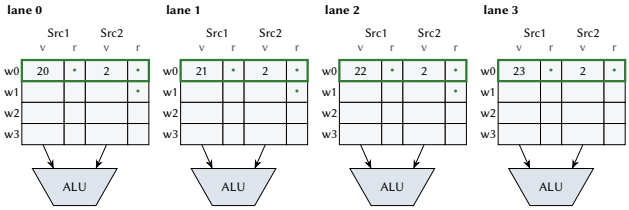
Definition 4.16 (Lane and active mask). A *lane* is one execution path through the pipeline: the operand-buffer entries, ALU and write-back slot that serve one thread of a warp instruction (the SIMT lane of Lesson 1 seen from the pipeline). The *active mask* of a warp is a bit vector with one bit per lane; a lane whose bit is 1 executes the instruction and writes its result back, a lane whose bit is 0 does no work and writes nothing.

Table 4.3: Active masks for a later instruction (the walk-through above ran with all masks 1111); lane 3 of W1 and lane 0 of W2 are switched off.

warp	mask
W0	1111
W1	1110
W2	0111
W3	1111

The walk-through above ran with every lane active. Figure 4.9 shows the per-lane operand buffers for a later instruction under the masks of Table 4.3, with W0 ready in all four lanes and selected, and W1 running under the mask 1110: its first three lanes receive operands and compute, its fourth lane is inactive and is never marked ready. A warp whose mask is all ones is the common case, and the mask is part of the warp's state alongside its PC.

Key point
A warp instruction is fetched once at the warp's PC, decoded once, and then read, executed and written back per lane; immediates and special registers like `%ctaid.x` are broadcast, while `%tid.x` and ordinary registers differ per lane. The active mask switches individual lanes off without changing the instruction stream.



w0 has both operands in every lane and is selected; lane 3 of w1 is inactive under mask 1110

Figure 4.9: Per-lane operand buffers and the scoreboard: w0 has both operands ready in every lane and is issued to the four ALUs; w1 runs under the mask 1110, so its fourth lane stays inactive.

4.4 Global memory coalescing

The execution side of the SM processes a warp instruction as one unit; the memory side cannot, because each thread of a load supplies its own address. This section counts the requests a warp can generate, reviews the memory technologies that must serve them, and shows how the hardware merges requests whose addresses are close.

Source M4, coalescing; K12 §1.4.3; A18 ch. 4; BPG §10.2.1 Coalesced Access to Global Memory; CPG §5.3.2 Device Memory Accesses; PMPP ch. 6.

4.4.1 The demand for bandwidth

One load instruction of a 32-thread warp can generate up to 32 distinct memory requests, one per thread. The *memory bandwidth* that would be needed if every SM issued such a load every cycle is easily computed.

Example 4.17. A GPU has 64 SMs at 1.5 GHz. If every SM issues one warp load per cycle and every thread's request fetches 32 B, the SMs demand $64 \times 32 \times 32\,\text{B} = 65\,536\,\text{B}$ per cycle, which at 1.5 GHz is about 98 TB/s. (With 32 SMs, 64 B per request and 1 GHz the same 64 KiB per cycle give about 66 TB/s, usually rounded to 64 TB/s.) The H100 SXM5 memory system delivers about 3 TB/s (preliminary figure), some thirty times less. The gap is closed by caches, by not every warp loading every cycle, and above all by coalescing: the 32 requests of a warp usually fall into a few adjacent blocks of memory.

HBM bandwidth by generation: P100 720 GB/s, V100 900 GB/s, A100 (40 GiB) 1555 GB/s [1]; H100 SXM5 3000 GB/s (preliminary, HWP V1.02) [2].

GPU caches are small relative to the thread count, so their bandwidth saturates easily, and the off-chip memory is the bottleneck for most kernels. Which memory technology sits there decides how wide the bottleneck is.

4.4.2 SRAM, DRAM and HBM

Figure 4.10 contrasts the three technologies.

SRAM. Static random-access memory, *SRAM*, stores a bit in a six-transistor cell: two cross-coupled inverters that hold the bit, connected to the bit lines through two access transistors gated by the word line. It is fast and needs no refresh, but six transistors per bit make it expensive in area, so it is used for the structures on the GPU die: registers, shared memory and caches.

DRAM. Dynamic random-access memory, *DRAM*, stores a bit as charge on a capacitor behind a single transistor (1T1C). The cell is far smaller, so DRAM gives the capacity of the GPU's main memory, but it is slower, needs periodic refresh, and its chips sit in separate packages: every bit that enters or leaves must pass through the package pins, whose number and speed bound the bandwidth. The SMs reach the L2 and DRAM through several *memory channels*, each with its own DRAM, and coalesced requests spread over them (K12 §1.4.3 [3]).

Earlier GPUs used GDDR DRAM over a wide bus, with an FR-FCFS (first-ready, first-come first-served) scheduler at each channel: K12 §1.4.3 [3]. Interface widths: P100 and V100 use a 4096-bit HBM2 interface, A100 a 5120-bit HBM2 interface [1], H100 SXM5 a 5120-bit HBM3 interface [2]. BPG §9.2.1 derives V100's 898 GB/s as $0.877\,\text{GHz} \times 2 \times 4096\,\text{bit}/8$ [8].

HBM. High bandwidth memory (Lesson 1) keeps the DRAM cell but changes the packaging. Several DRAM dies are stacked vertically and connected through the stack, which is *3D-stacked memory*, and the stack is placed next to the GPU die on a *silicon interposer*, a silicon substrate carrying thousands of fine wires inside the same package. The link to the GPU no longer crosses package pins or a circuit board, so it can be thousands of bits wide, and the bandwidth per package is several times that of a GDDR memory system.

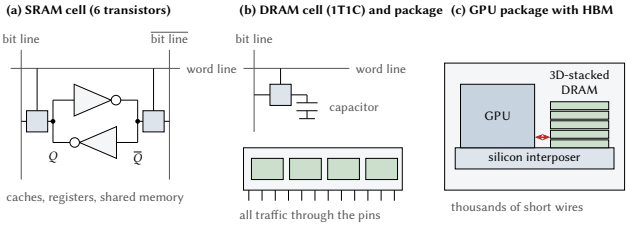


Figure 4.10: (a) A six-transistor SRAM cell. (b) A one-transistor, one-capacitor DRAM cell and a DRAM package whose traffic passes through its pins. (c) A GPU package in which a silicon interposer links the GPU die to a 3D stack of DRAM dies.

4.4.3 Coalescing

Even HBM cannot serve 32 separate requests per warp instruction, so the memory unit of the SM looks at the 32 addresses together.

Definition 4.18 (Memory coalescing). The combination, by the hardware, of the memory requests of the threads of one warp instruction into as few memory transactions as their addresses allow is *memory coalescing*. A *memory transaction* is one transfer between the memory system and the SM of a fixed, naturally aligned block of bytes. An access whose 32 requests fit into a small number of transactions is a *coalesced access*; one that needs a separate transaction for most threads is an *uncoalesced access*.

Figure 4.11 shows the two cases for eight threads executing two PTX loads,

```
ld.global.u32 %r3, [%r1];   // R1: 512, 516, ..., 540
ld.global.u32 %r3, [%r2];   // R2: 0, 128, 256, ..., 896
```

Caution Lecture vs guide: the 64 B request of the example is the granularity the lecture assumes ("64 to 128 B"). Since compute capability 6.0 the hardware counts 32 B sectors: BPG §10.2.1 states that a warp's accesses coalesce into "a number of transactions equal to the number of 32-byte transactions necessary to service all of the threads of the warp" [8]. Lesson 8 returns to sectored caches.

where a DRAM request is taken to be 64 B long. In the first load the eight addresses are consecutive 4-byte words from 512 to 540, 32 B in all, so they combine into one request for `Mem[512--575]`: the access is coalesced. In the second the addresses are 128 B apart, so no two of them share a request and eight requests are issued: the access is uncoalesced and consumes eight times the bandwidth for the same 32 B of useful data.

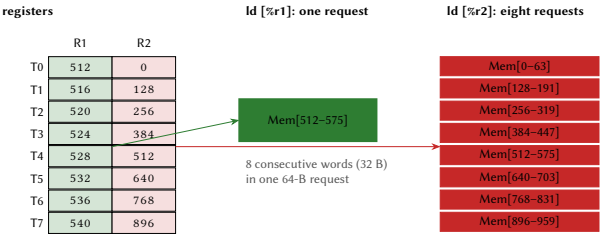


Figure 4.11: Eight threads load through `%r1` (consecutive words, one request) and through `%r2` (words 128 B apart, eight requests).

CPG §5.3.2 Device Memory Accesses: "device memory is accessed via 32-, 64-, or 128-byte memory transactions. These memory transactions must be naturally aligned" [5].

The precise rule has three ingredients. Global memory is accessed in transactions of 32, 64 or 128 B, and each transaction must be naturally aligned: its first address is a multiple of its size, which is the *alignment* requirement. Memory is cached in units of a *cache line*, 128 B on NVIDIA GPUs, made of four 32 B sectors, and a warp's accesses cost as many sectors as they touch. Finally the *stride* of an access, the distance between the addresses of consecutive threads, decides how many sectors the warp's 32 addresses span.

Example 4.19. A warp of 32 threads loads one 4-byte word per thread, 128 B of useful data, in the four patterns of Figure 4.12 and Table 4.4. The efficiency is the useful data divided by the bytes transferred.

Pattern (b) shows why alignment matters even for consecutive accesses: a shift of one word spills into a fifth sector. Pattern (d) is the column-wise access to a row-major matrix, and the

Table 4.4: Sectors transferred for a warp loading 32 words of 4 B (after BPG §10.2.1.1–10.2.1.4).

access pattern	sectors	bytes moved	efficiency
(a) consecutive, 128 B-aligned	4	128	100 %
(b) consecutive, offset by one word	5	160	80 %
(c) stride 2 words	8	256	50 %
(d) stride 8 words or more	32	1024	12.5 %

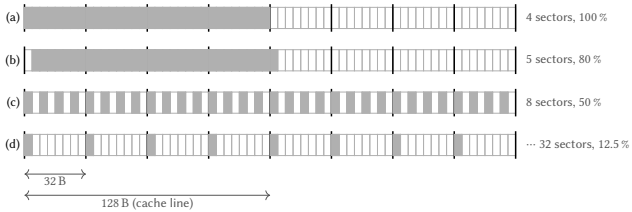


Figure 4.12: Words touched by a 32-thread warp (shaded) over 32 B sectors: (a) aligned and consecutive, (b) shifted by one word, (c) stride 2, (d) stride 8 (one word per sector).

uncoalesced load of Figure 4.11 is its extreme form; a kernel written so that consecutive threads touch consecutive words, pattern (a), is the single most important memory optimisation. In CUDA the pattern is determined by how the thread index enters the address:

K12 §1.4.3: GPUs before compute capability 1.2 imposed strict coalescing rules (consecutive threads, consecutive aligned words); later generations relaxed them and produce "one to four memory transactions" for a coalesced warp access [3].

```
// coalesced: consecutive threads, consecutive words
out[i] = in[blockIdx.x * blockDim.x + threadIdx.x];
// uncoalesced: stride of one row (n words) per thread
out[i] = in[threadIdx.x * n + blockIdx.x];
```

When the algorithm needs the strided view, the remedy is the shared memory of Section 4.2: load a tile coalesced, then read it from shared memory in the order the computation wants, which is the tiling of Lesson 3 and the subject of Lesson 5.

Tip Check a load by asking what addresses threads 0 and 1 of a warp touch: if they differ by the element size and thread 0's address is aligned, the access is coalesced.

Key point — Lesson summary

An SM hides memory latency with fine-grained multithreading: one PC per warp and a register set per thread stay on chip, so a warp switch is free and the number of warps is bounded by storage. The register file and shared memory are banked to supply a warp's operands per cycle; accesses that need more values from one bank than it has ports are bank conflicts, absorbed in the register file by operand collectors and a scoreboard that issues whichever warp is ready, and avoided in shared memory (32 banks of 4 B) by odd strides and padding. A warp instruction is fetched and decoded once and executed per lane under an active mask, with immediates and `%ctaid.x` broadcast and `%tid.x` per lane. On the memory side one warp load can issue 32 requests; the hardware coalesces those that fall into the same aligned 32 B sectors, so consecutive, aligned accesses move 4 sectors for 128 B of data while strided ones move up to 32, and HBM's stacked DRAM on an interposer exists to serve the remaining demand.

Lesson 5

Advanced GPU Programming

Lecture	M5, lesson 1 (Advanced GPU Programming)
Sources	[18] (required); [20] (required, concept level); [21], [22], [23], [24] (optional)
Textbook	A18 [4] ch. 3; BPG [8] §9 Performance Metrics, §10 Memory Optimizations, §11 Execution Configuration Optimizations; K12 [3] ch. 3, §4.1; CPG [5] §5 Performance Guidelines; PMPP [6] ch. 6
Keywords	profiling, hotspot, data transfer, pinned memory, asynchronous copy, CUDA stream, zero copy, UVA, effective bandwidth, intrinsic function, predicated execution, warp shuffle, warp vote, cooperative groups, independent thread scheduling, cuBLAS, sorting network

Lesson 3 introduced the CUDA programming model and Lesson 4 the hardware that runs it: the multithreaded SM, its register and shared-memory banks, the pipeline and the coalescing of global memory accesses. This lesson turns that knowledge into a method for making a CUDA program fast. It starts from the question of which programs suit a GPU at all, and from the profiler that shows where a program spends its time. The time of a GPU program has three parts, data transfer between host and device, memory access on the device, and computation, and the lesson gives a checklist for each: pinned memory, asynchronous copies and streams for the transfers; coalescing, alignment and bank conflicts for the memory accesses; cheaper instructions, lower precision, intrinsics, fewer branches and fewer atomics for the computation. It then surveys the newer CUDA features that remove old costs (warp shuffles and votes, cooperative groups, independent thread scheduling, clusters, split barriers), the libraries that should be preferred to hand-written kernels, the other GPU programming platforms, and finally the sorting networks of the assigned reading, the classic example of an algorithm reshaped for the GPU. Lessons 6 and 7 go below this level, into the hardware mechanisms behind divergence, register files and warp scheduling.

5.1 What suits a GPU, and where the time goes

Source M5, GPU-friendly programs and profiling; BPG §9.1 Timing; K12 §3.5.

Not every program benefits from a GPU. A program that will run well on one has three properties:

- **massive data parallelism**: the same operation applies to many independent data elements, enough to fill thousands of threads (Lessons 1 and 2);
- **little host–device communication** relative to the computation: the data moved over the I/O interface is a small share of the total cost;
- **coalesced global memory accesses**: neighbouring threads touch neighbouring addresses (Lesson 4).

A program that lacks the first property has no work for the lanes; one that lacks the second or third is limited by a bus or by the memory system, however fast the SMs are.

Amdahl's law (Lesson 1) is the quantitative form of the second property: the host–device transfer is part of the serial fraction unless it can be overlapped (Section 5.2).

5.1.1 Profiling

Optimisation starts with measurement.

Definition 5.1 (Profiling, hotspot). The measurement of where a program spends its time and what the hardware does while it runs is called *profiling*; a *profiler* is the tool that collects those measurements. A *hotspot* is a part of the program (a loop, a function, a kernel) that accounts for a large share of the total time and is therefore the part worth optimising.

Tip Measure before and after every change. Kernel launches and asynchronous copies return to the host before the work is done, so a CPU timer must be bracketed by `cudaDeviceSynchronize()`, or the time measured on the device with CUDA events (`cudaEventElapsedTime`); BPG §9.1 Timing.

Profiling comes first in two senses. Before any code is ported, the *CPU* program is profiled, to find the hotspots that a GPU can accelerate: only the data-parallel hotspots are moved, and the rest stays on the host. Once a kernel runs on the device, the GPU profiler reports the metrics that locate the bottleneck inside it (Table 5.1).

Definition 5.2 (Nsight Systems, Nsight Compute). NVIDIA's profilers are *Nsight Systems*, which records a timeline of the whole application (CPU threads, CUDA API calls, memory copies and kernels on every GPU) and so shows the transfer and overlap problems of Section 5.2 [23], and *Nsight Compute*, which profiles one kernel in detail (instruction throughput, memory

Table 5.1: Profiler metrics named in this lesson, what a bad value indicates, and where this book treats the remedy.

Metric	Bad value means	Remedy
throughput (instructions, FLOP/s)	the SMs are idle: too little parallelism, or stalls on memory	Section 5.4, Lesson 3
divergent branches	lanes of a warp follow different paths and execute one after the other	Section 5.4, Lesson 6
uncoalesced ("divergent") memory accesses	one warp load becomes many memory transactions	Section 5.3, Lesson 4
occupancy	too few resident warps to hide latency	Lesson 3
bandwidth utilisation	achieved bandwidth far below the peak: bad access pattern, or the kernel is not memory-bound	Section 5.3

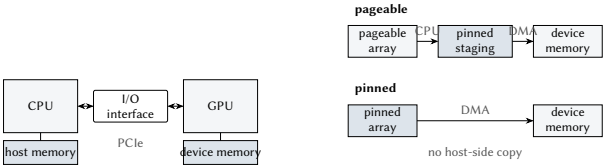


Figure 5.1: Left: host and device memories are separate, and every explicit transfer crosses the I/O interface. Right: a copy from pageable host memory is two copies, one by the CPU into a pinned staging buffer and one DMA; from pinned memory it is one DMA.

transactions, occupancy, bank conflicts, a Roofline chart) and shows the problems of Sections 5.3 and 5.4 [24].

The two Nsight tools replaced `nvprof` and the Visual Profiler, which do not support compute capability 8.0 and later [25].

5.1.2 Three parts of the execution time

The execution time of a GPU program decomposes as

$$T = T_{\text{transfer}} + T_{\text{memory}} + T_{\text{compute}}, \tag{5.1}$$

the time to move data between host and device, the time the kernel spends waiting for device memory, and the time it spends computing. All three must be attacked, and in that order of surprise: a program ported naively is often dominated by the first term, which does not appear in the kernel at all. The sum reflects the naive program, in which the three parts are serialised; the optimisations of Section 5.2 either shorten the first term or overlap it with the other two. The sections that follow take the three terms in turn; a fourth rule precedes them all: use the vendor's libraries (Section 5.6) wherever one covers the computation, because they already contain the optimisations below.

5.2 Data-transfer optimisations

A discrete GPU has its own memory, and the host reaches it only through the I/O interface (Figure 5.1, left).

Source M5, data transfer optimisations; BPG §10.1 Data Transfer Between Host and Device; [21]; [22].

Definition 5.3 (Data transfer, PCIe). A *data transfer* is the copy of a buffer between host memory and device memory (`cudaMemcpy` and its variants). The interface it crosses is *PCI Express (PCIe)*, a serial point-to-point bus of 1 to 16 lanes, whose bandwidth is one to two orders of magnitude below that of the device memory.[1]

[1] K12 §1.2 quotes 4 Gbit/s per lane for PCIe Gen 2 and 8 Gbit/s for Gen 3, i.e. about 8 GB/s and 16 GB/s per direction for 16 lanes; the V100's HBM2 delivers 898 GB/s (BPG §9.2.1). NVLink 4 is quoted as 7× PCIe Gen 5 [2]; Lesson 9.

The naive program copies its input to the device, runs the kernel and copies the result back, three steps in sequence (Figure 5.2, row 0). Four techniques shorten or hide the two copies.

5.2.1 Fast transfers: pinned memory

Definition 5.4 (Pinned memory). Host memory that the operating system has promised not to page out or move is called *pinned memory* (page-locked memory); it can be used by the GPU's DMA engine, which reads and writes it directly by physical address. It is allocated with `cudaMallocHost()` or `cudaHostAlloc()` and freed with `cudaFreeHost()` [21].

Ordinary (*pageable*) host memory, from `malloc` or `new`, cannot be the source of a DMA, because the operating system may move or evict its pages at any time. A `cudaMemcpy` from

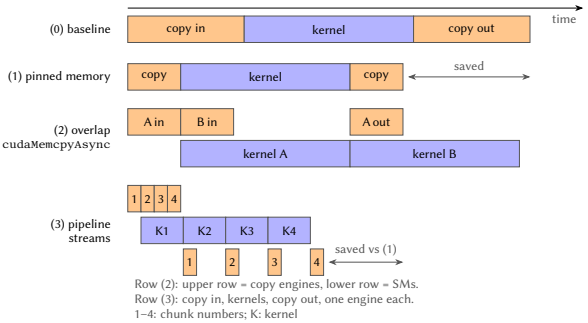


Figure 5.2: One job under the four data-transfer optimisations. (0) Copy in, kernel, copy out, in sequence. (1) Pinned memory shortens both copies. (2) An asynchronous copy of the next input runs on the copy engine while the SMs execute the current kernel. (3) Streams split input, kernel and output into chunks and pipeline them, so that the copy engines and the SMs are busy at the same time.

pageable memory therefore first copies the data, on the CPU, into a pinned staging buffer owned by the driver, and only then starts the DMA (Figure 5.1, right). A copy from pinned memory skips the staging step, which is why pinned transfers "attain the highest bandwidth between the host and the device" (BPG §10.1.1 Pinned Memory).[2]

[2] Measured with 16 MiB transfers [21]: on a laptop (NVS 4200M, PCIe Gen 2) pinned memory raised host-to-device bandwidth from 2.31 to 5.77 GB/s, 2.5×; on a desktop (GTX 680) only from 5.37 to 6.19 GB/s, because the faster CPU made the staging copy cheap.

Pinned memory is a scarce resource: every pinned page is one the operating system cannot reclaim, and "excessive use can reduce overall system performance" (BPG §10.1.1). The guide adds that how much is too much cannot be known in advance; the rule is to pin the transfer buffers, not every allocation. Nor is pinning free: allocating pinned memory is far slower than `malloc`, so the buffers are allocated once, at start-up, and reused, and a buffer transferred only once rarely pays for its pinning.

5.2.2 Overlapping computation and transfer: asynchronous copies

Definition 5.5 (Asynchronous copy, overlap). An *asynchronous copy* is a transfer that returns control to the host as soon as it has been *issued*, before the data has moved: `cudaMemcpyAsync()` is "a non-blocking variant of `cudaMemcpy()`" (BPG §10.1.2). The *overlap of computation and transfer* is the resulting ability of the copy engine and the SMs to work at the same time: while the kernel for one data set executes, the copy of the next data set is in flight (Figure 5.2, row 2). The copy engine is a DMA unit separate from the SMs; data-centre GPUs have at least one per direction, so a host-to-device and a device-to-host copy can also overlap each other.

The host, too, is free during an asynchronous copy, and can prepare the next input or consume the previous output. Overlap requires pinned memory: an asynchronous copy from pageable memory still performs the staging copy on the CPU before it returns, and is then largely synchronous with respect to the host.

Caution `cudaMemcpyAsync` with a pageable buffer compiles and runs, but gives no overlap; the profiler timeline (Nsight Systems) shows the difference immediately. Use pinned buffers for every asynchronous transfer.

5.2.3 Pipelining with streams: concurrent copy and execute

Overlap becomes a pipeline when the single job is split into chunks.

Definition 5.6 (CUDA stream). A *CUDA stream* is a sequence of device operations (copies, kernel launches, events) that execute in the order they were issued. Operations in *different* streams have no ordering between them and may run concurrently or interleaved [22]. Every runtime call that takes a stream argument (`cudaMemcpyAsync`, the fourth parameter of the execution configuration) is placed in that stream; calls without one go to the *default stream*.

The legacy default stream synchronises with every other stream of the device: an operation issued to it waits for all earlier work in all streams and blocks all later work (streams created with `cudaStreamNonBlocking` are exempt) [22]. One stray `cudaMemcpy` or `<<<grid, block>>>` without a stream argument therefore serialises a whole pipeline.

Caution CUDA 7 added a per-thread default stream (`nvcc --default-stream per-thread`) that behaves like an ordinary stream; without that flag, the default stream is the legacy one [22].

Definition 5.7 (Concurrent copy and execute). The pipelining of a job over n streams is called *concurrent copy and execute*: the input is divided into n chunks, and chunk k's copy-in, kernel

Table 5.2: Schedule of Example 5.8 (times in ms). Each column is one engine; within a stream the three steps stay in order.

Chunk (stream)	Copy in	Kernel	Copy out
1	0–2	2–3.5	3.5–5.5
2	2–4	4–5.5	5.5–7.5
3	4–6	6–7.5	7.5–9.5
4	6–8	8–9.5	9.5–11.5

and copy-out are issued to stream k. Within a stream the three steps stay in order, so each kernel waits for its own input; across streams the copy of chunk $k+1$ overlaps the kernel of chunk k, and the copy-out of chunk $k-1$ overlaps both (Figure 5.2, row 3; BPG §10.1.2 Asynchronous and Overlapping Transfers with Computation).

```
const int nStreams = 4;
size_t chunk = n / nStreams;  // per-chunk size
float *h_in, *h_out, *d_in, *d_out;
cudaMallocHost(&h_in,  n * sizeof(float)); // pinned
cudaMallocHost(&h_out, n * sizeof(float));
cudaMalloc(&d_in,  n * sizeof(float));
cudaMalloc(&d_out, n * sizeof(float));
cudaStream_t s[nStreams];
for (int i = 0; i < nStreams; ++i) cudaStreamCreate(&s[i]);
for (int i = 0; i < nStreams; ++i) {
  size_t off = i * chunk, bytes = chunk * sizeof(float);
  cudaMemcpyAsync(d_in + off, h_in + off, bytes,
                  cudaMemcpyHostToDevice, s[i]);
  scale<<<chunk / 256, 256, 0, s[i]>>>(d_in + off,
                                       d_out + off, chunk);
  cudaMemcpyAsync(h_out + off, d_out + off, bytes,
                  cudaMemcpyDeviceToHost, s[i]);
}
cudaDeviceSynchronize();      // wait for all
```

The loop issues twelve operations and returns at once; the device executes them subject only to the per-stream ordering. The `cudaDeviceSynchronize()` at the end waits for every stream before the host reads `h_out`.

Example 5.8 (A pipelined job). A job copies in 64 MiB, runs a kernel, and copies out 64 MiB. Sequentially the copy-in takes 8 ms, the kernel 6 ms and the copy-out 8 ms: 22 ms in all. Split into four chunks on four streams, each chunk needs 2 ms in, 1.5 ms of kernel and 2 ms out; the device has one copy engine per direction and runs one kernel at a time. Table 5.2 traces the schedule: a kernel starts when its input has arrived and the previous kernel has finished; a copy-out starts when its kernel has finished and the previous copy-out is done. The job completes at 11.5 ms, a speed-up of 1.9×, and the copy-in engine is the bottleneck: it is busy from 0 to 8 ms without a gap, and everything after that is the drain of the pipeline (the last kernel and the last copy-out, 3.5 ms). More chunks shorten the drain but add per-operation overhead.

Tip BPG §10.1.2 gives the estimate: with a transfer time t_T and a kernel time t_E, the sequential version takes $t_E + t_T$ and the staged version about $t_E + t_T/n$ when $t_E > t_T$ (the kernel is the long pole), or $t_T + t_E/n$ when the transfer is.

5.2.4 Direct host memory access: zero copy and UVA

The fourth technique removes the explicit copy altogether, by letting the kernel read host memory.

Definition 5.9 (Zero copy). The direct access of host memory by device threads is called *zero copy*. The host buffer is allocated pinned and *mapped* into the device's address space (`cudaHostAlloc` with `cudaHostAllocMapped`), the kernel receives the corresponding device pointer (`cudaHostGetDevicePointer`), and each load or store in the kernel travels over the I/O interface instead of to device memory (BPG §10.1.3 Zero Copy).

```
float *h_a, *d_a;
cudaHostAlloc(&h_a, bytes, cudaHostAllocMapped);
cudaHostGetDevicePointer(&d_a, h_a, 0);
kernel<<<grid, block>>>(d_a);  // loads cross the I/O bus
```

Zero copy is a clear win on an *integrated* GPU, whose memory is physically the CPU's memory: the "copy" it replaces was a copy from one region of the same DRAM to another. On a discrete GPU every access pays the I/O interface's bandwidth and latency, so it helps only when the data is read once, in a coalesced pattern, and the latency can be hidden by other warps; data that is reused belongs in device memory.

Definition 5.10 (Unified virtual addressing). Under *unified virtual addressing (UVA)*, host memory and the memories of all devices share one virtual address space: a pointer value identifies its memory by itself, the runtime can tell from the address which memory a pointer refers to (`cudaPointerGetAttributes`), and `cudaMemcpy` can be called with `cudaMemcpyDefault` without naming the direction (BPG §10.1.4 Unified Virtual Addressing). The driver and runtime hide the physically separate memories behind one addressing scheme.

UVA needs compute capability 2.0 and a 64-bit host; under it a pinned buffer has the same host and device pointer, and peer-to-peer copies between GPUs (Lesson 9) become possible. BPG §10.1.4.

Caution UVA is not unified memory. UVA (2011, CUDA 4) is one address space over separate memories; unified memory (`cudaMallocManaged`, CUDA 6) adds automatic migration of pages between them on demand, with page-fault costs of its own. Lesson 7 treats unified memory.

Ideally a unified address space would make data movement free. It does not: the memories are still separate, the data must still cross the interface, and the copy happens either explicitly or on the runtime's initiative, with its overhead. UVA changes the *naming* of memory, not its *location*.

Not every one of the four techniques fits every system: the gain of pinned memory depends on the host CPU, the gain of overlap on the ratio of transfer to kernel time, the gain of zero copy on whether the GPU is integrated. The rule is to try each technique and measure.

5.3 Memory-access optimisations

Source M5, memory-access optimisations; BPG §9.2 Bandwidth, §10.2 Device Memory Spaces; [18].

Shared memory latency is "roughly 100× lower than uncached global memory latency" [18]; on the Tesla and Fermi generations K12 Table 1.4 lists 4–32 cycles for shared memory against hundreds for global memory.

Once the data is on the device, the second term of Equation (5.1) is the time the kernel waits for device memory. The checklist repeats, as rules, the mechanisms of Lesson 4:

- use the on-chip memories: shared memory for data a block reuses, the L1 and L2 caches for data with locality, constant memory for values every thread reads;
- **coalesce** and **align** global accesses so that a warp's load touches as few memory transactions as possible: on current GPUs a transaction is a 32 B sector, and a warp of 32 threads reading consecutive 4-byte words touches four of them (BPG §10.2.1 Coalesced Access to Global Memory);
- **reduce the number of transactions**: a structure of arrays instead of an array of structures, data types sized to what is needed, loads of 8 or 16 bytes per thread where alignment allows;
- **compare the achieved bandwidth with the peak** (Definition 5.11): a gap that persists after coalescing means the kernel is not memory-bound and the next term of Equation (5.1) is the one to attack;
- **avoid shared-memory bank conflicts**: 32 banks of 32-bit words, one access per bank per cycle, so a warp whose lanes access different addresses in the same bank serialises (lanes reading the same word get a broadcast; Lesson 4; [18]).

Definition 5.11 (Theoretical and effective bandwidth). The *theoretical bandwidth* of a memory is computed from its hardware specification: memory clock rate × interface width in bytes × 2 for double data rate (BPG §9.2.1 Theoretical Bandwidth Calculation). The *effective bandwidth* of a kernel is the bandwidth it actually achieved,

$$\text{effective bandwidth} = \frac{(B_r + B_w)/10^9}{t}, \tag{5.2}$$

with B_r and B_w the bytes the kernel reads and writes and t its execution time in seconds, in GB/s (BPG §9.2.2 Effective Bandwidth Calculation).

BPG's own instance: the V100, HBM2 at 877 MHz on 4096 bits, $0.877 \times 10^9 \times 512 \times 2/10^9 = 898$ GB/s (BPG §9.2.1).

Both numbers must use the same divisor: bandwidths are decimal (10^9), and dividing the bytes by 1024^3 instead understates the effective bandwidth by 7 % (BPG §9.2.1). The Visual Profiler reported a "requested" throughput, corresponding to the effective bandwidth, and an "actual" throughput that includes the bytes wasted by uncoalesced accesses; the gap between the two is the coalescing loss (BPG §9.2.3 Throughput Reported by Visual Profiler). In Nsight Compute the same loss appears as sectors per request in the memory workload analysis: the ideal is four for 32 consecutive 4-byte loads.

Example 5.12 (Bandwidth utilisation). A GPU has HBM2 at a memory clock of 1.2 GHz on a 5120-bit interface. Its theoretical bandwidth is

$$1.2 \times 10^9 \times \frac{5120}{8} \times 2 \,/\, 10^9 = 1536 \text{ GB/s}.$$

A kernel reads an array of 256 MiB ($B_r = 268\,435\,456$) and writes one of the same size, and runs in 0.5 ms. By Equation (5.2),

$$\frac{(268\,435\,456 + 268\,435\,456)/10^9}{0.5 \times 10^{-3}} = 1074\ \text{GB/s},$$

70 % of the peak. The kernel is memory-bound and its access pattern is already good; the remaining 30 % is the usual gap between a specification and a streaming kernel, not a sign of uncoalesced accesses. Had the result been 300 GB/s with the same byte count, the first suspect would be the access pattern, and the sectors per request in Nsight Compute the first metric to read.

5.4 Reducing computation

The third term of Equation (5.1) is the computation itself.

Source M5, reducing computation; BPG §12 Instruction Optimization, §13 Control Flow; CPG §5.4 Maximize Instruction Throughput.

Definition 5.13 (Instruction optimisation). The replacement of the instructions a kernel executes by fewer or cheaper ones with the same (or an acceptable) result is called *instruction optimization*: a cheaper operation for the same function, a narrower data type, a hardware function unit for a software routine, a predicate for a branch.

Six techniques reduce computation.

Cheaper instructions. Integer division and modulo are multi-instruction sequences on the SM (BPG §12.1.4); 32-bit integer multiplication is a single instruction on Volta and later but was emulated on Maxwell and Pascal; shifts and masks are single instructions. For a power-of-two divisor $n = 2^k$, `i / n` is `i >> k` and `i % n` is `i & (n - 1)` for non-negative `i`.

```
int q1 = i / n;         // division sequence
int q2 = i >> k;        // one shift, if n == 1 << k
int r  = i & (n - 1);   // i % n, for i >= 0
```

The instruction-level form of strength reduction (Lesson 11). The compiler does this itself for a compile-time constant divisor; the rewrite matters for a run-time power of two.

Lower precision. Where the application tolerates it, 32-bit arithmetic instead of 64-bit, and 16- or 8-bit instead of 32-bit: each halving of the width doubles the throughput of the arithmetic units and halves the bytes moved. FP64 throughput on consumer GPUs is a small fraction of FP32; FP16 pairs (`half2`) execute two operations per instruction (BPG §12.1). Lesson 12 treats the formats and what the tensor cores do with them.

Caution Lower precision is a numerical decision before it is a performance one: a sum of many FP16 terms loses digits at every addition, and a reduction over a million elements in FP16 is wrong, not merely approximate. Keep accumulators wider than the operands.

Hardware special functions. The SM's special function units (Lesson 4) evaluate reciprocal, reciprocal square root, $\log_2$, 2^x, sine and cosine as single instructions, at reduced accuracy.

Definition 5.14 (Intrinsic function). An *intrinsic function* is a function the compiler translates directly into a hardware instruction rather than into a call to a library routine, such as `__sinf()`, `__expf()`, `__fdividef()`; the standard function `rsqrtf()` likewise maps to the SFU's reciprocal square root. The reciprocal square root "should always be invoked explicitly as `rsqrtf()`" rather than written as `1.0f / sqrtf(x)`, which the compiler cannot always merge (BPG §12.1.6).

```
float y1 = 1.0f / sqrtf(x);  // two ops
float y2 = rsqrtf(x);        // one SFU instruction
```

The math libraries (Section 5.6) use the intrinsics automatically; so does the compiler, for a whole file, under `nvcc -use_fast_math`, which also flushes denormals to zero and selects the approximate division and square root (BPG §12.1.9).

Caution `-use_fast_math` changes results, not only speed: check the output against the precise build.

Fewer branches. A branch whose condition differs between the lanes of a warp makes the warp execute both paths (Lesson 6). Two remedies: write the condition so that whole warps take the same path, as in a condition on `threadIdx.x / warpSize` rather than on `threadIdx.x` (CPG §5.4.2 Control Flow Instructions), and let the compiler replace short branches by predication.

Definition 5.15 (Predicated execution). In *predicated execution* both sides of a conditional are issued to every lane, and each instruction carries a predicate register that decides, per lane, whether its result is written. No branch instruction is executed and no warp diverges; the cost is that every lane spends the cycles of both sides. The compiler chooses predication only when the number of instructions controlled by the condition is below a threshold (BPG §13.2 Branch Predication).

Table 5.3: The warp shuffle functions. `mask` names the participating lanes (`0xffffffff` for all 32); the optional `width` splits the warp into independent segments.

Function	Lane i receives the value of lane
`__shfl_sync(mask, v, src)`	`src` (one lane broadcasts to all)
`__shfl_up_sync(mask, v, d)`	$i - d$ (lanes below d keep their own value)
`__shfl_down_sync(mask, v, d)`	$i + d$ (the top d lanes keep their own value)
`__shfl_xor_sync(mask, v, m)`	$i \oplus m$ (a butterfly exchange)

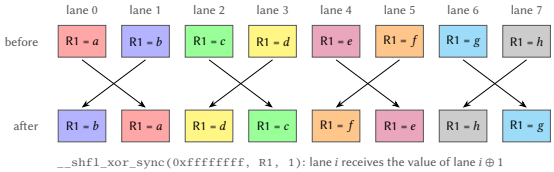


Figure 5.3: `__shfl_xor_sync` with lane mask 1: the eight lanes shown swap R1 with their neighbour in one instruction.

Lesson 6 shows the hardware side: the SIMT stack, the active mask, and the PTX guard predicates of Lesson 10.

Avoid atomics. An atomic read-modify-write on a location that many threads update serialises them at the memory or in the L2 cache. Atomics have become much cheaper over the GPU generations, but a reduction into one counter by thousands of threads is still a serialisation; reduce within the block first (shared memory, or the warp shuffles of Section 5.5) and issue one atomic per block.

Kepler implemented shared-memory atomics as a lock/update/unlock sequence; Maxwell added native shared-memory atomic instructions for 32-bit integers [26]: atomics became "much cheaper than before" through changes of this kind.

Tensor cores. A matrix multiply-accumulate on the tensor cores (Lesson 1) runs an order of magnitude faster than the same operation on the FP32 lanes. Any computation that can be phrased as a matrix product should be, and the libraries do so when the shapes and types allow (Lesson 12).

> **Key point**
> The three terms of Equation (5.1) have three checklists. Transfers: pin, overlap, pipeline, or map. Memory: cache, coalesce, align, count transactions, avoid bank conflicts, compare with the peak. Computation: cheaper instructions, narrower types, intrinsics, warp-uniform conditions, fewer atomics, tensor cores. Profile before and after each step.

5.5 Newer CUDA features

Source M5, newer CUDA features; CPG §4.1 SIMT Architecture, §7.19 Warp Vote Functions, §7.22 Warp Shuffle Functions, §7.26 Asynchronous Barrier, §8 Cooperative Groups; [2].

Several costs of the checklists above have been removed by hardware and API additions since the original CUDA model.

5.5.1 Warp-level operations: shuffle and vote

Registers are private to a thread. In the original model two threads of the same warp could exchange a value only through shared memory: a store, a `__syncthreads()`, a load, with the risk of a bank conflict on the way. Warp-level operations move the value directly between lanes.

Definition 5.16 (Warp shuffle). A *warp shuffle* is an instruction by which each lane of a warp reads a register of another lane of the same warp, with no shared memory and no barrier (CPG §7.22 Warp Shuffle Functions). The four variants differ in which lane is read (Table 5.3); every variant takes a *mask* of the lanes that participate, the value, and the lane selector.

Figure 5.3 shows the simplest case, a pairwise swap. The important use is the reduction: a sum over the 32 lanes of a warp takes five shuffles, one per bit of the lane index, and every lane ends with the total (Figure 5.4, Example 5.17).

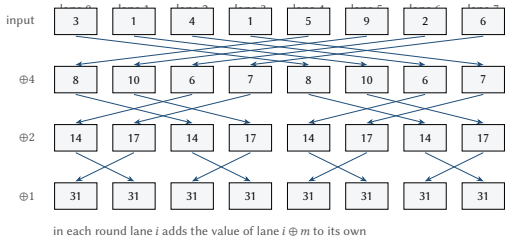


Figure 5.4: The butterfly reduction of Example 5.17: three rounds of `__shfl_xor_sync` with masks 4, 2, 1 leave the sum 31 in every lane.

```
__device__ float warp_sum(float v) {
  for (int m = 16; m > 0; m >>= 1)
    v += __shfl_xor_sync(0xffffffff, v, m);
  return v;   // every lane holds the warp total
}
```

A block reduction then needs shared memory only once, for the per-warp totals, and a grid reduction one atomic per block.

Example 5.17 (Butterfly reduction). Eight lanes hold the values 3, 1, 4, 1, 5, 9, 2, 6 (sum 31), and the loop runs with masks 4, 2, 1. After the first round lane i holds $v_i + v_{i\oplus 4}$: lanes 0–3 hold 8, 10, 6, 7 and lanes 4–7 the same values. After mask 2, lanes 0 and 1 hold $8 + 6 = 14$ and $10 + 7 = 17$, repeated across the warp; after mask 1, every lane holds $14 + 17 = 31$ (Figure 5.4). Three rounds for 8 lanes, $\log_2 32 = 5$ for a full warp.

The `_sync` forms date from CUDA 9, which deprecated the old `__shfl()` family without a mask when Volta introduced independent thread scheduling (below); the mask tells the hardware which lanes must have arrived before the exchange. Compute capability 8.0 adds warp reductions as single functions, `__reduce_add_sync` and relatives (CPG §7.21 Warp Reduce Functions).

Definition 5.18 (Warp vote function). A *warp vote function* evaluates a predicate p in every participating lane and combines the results across the warp (CPG §7.19 Warp Vote Functions):

- `__all_sync(mask, p)` is true if p holds in every lane;
- `__any_sync(mask, p)` is true if it holds in at least one;
- `__ballot_sync(mask, p)` returns the 32-bit mask of the lanes in which it holds.

```
unsigned m = __ballot_sync(0xffffffff, x > 0);
int count = __popc(m);                  // lanes with x > 0
if (__all_sync(0xffffffff, x > 0))  // warp-uniform branch
  ...
```

A ballot followed by `__popc` counts in two instructions, with no barrier, what a shared-memory histogram would take a barrier for; a condition on the result of `__all_sync` or `__any_sync` is uniform across the warp by construction and never diverges.

5.5.2 Cooperative groups

Definition 5.19 (Cooperative groups). The CUDA API that names groups of threads other than the fixed warp and block is called *cooperative groups* (CPG §8 Cooperative Groups). A group is the whole block (`this_thread_block`), a warp or a tile of a warp (`tiled_partition`), the set of lanes of a warp that are active at the call (`coalesced_threads`), the whole grid, or a cluster. Every group offers `sync()` and its size and rank; the warp-sized groups (tiles and coalesced groups) also offer `shfl`, `ballot`, `any`, `all` and `cg::reduce`; the grid group needs a cooperative launch.

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;
auto block = cg::this_thread_block();
auto tile  = cg::tiled_partition<8>(block);  // 8-lane tiles
v += tile.shfl_xor(v, 4);     // shuffle within the tile
tile.sync();                  // barrier for the 8 lanes
```

The API makes the warp size and the block size parameters of a function rather than assumptions baked into it: a routine that takes a tile as a parameter works unchanged for tiles of 4, 8 or 32 lanes, and one that uses only `sync()`, size and rank works for a tile and for a block of

Table 5.4: The NVIDIA libraries named in this lesson.

Library	Domain
cuBLAS	dense linear algebra (BLAS: vector, matrix–vector and matrix–matrix operations); uses the tensor cores for GEMM
cuSPARSE	sparse matrices and sparse–dense products
cuTENSOR	tensor contractions, reductions and elementwise operations
Thrust	C++ parallel algorithms on device vectors (sort, scan, reduce, transform), in the style of the C++ standard library
NCCL	collective communication between GPUs (all-reduce, broadcast, all-gather), the MPI collectives of Lesson 2 for GPUs
NVSHMEM	one-sided, partitioned global address space communication between GPUs
cuDNN	deep-learning primitives: convolutions, pooling, normalisation, attention

1024 threads alike.

5.5.3 Independent thread scheduling

Definition 5.20 (Independent thread scheduling). The execution model introduced with Volta, called *independent thread scheduling*, gives every thread of a warp its own program counter and call stack, so that the threads of one warp can diverge, interleave and reconverge at sub-warp granularity, and a thread waiting in a spin loop no longer blocks the other threads of its warp from making progress.

Before Volta a warp had one program counter: on a divergent branch the hardware ran one path to its reconvergence point before starting the other (Lesson 6), and a thread that waited for a thread on the other path waited for ever. With per-thread state the scheduler may interleave the two paths, which makes such fine-grained synchronisation within a warp correct, at the price of giving up the guarantee that the lanes of a warp execute in lockstep.

Caution Lecture vs document: the lesson describes this feature as smaller warp-level operations in place of a fixed warp size. CPG §4.1 SIMT Architecture is more specific: "Starting with the NVIDIA Volta architecture, Independent Thread Scheduling allows full concurrency between threads, regardless of warp … the GPU maintains execution state per thread, including a program counter and call stack". The warp is still 32 threads; what changed is that code may no longer assume the lanes are in lockstep, which is why the warp-level functions take a mask.

5.5.4 Clusters, unified on-chip memory and split barriers

Three further additions concern the on-chip memories and synchronisation.

Shared memory across blocks. On Hopper the thread block clusters of Lesson 3 give a block access to the shared memory of the other blocks of its cluster, which the hardware schedules together on one GPU processing cluster; NVIDIA calls this *distributed shared memory* [2]. A block can thus exchange data with its neighbours without a round trip through global memory.

Unified L1, shared and texture storage. Since Volta the L1 data cache, the shared memory and the texture cache are one SRAM array per SM, split between cache and scratchpad per kernel; a kernel that uses little shared memory gets a large L1. Lesson 6 treats the carve-out and the research that led to it.

H100: a cluster holds up to 16 blocks (a non-portable size), and an SM has 256 KiB of combined L1 and shared memory, of which up to 228 KiB can be shared memory [2, Table 4].

Hardware split arrive/wait barriers.

Definition 5.21 (Split arrive/wait barrier). A *split arrive/wait barrier* separates the two halves of a barrier: a thread *arrives* when its data is ready and continues with independent work, and *waits* only when it needs the others' data. `__syncthreads()` is an arrive immediately followed by a wait; the asynchronous barrier of CPG §7.26 (`cuda::barrier`) exposes the two halves, and compute capability 8.0 accelerates them in hardware.

The gap between arrive and wait is time the thread would otherwise spend stalled; with the asynchronous copies of CPG §7.27, a thread can start the load of the next tile into shared memory, arrive, and compute on the current tile until the copy has landed.

5.6 Libraries and other platforms

Source M5, libraries and other platforms.

The first rule, before any of the checklists, is to use the libraries.

Definition 5.22 (Math library). A *math library* is a vendor- or community-provided set of GPU routines for a domain (dense linear algebra, sparse matrices, tensors, deep-learning layers, communication) that is already tuned for each GPU generation: coalesced, tiled, tensor-core-aware, pipelined. A program that calls one inherits those optimisations and follows the hardware across generations by relinking.

Table 5.4 lists the ones named. The three most widely used are *cuBLAS*, the dense linear

Table 5.5: Other GPU programming platforms named in this lesson.

Platform	What it is
OpenCL	the Khronos open standard for heterogeneous computing: kernels in a C dialect, portable across GPUs, CPUs and FPGAs
oneAPI / SYCL	Khronos's single-source C++ model (SYCL) and Intel's implementation and libraries (oneAPI)
OpenACC	directive-based offloading, in the style of OpenMP (`#pragma acc`)
HIP	AMD's C++ runtime API, source-compatible with CUDA (`hipMalloc`, `hipLaunchKernelGGL`); CUDA code is ported mechanically with `hipify`
Kokkos	a C++ performance-portability layer that compiles one source to CUDA, HIP, SYCL or OpenMP back ends
Python, Julia	high-level languages with GPU array libraries and kernel compilers (CuPy, Numba; CUDA.jl)

algebra library whose GEMM is the kernel behind most of machine learning (Lesson 12); *cuDNN*, the deep-learning primitives library that frameworks such as PyTorch call; and *NCCL*, the collective-communication library for multi-GPU training (Lesson 9). The library *Thrust* is the odd one out: not a domain library but a C++ template library of parallel algorithms, so that a sort or a reduction on the device is one call rather than a kernel.

`thrust::sort` on a `device_vector` is the quickest correct GPU sort a program can have; writing one's own only pays when the data has structure the library cannot know (Section 5.7).

CUDA is NVIDIA's platform; the same ideas exist elsewhere (Table 5.5).

The standard *OpenCL* has the same execution model as CUDA under other names (work-item, work-group, NDRange for thread, block, grid; K12 §1.3.10), *SYCL* puts host and device code in one C++ source as CUDA does, and *HIP* is close enough to CUDA that the programs of this book translate function by function. What transfers between them is everything in this lesson: the three terms of Equation (5.1) and their checklists are properties of the hardware, not of the language.

5.7 Sorting networks

The sorting networks of the assigned reading [20] are an example of an algorithm reshaped for the GPU.

Source M5, required readings; [20]; [27].

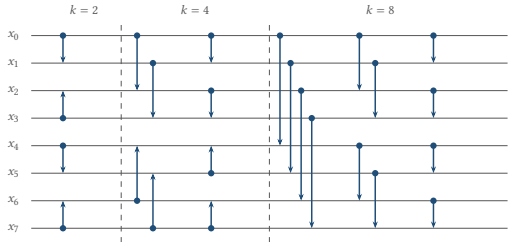
Definition 5.23 (Merge sort). The algorithm called *merge sort* sorts by recursively sorting the two halves of the sequence and *merging* them: two sorted runs are combined by repeatedly taking the smaller of their heads. It is $O(n \log n)$ and the standard sequential algorithm; but the merge is a data-dependent loop, which head is taken next depends on the values, so its control flow and its memory accesses differ from thread to thread.

On a GPU, data-dependent control flow is divergence and data-dependent addresses are uncoalesced accesses (Table 5.1). The algorithms that map well are the ones whose sequence of comparisons is fixed in advance.

Definition 5.24 (Sorting network). A *sorting network* is a fixed sequence of *comparators*, each of which compares two positions of the array and swaps them if they are out of order, that sorts every input of a given size. Which positions are compared at each step depends only on the step and the size, never on the data, so every comparator of a stage can execute in parallel, every thread performs the same operation, and the addresses are known before the kernel runs.

Batcher introduced the two classic networks, odd-even merge sort and bitonic merge sort, in 1968 for hardware sorters [27]; [20] brought them to the GPU in 2005, before CUDA, as fragment (pixel) shader programs over textures (Lesson 1).

Definition 5.25 (Bitonic sort). A *bitonic sequence* first rises then falls (or is a rotation of such a sequence). The sorting network called *bitonic sort* builds bitonic sequences of length $2, 4, 8, \ldots$ by merging sorted runs of alternating direction, and then sorts each bitonic sequence by a *half-cleaner*: comparing element i with element $i + k/2$ splits the sequence into two bitonic halves with every element of the first below every element of the second, and the halves are cleaned recursively (Figure 5.5).



the arrow head receives the larger value; the $k = 2$ and $k = 4$ phases build alternating ascending and descending runs, the $k = 8$ phase sorts the whole sequence ascending

Figure 5.5: A bitonic sorting network for $n = 8$: six stages, four comparators each. Phase k merges pairs of runs of length $k/2$ into sorted runs of length k; within a phase the comparator distance is $k/2, k/4, \ldots, 1$.

For $n = 2^p$ inputs the network has $p(p+1)/2$ stages of $n/2$ comparators, i.e. $\frac{n}{4}\log_2 n(\log_2 n + 1) = O(n\log^2 n)$ comparisons, more than merge sort's $O(n \log n)$; the GPU accepts the extra work for the regularity. Each stage is one data-independent pass in which thread i performs the comparator at distance j on its pair. The same idea underlies the other data-independent primitives of GPU programming: the parallel prefix sum, the butterfly reduction of Figure 5.4, and the FFT.

For $n = 2^{20}$: $p = 20$, 210 stages of 524 288 comparators, about 1.1×10^8 comparisons, against $n \log_2 n \approx 2.1 \times 10^7$ for merge sort: five times the work, all of it regular.

Key point — Lesson summary

A program suits a GPU when it is massively data-parallel, moves little data between host and device relative to its computation, and accesses global memory in a coalesced pattern; the profiler (Nsight Systems for the timeline, Nsight Compute for the kernel) finds the hotspots, on the CPU before porting and on the GPU after. Execution time is transfer plus memory plus compute. Transfers are shortened by pinned memory and hidden by asynchronous copies and a stream pipeline of chunks (concurrent copy and execute); zero copy and UVA let a kernel address host memory, but the data still crosses the bus. Memory accesses are made coalesced, aligned and bank-conflict-free, and the effective bandwidth is compared with the theoretical one. Computation is reduced with cheaper instructions, lower precision, intrinsics, warp-uniform conditions or predication, fewer atomics and the tensor cores. Warp shuffles and votes, cooperative groups, independent thread scheduling, distributed shared memory and split barriers remove old detours through shared memory and old restrictions of the lockstep warp. Libraries (cuBLAS, cuDNN, NCCL, Thrust) carry these optimisations already, and the same rules hold on OpenCL, SYCL and HIP. Sorting networks such as bitonic sort trade extra comparisons for a data-independent schedule, the shape a GPU algorithm wants.

GPU Architecture Optimizations - I

Lecture	M6, lessons 1–3 (Handling Divergent Branch; Register Optimizations; Unified Register File and Shared Memory)
Sources	Fung et al. [28] (dynamic warp formation); Narasiman et al. [29] (large warps); Gebhart et al. [30] (register file cache), [31] (compiler-managed register hierarchy), [32] (unified local memory); Jeon et al. [33] (register file virtualization); NVIDIA A100 [1] and H100 [2] whitepapers; Harris [18] (shared memory latency)
Textbook	A18 [4] ch. 3 (assigned), ch. 4; K12 [3] §1.4.1 GPU Pipeline, §1.4.2 Handling Branch Instructions; CPG [5] §4.1 SIMT Architecture, Compute Capability 7.x–9.0 Shared Memory; PMPP [6] ch. 10
Keywords	divergent branch, divergent warp, reconvergence point, immediate post-dominator, if-conversion, SIMT stack, large warp, dynamic warp formation, SIMD efficiency, live and dead registers, register file cache, register file virtualization, software-managed cache, unified local memory, shared memory carve-out

Lesson 4 followed a warp instruction through the pipeline of an SM and noted two facts that this lesson builds on: a warp carries an active mask that can switch lanes off, and its registers sit in a register file so large that it must be banked. The first part of this lesson asks what happens when the threads of one warp disagree about a branch: how the hardware notices, which lanes run which path, and how the paths are joined again with a stack of masks and program counters. Two research ideas then trade the warp's fixed composition for better lane utilisation. The second part looks at the register file itself, why it is large, slow and power-hungry, and what four observations about register values suggest: a register file cache in front of it, and a virtualized file smaller than the names it serves. The third part compares registers with shared memory and caches and asks whether one on-chip pool could serve all three, a research generalisation of the configurable L1 and shared memory of NVIDIA GPUs. Like Lesson 7, this lesson is organised around research papers, and a second aim is to read such papers.

6.1 Handling divergent branches

The SPMD model of Lesson 1 runs one program in every thread, but nothing forces the threads to follow the same path through it. This section defines divergence, shows two ways to execute a divergent branch (predication and the SIMT stack), and presents two microarchitectural responses to its cost: large warps and dynamic warp formation.

Source M6, divergent branches; K12 §1.4.2 Handling Branch Instructions; A18 ch. 3; CPG §4.1 SIMT Architecture; [28].

6.1.1 Divergent branches and warps

The kernel below lets the first two threads of each block do one thing and the others another; with a warp of four threads, as in the figures of this lesson, the warp of threads 0–3 splits in half.

The figures of this lesson draw warps of four threads so that masks stay short; the real warp size is 32 on every NVIDIA GPU from compute capability 5.0 to 9.0 (CPG Table 21 [5]).

```
__global__ void split(float *a) {
  int i = threadIdx.x;       // BB1: compute the condition
  if (i < 2)
    a[i] = a[i] * 2.0f;      // BB2: Work-1
  else
    a[i] = a[i] + 1.0f;      // BB3: Work-2
  a[i] = a[i] - 0.5f;        // BB4: both paths again
}
```

The compiler cuts such code into a *basic block*, a maximal sequence of instructions that is entered only at its first instruction and left only after its last, so that either all of its instructions execute or none. The blocks and the possible transfers of control between them form the control-flow graph of Figure 6.1, which Lesson 10 defines formally.

Definition 6.1 (Divergent branch and divergent warp). A *divergent branch* is a branch whose outcome differs among the threads of one warp, so that after it the threads of the warp would fetch from different program counters. The warp is then a *divergent warp*, and the phenomenon is called *warp divergence*. A branch on which all threads of every warp agree is warp-uniform

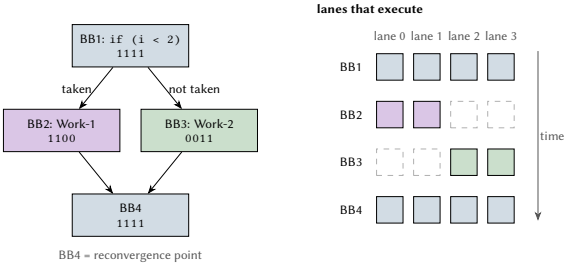


Figure 6.1: A divergent branch for a warp of four threads. Left: the control-flow graph, each basic block with the active mask under which it runs (bit k from the left is lane k). Right: the lanes that execute each block; BB2 and BB3 run one after the other, each with half of the lanes idle.

and costs no more than on a scalar processor.

The active mask of Lesson 4 records which lanes take part in each block (Figure 6.1): BB1 runs with `1111`, BB2 with `1100`, BB3 with `0011` and BB4 again with `1111`. The warp has one program counter, so the two paths cannot run at the same time; they run one after the other, each with the lanes of the other path switched off, and the two paths' times add up. The point where the paths meet again, BB4, is where the warp can resume with its full mask.

Caution A divergent branch does not make the code wrong, only slower: the result is the one a scalar machine would compute. Divergence between *different* warps costs nothing, since each warp has its own program counter.

Definition 6.2 (Reconvergence point). The *reconvergence point* of a divergent branch is the first instruction at which all of the warp's paths from the branch are guaranteed to meet again, so that the warp can continue as a whole. In compiler terms, a block Y is a *post-dominator* of a block X if every path from X to the exit of the program passes through Y, and the *immediate post-dominator (IPDOM)* of X is its closest post-dominator other than X itself. The IPDOM of the branch block is the usual reconvergence point; in Figure 6.1 it is BB4.

Fung et al. report that reconverging at the immediate post-dominator, rather than letting the diverged parts of a warp run apart until the end of the kernel, gives a speed-up of 93.4 % [28].

Reconverging as early as possible matters because every block executed after the branch and before the reconvergence point runs with a partial mask.

6.1.2 If-conversion and predication

The oldest way to remove a branch is to remove the jump.

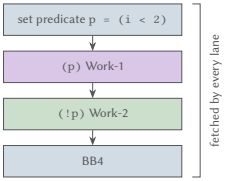


Figure 6.2: The branch of Figure 6.1 after if-conversion: one basic block whose guarded instructions every lane fetches.

Definition 6.3 (If-conversion). The transformation called *if-conversion* replaces a branch and its two paths by straight-line code: an instruction computes the condition into a *predicate*, a one-bit register per lane, and each instruction of the former paths is guarded by the predicate or its negation (Figure 6.2). A lane whose guard is false does not write the result. The resulting code is executed by the predicated execution of Lesson 5.

In PTX the guard is written `@p` or `@!p` in front of the instruction; Lesson 10 shows the predicate registers.

If-conversion is standard on vector processors, and CPUs use it for short branch bodies, where a conditional move is cheaper than a possible misprediction. On a GPU it removes divergence altogether: no lane ever jumps, so the warp keeps one program counter. The price is that the instructions of both paths are fetched and issued even when no lane needs one of them, so the compiler applies if-conversion only to short bodies.

Example 6.4. In the kernel above let BB1 and BB4 have 2 instructions each and BB2 and BB3 have 6 each. Take a warp of four lanes and count lane-slots, one per lane per issued instruction.

- *Divergent warp, branch code*: $2 + 6 + 6 + 2 = 16$ issued instructions, 64 lane-slots, of which $2 \cdot 4 + 6 \cdot 2 + 6 \cdot 2 + 2 \cdot 4 = 40$ do work.
- *Divergent warp, if-converted*: the same 16 instructions (the predicate is computed in BB1) and 40 useful lane-slots.
- *Uniform warp (all lanes take BB2), branch code*: $2 + 6 + 2 = 10$ instructions, all lanes busy.
- *Uniform warp, if-converted*: still 16 instructions; the 6 of BB3 do nothing in any lane.

If-conversion makes the cost independent of the data. In this count it ties when the warp diverges and loses when the warp is uniform; what it saves is the overhead the count leaves out (the branch instruction, the stack push and pop, the reconvergence check and the fetch redirect), which matters only for short bodies.

Tip Cost of if-conversion: the sum of both bodies, always. Cost of a branch: the body taken if the warp is uniform, the sum of both if it diverges.

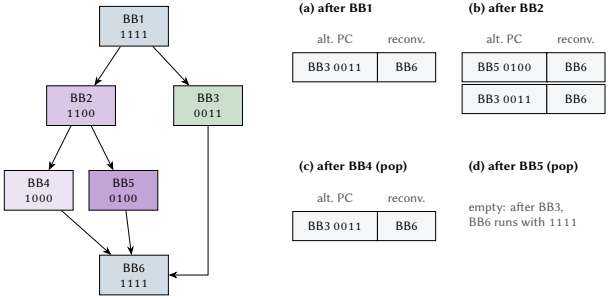


Figure 6.3: The SIMT stack on a nested branch. Left: BB2 diverges again and all paths reconverge at BB6. Right: the stack (top entry first) after the branch in BB1, after the branch in BB2, after the pop at the end of BB4 and after the pop at the end of BB5.

Table 6.1: Execution of the nested branch of Figure 6.3 with a SIMT stack. Stack entries are written (alternate PC, mask; reconvergence PC), top first.

block	mask	next PC	action	stack afterwards
BB1	1111	BB2 / BB3	diverges: push BB3, go to BB2	(BB3, 0011; BB6)
BB2	1100	BB4 / BB5	diverges: push BB5, go to BB4	(BB5, 0100; BB6), (BB3, 0011; BB6)
BB4	1000	BB6	equals top reconv.: pop, fetch BB5	(BB3, 0011; BB6)
BB5	0100	BB6	equals top reconv.: pop, fetch BB3	empty
BB3	0011	BB6	stack empty: full mask (implicit)	empty
BB6	1111	—	—	empty

6.1.3 The SIMT stack

Without predication the hardware itself must run both paths and join them. The mechanism is easiest to understand by building it in steps, following the warp of Figure 6.1.

1. After BB1 the next PCs of the four threads differ (BB2 for lanes 0–1, BB3 for lanes 2–3): the branch is divergent. One PC and one mask cannot express both destinations.
2. Following one path only (BB1, BB2, BB4) is wrong: the threads of lanes 2–3 never execute BB3.
3. The fix is to remember the path not taken: an *alternate PC* (BB3) with its mask (0011), executed after the taken path. The order becomes BB1, BB2, BB3, BB4.
4. One alternate slot is not enough when branches nest. If BB2 itself diverges (Figure 6.3, left: BB4 with 1000, BB5 with 0100), its alternate PC BB5 would overwrite BB3 in the single slot, and BB3 would be lost.
5. The pending paths therefore go onto a stack.

Definition 6.5 (SIMT stack). The *SIMT stack* of a warp holds one entry per pending path of a divergent branch: the alternate PC with its active mask, and the reconvergence PC at which the current path ends. On a divergent branch the hardware pushes an entry for the path not taken and continues with the taken path under the reduced mask. Before every fetch it compares the next PC with the reconvergence PC of the top entry; if they are equal, it *pops* the entry and fetches from the alternate PC with the entry's mask instead. A *nested branch*, a divergent branch inside a path of another, simply pushes another entry.

Table 6.1 traces the nested example. BB4 ends with the next PC BB6, which equals the reconvergence PC on top of the stack, so the warp pops and fetches BB5 with 0100; BB5 ends at BB6 again, the warp pops the second entry and fetches BB3 with 0011; when BB3 ends the stack is empty and BB6 runs with the full mask. The two-field stack does not say where 1111 comes from; the three-field stack of the sidenote restores it from the pre-branch entry. Every thread has executed exactly its own path, and the six blocks took six fetches.[1]

The stack has a hardware cost of a few entries per warp and a reconvergence comparison per

Caution Lecture vs textbook: the two-field entry built below leaves implicit how the full mask returns at the reconvergence point. K12 §1.4.2 gives the entry three fields, "the PC address of the reconvergence point, active mask bits, and the next PC address" [3].

[1] In the three-field formulation of K12 §1.4.2 and A18 ch. 3 [3, 4] the top of the stack always describes what the warp is executing. At a divergent branch the top entry's next PC is replaced by the reconvergence PC (keeping the pre-branch mask), and one entry per path is pushed, each with its own next PC, mask and the common reconvergence PC. When the warp reaches the reconvergence PC of the top entry, that entry is popped and the next one supplies PC and mask, so the pre-branch entry restores 1111 at BB6.

fetch; the number of entries it must hold grows with the nesting depth of divergent branches.

6.1.4 Finding the reconvergence point

The reconvergence PC is computed at compile time: the compiler builds the control-flow graph, finds the immediate post-dominator of every branch, and either marks it in the code or inserts explicit instructions that push and pop the stack. The stack itself is kept in hardware, because only at run time is it known which branches actually diverge. The compiler could in principle avoid the stack operations for branches that never diverge, but deciding statically whether a branch diverges is hard, so a compiler must treat a branch as divergent whenever it cannot prove otherwise, and over-reports divergent branches.

Lesson 11 presents divergence analysis, the compiler analysis that decides which values, and hence which branch conditions, can differ between the threads of a warp.

Note 6.6. The SIMT stack serialises the paths of a branch: the warp runs one path to the reconvergence point before it starts the other. A thread that spins on a value written by a thread on the other path hangs if its own path is scheduled first, since the writer runs only after the spinning path reaches the reconvergence point, which it never does: a *SIMT deadlock*. For example, lane 0 takes the `then` path and loops until a flag in memory is set, while lane 1 takes the `else` path, which sets the flag. If the stack runs the `then` path first, lane 0 loops forever and the `else` path is never fetched; a spin lock taken by several lanes of one warp fails the same way. A18 ch. 3 discusses the problem together with stackless designs that keep a program counter per thread, so that the scheduler can switch to the other path. The rest of this book keeps the per-warp stack model of this section.

Since Volta, NVIDIA GPUs keep a program counter and call stack per thread (independent thread scheduling, Lesson 5) and no longer depend on the classic stack for correctness (CPG §4.1 SIMT Architecture [5]).

6.1.5 Large warps

The warp size is a microarchitectural choice, not a law: one fetch and decode can drive 32, 64 or 128 lanes, and a very wide warp turns the SM into a wide vector machine with little front-end cost per lane. The cost is divergence: the more threads a warp holds, the more likely it is that one of them disagrees with the others, and every divergence serialises paths and occupies the SIMT stack. The right width depends on how uniform the application's control flow is.

Example 6.7. Suppose each thread takes a branch independently with probability $p = 0.1$. A warp of W threads is uniform only if all threads agree, which happens with probability $0.9^W + 0.1^W$: about 0.66 for $W = 4$, 0.034 for $W = 32$ and 0.0012 for $W = 64$. Under this model almost every branch of a 32-thread warp diverges.

Caution Real branch outcomes are correlated (conditions on `threadIdx.x / 32` are uniform per warp by construction), so independent outcomes are a pessimistic model; it shows the trend, not a real divergence rate.

Definition 6.8 (Large warp). A *large warp* is a warp with more threads than the conventional 32 (64, 128 or more), whose instructions are fetched and decoded once for all of them, so that the front-end cost is shared by more lanes.

Caution Lecture vs paper: this chapter, following the lecture, treats a large warp as a wider warp. Narasiman et al. [29] pack the *active* threads of a large warp into SIMD-wide sub-warps every cycle, so divergence leaves fewer idle lanes; combined with two-level warp scheduling (Lesson 7) they report 19.1 % over traditional GPU cores.

The large warp of the paper is thus also a way of fighting divergence, which leads to the next idea.

6.1.6 Dynamic warp formation

After a divergent branch each path runs with only part of its lanes busy; in Figure 6.1 half of the lanes idle in BB2 and in BB3. The fraction of busy lanes is the measure of this loss.

Definition 6.9 (SIMD efficiency). The *SIMD efficiency* of an execution is the number of lane-slots that do work divided by the number of lane-slots issued (warp size × warp instructions issued). A program without divergence has a SIMD efficiency of 100 %; the divergent warp of Example 6.4 has $40/64 = 62.5$ %.

PMPP ch. 10 [6] applies the same measure in software: it rewrites a parallel reduction so that the active threads stay packed in few warps, keeping SIMD efficiency high.

Fung et al. report an average speed-up of 20.7 % for an estimated area increase of 4.7 % from a realistic DWF implementation [28].

The many warps of an SM are often at the same point of the program, and after the same divergent branch, different warps are waiting at the same PC with complementary masks. Their threads can be combined.

Definition 6.10 (Dynamic warp formation). The technique of *dynamic warp formation (DWF)* regroups, at run time, threads of different warps that are about to execute the same PC into new warps with fuller active masks (Figure 6.4).

The constraint is the register file. Lesson 4 assigned the registers of thread k to bank k, so that lane k reads from its own bank; a thread moved into another lane would need its registers from a bank that belongs to another lane, creating the register bank conflicts the layout was designed to avoid. DWF therefore keeps every thread in its home lane and merges only warps

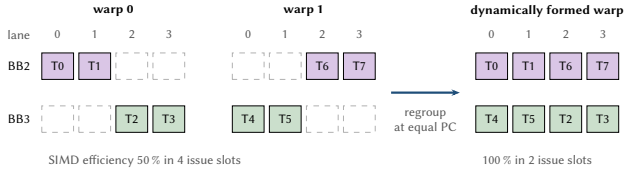


Figure 6.4: Dynamic warp formation. Warps 0 and 1 diverge on the same branch; at BB2 the active threads of warp 0 sit in lanes 0–1 and those of warp 1 in lanes 2–3, so they form one full warp without leaving their lanes. BB3 is merged the same way.

whose active lanes do not overlap, as in Figure 6.4; two half warps that both use lanes 0–1 cannot be merged.

Tip DWF can merge two warps at the same PC only if the bitwise AND of their masks is zero; the new mask is their OR.

Example 6.11. Take two warps executing a branch with the block sizes of Example 6.4, whose condition depends on the data so that warp 0 takes BB2 in lanes 0–1 and warp 1 in lanes 2–3, as in Figure 6.4. Without DWF, BB2 and BB3 cost $2 \times (6 + 6) = 24$ issue slots at 50 % SIMD efficiency. With DWF the merged warps need $6 + 6 = 12$ issue slots at 100 %. The whole kernel drops from $2 \times 16 = 32$ to $2 \cdot 2 + 12 + 2 \cdot 2 = 20$ warp instructions, and its SIMD efficiency rises from 62.5 % to $80/80 = 100$ %.

> **Key point**
> A divergent branch makes a warp run both paths under partial masks. If-conversion trades it for always fetching both paths; the SIMT stack runs the paths one after the other and reconverges at the immediate post-dominator. Wider warps share the front end but diverge more (the large warp microarchitecture of Narasiman et al. repacks active threads into sub-warps), and dynamic warp formation recovers idle lanes, keeping each thread in its home lane.

6.2 Register optimizations

The register file of an SM holds a private register set for every resident thread (Lesson 4), so its size is threads times registers per thread: 64 KiB, 128 KiB or 256 KiB per SM in successive generations. A large SRAM is slow and consumes power, and a GPU cannot accept a slow one: the register file must deliver operands at the rate at which the lanes consume them, so its read bandwidth must equal the execution bandwidth. Two remedies come from CPU design: a hierarchy, to make the common access fast, and sharing, to need less storage. Four observations about how GPU programs use registers show where they apply.

Source M6, register optimizations; K12 §1.4.1 GPU Pipeline; [30, 31, 33].

256 KiB per SM on V100, A100 and H100; per GPU 27 648 KiB on A100 and 33 792 KiB on H100 SXM5 (preliminary figure) [2, Table 3].

6.2.1 Four observations

Not all threads are active. Threads of the same kernel use the same register names, but they do not finish at the same time: execution is asynchronous, and a thread that has finished still occupies its registers until its warp or block is released. On a divergent branch, the two paths may also use different registers.

Not all registers are live. In `add r1, r2, #1` followed by `mov r2, #10`, the value in `r2` is never read again after the `add`: it is dead until the `mov` overwrites it.

Producer and consumer are close. In `mov r2, #10` followed by `add r3, r2, #20`, the value is consumed by the very next instruction, and a scheduler can keep the two together. Such a value may not need to reach the large register file at all.

Most values are read only once. The first value of `r2` in `mov r2, #10` followed by `add r2, r2, #20` is read by one instruction and then overwritten. Many values follow this pattern; storing them in the main register file is questionable.

Caution Lecture vs paper: the lecture gives almost 70 % of register values as read only once. Gebhart et al. measure "up to 70 %" across their workloads, with HPC codes reusing more (40 % of values read more than once), and "up to 40 %" read once *and* within 3 instructions of being produced [30].

Definition 6.12 (Live and dead registers). A register is a *live register* at a point of the program if the value it holds will be read later before being overwritten, and a *dead register* otherwise. The *value lifetime* of a value is the interval from the instruction that produces it to its last read.

The second observation weighs more on a GPU than on a CPU (Figure 6.5). On a CPU, the `add` and the `mov` execute in consecutive cycles, and `r2` is dead for a cycle or less. On a GPU the scheduler interleaves warps, so the next instruction of the same warp may issue tens of

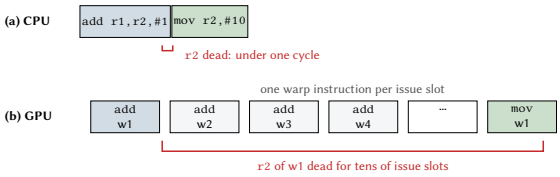


Figure 6.5: After `add r1,r2,#1`, register `r2` is dead until `mov r2,#10`. (a) On a CPU the two instructions are adjacent. (b) On a GPU the issue slots in between belong to other warps, so the dead interval of warp `w1`'s `r2` spans tens of instructions.

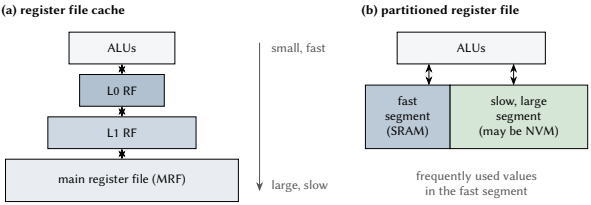


Figure 6.6: Two hierarchical register files. (a) A register file cache: small L0 and L1 register files between the ALUs and the main register file. (b) A partitioned register file with a fast segment and a slow, large segment that may use non-volatile memory.

warp instructions later; for that whole time the dead value of `r2` occupies a register of every thread of the warp.

Lesson 7 discusses the warp schedulers that decide this interleaving; a greedy scheduler shortens the gap, a round-robin one stretches it to the number of ready warps.

Example 6.13. An SM interleaves 24 warps round-robin, one instruction each per round, so consecutive instructions of one warp are 24 cycles apart. A value produced by instruction k, last read by instruction $k+1$ and overwritten by instruction $k+5$ is live for 24 cycles and dead for $4 \times 24 = 96$: its register holds useful data $24/120 = 20\,\%$ of the time. On a CPU issuing one instruction per cycle the same value is live for 1 cycle and dead for 4, with the same 20 %, but the dead register costs one entry of a small file; on the GPU it costs one entry per thread for 96 cycles in a file of hundreds of kibibytes.

6.2.2 Hierarchical register files

The third and fourth observations suggest a hierarchy: values that are read soon after being produced, and only once, can be served from a small structure close to the ALUs.

Tip A value read once, by an instruction issued soon after its producer, is the ideal cache candidate: it is read and dies in the small file and never costs a write to the main register file.

Definition 6.14 (Hierarchical register file). A *hierarchical register file* places smaller, faster register storage in front of the *main register file (MRF)*. The two forms of Figure 6.6 are a *register file cache (RFC)*, one or two small levels (L0 and L1 RF) that hold recently produced values and filter reads and writes to the MRF, and a *partitioned* register file with a fast and a slow segment, in which the slow, large segment can be built in a denser technology.

With an RFC, most reads hit in the small file, and a value that dies before it is evicted never has to be written to the MRF. The MRF can then be smaller, or kept at the same size and accessed less often, which saves energy because a small SRAM costs less per access than a large one. GPUs make this easier than CPUs: K12 §1.4.1 notes that GPUs do not support precise exceptions, so "register files do not have to maintain architectural states" [3], and a value can live only in the cache.

Gebhart et al.: a 6-entry-per-thread RFC cuts MRF reads by 50 % and writes by 59 %; with a two-level scheduler that keeps a quarter of the threads active, register file energy falls by 36 % [30].

A candidate for the slow segment of a partitioned file is *non-volatile memory (NVM)*, memory that keeps its contents without power. Built in NVM, the slow, large segment is meant to save energy, while the values in frequent use stay in the fast segment.

NVM in a register file is a research direction, not a shipped design; current GPUs build the whole register file in SRAM.

The caching decision can also be moved to the compiler, which knows the lifetimes of values. Gebhart, Keckler and Dally [31] replace the hardware RFC by an *operand register file (ORF)*, a small per-thread file whose contents the compiler allocates, and a *last result file (LRF)* of one entry per thread that holds the most recent result for immediate reuse. Because the compiler places every value explicitly, the ORF needs no tags and no hardware replacement, the same property that distinguishes a scratchpad from a cache (Section 6.3).

The compiler-managed hierarchy is reported to cut register file energy by up to 54 %, against 34 % for hardware-only caching; in their baseline the register file consumes 15–20 % of the processor's dynamic energy [31].

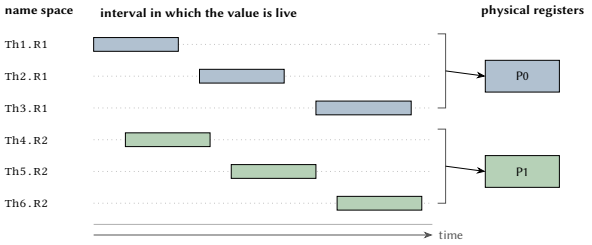


Figure 6.7: Register file virtualization. The values of R1 in threads 1–3 are live in disjoint intervals and share physical register P0; those of R2 in threads 4–6 share P1. Six names need two physical registers.

6.2.3 Register file virtualization

The first two observations suggest sharing: if many registers are dead or belong to finished threads at any moment, fewer physical registers than names suffice [33].

Definition 6.15 (Architectural and physical registers). An *architectural register* is a register name as the program sees it (R1 of thread 3); a *physical register* is a storage entry of the register file. The mapping of architectural to physical registers at run time is *register renaming*. An out-of-order CPU renames a few architectural registers onto *more* physical registers, to remove false dependences.

Definition 6.16 (Register file virtualization). The technique of *register file virtualization* keeps the name space of all threads' architectural registers but maps it onto a *smaller* physical register file: a physical register is assigned when a value is produced and released when the value dies, so that names whose values are live at different times share storage (Figure 6.7). The program semantics are unchanged; only the hardware is smaller.

Caution The direction is the opposite of CPU renaming: a CPU has more physical than architectural registers, register file virtualization on a GPU has fewer. A virtualized file is resource sharing: only the name space has to be maintained.

The mapping costs a table lookup per operand, and the design must release a register exactly when its value dies, which requires lifetime information from the compiler (the last read of each value). It must also guarantee progress when the physical registers run out: a warp that needs a new register while all are taken must wait for others to release theirs, and if every warp waits, none progresses.

Example 6.17. A kernel uses 32 registers per thread and the SM keeps 1024 threads resident, so the name space holds $32 \times 1024 = 32\,768$ registers, 128 KiB at 4 B each. Suppose profiling shows that at most 60 % of them hold live values at any moment, partly because of dead registers and partly because of finished threads. A virtualized file of $0.6 \times 32\,768 \approx 19\,661$ physical registers, about 77 KiB, then holds every live value; the remaining 51 KiB can be removed, or kept to raise the number of resident threads.

Our numbers, chosen for the arithmetic; no measured live fraction is implied.

> **Key point**
> The GPU register file is large because it holds every resident thread's registers, and must be fast because it feeds every lane. Most values are dead for long intervals or read once soon after being produced, which justifies a hierarchy (register file cache, ORF/LRF, fast and slow partitions) for latency and energy, and virtualization (fewer physical registers than names) for capacity.

6.3 Unified register file and shared memory

Lesson 3 showed that occupancy is limited by whichever resource a kernel exhausts first: thread slots, registers or shared memory. Kernels differ widely in how they use the latter two, so a fixed split between them wastes storage. This section compares the three kinds of on-chip storage and presents the unified design.

Source M6, unified register file and shared memory; [32]; A18 ch. 4; CPG Compute Capability 7.x–9.0, Shared Memory.

6.3.1 Occupancy revisited

Consider an SM with a 128 KiB register file (32 768 registers) and 64 KiB of shared memory,

A configurable split predates the unified design: a Fermi SM (2009) splits 64 KiB into 48 KiB of shared memory and 16 KiB of L1 or the reverse [34].

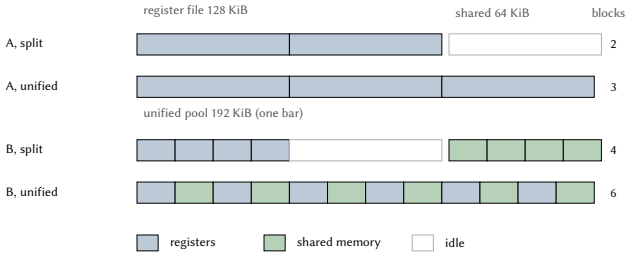


Figure 6.8: Split versus unified storage for two kernels with 256 threads per block. Kernel A fills the register file and leaves shared memory idle; kernel B fills shared memory and leaves half the register file idle. One 192 KiB pool admits three blocks of A and six of B.

Table 6.3: Three kinds of on-chip storage.

	hardware cache (L1)	scratchpad (shared memory)	register file
contents chosen by	hardware	program	compiler
tags	yes	no	no
located by	address and tag	location	location (register name)
accesses per instruction	one address	one address	2–3 reads, one write
pattern known	at run time	at run time	at compile time

and two kernels launched with 256 threads per block (Figure 6.8).

- Kernel A uses 64 registers per thread and no shared memory: $64 \times 256 \times 4\,\text{B} = 64\,\text{KiB}$ of registers per block, so two blocks fill the register file and the whole shared memory idles.
- Kernel B uses 16 registers per thread and 16 KiB of shared memory per block: 16 KiB of registers per block. Shared memory admits four blocks, which use only half of the register file.

If the 192 KiB were one pool from which each block takes its registers and its shared memory, kernel A would fit $192/64 = 3$ blocks and kernel B $192/32 = 6$ blocks: 50 % more resident threads for both (Table 6.2). In the unified design, occupancy is limited by registers and shared memory *together*.

The limit on resident threads (2048 per SM on compute capabilities 7.0, 8.0 and 9.0, CPG Table 21 [5]) still applies; kernel B's six blocks are 1536 threads.

Table 6.2: Resident blocks (threads) per SM for the kernels of Figure 6.8.

kernel	split	unified
A	2 (512)	3 (768)
B	4 (1024)	6 (1536)

6.3.2 Caches, scratchpads and register files

Whether registers and shared memory can share storage depends on how each is addressed. A cache and a scratchpad differ in who decides what they hold.

Definition 6.18 (Hardware- and software-managed caches). A *hardware-managed cache* decides by itself which memory blocks it holds; each entry therefore carries a *tag*, the upper address bits of the block it holds, compared on every access to tell whether the requested address is present, and a full cache evicts a block chosen by its *replacement policy*, for example *LRU (least recently used)*, which evicts the block whose last access lies furthest in the past. A *software-managed cache* (the scratchpad memory of Lesson 3, CUDA's shared memory) holds what the program puts there: the program knows where every datum lives, so there are no tags, no misses and no replacement.

NVIDIA GPUs cache global memory in 128 B lines of four 32 B sectors (Lesson 4), and a warp's accesses are served in 32 B transactions (BPG §10.2.1 Coalesced Access to Global Memory [8]).

Shared memory latency is roughly 100 times lower than that of uncached global memory [18].

A register file resembles a scratchpad: both store data only, have no tags, and select an entry by its location rather than by a memory address. They differ in two respects (Table 6.3). An arithmetic instruction reads two source registers, three for a fused multiply-add $d = a \cdot b + c$, and writes one, so a register file needs more ports than a memory, which serves one address per access. And the registers an instruction uses are named in the instruction, so its access pattern is known statically, before execution, whereas a scratchpad address is computed at run time. The static knowledge is what allows the bank layouts of Lesson 4 and the compiler-managed ORF above, despite the higher bandwidth.

Gebhart et al. build on their earlier register hierarchy and two-level scheduler, which "reduce accesses to the main register file by 60 %", so that a pool with fewer ports per byte than a dedicated register file can still supply the operands [32].

6.3.3 A unified structure

Since all three structures are SRAM arrays on the same SM, one array can serve all of them, partitioned per kernel.

Definition 6.19 (Unified local memory). The design called *unified local memory* is a single on-chip SRAM pool per SM whose capacity is divided, for each kernel, among the register file, the shared memory and, in the full proposal, the *L1 data cache*, the hardware-managed first-level cache for global and local memory accesses.

The unified pool must meet the strictest demand of each user: the bandwidth of the register file, hence enough read and write ports, and more banks than either structure alone, because register accesses (the same register in every lane) and shared-memory accesses (by computed address) have different patterns and must both find their banks free. In exchange, each kernel gets the split it needs, relaxing its occupancy limit.

Shipping GPUs share one array only between the L1 data cache and shared memory, as Fermi did and Volta and later GPUs do again, while the register file stays separate.

Definition 6.20 (Shared memory carve-out). The *shared memory carve-out* is the part of an SM's combined L1 data cache and shared memory array that is configured as shared memory; the rest serves as L1 cache. The CUDA runtime chooses it per kernel from a fixed set of sizes, guided by the kernel's shared memory use and an optional preference.

The preference is set per kernel with one runtime call.

```
// Prefer about half of the maximum shared memory
// capacity for kernel k (a hint; the runtime rounds
// up to a supported carve-out).
cudaFuncSetAttribute(k,
    cudaFuncAttributePreferredSharedMemoryCarveout, 50);
```

A kernel that uses little shared memory thus gets a large L1, one that tiles its data gets most of the array as scratchpad: the flexibility of Figure 6.8 for two structures.

Key point — Lesson summary

A divergent branch makes a warp run its paths one after the other under partial masks until they reconverge at the immediate post-dominator. If-conversion turns the branch into predicated code that fetches both paths; the SIMT stack pushes (alternate PC, mask, reconvergence PC) entries at divergent, also nested, branches and pops them at the reconvergence PC. Wider warps save fetches but diverge more; dynamic warp formation merges threads of different warps at one PC, each in its home lane. Dead, short-lived and read-once register values justify register file caches, ORF/LRF levels and fast/slow partitions; virtualization maps the names onto fewer physical registers. Unified local memory partitions one pool among registers, shared memory and the L1 per kernel; current GPUs do so for L1 and shared memory through the carve-out.

Caution Lecture vs paper: this chapter, following the lecture, unifies the register file and shared memory. Gebhart et al. unify *three* structures, "the register file, scratchpad (shared memory in NVIDIA terminology), and cache", and report "a performance benefit as high as 71 % along with an energy reduction up to 33 %" [32].

Tip A kernel that needs more than 48 KiB of dynamic shared memory per block must opt in with `cudaFuncAttributeMaxDynamicSharedMemorySize` (CPG, Compute Capability 7.x, Shared Memory [5]).

Combined L1 and shared memory per SM: 128 KiB on V100, 192 KiB on A100, 256 KiB on H100. Carve-outs: V100 0, 8, 16, 32, 64 or 96 KiB; A100 up to 164 KiB; H100 0, 8, 16, 32, 64, 100, 132, 164, 196 or 228 KiB (CPG, Compute Capability 7.x, 8.x and 9.0 [5]).

GPU Architecture Optimizations - II

Lecture	M7, lessons 1–3 (GPU Virtual Memory; GPU Virtual Memory - UVA; GPU Warp Scheduling)
Sources	Power et al. [35] (GPU MMU); Ausavarungnirun et al. [36] (multiple page sizes); Li et al. [37] (multi-GPU TLBs); Ganguly et al. [38] (prefetching and eviction in unified memory); Kim et al. [39] (batched fault handling); Rogers et al. [40] (cache-conscious wavefront scheduling); Narasiman et al. [29] and Gebhart et al. [30] (two-level scheduling); Jog et al. [41] (prefetch-aware scheduling), [42] (OWL)
Textbook	A18 [4] ch. 3, ch. 4; K12 [3] §1.4.1 GPU Pipeline; CPG [5] §4.2 Hardware Multi-threading, §19 Unified Memory Programming; BPG [8] §10.1.4 Unified Virtual Addressing
Keywords	virtual memory, address translation, TLB, multi-level page table, page table walker, large page, segment mapping, copy-then-execute, shared virtual memory, on-demand paging, far fault, tree-based neighborhood prefetching, oversubscription, warp scheduler, round-robin, greedy-then-oldest, two-level scheduling, working set, cache-conscious wavefront scheduling

Lesson 6 changed the inside of the SM: how a warp survives a divergent branch and how its registers are stored. This lesson continues with two further parts of the SM's machinery. The first two sections concern memory addresses. A GPU that shares data structures with the CPU needs virtual memory, and translating the addresses of a whole warp at once strains the translation hardware in a way a CPU core never does; when CPU and GPU share one address space, pages also have to be allocated and moved on demand, across PCIe, while the kernel runs. The third section returns to the scheduler stage of the pipeline of Lesson 4 and asks which warp should issue when several are ready, a choice that decides how well memory latency is hidden and how much of the L1 cache each warp can keep. Like Lesson 6, the lesson is organised around research papers; each section names the ones it draws on.

7.1 GPU virtual memory

Source M7, GPU virtual memory; A18 ch. 4; [35]; CPG §19.2.2.1 Memory Paging and Page Sizes.

Virtual memory has two sides: the translation of every address the program issues, and the allocation of the physical pages behind them. On a CPU the operating system allocates pages. On a GPU the allocation was originally done by the CPU together with the GPU driver, through `cudaMalloc`, `cudaMallocManaged` and `cudaMemcpy`; Section 7.2 returns to it. This section reviews address translation, shows what is different when a warp rather than a thread issues the addresses, and presents three ways to make translation cheaper.

7.1.1 Address translation

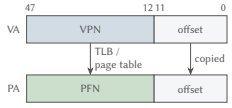


Figure 7.1: Translation keeps the page offset and replaces the VPN by a PFN (bit positions for 4 KiB pages and a 48-bit virtual address).

Caution A TLB miss and a page fault are different events. A TLB miss is resolved by hardware (or a short handler) reading the page table; a page fault needs the operating system and, for a GPU, the host (Section 7.2).

Definition 7.1 (Virtual memory). With *virtual memory* a program issues *virtual addresses* in a private address space, and the hardware maps each one, at every access, to a *physical address* in the memory. The mapping is called *address translation*. It is kept at the granularity of a *page*, a fixed-size, aligned block of the address space, typically 4 KiB or 2 MiB.

A virtual address splits into two fields (Figure 7.1). The upper bits are the *virtual page number (VPN)*, the lower bits the *page offset*, the position inside the page. Translation replaces the VPN by a *physical frame number (PFN)*, also called the physical page number (PPN), and keeps the offset unchanged; a page of 2^k bytes has a k-bit offset.

Definition 7.2 (Page table, TLB and page fault). The *page table* is the data structure, kept in memory, that holds the mapping; each *page table entry (PTE)* maps one VPN to a PFN and carries metadata such as valid and permission bits. A *TLB (translation lookaside buffer)* is a small cache of recently used translations. A *TLB miss* occurs when the TLB does not hold the translation, so the page table must be read; a *page fault* occurs when the page table holds no valid mapping because no physical page has been allocated yet, so that the operating system must allocate one.

The TLB exists because the PTE itself is in memory: without it, every load would first need

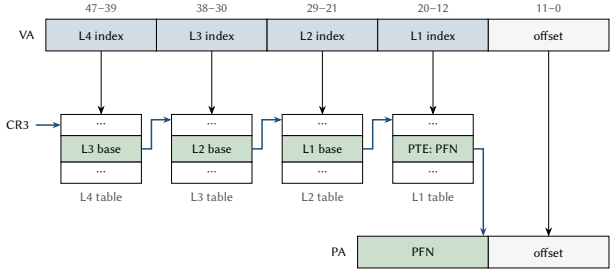


Figure 7.2: A four-level page table walk with the x86-64 layout. Each index selects an entry holding the base of the next table; the level-1 entry is the PTE, whose PFN joined with the offset is the physical address.

one or more memory accesses to find its translation.

7.1.2 Multi-level page tables

A single table indexed by the VPN would need 2^v entries for a v-bit VPN, whether or not the program uses the pages.

Definition 7.3 (Multi-level page table and page table walk). A *multi-level page table* splits the VPN into several indices, one per level. The index of the top level selects an entry of the top-level table that holds the base address of a next-level table, whose entry is selected by the next index, and so on; the entry of the last level is the PTE. Following this chain is a *page table walk*. It is performed either by a *page table walker (PTW)*, a hardware state machine, or by a software miss handler. Tables that no used address reaches are never allocated.

Figure 7.2 shows the four-level form used by x86-64, ARM and GPUs. The walk starts at a root register, uses the level-4 index to read the level-3 table base, and so on down to the level-1 entry, the PTE; PFN and offset together form the physical address. The price of the saved space is time: one translation that misses in the TLB costs at least four dependent memory accesses.

x86-64 with four levels: 48-bit virtual addresses, four 9-bit indices and a 12-bit offset; each table has 512 entries of 8 B, exactly one 4 KiB page; the root register is CR3.

Example 7.4. Take the x86-64 layout: a 36-bit VPN and 8 B entries.

- *One level*: $2^{36} \approx 6.9 \times 10^{10}$ entries, 512 GiB of page table per process, nearly all of it for pages that are never used.
- *Four levels, best case*: a program whose pages all lie under one level-1 table needs one table per level, $2^9 + 2^9 + 2^9 + 2^9 = 2048$ entries in 16 KiB, and maps up to 512 pages (2 MiB).
- *A sparse program*: two arrays far apart in the address space need two chains below the level-4 table, $1 + 2 \cdot 3 = 7$ tables, still only 28 KiB.

Thousands of entries replace tens of billions.

Caution The size of a single-level table is $2^{\text{VPN bits}}$, not $2^{\text{address bits}}$: the offset bits do not index the table.

7.1.3 Address translation on a GPU

Early GPUs had no virtual memory, or only a fixed one-to-one mapping. Virtual memory became necessary as GPUs became programmable and began to share data with the CPU, most visibly with the unified virtual addressing of Lesson 5. Translation works as on a CPU, with one difference: it happens after *address coalescing*.

In the pipeline of Lesson 4 the memory stage now contains the steps of Figure 7.3. The coalescer merges the requests of the warp's threads (Lesson 4); each resulting request needs a translation, looked up in the TLB; a miss is resolved by the PTW or by a TLB miss handler. In this organisation the L1 data cache is accessed with virtual addresses and the L2 cache, shared by all SMs, with physical addresses.

Power et al. point out that an AMD A10 APU with 400 lanes "can require up to 400 unique translations in a single cycle" [35].

Coalescing decides how many translations a warp instruction needs. Two pages that hold requests of the same instruction need two TLB lookups, however close the requests are; two requests in one page need one.

Both placements exist: the proof-of-concept MMU of Power et al. translates after the coalescer and *before* the L1, so its L1 is physically addressed [35].

Example 7.5. Take a warp of eight threads, 64 B memory requests and a toy page size of 256 B, so that the VPN is the address shifted right by 8 bits. A load through `R1` reads the consecutive words 64, 68, ..., 92: one request, one page, one TLB lookup. A load through `R2` reads with a

The toy page of 256 B keeps the numbers small; with 4 KiB pages the same effect appears with strides of kilobytes, such as the column of a row-major matrix with 4 KiB rows.

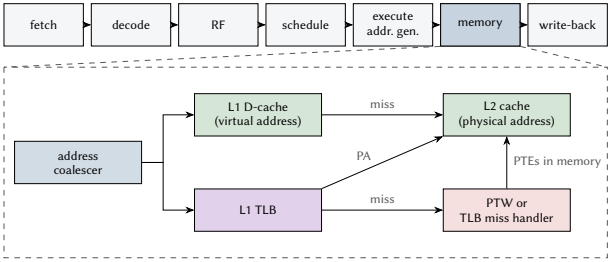


Figure 7.3: The memory stage of the GPU pipeline with address translation. The coalescer merges the warp's requests; the L1 data cache is accessed with virtual addresses while the TLB translates; a TLB miss goes to the PTW or a miss handler; the L2 cache is accessed with physical addresses.

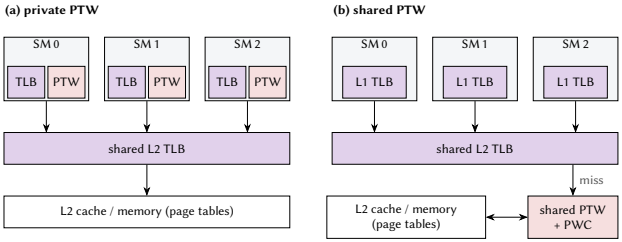


Figure 7.4: (a) A private PTW in every SM, with a shared L2 TLB. (b) SMs with L1 TLBs only and one shared, high-bandwidth PTW with a page walk cache (PWC) next to the shared L2 TLB. The design of Power et al. is (b) without the L2 TLB.

stride of 160 B:

thread	0	1	2	3	4	5	6	7
address	0	160	320	480	640	800	960	1120
hex	000	0A0	140	1E0	280	320	3C0	460
VPN	0	0	1	1	2	3	3	4

No two addresses fall into one 64 B request, so the load issues eight requests; they touch five distinct pages, so after merging requests to the same page it needs five TLB lookups for a single instruction. With a stride of 256 B or more every thread would touch its own page.

Tip TLB lookups per warp instruction: the number of distinct pages its addresses touch, at most the number of threads in the warp; a coalesced access needs one or two.

In the worst case an uncoalesced load needs as many translations as the warp has threads. Uncoalesced accesses therefore cost twice: once in memory bandwidth (Lesson 4) and once in translation bandwidth.

7.1.4 Scalability of translation

The challenge of GPU address translation is scalability. One warp instruction can produce many TLB misses at once, and dozens of warps per SM and dozens of SMs issue them concurrently; the page table walker must therefore sustain a high bandwidth of walks. Figure 7.4 shows two organisations. With a private PTW per SM each SM walks its own misses, but every SM pays for a walker that idles most of the time. With a shared PTW the SMs keep only their L1 TLBs and send their misses to one high-bandwidth walker next to a shared L2 TLB.

Power, Hill and Wood build the shared organisation from known CPU parts [35]: per-compute-unit TLBs placed after the coalescer, a highly threaded PTW that keeps the state of many outstanding walks in a set of buffers, and a *page walk cache*, a small cache in the MMU that holds entries of the upper (non-leaf) levels of the page table, so that most walks need fewer than four memory accesses. Their design has no L2 TLB: the per-CU TLBs feed the shared walker and its page walk cache directly. A shared L2 TLB in place of the page walk cache was more than 2× slower on average, and one added beside it changed performance by about 0.1 % [35, §6].

Measured on the Rodinia benchmarks and a database sort: coalescing and the scratchpad cut the translation rate by 6.8×, TLB misses come in bursts of 60 concurrent misses on average, and post-coalescer TLBs miss 29 % of the time [35].

The page walk cache cuts the average walk latency by over 95 %; the walker needs it because the GPU's L2, where the PTEs are cached, takes nearly 300 cycles. The complete design comes within 2 % of an ideal MMU on average, with 4 KiB pages and unmodified x86-64 page tables [35].

7.1.5 Reducing the cost of translation

Three techniques reduce the number or the cost of translations.

Segment mapping. In *segment mapping* each program's virtual space maps onto one contiguous region of physical memory, so translation is the addition of a base (Figure 7.5): in the figure, virtual address `0x1234` of program 2 is physical address `0x011234`. No table and no walk are needed, but each program's space must be contiguous in physical memory.

Large pages. Fewer, larger pages need fewer translations (below).

Multi-level TLBs. A small L1 TLB per SM backed by a larger, shared L2 TLB, as in Figure 7.4, can serve an L1 miss with a translation that another SM already brought in, before a walk starts (with a page walk cache present, Power et al. saw no gain from it, above).

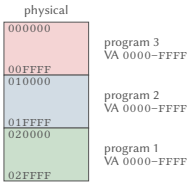


Figure 7.5: Segment mapping: three programs with the same virtual range in three 64 KiB physical segments.

Definition 7.6 (Large page). A *large page* is a page much larger than the base page, typically 2 MiB instead of 4 KiB. Its offset is 21 bits instead of 12, so the VPN is 9 bits shorter, and on x86-64 the walk ends one level early. One TLB entry then covers 2 MiB/4 KiB = 512 times as much memory.

The amount of memory a TLB can map at once, entries times page size, is its *reach*. The cost of a large page is *internal fragmentation*: memory allocated in whole pages but not used, lost inside the last page of an allocation.

Example 7.7. An L1 TLB of 32 entries reaches 32 × 4 KiB = 128 KiB with base pages and 32 × 2 MiB = 64 MiB with large pages. An allocation of 10 KiB occupies three 4 KiB pages and wastes 2 KiB, but one 2 MiB page wastes 2038 KiB, over 99 % of the page (our numbers, chosen for the arithmetic).

Large and small pages can also be mixed, so that each region of memory gets the page size that suits it.[1]

> **Key point**
> A GPU translates the addresses of whole warps: coalescing decides how many translations an instruction needs, an uncoalesced access can need one per thread, and the walker must handle many misses at once. Shared high-bandwidth walkers with page walk caches, multi-level TLBs, segments and large pages reduce the cost; large pages trade TLB reach against internal fragmentation.

CPG §19.2.2.1 Memory Paging and Page Sizes: x86-64 CPUs use 4 KiB physical pages, while "NVIDIA GPUs support multiple physical page sizes, but prefer 2MiB physical pages or larger"; TLB misses "are generally significantly more expensive on the GPU compared to the CPU" [5].

[1] Research has continued along these lines. Mosaic [36] supports several page sizes at once without application changes and reports 55.5 % (homogeneous) and 29.7 % (heterogeneous) average gains for multi-application workloads, within 6.8 % and 15.4 % of an ideal TLB. For several GPUs, Li et al. [37] share translations among peer GPUs and spill TLB entries to the IOMMU's TLB, gaining 23.5 % and 16.3 % on average for single- and multi-application workloads.

7.2 Unified virtual address space and on-demand paging

The second side of virtual memory is page allocation, and here the GPU differs from the CPU most. This section compares explicit copying with a shared address space, follows a GPU page fault to the host and back, and presents the prefetching and eviction policies that make on-demand paging affordable.

Source M7, UVA and on-demand paging; CPG §19 Unified Memory Programming; BPG §10.1.4 Unified Virtual Addressing; [38, 39].

7.2.1 Copy-then-execute versus a shared address space

Definition 7.8 (Copy-then-execute model). In the *copy-then-execute model* the host allocates device memory (`cudaMalloc`) and copies the input into it (`cudaMemcpy`) before it launches the kernel, and copies the results back afterwards. All data a kernel touches is in device memory before the kernel starts.

The data transfer is explicit and happens in the preparation before the kernel launch. It has one strength: bulk transfers that the programmer can schedule use the PCIe bandwidth fully, as the pinned buffers and streams of Lesson 5 showed. Its weakness is that the programmer manages every buffer and every copy by hand, and that data structures with pointers cannot be shared at all.

Definition 7.9 (Shared virtual memory). With *shared virtual memory (SVM)*, the name used in the OpenCL/CUDA programming model, the CPU and the GPU use one virtual address space, so that one pointer means the same datum on both (Figure 7.6).

The *unified virtual addressing (UVA)* of Lesson 5 is one address space over separate memories; a shared address space raises programmability further when the pages also move by themselves. Most GPUs in high-performance computing are discrete: they have their own physical memory,

Caution "SVM" is the term of OpenCL 2.0. CUDA does not use it: it calls the shared address space unified virtual addressing and the migrating variant unified (managed) memory.

Lesson 3 introduced the programming side of `cudaMallocManaged`; Lesson 5 separated UVA (one address space) from unified memory (automatic page migration).

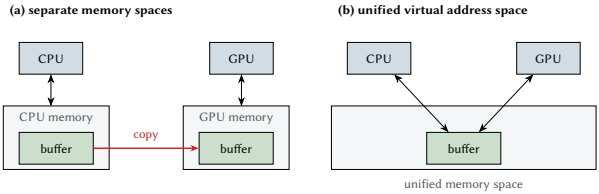


Figure 7.6: (a) Separate memory spaces: each processor reads and writes its own buffer and explicit copies move the data. (b) A unified virtual address space: one logical buffer that both processors read and write.

so a shared address space still requires copies between host and device memory, now performed on demand. That needs *dynamic page allocation*: the decision where a page lives is taken while the kernel runs.

Definition 7.10 (On-demand paging). Under *on-demand paging* memory allocated with `cudaMallocManaged` is not placed in device memory in advance; the first GPU access to a page that is not resident faults, and the page is then allocated in device memory and its data migrated from the host.

The two models side by side, for a kernel `scale` that multiplies n floats in place:

```
// copy-then-execute
float *d;
cudaMalloc(&d, n * sizeof(float));
cudaMemcpy(d, h, n * sizeof(float),
           cudaMemcpyHostToDevice);
scale<<<(n + 255) / 256, 256>>>(d, n);
cudaMemcpy(h, d, n * sizeof(float),
           cudaMemcpyDeviceToHost);

// on-demand paging
float *m;
cudaMallocManaged(&m, n * sizeof(float));
for (int i = 0; i < n; ++i) m[i] = i;  // CPU touches
scale<<<(n + 255) / 256, 256>>>(m, n);  // GPU faults
cudaDeviceSynchronize();      // m is valid on the CPU
```

CPG §19.3.2.1 Data Migration and Coherency: before compute capability 6.x, all managed memory was moved to the GPU at every kernel launch; compute capability 6.x (Pascal) introduced GPU page faulting, which migrates pages on demand [5].

The second version has no copy in the program, but the copies still happen: each page crosses PCIe when the kernel first touches it.

7.2.2 The page-fault path

Definition 7.11 (Far fault and GMMU). In a GPU with unified memory, the *GPU memory management unit (GMMU)* receives the TLB misses of the SMs and walks the GPU page table. A *far fault* is a GPU page fault on a page that is not present in device memory, which the host driver must resolve by allocating a page and migrating its data, in contrast to a *near fault*, an L2 miss that device memory serves.

Figure 7.7 follows a far fault.

1. The GMMU's walk finds no valid PTE; the fault is recorded in the GPU's page-fault MSHRs (miss status holding registers), and the faulting warp waits.
2. The fault is sent over PCIe to the CPU.
3. The operating system and the GPU driver handle it: they allocate a page in device memory and migrate the data.
4. The new mapping is sent back; the PTE is validated, and the access is replayed.

The round trip takes several tens of microseconds.

Caution Lecture vs paper: one passage of the source puts this latency at "tens of milliseconds"; it is tens of microseconds. Prior work reports 20 µs to 50 µs per fault, and Kim et al. measure 223 µs to 553 µs for a whole batch of faults (median 313 µs) [39].

Both processors reach memory through a translation unit for devices.

Definition 7.12 (IOMMU and DMA). *DMA (direct memory access)* is the transfer of data between a device and memory without the CPU executing the copy. An *IOMMU* (input–output memory management unit) translates the addresses that devices use for DMA, so that devices can work with virtual instead of physical addresses (I/O virtualisation).

GPU memory was once managed only through the IOMMU, as the memory of a peripheral device. With a shared address space the CPU takes part in every mapping change, and this is the cost of on-demand GPU memory: page management is performed by the host, and fetching a mapping from the CPU costs a PCIe round trip plus the CPU's interrupt service routine, often several tens of microseconds.[2] Three remedies follow.

- *Prefetch pages*: predict the pages that will be needed and migrate them before they fault.
- *Move page-mapping management to the GPU side*, so that a fault does not need the host.
- *Let CPU and GPU cooperate*: the CPU manages large chunks of memory, the GPU the small pages inside a chunk it has been given.

Prefetching is the remedy that NVIDIA's driver implements [38].

Ganguly et al. give a far-fault handling latency of typically 45 µs on Pascal GPUs, before the PCIe transfer of the data [38].

[2] The driver does not handle one fault at a time: it collects faults in a buffer and processes them in batches, and faults raised while a batch is processed wait for the next one. Kim et al. [39] show that this serialisation hurts throughput and propose larger batches (by switching thread-block contexts so that more faults are raised together) and eviction overlapped with migration: 2× average speed-up over state-of-the-art prefetching, batches 2.27× larger and 51 % fewer, batch processing time 27 % shorter.

7.2.3 Page prefetching

Definition 7.13 (Prefetching). Fetching data before it is requested, on the strength of a prediction of future addresses, is called *prefetching*. Applied to the pages of a unified address space it is *page prefetching*: migrating pages to device memory before the GPU faults on them.

A sequential predictor that sees the addresses `0x40`, `0x80`, `0xC0` predicts `0x100` and `0x140` next. A vector addition that reads `a[idx]` and `b[idx]` with consecutive indices produces exactly such a sequence, and page addresses of such an array are as predictable: a sequential page predictor fetches page $p+1$ when page p faults.

On a GPU the prediction breaks down, because thread blocks do not run in order (Lesson 3). In Figure 7.8 each block reads three consecutive pages, but the blocks run in the order 2, 0, 1, and the faults arrive as 6, 7, 8, 0, 1, 2, 3, 4, 5: after page 8 a "next page" predictor fetches page 9, which no block reads, and the jump back to page 0 is a fault it could not foresee. Spatial locality is there, in the neighbourhood of a fault, but not in the order of the faults.

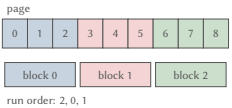


Figure 7.8: Nine pages read by three thread blocks that run in the order 2, 0, 1: the faults arrive as 6, 7, 8, 0, 1, ...

Definition 7.14 (Tree-based neighborhood prefetching). The policy of NVIDIA's unified-memory driver, called *tree-based neighborhood prefetching* (TBNp), divides each 2 MiB region of a managed allocation into two 1 MiB halves and those into 64 KiB nodes. A 4 KiB fault migrates the whole 64 KiB node, 16 pages; when several nodes under one parent have faulted, the parent node, up to 1 MiB, is fetched.

The tree captures spatial locality around each fault, whatever the order of the faults, and bounds the size of a single prefetch.

Example 7.15. Apply the rule of Ganguly et al. to far faults in the leaves 0, 1, 2 and 4 of a 2 MiB region, in this order (Figure 7.9). After each fault the GMMU adds up the valid size of every ancestor; a node more than half valid has its emptier child filled up to the size of the fuller one.

fault	ancestors above 50 %	migrated
leaf 0	none (128-KiB node: 64 of 128)	64 KiB
leaf 1	128-KiB node full, nothing to fill (256-KiB node: 128 of 256)	64 KiB
leaf 2	256-KiB node: 192 of 256; fill leaf 3	128 KiB
leaf 4	512-KiB node: 320 of 512; fill leaves 5–7	256 KiB

Four faults bring in 512 KiB, eight leaves, instead of the 16 KiB that four on-demand 4 KiB migrations would. Leaf 0 alone did not trigger a prefetch beyond its own node: at exactly 50 %

Caution Lecture vs paper: Ganguly et al. [38] measured the exact rule on a GTX 1080 Ti. The tree is a full binary tree with 64 KiB leaves, so it has the levels 128, 256 and 512 KiB in between; a node is filled up when its "to-be valid size" is "strictly greater than 50 %" of its capacity, so a prefetch ranges from 64 KiB to 1 MiB.

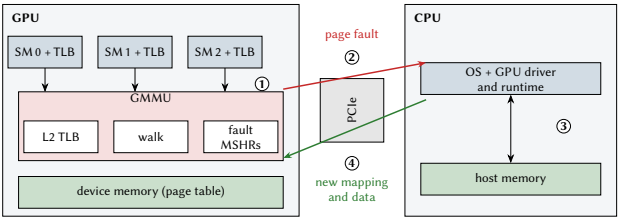


Figure 7.7: A far fault on a discrete GPU: (1) the GMMU finds no valid PTE and records the fault; (2) the fault crosses PCIe; (3) the OS and the GPU driver allocate and migrate the page; (4) the mapping returns and the access is replayed.

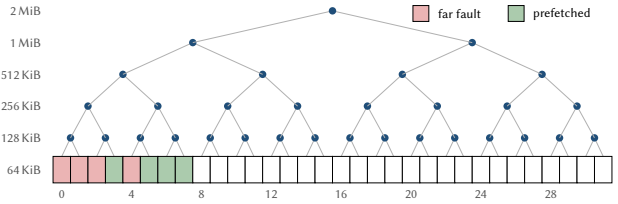


Figure 7.9: The prefetch tree of a 2 MiB region: 32 leaves of 64 KiB (16 pages each) under binary levels up to the root. Shaded: the end state of Example 7.15, with far faults in leaves 0, 1, 2 and 4 and the leaves prefetched.

a node is not filled.

7.2.4 Oversubscription and eviction

Definition 7.16 (Oversubscription and eviction). The allocation of more managed memory than the GPU's physical memory holds is called *memory oversubscription*. Pages must then leave device memory to make room, which is *page eviction*; the policy decides which pages go.

CPG §19.1.2.7 GPU Memory Oversubscription: unified memory lets applications "allocate and share arrays larger than the memory capacity of any individual processor in the system" [5].

Prefetching and eviction interact. Aggressive prefetching evicts pages that are still needed, and an eviction policy that ignores the prefetcher evicts scattered 4 KiB pages, which breaks the contiguous regions the prefetcher relies on. The problem grows with the memory demand of kernels. One answer is *pre-eviction*: together with the page that the LRU list selects for eviction, the contiguous pages around it are evicted in multiples of the 64 KiB blocks in which the prefetcher brought them in, so that free space appears in units the prefetcher can use again.

Ganguly et al.: a tree-based pre-eviction combined with the hardware prefetcher gives 93 % and 18.5 % average speed-ups over LRU replacement of 4 KiB and 2 MiB pages [38].

> **Key point**
> A shared address space removes the explicit copies, not the data movement: on a discrete GPU every first touch of a page is a far fault that crosses PCIe to the host and back, tens of microseconds. Prefetching around each fault (the tree-based scheme) amortises the round trips despite the out-of-order execution of thread blocks; under oversubscription the eviction policy must respect the units the prefetcher works in.

7.3 Warp scheduling

Source M7, GTO vs round-robin; A18 ch. 3; K12 §1.4.1 GPU Pipeline; CPG §4.2 Hardware Multithreading; [40, 29, 30].

The pipeline walk-through of Lesson 4 assumed one ready warp at a time. Usually several warps have all their operands ready in the scoreboard at once, and one of them must be chosen. This section defines the basic policies, shows how the choice affects latency hiding and the L1 cache, and presents research schedulers built on these observations.

Definition 7.17 (Warp scheduler). The *warp scheduler* is the pipeline stage that, every cycle, selects one warp among those whose next instruction is ready and issues that instruction.

CPG §16.6.1 (Compute Capability 7.x, Architecture): an SM has 4 warp schedulers; it distributes its warps statically among them, and each issues one instruction per cycle for one of its ready warps [5]. With the 64 resident warps of CC 7.0 (CPG Table 21; 32 on CC 7.5) that is up to 16 warps per scheduler (our arithmetic).

7.3.1 From instruction scheduling to warp scheduling

An out-of-order CPU has a similar unit. Its instruction scheduler selects among the ready instructions of a scheduling queue. The default policy is *oldest-first*: among the ready instructions, the one that entered the queue first issues. Alternatives give priority to *critical* instructions: loads first, so that memory requests start as soon as their addresses are known, or instructions with many dependants first, so that more instructions become ready sooner.

The GPU warp scheduler is in order within each warp: a warp's instructions issue in program order. Across warps it may pick any ready warp, so the instruction stream of the SM as a whole looks out of order. Four warps that each run the instructions I1, I2, I3 may, for example, issue as

I1 w1, I1 w2, I2 w1, I1 w3, I2 w2, I3 w1, I1 w4, I2 w3, I2 w4, I3 w2, I3 w4, I3 w3,

which keeps the order I1, I2, I3 inside every warp. Which ready warp goes first is the policy.

K12 §1.4.1: the scheduler is in order but "can select any warps that are ready", so the SM's execution "might look like an out-of-order execution" [3].

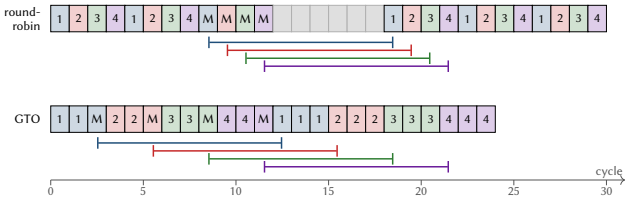


Figure 7.10: Issue slots of Example 7.19: the number is the warp, M its load, grey a stall cycle; the bars below each row are the 10-cycle load latencies. Round-robin issues the four loads back to back and stalls; GTO staggers them.

7.3.2 Round-robin and greedy-then-oldest

Definition 7.18 (Round-robin and GTO). A *round-robin (RR)* scheduler gives the warps turns in a fixed cyclic order, one instruction each. In the *loose round-robin (LRR)* form, a warp that cannot issue in its turn is skipped and the next one in order gets the chance. A *greedy-then-oldest (GTO)* scheduler keeps issuing from one warp until it stalls on a long-latency event (a cache miss, a branch, a long tensor-core operation), then switches to the oldest ready warp, the age of a warp being the time it was assigned to the SM.

Rogers et al. define both baselines: GTO "runs a single wavefront until it stalls then picks the oldest ready wavefront", the age being "the time it is assigned to the core" [40].

The two policies differ in when the warps reach their memory instructions. Under round-robin all warps advance together and reach their loads at nearly the same time; under GTO one warp runs ahead and starts its load early, and the others follow one by one.

Example 7.19. Four warps run the same program: two arithmetic instructions `c1`, `c2`, a load `m3` that misses in the cache and returns after 10 cycles, and three arithmetic instructions `c4`–`c6`, of which `c4` uses the loaded value. Figure 7.10 shows the issue slots.

- *Round-robin*: cycles 0–7 issue `c1` and `c2` of all warps, cycles 8–11 the four loads. Every warp now waits for its load; the first returns in cycle 18, so cycles 12–17 are stalls. The 12 remaining instructions take cycles 18–29: 30 cycles, 6 of them stalls.
- *GTO*: warp 1 issues `c1`, `c2`, `m3` in cycles 0–2 and stalls; warp 2 follows in cycles 3–5, warp 3 in 6–8, warp 4 in 9–11. Warp 1's load returns in cycle 12, one cycle after warp 4's load issues, and each warp then finishes its three instructions as its data arrives: 24 cycles, no stall.

The loads under GTO are spread out, and the other warps' arithmetic hides each one.

Notation: one instruction issues per cycle; a miss issued in cycle t lets the dependent instruction issue in cycle $t + 10$. Our numbers.

GTO is not always better: the gain depends on the program. In the example it comes from the arithmetic after the load. A program whose warps all stall at the same barrier right after their loads gains nothing; with a load latency longer than the arithmetic of all other warps together GTO stalls too; and when warps share data, issuing them together under round-robin may let one warp's miss serve the others.

7.3.3 Two-level scheduling

A scheduler that searches all ready warps every cycle compares tens of candidates for one issue slot, and this search costs power.

Definition 7.20 (Two-level warp scheduling). In *two-level warp scheduling* the warps are divided into an *active warp set* / *pending warp set* pair. Only the few active warps are searched; a warp that meets a long-latency operation moves to the pending set, and a pending warp whose operation has completed moves back to the active set when there is room (Figure 7.11).

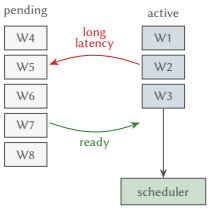


Figure 7.11: Two-level scheduling: the scheduler searches only the active set.

Two papers are cited for this idea, with different motives. Narasiman et al. divide the warps into *fetch groups*, for example 32 warps into 4 groups of 8; one group has priority until all its warps stall, with round-robin inside and across groups. The groups thus reach their long-latency operations at different times, the same staggering as in Example 7.19, and some group always has work while another waits for memory.

Caution Lecture vs paper: Gebhart et al. [30] introduce the active/pending split for *energy* (with a register file cache, 36 % less register file energy). Narasiman et al. [29] split the warps into fetch groups for *performance*: 19.1 % together with large warps.

7.3.4 Scheduling and cache behaviour

The scheduler also decides which data must be in the L1 cache at the same time.

Definition 7.21 (Working set, thrashing and compulsory misses). The *working set* of a program over an interval is the set of memory blocks it uses in that interval. The cache suffers *cache thrashing* when the working set exceeds it, so that blocks are evicted before they are reused and most accesses miss. A *compulsory miss* is the miss of the first access to a block, which no cache can avoid.

Rogers et al. classify the hits of their benchmarks this way: in the highly cache-sensitive ones most of the reuse is intra-warp [40].

A hit on a block that the same warp brought into the cache exploits *intra-warp locality*; a hit on a block another warp brought in exploits *inter-warp locality*. When each warp reuses its own data, as in the next example, round-robin is the worst order.

Example 7.22. Three warps each read their own block four times: warp k reads block Bk. The L1 holds two blocks with LRU replacement (Lesson 6), and each access hits or misses before the next one issues.

order	accesses (bold = miss)
RR	**B1 B2 B3 B1 B2 B3 B1 B2 B3 B1 B2 B3**
GTO	**B1** B1 B1 **B2** B2 B2 **B3** B3 B3 B3

Under round-robin each block is evicted by the two others before its warp returns to it: 12 misses in 12 accesses. Under GTO each warp finishes its block before the next starts: only the 3 compulsory misses.

Caution The GTO row assumes that a warp keeps issuing after its first miss. When other warps are ready at that miss, GTO switches, and its working set holds more than one warp's data; hence $SW \geq 1$ for GTO in the formula below.

In general, let each warp have a working set of size SS_W and let N warps be resident on the SM. Under round-robin all N warps are in flight at once and the L1 must hold $SS_W \times N$. A scheduler that limits the number of schedulable warps to SW needs only $SS_W \times SW$; for GTO, SW is not necessarily 1, because it switches warps at every long-latency event. In Example 7.22, $SS_W = 1$ block and $N = 3$, which exceeds the two-block cache; with $SW = 2$ the working set fits.

Tip L1 needed $\approx SS_W$ times the number of warps that may issue: N for round-robin, SW under a limit. To fit a cache of size C, limit to $SW \leq C/SS_W$.

Definition 7.23 (Cache-conscious wavefront scheduling). The scheduler of Rogers et al., called *cache-conscious wavefront scheduling (CCWS)* [40], limits the number of warps that may issue so that their working set fits in the L1: on a stall it chooses among a small set of warps (W1–W3, say) and starts another warp (W4) only when those have completed.

Caution Lecture vs paper: the fixed limit described here is the paper's baseline "static wavefront limiting". CCWS itself is adaptive: a detector notices when evictions destroy a warp's locality, and the scheduler then gives that warp more exclusive access to the L1D by throttling the others.

The paper's mechanism is the *lost intra-wavefront locality detector (LLD)*, which keeps, for every warp, the tags of the lines that warp brought into the L1 and lost to eviction (a victim tag array without data). An L1 miss that hits in the warp's own victim tags is lost intra-warp locality; it raises the warp's lost-locality score, and warps pushed past a cutoff in the score stack (in the paper, above it) by the high scores of others may not issue loads to the L1D until the scores decay. The reported results are 25 % fewer L1 data cache misses on average and a harmonic-mean speed-up of 24 % over earlier scheduling policies.

The CCWS hardware adds 0.17 % to the total chip area; against a simple greedy scheduler the harmonic-mean gain on the highly cache-sensitive benchmarks is 63 % [40].

Limiting the warps raises two problems.

- *Synchronisation.* A barrier (Lesson 3) releases the warps of a block only when all of them have arrived. If the scheduler holds back a warp of a block whose other warps wait at `__syncthreads()`, no warp of that block can progress: a deadlock. The limit must therefore never keep a warp out that others are waiting for.
- *The size of the limit.* More warps give more thread-level parallelism to hide latency; fewer warps give a smaller working set. The best limit depends on the cache and the program, which is why CCWS adapts it at run time.

In a kernel of the following shape every warp of the block must reach the barrier before any warp starts phase 2:

```
__global__ void twoPhase(float *a) {
  // phase 1: every warp of the block works
  __syncthreads();   // waits for ALL warps
  // phase 2
}
```

7.3.5 Other scheduling policies

GPUMech (Lesson 8) models the performance of both RR and GTO analytically, with average errors of 13.2 % and 14.0 % [43].

The scheduler can also serve the memory system beyond the L1. Like the memory channels of Lesson 4, the banks inside each DRAM can serve requests in parallel; the number of banks busy at the same time is its *bank-level parallelism*.

- *Prefetch-aware scheduling* [41] builds on the two-level scheduler but forms its groups from *non-consecutive* warps. Consecutive warps access neighbouring data, so a simple

prefetcher triggered by one warp can fetch the data of the next; scheduled apart, the next warp runs late enough for the prefetch to arrive in time, and the requests in flight at once spread over more DRAM banks.

- *CTA-aware scheduling* (OWL [42]) decides which CTAs (thread blocks, Lesson 3) are scheduled together, grouping the warps of a few CTAs so that their data stays in the cache, and combines this with locality- and bank-level-parallelism-aware warp selection.

Jog et al. report 25 % and 7 % average gains over round-robin and two-level scheduling, each with prefetching, on a simulated 30-core GPU [41].

OWL: a 33 % average gain over round-robin warp scheduling on a simulated 28-core GPU; in its motivation, 19 of 38 applications spend on average 62 % of their cycles with no warp executing [42].

Key point — Lesson summary

GPU virtual memory translates warps, not threads: coalescing decides the number of translations, uncoalesced loads can need one per thread, and shared high-bandwidth page table walkers with page walk caches, multi-level TLBs, segment mapping and large pages keep the cost down; large pages trade TLB reach for internal fragmentation. A shared virtual address space replaces copy-then-execute by on-demand paging, whose far faults cost tens of microseconds over PCIe; tree-based neighbourhood prefetching amortises them, and eviction under oversubscription must respect the prefetcher's units. The warp scheduler issues in order within a warp but chooses among ready warps: round-robin advances all warps together and bunches their misses, GTO staggers them; two-level schedulers search a small active set for energy and performance; and limiting the schedulable warps (CCWS) shrinks the L1 working set, provided barriers are respected.

Lesson 8

GPU Simulation

Lecture	M8, lessons 1–4 (GPU Cycle Level Simulation, Parts 1 and 2; Analytical Models of GPUs; Accelerating GPU Simulation)
Sources	Liu et al. [44] (Photon, assigned); Williams et al. [45] (Roofline, assigned); Hong and Kim [46] (MWP/CWP model); Huang et al. [43] (GPUMech); Karkhanis and Smith [47] (first-order model); simulators [48, 49, 50, 51, 52]; Sherwood et al. [53] (SimPoint); NVIDIA GEMM guide [54]
Textbook	Eeckhout [55] (optional); K12 [3] ch. 2 Performance Principles, §4.1.1 Execution Time Modeling, §4.2.4 Roofline Model, §4.2.6 Other GPGPU Performance Modeling Techniques; BPG [8] §10.2.1 Coalesced Access to Global Memory
Keywords	performance modeling, cycle-level simulation, execution-driven and trace-driven simulation, queue-based model, sectored cache, analytical model, Roofline model, arithmetic intensity, ridge point, CPI stack, interval analysis, MWP and CWP, sub-core, event-driven simulation, sampled simulation, SimPoint, data-driven model

The mechanisms of the previous lessons (the SIMT stack and dynamic warp formation of Lesson 6, register file caches, the warp schedulers of Lesson 7) were all proposed and evaluated before any of them existed in silicon. This lesson is about the tools that make such evaluations possible. It first builds a cycle-level simulator of a GPU from queues and a clock, and then adds what is specific to GPUs: warps, divergence, coalescing and sectored caches. It then turns to analytical models, which predict performance from a few parameters without simulating anything: a first-order design of a GPU, the Roofline model, CPI stacks and interval analysis. The last part asks why detailed simulation is so slow and what can be done about it: parallel and event-driven simulators, simpler models, sampling, smaller workloads and models learned from data.

8.1 Cycle-level simulation

Source M8, cycle-level simulation (part 1); Eeckhout [55].

Before a processor is built, its designers must predict how fast it will run real programs and how each design choice changes that speed. This section lists the techniques for doing so, defines cycle-level simulation and its two flavours, and shows how a simulator of a pipeline is organised as queues advanced by a clock.

8.1.1 Six techniques

Definition 8.1 (Performance modeling). The prediction of the performance of a processor design, or of a program on a design, without measuring it on the finished hardware, is *performance modeling*. A program that imitates the behaviour of the hardware in enough detail to produce such a prediction is a *simulator*.

Six techniques are in common use:

1. *cycle-level simulation*: model what every part of the machine does in every cycle (this section and the next);
2. *event-driven simulation*: model only the moments at which something changes (Section 8.4);
3. *analytical models*: equations in a few parameters (Section 8.3);
4. *sampling-based techniques*: simulate in detail only selected parts of a program (Section 8.4);
5. *data-driven, statistical or machine-learning models*: fit a model to measured or simulated data (Section 8.4);
6. *FPGA emulation*: implement the design, or a model of it, on a field-programmable gate array, which runs it orders of magnitude faster than a software simulator at the cost of much more design effort.

FPGA emulation is as detailed as cycle-level simulation, only faster: FPGA-based simulators are "fast and accurate, but they take significant effort to develop and use" [56]. This chapter does not treat them further.

8.1.2 The cycle-level approach

Definition 8.2 (Cycle-level simulation). In *cycle-level simulation* the simulator keeps a global clock and, in every simulated cycle, models the events of that cycle in each unit: which

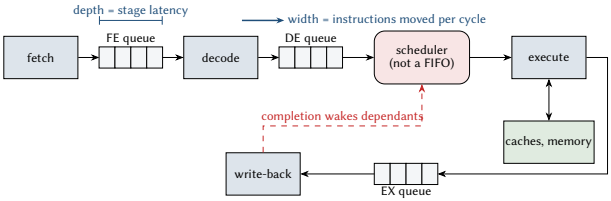


Figure 8.1: A queue-based model of a pipeline. Fetch, decode, schedule, execute and write-back are connected by FIFO queues; the scheduler is a pool from which ready instructions are picked. The execute stage obtains memory latencies from a model of the caches and memory, and a completing instruction wakes up its dependants.

instruction is fetched, which is decoded, which leaves the scheduler, which request reaches the cache.

Cycle-level simulators are the standard tool of architecture research and are used especially early in a design, when there is no hardware to measure. A single clock is a simplification: a real chip runs its cores, its memory and its on-chip network at different frequencies. A part of a design driven by one clock is a *clock domain*, and a simulator can keep one clock per domain (core, memory, network on chip) and advance each at its own rate.

On NVIDIA GPUs the SM clock and the memory clock are independent: `nvidia-smi -q -d CLOCK` reports "Graphics", "SM" and "Memory" clocks separately.

8.1.3 Execution-driven and trace-driven simulation

A cycle-level simulator models timing. It must also know *which* instructions run, with which addresses and which branch outcomes, and there are two ways to obtain them.

Definition 8.3 (Execution-driven and trace-driven simulation). In *execution-driven simulation* the simulator executes the program's instructions itself, computing their results as the simulation proceeds. In *trace-driven simulation* it replays a *trace*, a record of the instructions (and their addresses and branch outcomes) collected beforehand from a real or functional execution, so that execution and timing are decoupled.

An execution-driven simulator still chooses *when* to execute an instruction functionally. It can do it when the instruction is fetched (*execute-at-fetch*), which keeps the functional model simple, or when the timing model reaches the execute stage (*execute-at-execute*), which lets values depend on the simulated order of events. The two flavours trade off as follows.

Execution-driven. No traces have to be stored, and behaviour that depends on the run itself is modeled faithfully: which thread acquires a lock first, how long a thread waits at a barrier, which path a data-dependent branch takes under a different schedule.

Trace-driven. The simulator needs no functional model, so it is simpler, lighter and easier to develop. A study of caches alone needs only the memory addresses (a memory trace). But the trace fixes the order of events as it was recorded: lock acquisition, barriers and anything else that depends on run-time timing cannot change in the simulated machine.

Caution The two choices usually give the same result. They differ when the simulated timing changes the value, for example when two threads race for a lock: execute-at-fetch fixes the winner in fetch order, execute-at-execute in the simulated execution order.

Tip Rule of thumb: if the question is "how fast is this design", a trace suffices; if the design can change *what* the program does (its synchronisation or the paths its threads take), the simulation must be execution-driven.

8.1.4 Queue-based modeling

The pipeline of Lesson 4 is modeled most easily as stages connected by queues (Figure 8.1).

Definition 8.4 (Queue-based model). A *queue-based model* represents a pipeline as a set of stages connected by queues. Each queue is a *FIFO* (first in, first out): instructions leave in the order in which they entered. The exception is the scheduler, which picks any *ready* instruction, so it is a pool rather than a FIFO. Two parameters describe each connection: the *depth* of a queue is the number of cycles an instruction spends in that stage, and the *pipeline width* is the number of instructions that can move from one queue to the next per cycle (the fetch, issue or execute bandwidth).

Three things give the model its timing. First, dependences: when an instruction completes, its result must be broadcast to the instructions waiting for it, which then become ready, exactly as the scoreboard of Lesson 4 does in hardware. Second, the latency of memory instructions comes from a model of the caches and memory, which decides hit or miss at each level. Third, the latency of other instructions is a parameter, for example 1 cycle for an integer add and

Table 8.1: Completion times computed by a simulator for four instructions (our own latencies). The load's latency is known only after the cache model has decided where it hits.

instruction	schedule	latency	done	latency from
`mul r1, r2, r3`	10	3	13	table
`ld r4, [r5]`	11	30	41	cache model: L1 miss, L2 hit
`mul r6, r1, r1`	13	3	16	table; waits for `r1`
`add r7, r4, 1`	41	1	42	table; waits for `r4`

3 cycles for a multiply, taken from the literature, from a circuit model such as CACTI, or from the register-transfer-level (RTL) description of the design; the simulator itself simulates no circuits. Execution latencies that do not depend on the run are therefore a lookup table.

CACTI is a widely used model of the access time, area and energy of SRAM arrays such as caches and register files, from their size and organisation.

8.1.5 Structure of a simulator

A cycle-level simulator is a loop that advances the clock by one cycle per iteration and asks every component to do one cycle's work. The components form a hierarchy: the simulator ticks the cores, the memory system and the network on chip, and each core ticks its pipeline stages.

MacSim, a CPU and GPU simulator from Georgia Tech [51], is organised this way; the listing is our own sketch, not its code.

```
// Top level: one iteration = one simulated cycle.
while (!sim.all_done()) {
  ++sim.cycle;
  for (Core &c : sim.cores) c.tick();
  sim.memory.tick();
  sim.noc.tick();
}

// One core: stages in REVERSE pipeline order.
void Core::tick() {
  writeback(); execute(); schedule();
  decode(); fetch();
}
```

The stages are called from the last to the first. When `execute()` runs, it consumes the instructions that `schedule()` produced in the *previous* cycle, before `schedule()` overwrites them with this cycle's output. Calling them in pipeline order would let an instruction pass through all five stages in a single cycle.

Tip Reverse order is the software equivalent of the pipeline registers between stages: each stage reads what the stage before it wrote one cycle earlier.

Each simulated instruction is represented by a small record (an *op*) that carries it through the stages, together with its timing state. The key field is the cycle at which it completes:

$$done_cycle = schedule_cycle + latency. \tag{8.1}$$

For an instruction of fixed latency the scheduler can compute it at the moment of issue. The latency of a load is not fixed, since it depends on whether the access hits in the L1 cache, the L2 cache or neither; it becomes known only after the access has been simulated in the cache hierarchy, and is then added to the issue cycle.

Example 8.5. With latencies of 1 cycle for `add`, 3 for `mul`, and 4, 30 or 200 cycles for a load that hits in L1, hits in L2 or misses in both (our own numbers), a simulator schedules four instructions as in Table 8.1. The `add` depends on the load, so it becomes ready only at the load's *done_cycle*.

The scheduler is the most detailed part of the model. Each cycle it selects instructions that are ready, subject to the resources they need: a free floating-point unit, an entry in the load or store queue, a free execution or cache port. An instruction that occupies a unit for several cycles keeps the resource for all of them, and the scheduler must not issue a second instruction to it in the meantime.

8.1.6 Modeling a GPU

A cycle-level GPU simulator has the same structure, with one change of unit: it models *warps*, not threads, because the SIMT hardware fetches, decodes, schedules and executes one warp instruction for all its threads together (Lesson 4). The scheduler considers the instruction at the head of each warp; instructions of one warp issue in order, those of different warps in any

order. Three things have no CPU counterpart: divergent warps (Section 8.2), the hierarchy of warps, thread blocks and kernels, and the warp scheduling policies of Lesson 7.

The hierarchy also decides when a simulation ends. A whole thread block is assigned to one SM (Lesson 3), and the simulator tracks every thread until it executes its last instruction. A block finishes when all of its threads have finished, which frees its SM resources for the next block; a kernel finishes when all of its blocks have finished; and the application finishes when all of its kernels have.

A GPU simulator has far more to model per cycle than a CPU simulator: every SM can hold up to 64 resident warps (CPG Table 21 [5]), and a GPU has tens of SMs.
How many blocks an SM holds at once is itself a modeled limit: at most 32 resident blocks per SM on compute capabilities 7.0, 8.0 and 9.0, 16 on 7.5 and 8.6 (CPG Table 21 [5]).

> **Key point**
> A cycle-level simulator advances a global clock (one per clock domain) and models every unit in every cycle. It obtains the instruction stream by executing the program or by replaying a trace. The pipeline is a set of FIFO queues whose depths are stage latencies and whose transfer rates are the pipeline widths, plus a scheduler that picks ready instructions under resource constraints. A GPU simulator models warps instead of threads.

8.2 Divergence and memory in GPU simulators

The modeling unit of a GPU simulator is the warp: an instruction-cache access and a program counter belong to a warp, not to a thread. Not every thread of a warp takes part in every instruction, so the simulator also keeps the active mask of Lesson 4, both to compute results and to count the resources an instruction really uses. This section shows how a simulator handles the two GPU-specific effects of the previous lessons, divergence and coalescing, and then lists the open-source GPU simulators.

Source M8, cycle-level simulation (part 2); BPG §10.2.1 Coalesced Access to Global Memory [8].

8.2.1 Divergent branches

How a simulator handles a divergent branch (Lesson 6) depends on how it obtains its instructions.

Execution-driven. The simulator computes each thread's branch outcome and maintains the SIMT stack itself, exactly as the hardware would. Because the stack is part of the model, a different mechanism (dynamic warp formation, another reconvergence rule) can be modeled by replacing it.

Trace-driven. The trace must record every path the warp executed, with the active mask of each basic block, for example BB1 `1111`, BB2 `1100`, BB3 `0011` and BB4 `1111` for the branch of Lesson 6. The simulator replays these blocks in the recorded order.

A trace-driven simulator is therefore poorly suited to studying divergence handling itself; it is adequate when divergence is only one of the effects that shape the timing.

Caution A trace records the divergence behaviour of the machine on which it was collected. Simulating a different divergence-handling mechanism from such a trace is difficult, because the trace contains neither the branch conditions nor the order the new mechanism would choose.

8.2.2 Coalescing

The number of memory transactions a warp instruction generates, not the number of threads, drives everything downstream: cache accesses, misses, DRAM traffic and bandwidth. Modeling memory coalescing (Lesson 4) is therefore one of the most important parts of a GPU simulator. Requests are merged at the granularity of the first cache they reach, typically one cache line. In the toy configuration of this section, with 64 B lines and eight threads per warp, a load `R1` whose threads read consecutive 4 B words is merged into one 64 B request, while a load `R2` whose threads read words 128 B apart produces eight requests, one per thread, each for a full line.

A trace can represent the memory accesses of a warp instruction in two ways. It can list the address of every thread (`0x00`, `0x04`, `0x08`, ...), leaving the coalescing to the simulator, or it can store the result of coalescing, one base address and a size per transaction. The second form is much smaller, but it fixes the transaction size of the machine on which the trace was collected.

Caution A pre-coalesced trace is tied to one microarchitecture: to study a cache with another line or transaction size, new traces must be collected.

8.2.3 Cache hierarchy and sectored caches

After coalescing, the requests enter a cache hierarchy that is modeled much as for a CPU: the address is translated by a TLB (with a page table walk on a miss, Lesson 7), then the L1 and L2 caches are accessed, depending on the microarchitecture being modeled. What differs is the importance of coalescing in front of it, and one feature of the caches themselves.

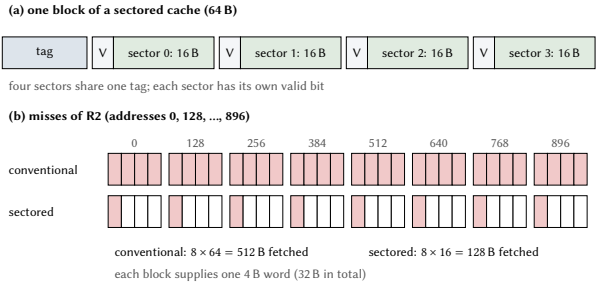


Figure 8.2: A sectored cache (toy sizes: 64 B block, 16 B sectors). (a) The sectors share one tag. (b) The uncoalesced load R2 misses in eight blocks; a conventional cache fills eight whole blocks, a sectored cache only one sector in each.

Table 8.2: Open-source GPU simulators, as classified in this chapter.

simulator	ISA	type	scope
GPGPU-Sim [48]	NVIDIA PTX, SASS	execution-driven	a GPU running general-purpose code
Accel-Sim [49]	NVIDIA PTX, SASS	trace-driven	GPUs and accelerators
MGPUSim [50]	AMD GPU	execution-driven	multiple GPUs
MacSim [51]	NVIDIA and Intel GPUs	trace-driven	heterogeneous CPU and GPU systems
gem5-gpu [52]	AMD GPU or NVIDIA PTX	execution-driven	heterogeneous CPU and GPU systems

Definition 8.6 (Sectored cache). A *sectored cache* divides each cache block into several sectors, each with its own valid bit, all sharing the block's single tag. A *sector* is the unit that is filled from memory on a miss: the cache brings in only the sector that holds the requested data rather than the whole block (Figure 8.2).

For the coalesced load R1 (eight consecutive 4 B words, 32 B at addresses 0 to 31) the conventional cache fills the whole 64 B block, while the sectored cache fills only sectors 0 and 1, the 32 B that hold the data. The saving is a factor of two, and every fetched byte is used. For the uncoalesced load R2 the conventional cache fetches $8 \times 64 = 512$ B to deliver $8 \times 4 = 32$ B of useful data, the sectored cache $8 \times 16 = 128$ B. The benefit of sectoring is therefore lower memory bandwidth for scattered accesses, and sharing one tag among several sectors keeps the tag storage of a large block. The cost is less implicit prefetching: a block that is filled sector by sector no longer brings in the neighbouring data that spatial locality would have used soon after.

Caution Lecture vs guide: the 16 B sector in a 64 B block is a toy size. NVIDIA GPUs use 32 B sectors in 128 B lines, and since compute capability 6.0 a warp's accesses cost "the number of 32-byte transactions necessary to service all of the threads of the warp" (BPG §10.2.1 [8]); Lesson 4 counts these sectors.

Note 8.7. Simulating a sectored cache changes what the simulator counts. A block can be present with only some sectors valid, so an access can hit the tag and still miss the sector; a simulator that counts only tag hits overestimates the hit rate and underestimates DRAM traffic.

8.2.4 Open-source GPU simulators

Table 8.2 lists the open-source GPU simulators that are widely used in research. They differ in the ISA they model, in how they obtain instructions, and in what they model beyond one GPU.

GPGPU-Sim is the most widely used of them. gem5-gpu combines it with gem5, the most widely used CPU simulator, so that a CPU and a GPU execute together and share a memory system. MGPUSim models AMD GPUs and is the base on which the Photon sampling methodology of Section 8.4 was built.[1]

[1] GPGPU-Sim was introduced with a study of CUDA workloads at ISPASS 2009 [48]. The table follows the classification of this chapter's source. In detail, Accel-Sim keeps GPGPU-Sim's execution-driven PTX mode next to its trace-driven SASS mode [49], and gem5-gpu, the combination also listed as "Gem5-GPGPU-Sim", couples gem5 with GPGPU-Sim's NVIDIA model; the AMD GPU model of gem5 is a separate development. MacSim's documentation describes it as configurable as "either a trace driven or execution-driven cycle level simulator" for x86, ARM64, PTX and Intel GEN instructions [51].

Key point

A GPU simulator models warps with their active masks. It either maintains the SIMT stack itself (execution-driven) or replays the recorded blocks and masks (trace-driven, tied to the trace machine's divergence handling). Coalescing decides the number of transactions and is among the most important things to model; pre-coalesced traces fix the transaction size. Sectored caches fill one sector per miss: less bandwidth, less implicit prefetching.

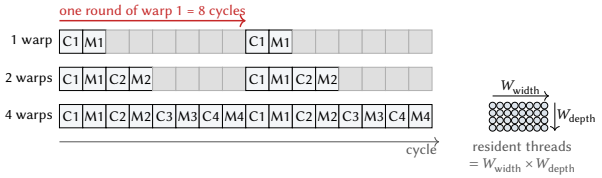


Figure 8.3: Hiding a memory stall. Each warp issues a compute (C) and a memory (M) instruction and can issue again only 8 cycles after its compute instruction. One warp keeps the issue slot busy 2 cycles in 8, two warps 4 in 8, four warps all the time: $W_{depth} = 4$.

8.3 Analytical models of GPUs

A cycle-level simulator answers a question about one design and one program by running the program. An analytical model answers it with a formula.

Source M8, analytical models; K12 ch. 2 Performance Principles, §4.1.1 Execution Time Modeling, §4.2.4 Roofline Model, §4.2.6 Other GPGPU Performance Modeling Techniques; [45, 46, 43, 47].

Definition 8.8 (Analytical model). An *analytical model* predicts performance from a small number of parameters of the design and the program, by equations rather than by executing the program. A *first-order model* keeps only the effects that dominate performance and ignores second-order interactions.

Analytical models need no full execution, are simple enough to reason about, and give insight: they say *why* a design is fast or slow and which parameter to change. Their accuracy is lower than that of a simulator, so they are used to explore a design space before simulating its best points, and to explain simulation results.

8.3.1 A first-order GPU design

We size a GPU for one target: 10^{12} dot-product steps per second, each step being `sum += x[i] * y[i]`.

Definition 8.9 (Fused multiply-add). A *fused multiply-add (FMA)* computes $a \cdot b + c$ as one instruction with a single rounding. It is counted as two floating-point operations (FLOPs).

Compute. Each step is one multiply and one add, so the target is 2 TFLOP/s, or 10^{12} FMAs per second. At 1 GHz this needs 1000 FMA units, at 2 GHz 500.

Memory. Each step reads one element of x and one of y. Counting one byte per element, as this design exercise does, the GPU needs 2 TB/s of memory bandwidth.

Warps. 500 FMA units correspond to about 16 warps of width 32 ($16 \times 32 = 512$) issuing one FMA per lane per cycle. The GPU can be built as 16 SMs that each issue one warp per cycle, or as 8 SMs that each issue two, both at 2 GHz.

Caution Lecture vs reality: the "two bytes per step" assume one-byte elements. With FP32 elements a step reads 8 B for 2 FLOPs, and the same design needs 8 TB/s, about 2.7 times the 3 TB/s of an H100 SXM5 (preliminary figure, [2]).

8.3.2 Width and depth of multithreading

How many warps an SM must hold follows from two parameters.

Definition 8.10 (W_{width} and W_{depth}). W_{width} is the number of threads that execute in one cycle (for example 32, one warp). W_{depth} is the number of warps that the SM can schedule while one warp stalls, so that the stall is hidden. An SM holds $W_{width} \times W_{depth}$ resident threads.

Figure 8.3 shows the idea for a program whose warps alternate one compute and one memory instruction. With one warp, the SM issues two instructions and then idles until the memory instruction completes; with two warps, the second warp fills two of the idle slots; with four, the idle slots disappear. In general the stall must be covered by the work of the other warps: if each warp issues n instructions and then cannot issue again until L cycles after the first of them, about

$$W_{depth} \approx \frac{L}{n} \tag{8.2}$$

warps are needed. With a memory latency of 200 cycles and one compute plus one memory instruction per warp, $W_{depth} = 200/(1+1) = 100$; with one memory instruction per four compute instructions, $200/(1+4) = 40$. The more compute per memory access, the fewer warps are needed to hide the latency.

Tip Count the cycles a warp keeps the issue slot busy between two waits, and divide the wait by it.

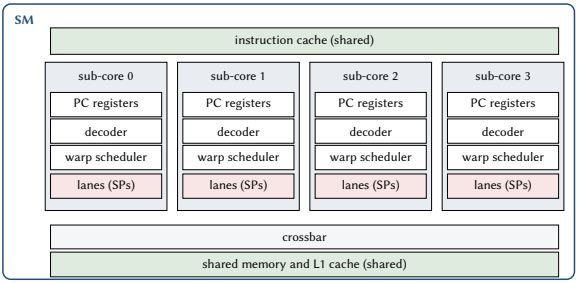


Figure 8.4: An SM divided into four sub-cores. Each has its own front end and lanes; the instruction cache and the data caches are shared.

Both parameters are set by hardware. W_{width} is bounded by the number of ALUs (and the width of the scheduler). W_{depth} is bounded by the storage for the warps' state: a W_{depth} of 20 needs $20 \times 32 \times r$ registers for r registers per thread, and at least 20 program counters, one per warp. Equation (8.2) with a latency of hundreds of cycles explains why GPUs keep dozens of warps per SM and why their register files are so large (Lesson 6).

Real limits: at most 64 resident warps per SM (compute capability 7.0, 8.0, 9.0) and 65 536 32-bit registers per SM, so 64 warps of 32 threads get at most 32 registers each (CPG Table 21 [5]).

8.3.3 How many SMs, and sub-cores

The two designs above, 16 SMs issuing one warp per cycle and 8 SMs issuing two, have the same execution bandwidth and, for the same chip area, the same total cache capacity. They differ in how that capacity is split. Fewer, larger SMs have larger caches with longer access times, but the thread blocks on one SM share more data, which can raise the hit rate; more, smaller SMs have smaller, faster caches. Which is better depends on how much data the blocks share, a trade-off that a first-order model cannot settle; it needs a finer model or a simulation.

Definition 8.11 (Sub-core). A *sub-core* is one of several partitions of an SM, each with its own program counters, decoder, warp scheduler and execution lanes, which share the SM's instruction cache and, through a crossbar, its large caches and shared memory (Figure 8.4).

Sub-cores are a middle way between the two designs: each schedules a small number of warps over its own lanes, while large structures stay shared. For a model this adds resource management: two warps on different sub-cores compete for the shared caches and crossbar but not for an issue slot.

Current NVIDIA SMs have 128 FP32 lanes split into four partitions [2], so each sub-core issues for 32 lanes.

> **Key point**
> A first-order design starts from the target throughput: FLOPs fix the number of FMA units (at a given clock), bytes per FLOP fix the memory bandwidth, and the latency to hide fixes $W_{\text{depth}} \approx L/n$, hence the registers and program counters per SM.

8.3.4 The Roofline model

The first-order design asked how much compute and how much bandwidth a program needs. The Roofline model asks the reverse question: given a machine, which of the two limits a program.

Definition 8.12 (Arithmetic intensity). The *arithmetic intensity* of a program or kernel is the number of floating-point operations it performs per byte of memory it accesses, in FLOP/B.

Definition 8.13 (Roofline model). The *Roofline model* bounds the attainable performance of a kernel with arithmetic intensity I on a machine with peak compute performance P_{peak} and peak memory bandwidth B_{peak} by

$$P(I) = \min\bigl(P_{\text{peak}},\ B_{\text{peak}} \cdot I\bigr). \tag{8.3}$$

On log-log axes the bound is a slanted line (slope one) followed by a flat roof (Figure 8.5). A kernel left of the *ridge point*, the intensity $I^* = P_{\text{peak}}/B_{\text{peak}}$ at which the two lines meet, is *memory-bound*: its performance is limited by bandwidth. A kernel right of it is *compute-bound*: limited by the peak FLOP rate.

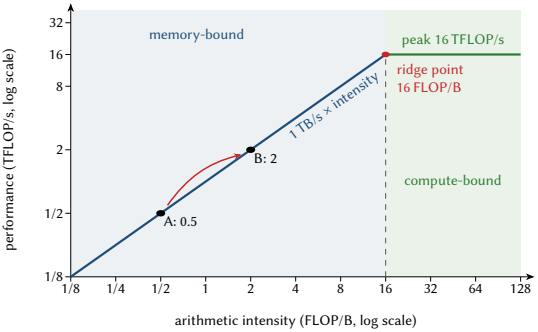


Figure 8.5: The Roofline model of a machine with 16 TFLOP/s and 1 TB/s (ridge point 16 FLOP/B). Kernel A (0.5 FLOP/B) reaches at most 0.5 TFLOP/s; raising the intensity to 2 FLOP/B (kernel B) quadruples the bound.

Williams et al., who introduced the model, measure the intensity after the caches: their *operational intensity* counts the operations per byte of DRAM traffic, so a kernel whose data stay in the caches has a higher operational than arithmetic intensity [45].

Two examples in the units of the first-order design, one byte per element, show how a kernel moves on the chart. A vector sum `z[i] = x[i] + y[i]`, counting only its two loads (one byte each), performs 1 FLOP per 2 bytes: its intensity is 0.5 FLOP/B, and the kernel is memory-bound. The variant `sum += x[i]*x[i]*y[i]*y[i]` of the dot product reads the same 2 bytes but performs 4 FLOPs (three multiplications and one addition), an intensity of 2 FLOP/B: it moves right on the chart, toward the compute-bound region. The model's practical use is to judge whether a kernel has room to improve: a kernel far below its roof is limited by something else (latency, divergence, bank conflicts), one on the roof can only be improved by raising its intensity or by moving to another machine. The roofs are drawn once per machine, "not once per kernel" [45]; each kernel is then a point on the intensity axis.

Caution Lecture vs paper: this chapter, like the lecture, counts the bytes the program requests. The paper uses "the term operational intensity to mean operations per byte of DRAM traffic" and states the bound as "Attainable GFlops/sec = Min(Peak Floating Point Performance, Peak Memory Bandwidth x Operational Intensity)" [45].

Example 8.14. Our machine of Figure 8.5 has $P_{\text{peak}} = 16$ TFLOP/s and $B_{\text{peak}} = 1$ TB/s, so $I^* = 16/1 = 16$ FLOP/B. An FP32 dot product reads 8 B per FMA (2 FLOPs), $I = 2/8 = 0.25$ FLOP/B, and attains at most $1 \times 0.25 = 0.25$ TFLOP/s, about 1.6 % of the peak; the four-FLOP variant above reaches 0.5 FLOP/B in FP32. A kernel with $I = 32$ FLOP/B, such as a large tiled matrix multiplication, attains $\min(16, 32) = 16$ TFLOP/s, the full peak.

Ridge points of real GPUs (FP16 tensor peak divided by HBM bandwidth): V100 138.9 FLOP/B [54]; A100 $312/1.555 \approx 201$; H100 SXM5 $1000/3 \approx 333$ (preliminary figures [1, 2]).

8.3.5 CPI models

A second family of analytical models starts from the time per instruction.

Definition 8.15 (CPI). The *cycles per instruction (CPI)* of a program on a processor is the number of cycles it takes divided by the number of instructions it executes, the reciprocal of the IPC of Lesson 1.

The simplest CPI model adds to the CPI of an ideal pipeline the cost of every kind of disruptive event, weighted by how often it occurs:

$$\text{CPI} = \text{CPI}_{\text{steady}} + \sum_i f_i \cdot p_i, \tag{8.4}$$

where f_i is the number of events of kind i per instruction and p_i the penalty of one such event in cycles. The model assumes that penalties do not overlap: they are serialised.

Caution The serialised sum is pessimistic for out-of-order and multi-threaded processors, which overlap penalties with each other and with useful work.

Example 8.16. A five-stage in-order pipeline has $\text{CPI}_{\text{steady}} = 1$. Branch mispredictions cost 3 cycles and occur in 2 % of the instructions, cache misses cost 5 cycles and occur in 5 %:

$$\text{CPI} = 1 + 0.02 \cdot 3 + 0.05 \cdot 5 = 1 + 0.06 + 0.25 = 1.31.$$

The terms of the sum are worth showing separately, because they say where the time goes.

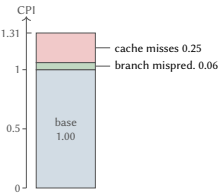


Figure 8.6: CPI stack of Example 8.16.

Definition 8.17 (CPI stack). A *CPI stack* is a stacked bar of the components of the CPI: the base (steady-state) component and one component per kind of event (Figure 8.6).

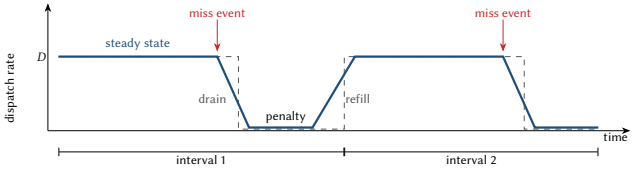


Figure 8.7: Interval analysis. Each interval is a steady state at the effective dispatch rate D followed by one miss event. Dashed: the abrupt stop of the simple model. Solid: the first-order model with the pipeline drain before and the refill after the penalty.

Tip IPC is the reciprocal of CPI: a CPI of 1.31 is an IPC of $1/1.31 \approx 0.76$.

In the example, cache misses account for $0.25/1.31 \approx 19\,\%$ of the time and mispredictions for under 5 %: a better cache helps more than a better predictor.

A multithreaded processor overlaps the stalls of one thread with the work of others. Ideally, with W_{depth} threads, the stalls are divided among them:

$$\text{CPI}_{\text{MT}} \approx \frac{\text{CPI}_{\text{single}}}{W_{\text{depth}}} + \text{contention terms.} \tag{8.5}$$

The contention terms account for shared resources that run out when many threads are active: busy execution units, DRAM bandwidth, and the registers that track outstanding misses.

Definition 8.18 (MSHR). An *MSHR (miss status holding register)* is an entry that records one outstanding cache miss (the missing block and the requests waiting for it), so that the cache can keep serving other accesses while the miss is serviced. The number of MSHRs bounds the number of misses in flight.

The MSHRs therefore cap the memory-level parallelism of Lesson 4: with, for example, 32 MSHRs (an illustrative number), a 33rd outstanding miss stalls until one returns, however many warps are ready.

Note 8.19. Read (8.5) as cycles per instruction of the whole processor, counting the instructions of all threads. The ideal term cannot fall below the issue limit (1 for a pipeline that issues one instruction per cycle): with the CPI of 1.31 of Example 8.16, two threads already reach that limit. A GPU is a multithreaded processor with many warps, so the CPI models apply to it, but the contention terms, especially MSHRs and DRAM bandwidth, dominate.

8.3.6 Interval analysis

The CPI model treats every penalty as a fixed number of cycles added to an otherwise perfect pipeline. Interval analysis looks at the shape of the execution over time.

Definition 8.20 (Interval analysis). The method of *interval analysis* divides an execution into intervals. Each interval consists of a steady state, in which instructions are dispatched at the *effective dispatch rate*, followed by one *miss event*, a long-latency event (a branch misprediction, a cache or TLB miss) during whose penalty no instruction is dispatched (Figure 8.7). The execution time is the sum of the steady-state times and the penalties.

A pipeline does not stop and restart abruptly. When a miss event occurs, instructions already in flight still drain out of the pipeline; after it is resolved, the pipeline needs time to fill again before dispatch reaches the steady rate. Karkhanis and Smith's first-order superscalar model adds these *drain* and *refill* phases, whose shapes depend on the kind of event: a misprediction is detected only when the branch executes, while a long cache miss blocks the head of the reorder buffer and stalls dispatch once it fills [47].

Eeckhout [55] presents interval analysis and the first-order model in detail, including the penalties of each kind of miss event.

Applied to a GPU, the naive approach treats the GPU as one more multithreaded processor. That misses the two effects that most distinguish GPUs:

Branch divergence. Not every lane of a warp is active, and the paths of a divergent branch are serialised (Lesson 6), so a warp instruction does not represent a fixed amount of work.

Memory divergence. The latency of a warp's memory instruction depends on whether its accesses coalesce: an uncoalesced access generates many transactions and waits for the last of them (Lesson 4).

GPUMech applies interval analysis to GPUs, models the round-robin and greedy-then-oldest schedulers of Lesson 7 with 13.2 % and 14.0 % error at 97 times the speed of simulation, and produces CPI stacks [43].

More recent GPU interval models add sub-cores, sectored caches and the contention for other shared resources.

Note 8.21 (A GPU model of memory and computation parallelism). Hong and Kim's analytical model of a GPU, one of this unit's optional readings, rests on two quantities [46]. *memory*

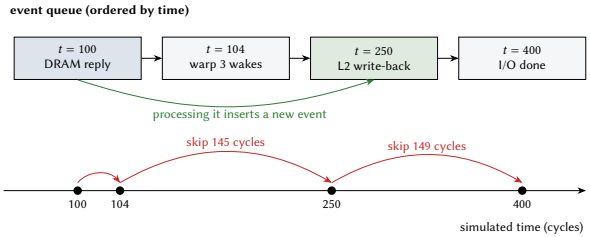


Figure 8.8: Event-driven simulation. The event queue is ordered by time stamp; processing an event can insert a new one. Simulated time jumps from event to event and skips the cycles in between.

warp parallelism (MWP) is the maximum number of warps per SM that can access memory at the same time, during one warp's memory waiting period; it is limited by the latency, the departure delay between consecutive requests, the bandwidth and the number of active warps N:

$$\mathrm{MWP} = \min(\mathrm{MWP}_{\text{without BW}}, \mathrm{MWP}_{\text{peak BW}}, N),$$

where $\mathrm{MWP}_{\text{without BW}} = \mathit{Mem_L}/\mathit{Departure_delay}$ and $\mathrm{MWP}_{\text{peak BW}}$ is the peak bandwidth divided by the bandwidth one warp needs times the number of active SMs. *computation warp parallelism (CWP)* is the number of warps that the SM can execute during one memory waiting period, plus one: $\mathrm{CWP} = \min\big((\mathit{Mem_cycles} + \mathit{Comp_cycles})/\mathit{Comp_cycles},\ N\big)$. If CWP $\geq$ MWP the computation of the other warps covers more than the memory system can overlap: the kernel is memory-bound and its time grows with N/MWP memory periods. If MWP $>$ CWP the memory latency is hidden and the time is about one memory latency plus $N \times \mathit{Comp_cycles}$.

Hong and Kim report a geometric-mean error of 5.4 % on micro-benchmarks and 13.3 % on applications [46].

Key point
Analytical models trade accuracy for insight. Roofline compares a kernel's arithmetic intensity with the machine's ridge point to decide whether it is compute-bound or memory-bound. CPI models add frequency-weighted penalties to a steady-state CPI, and a CPI stack shows where the time goes; interval analysis refines this with drain and refill, and GPU versions must add branch and memory divergence.

8.4 Accelerating GPU simulation

Detailed simulation is slow. A cycle-level simulator that models every unit in every cycle runs at the order of a thousand simulated instructions per second (1 KIPS). At that speed the 10^9 instructions that a processor executes in about a second take about 28 hours to simulate, and a machine-learning workload that runs for hours on real hardware would take on the order of a century. There are two ways out: make the simulator faster, or give it less to simulate.

Source M8, accelerating simulation; Eeckhout [55] (sampling); [44, 53].

Caution Lecture vs arithmetic: $10^9/10^3 = 10^6$ s, about 11.6 days, not 28 hours; the 28 hours correspond to 10 KIPS (10^5 s). At 1 KIPS one hour of native execution takes about 3600×11.6 days $\approx$ 114 years, which is the century.

Speeds of CPU simulators: gem5, Flexus and MARSS reach about 200 KIPS with detailed models; zsim simulates a detailed out-of-order core at 20 MIPS and a 1024-core chip at up to 300 MIPS on a 16-core host [56].

8.4.1 Faster simulators

Parallel simulation. A simulator can run on several host threads, for example one thread per simulated core. The cores are nearly independent, but the memory system and the network on chip are shared: the threads must communicate, or serialise their accesses to the shared model, at every interaction. Most workloads are memory-bound, so these interactions are frequent and the memory model becomes the bottleneck of the parallel simulator.

Event-driven simulation.

Definition 8.22 (Event-driven simulation). In *event-driven simulation* the simulator keeps a queue of future events, each with a time stamp, ordered by time. It repeatedly removes the earliest event, advances the simulated time directly to its time stamp, and processes it; processing may insert new events (Figure 8.8).

The cycles between events cost nothing, so event-driven simulation pays off when events are sparse, as with long latencies: an I/O operation or a write to non-volatile memory that takes thousands of cycles is one event instead of thousands of ticks.

Tip Cycle-level simulation costs time per cycle, event-driven simulation per event. A busy pipeline has events in nearly every cycle, so its core gains little; long-latency devices gain most.

Simpler models. Not every event matters equally. A simulator can use average latencies or throughputs for non-critical events and keep detail only where the question requires it. To study the memory hierarchy, the core can be reduced to issuing instructions at an IPC equal to its issue width, with only the caches and memory modeled; zsim, a fast simulator of many-core CPUs, offers such a simple core (one instruction per cycle) next to its detailed out-of-order model [56]. To study the core, the memory can be given a fixed latency.

Caution Every simplification removes an interaction. A fixed memory latency, for example, hides the queueing delays that a bandwidth-hungry design creates, exactly the contention terms of (8.5).

The remaining two accelerations, sampling and statistical models, are the subject of the rest of this section.

8.4.2 Sampling

Definition 8.23 (Sampled simulation). In *sampled simulation* only selected parts of a program's execution, the samples, are simulated in detail, and the performance of the whole is estimated from them. The samples may be chosen at random or by analysing the program.

The parts between samples still have to be skipped somehow. An execution-driven simulator fast-forwards through them by executing functionally, without timing, which is much faster, or starts each sample from a *checkpoint*, a saved architectural state (registers and memory) from which the simulation can resume without executing what came before. A trace-driven simulator simply collects traces of the samples only.

A sample does not start from a realistic microarchitectural state: its caches and branch predictors are empty or hold the contents left by the previous sample. The *warm-up* of a sample is a stretch of execution simulated before it, whose timing is discarded, during which these structures are filled. Alternatively the samples are made long enough for the cold start to be negligible.

Random samples need many draws for an accurate estimate. Programs, however, are not random: they repeat the same behaviour.

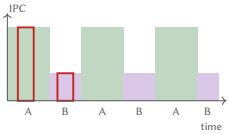


Figure 8.9: IPC over time of a program alternating a high-IPC phase A and a low-IPC phase B. Phase-based sampling simulates one representative of each (outlined).

Definition 8.24 (Program phase and SimPoint). A *program phase* is a stretch of execution with homogeneous behaviour (IPC, miss rates), which may recur many times. *SimPoint* identifies the phases of a program by clustering intervals of its execution by the code they execute, and selects one representative interval per phase, weighted by the fraction of execution the phase accounts for (Figure 8.9) [53]. Each interval is described by a *basic block vector*, how often each basic block executed in it, and the vectors are grouped by k-means clustering.

With a few phases recurring hundreds of times, simulating one interval of each phase and weighting it gives most of the accuracy of a full simulation at a small fraction of its cost.

8.4.3 Sampling on GPUs

A single GPU thread is too short to sample within, but a GPU program consists of very many threads executing the same code (SPMD, Lesson 1), which offers other granularities:
Block level. Simulate about 100 thread blocks instead of the thousands of the full grid.
Kernel level. Simulate one or two kernels instead of tens.
Warp level. Simulate fewer warps of a kernel in detail.

Definition 8.25 (Kernel-level sampling). *kernel-level sampling* simulates in detail one launch of each distinct kernel, or of each group of similar launches, and uses its time for the other launches.

Kernel-level sampling is especially effective for machine-learning workloads, which launch the same kernels hundreds of times, once per layer and iteration. Photon, one of this unit's assigned readings, combines sampling at three levels, kernels, warps and basic blocks, and finds the repetitive behaviour by a lightweight online analysis instead of profiling the workload beforehand [44].

Photon reduces the simulation of one ResNet-152 inference (batch size 1) from 7.05 days to 1.7 hours with a sampling error of 10.7 %; it is built on MGPUSim [44].

8.4.4 Reducing the workload

The other way out is to simulate less work.

- *Fewer iterations*: one training iteration instead of a thousand, since the iterations of a machine-learning training run behave alike.
- *Smaller inputs*: a graph of 10^6 nodes instead of 10^9.
- *Only the hot code*: when one or two of hundreds of functions take 90 % of the time, simulate only those.

Caution A smaller input can change the behaviour, not only shorten it: a working set that fits in the L2 cache at reduced size may not fit at full size, so the reduced run can hide the memory-bound behaviour of the real one.

Definition 8.26 (Microbenchmark and representative workload). A *microbenchmark* is a small program written to exercise one feature of a machine (a cache level, a memory access pattern, an instruction's latency) in isolation. A *representative workload* is a reduced program or input built so that its behaviour matches that of the full workload on the features under study.

8.4.5 Data-driven models

The last approach does not model the machine at all.

Definition 8.27 (Data-driven model). A *data-driven model* predicts performance from features of a program (instruction count, number of loads, number of divergent branches, …) by a statistical or machine-learning model fitted to data, without encoding how the hardware works.

The procedure has three steps: run a set of programs on the hardware (or a detailed simulator) and record their performance, for example their IPC; extract features from hardware performance counters or from the simulator; and fit a model. A simple form of such a model is

$$cycles = C_1 N_{\text{inst}}^{e_1} + C_2 N_{\text{FP}}^{e_2} + C_3 N_{\text{mem}}^{e_3}, \tag{8.6}$$

where N_{inst}, N_{FP} and N_{mem} are the counts of all, floating-point and memory instructions and the coefficients C_k and exponents e_k are fitted to the data.

The hardware counters are the ones that Nsight Compute (Lesson 5) reads: instructions executed per type, memory transactions, branch divergence.

Caution An analytical model is built from an understanding of the machine; a data-driven model only from data. It is accurate for programs like those it was fitted on and gives no guarantee, and little insight, outside them.

Key point — Lesson summary

Cycle-level simulators advance a clock and model every unit every cycle, driven by execution or by traces; the pipeline is a set of FIFO queues with depths and widths plus a scheduler, and a GPU simulator models warps, their masks or SIMT stack, coalescing and sectored caches. Analytical models give first-order answers: the FMA units, bandwidth and $W_{\text{depth}} \approx L/n$ of a design, the Roofline bound $\min(P_{\text{peak}}, B_{\text{peak}} \cdot I)$ with its ridge point, CPI stacks and interval analysis with branch and memory divergence. Simulation at about 1 KIPS is too slow for real workloads; parallel and event-driven simulation, simplified models, sampling (phases, SimPoint, block-, kernel- and warp-level), reduced workloads and data-driven models make it tractable.

Lesson 9

Multi GPU

Lecture	M9, lessons 1–2 (Multi-GPU Hardware; Concurrency on GPUs)
Sources	Arunkumar et al. [57] (required); NVIDIA NVLink page [58], GPUDirect RDMA documentation [59], MPS overview [60], Yu et al. [61], Ye et al. [62] (optional); NVIDIA A100 [1] and H100 [2] whitepapers
Textbook	A18 [4] ch. 5; PMPP [6] ch. 20; CPG [5] §3.2.8 Asynchronous Concurrent Execution, §3.2.9 Multi-Device System
Keywords	multi-GPU, monolithic GPU, multi-chip module, GPU module, NUMA, near and far memory, first-touch page placement, page migration, thread block scheduling, NVLink, NVSwitch, RDMA, GPUDirect RDMA, utilization, multi-tenancy, MPS, MIG, concurrent kernel execution, CUDA Graph

The lessons so far have treated one GPU: its programming model (Lessons 3 and 5), its SMs and memory system (Lessons 4 and 6), and its virtual memory and warp scheduling (Lesson 7). Many workloads, datacenter training of large models above all, need more computing power than one chip provides, and this lesson asks how GPUs are combined. The first part follows the hardware from one die to several dies in a package, several GPUs in a node and several nodes on a network: the memory becomes non-uniform, so where a thread block runs and where its pages live start to matter, and new interconnects (NVLink, NVSwitch, RDMA) carry the traffic that PCIe cannot. The second part asks the opposite question: when one GPU is more than a job needs, how can several jobs share it? Streams, the Multi-Process Service and the Multi-Instance GPU are three answers that differ in how much of the GPU they partition and how much isolation they give. The distributed-memory programming of Lesson 2 returns at the largest scale.

9.1 Multi-GPU hardware

Source M9, multi-GPU hardware; A18 ch. 5; [57].

The demand for computing power has grown faster than a single GPU, and GPUs have answered with more chips. This section follows the steps of that scaling, the memory organisation they produce, the two placement decisions it raises, and the interconnects that join GPUs inside a node and across nodes.

9.1.1 Scaling beyond one chip

The first way to a faster GPU is more cores on one die.

SMs per data-centre GPU: 56 (P100), 80 (V100), 108 (A100), 132 (H100 SXM5, preliminary); transistors from 15.3 to 80 billion over the same four generations (AWP Table 4 [1]; HWP Table 3 [2]).

Definition 9.1 (Monolithic GPU, multi-GPU). A *monolithic GPU* is a GPU built as a single die: all SMs, caches and memory controllers on one piece of silicon. A *multi-GPU* system combines several GPU chips, in one package, on one board or across boards, to act on one workload.

Every GPU generation of Lesson 1 added SMs to a monolithic die, and Figure 9.1 shows where that path ends. A larger die costs more, and its *yield*, the fraction of manufactured dies that work, falls, because every die is more likely to contain a fatal defect; and the lithography sets a hard maximum die size. The next step therefore keeps the dies small and connects several of them, either as separate GPU chips in several sockets or as several dies on a silicon interposer (Lesson 4) inside one package, joined by a high-bandwidth switch. The last step connects whole GPU boards through PCIe.

Data-centre dies are already large: 815 mm^2 (V100), 826 mm^2 (A100) and 814 mm^2 (H100) (AWP Table 4 [1]; HWP Table 3 [2]). Arunkumar et al. assume that a GPU of more than 128 SMs cannot be built as one die [57].

Definition 9.2 (Multi-chip module, GPU module, chiplet). A *multi-chip module (MCM)* is a package that holds several dies connected by short on-package links. In the MCM-GPU of Arunkumar et al. each die is a *GPU module (GPM)*, a smaller GPU with its own SMs, L2 slice and DRAM partition, and the package of GPMs appears to software as one larger logical GPU [57]. A die designed to be combined with others in this way is called a *chiplet*.

The paper calls this "package-level integration of multiple GPU modules to build larger logical GPUs", and argues that it allows "continuous performance scaling beyond Moore's law", because each GPM stays small enough to be manufactured with good yield [57].[1]

[1] The MCM-GPU studied in the paper has four GPMs of 64 SMs each (256 SMs), 3 TB/s of DRAM bandwidth in total, 768 GB/s per local partition. Its optimised version is reported as "45.5 % faster than the largest implementable monolithic GPU", within 10 % of an unbuildable monolithic GPU of the same size, and 26.8 % faster than a multi-GPU system with the same SMs and DRAM bandwidth [57].

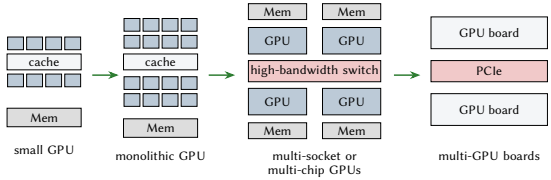


Figure 9.1: Four steps of GPU scaling: a small GPU; a monolithic GPU with more cores on one die; several GPU chips, each with its own memory, joined by a high-bandwidth switch (multi-socket or multi-chip); and several GPU boards joined by PCIe.

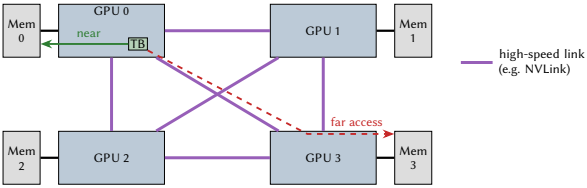


Figure 9.2: A four-GPU NUMA system. Each GPU has its own memory, and the six links connect every pair. A thread block on GPU 0 reaches its own memory directly (near access) and the memory of GPU 3 over a link (far access), at lower bandwidth and higher latency.

9.1.2 One logical GPU with non-uniform memory

To the programmer, a package or node of several GPUs joined by high-speed links (Figure 9.2) is still a large number of SMs, and all memories are reachable as global memory, one shared address space. Each GPU keeps its own DRAM, typically HBM stacks (Lesson 1) for the bandwidth its SMs demand, while the I/O bandwidth to the rest of the system is shared by all of them. What changes is the cost of an access.

Definition 9.3 (NUMA, near and far memory). A memory system has *non-uniform memory access (NUMA)* when one address space is spread over several memories and the latency and bandwidth of an access depend on which memory holds the address relative to the processor that issues it. The two cases are called *near memory / far memory*: near memory is attached to the GPU that issues the access, far memory to another GPU, so that the access must cross the inter-GPU link.

Example 9.4 (Near and far, in numbers). A kernel on an H100 reads a 1 GiB buffer (1.074 GB) once. From its own HBM3 at the preliminary 3 TB/s the transfer takes at least 0.36 ms. If the buffer lives on a peer GPU, it arrives over NVLink 4 at 450 GB/s per direction, half of the 900 GB/s total, in at least 2.4 ms; over PCIe Gen 5 at 64 GB/s per direction it takes at least 16.8 ms [2]. The far access is 6.7× slower than the near one over NVLink and 47× slower over PCIe, before any difference in latency is counted.

The figures in Example 9.4 are peak bandwidths; a real transfer reaches less, and a pattern of small far accesses suffers the link latency as well.

The ratio, not the absolute figure, is what NUMA means for a program: data should be near the SMs that use it. Two questions follow, and in this chapter they are asked for thread blocks and pages.

9.1.3 Where to run a block and where to put a page

Definition 9.5 (Thread block scheduling, first-touch page placement). The decision on which GPU (or GPM), and on which of its SMs, each thread block of a kernel runs is called *thread block scheduling*. Under *first-touch page placement* a page is allocated in the memory near the GPU whose thread block accesses it first.

First-touch needs no knowledge of the program: the page goes where it is first wanted. It works well as long as each page is used by blocks on the same GPU; it fails when a later block that runs elsewhere needs the same page. Suppose blocks are dealt out round-robin, one per GPU, and each brings its pages into its GPU's memory by first touch. A block scheduled later on GPU 0 that shares a page with the block on GPU 3 finds that page already placed in GPU 3's

Table 9.1: Trace of Example 9.7. Each entry is a block, its GPU, and its two page accesses; "far" marks a page that an earlier block has already placed on another GPU.

	Round-robin		Grouped	
Wave	Block, GPU	Pages	Block, GPU	Pages
1	TB0, G0	P0 new, P1 new	TB0, G0	P0 new, P1 new
	TB1, G1	P1 **far**, P2 new	TB2, G1	P2 new, P3 new
	TB2, G2	P2 **far**, P3 new	TB4, G2	P4 new, P5 new
	TB3, G3	P3 **far**, P4 new	TB6, G3	P6 new, P7 new
2	TB4, G0	P4 **far**, P5 new	TB1, G0	P1 near, P2 **far**
	TB5, G1	P5 **far**, P6 new	TB3, G1	P3 near, P4 **far**
	TB6, G2	P6 **far**, P7 new	TB5, G2	P5 near, P6 **far**
	TB7, G3	P7 **far**, P8 new	TB7, G3	P7 near, P8 new
Far accesses: 7 of 16			3 of 16	

memory, and every access to it is a far access (Figure 9.2). There are two remedies: schedule the block on the GPU where its data lives, or move the data.

Definition 9.6 (Page migration). The move of a page from one GPU's memory to another's while the program runs is called *page migration*; it is typically done when the page has received enough far accesses from that other GPU to repay the cost of the copy.

Migration is the multi-GPU form of the page movement of Lesson 7, where unified memory moves pages between host and device on a page fault; Li et al. study the address translation of such multi-GPU systems [37].

A migration is a copy of a whole page over the link plus an update of the page tables, so a policy that migrates on the first far access can do more harm than good when two GPUs take turns at the same page.

Example 9.7 (Round-robin versus grouped block scheduling). A kernel of eight thread blocks TB0–TB7 runs on four GPUs, two blocks per GPU in two waves. Block TBi reads page Pi and then page P(i+1), as in a one-dimensional stencil in which neighbouring blocks share a boundary page; pages are placed by first touch, and within a wave the blocks touch their pages in index order. Table 9.1 traces two schedules. Round-robin puts TBi on GPU i mod 4, so neighbours always sit on different GPUs: 7 of the 16 page accesses are far. Grouping contiguous blocks, TB0 and TB1 on GPU 0, TB2 and TB3 on GPU 1 and so on, leaves a shared page far only at the boundary between two groups: 3 of 16.

Caution Lecture vs paper: this chapter, like the lecture, reduces the problem to first-touch placement and locality-aware block scheduling. The MCM-GPU paper proposes three optimisations together: a GPM-side "L1.5" cache that holds only remote data, "distributed and batched" CTA scheduling that sends a group of contiguous CTAs to one GPM, and first-touch page allocation; combined they give a 22.8 % speed-up and a 5× reduction of inter-GPM traffic over the basic MCM-GPU [57].

The grouped schedule of Example 9.7 is the idea of the paper's distributed scheduler: contiguous blocks tend to share data, so they should share a GPU. It also pays across kernel launches: a kernel launched again in a loop sends the same block indices to the same GPUs, whose memories already hold the pages that first touch put there [57].

9.1.4 NVLink and NVSwitch

The link between the GPUs decides how much a far access costs.

Definition 9.8 (Interconnect, NVLink, NVSwitch). An *interconnect* is the network of links (and switches) over which processors and memories exchange data. NVIDIA's *NVLink* is a point-to-point GPU-to-GPU interconnect and protocol, an alternative to PCIe for traffic between GPUs. An *NVSwitch* is a switch chip with many NVLink ports; with NVSwitches between them, every GPU of a node reaches every other at full link bandwidth.

NVLink per GPU, by generation: 900 GB/s and 18 links (4th, Hopper); 1800 GB/s and 18 links (5th, Blackwell); 3000 GB/s and 36 links (6th, Vera Rubin, preliminary). An NVLink Switch domain spans 8 GPUs, or 72 in an NVL72 rack [58].

Caution Lecture vs reality: the lecture calls this about a quarter. Per direction NVLink 4 carries 450 GB/s (18 links of 25 GB/s), 15 % of HBM [2].

Lesson 5's unified virtual addressing is what lets a kernel on one GPU dereference a pointer into a peer GPU's memory, once `cudaDeviceEnablePeerAccess` has enabled peer-to-peer access over PCIe or NVLink (CPG §3.2.9.4 Peer-to-Peer Memory Access).

Without NVLink, data between two GPUs crosses the PCIe switch that they share with the CPU (Figure 9.3a); a direct NVLink bypasses it (b). Fourth-generation NVLink, in the H100, provides 900 GB/s per GPU over 18 links, which the whitepaper quotes as 7× the bandwidth of PCIe Gen 5 [2]; against the H100's 3 TB/s of HBM3 bandwidth, NVLink offers 30 % of the local memory bandwidth (both directions counted). Point-to-point links do not scale, however: a GPU has a fixed number of links, and connecting every pair of n GPUs directly needs $n(n-1)/2$ links. NVSwitch chips solve this (Figure 9.3c): each GPU spreads its links over the switches, and the switches connect every pair of GPUs with higher bandwidth and lower latency than a chain of direct links could.

Within a node, then, the GPUs form one NUMA machine: one address space, with near and far memory.

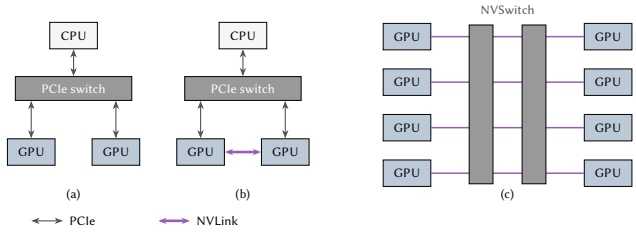


Figure 9.3: (a) Two GPUs that communicate only through the PCIe switch they share with the CPU. (b) The same GPUs with a direct NVLink. (c) Eight GPUs, each linked to every NVSwitch chip, so that every pair of GPUs is connected through a switch.

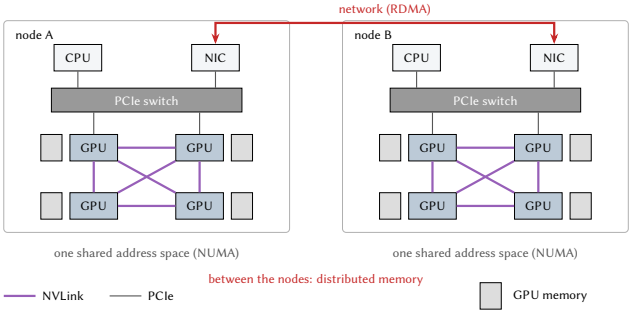


Figure 9.4: Two multi-GPU nodes. Inside a node, NVLink joins the four GPUs into one NUMA address space; the CPU and the network interface card sit on the PCIe switch (only two GPU connections drawn). Between nodes the memories are separate, and data crosses the network by RDMA.

9.1.5 Across boards: RDMA

GPUs on different boards or in different nodes are attached to their hosts through PCIe (Figure 9.4), and PCIe through the CPU does not provide enough bandwidth to share memory between them. Their memories are separate spaces, and the system is a distributed-memory system as in Lesson 2: data moves by explicit messages, here by RDMA.

Traditional networking sends a buffer through the TCP/IP software stack (Figure 9.5a): the sending CPU copies the data from the application buffer through the operating system to the network, and the receiving CPU copies it up again into the destination buffer. Both CPUs spend cycles on every message, and a busy CPU delays the transfer.

Programs rarely issue these transfers themselves: collective libraries such as NCCL (Lesson 5) choose NVLink, NVSwitch or GPUDirect RDMA for each pair of GPUs.

Definition 9.9 (RDMA, host bypass, NIC). A *network interface card (NIC)* is the device that connects a host to the network. The technique called *remote direct memory access (RDMA)* extends the DMA of Lesson 7 across the network: the RDMA engine of one NIC reads the source buffer from memory and the RDMA engine of the other NIC writes it into the destination memory, without either CPU copying the data (Figure 9.5b). Such a transfer is a *host bypass* technique: the host processors and their software stacks are left out of the data path.

Definition 9.10 (GPUDirect RDMA). NVIDIA's technology *GPUDirect RDMA* provides a direct data path between GPU memory and a third-party PCIe device, such as a NIC or a storage adapter, "using standard features of PCI Express" [59]. A GPU buffer can then be sent through the network to a remote GPU, or to another device, without being staged in host memory (Figure 9.5c).

Caution GPUDirect RDMA needs the GPU and the third-party device to "share the same upstream PCI Express root complex"; it was introduced with Kepler-class GPUs and CUDA 5.0 [59].

The two kinds of link work together: inside a node, NVLink and NVSwitch carry the GPU-to-GPU traffic; between nodes, GPUDirect RDMA moves it over the network. PCIe alone cannot meet the demand of so many GPUs exchanging data.

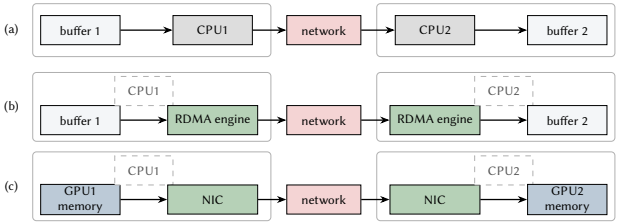


Figure 9.5: (a) Traditional networking: the data passes through both CPUs. (b) RDMA: the RDMA engines move it memory to memory; the CPUs (dashed) stay out of the path. (c) GPUDirect RDMA: the NIC reads and writes GPU memory directly, so a GPU buffer reaches a remote GPU without a copy through host memory.

Key point
Scaling past one die leads to multi-chip modules, multi-GPU nodes and clusters. Within a package or node the memory is one NUMA space, so block scheduling and page placement decide how many accesses are far; NVLink and NVSwitch make far accesses cheaper. Between nodes the memory is distributed and RDMA, with GPUDirect RDMA for GPU memory, moves the data without the CPUs.

9.2 Concurrency on GPUs

Source M9, concurrency on GPUs; CPG §3.2.8 Asynchronous Concurrent Execution, §3.2.9 Multi-Device System; [1]; [60].

A large GPU is not always used well. This section asks how several jobs can share one GPU, compares three mechanisms that let them, and shows how a program drives several streams and several GPUs.

9.2.1 Utilization and multi-tenancy

Definition 9.11 (Utilization, datacenter, multi-tenancy). The *utilization* of a GPU is the fraction of its resources (SMs, memory bandwidth, time) that does useful work. A *datacenter* is a facility of many servers shared by many users, and *multi-tenancy* is the sharing of one machine, here one GPU, by the jobs of several independent users (tenants).

Not every job can use all of a large GPU: an inference service with small batches, a development session or a small training job may keep a fraction of the SMs busy. On a dedicated GPU the rest is wasted, which matters most in datacenters, where GPUs are the expensive resource. Traditional datacenters serve many tenants with jobs of this size. Some AI workloads behave differently: training a large language model takes whole GPUs, many of them, for a single tenant, and such a job advances only as fast as its slowest task, since every step waits for all of its GPUs. Both patterns call for ways to place more than one stream of work on a GPU.

Ye et al. survey the scheduling of deep-learning jobs in GPU datacenters and note that "traditional approaches designed for big data or high performance computing workloads can not support DL workloads to fully utilize the GPU resources" [62]; Yu et al. survey multi-tenant inference [61].

9.2.2 Three mechanisms

Figure 9.6 compares the three mechanisms by what they partition.

Streams. One client (one process) issues work to several CUDA streams (Lesson 5). Only independent streams run concurrently, and their kernels share all resources: SMs, L2 and DRAM.

MPS. Several clients, separate processes, run at the same time on different SMs, while the L2 cache and DRAM bandwidth remain shared.

MIG. The GPU is divided physically: each client receives its own SMs, L2 slices and DRAM bandwidth.

9.2.3 Multi-Instance GPU

Definition 9.12 (Multi-Instance GPU, GPU instance, QoS). The technology called *Multi-Instance GPU (MIG)* divides one GPU into several isolated partitions, each a *GPU instance* that behaves like a smaller GPU with its own SMs, L2 cache slices, memory controllers and DRAM [1]. MIG partitions memory, the L2 cache ports, DRAM bandwidth and the on-chip crossbar

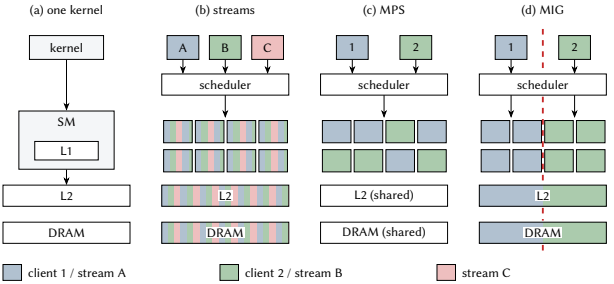


Figure 9.6: GPU concurrency mechanisms. (a) One kernel uses the SMs, the L2 and DRAM. (b) Streams of one client: their work mixes on every SM and in the L2 and DRAM; nothing is partitioned. (c) MPS: the clients' blocks are spread over the SMs (drawn here with a per-client SM share); the L2 and DRAM are shared. (d) MIG: SMs, L2 and DRAM are split physically between the clients.

ports, so that each instance has a guaranteed *quality of service (QoS)*: predictable throughput and latency whatever the other instances do.

MIG thus makes one large GPU an efficient host for several jobs, with memory and core isolation for several users. The A100 "can be securely partitioned into as many as seven separate GPU Instances" [1], which is what a cloud provider needs to rent parts of one GPU to different customers (Figure 9.7). The price of isolation is rigidity: an instance has a fixed size, and work cannot spill into an idle neighbour.

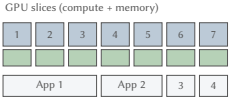


Figure 9.7: Four applications on a GPU of seven MIG slices, with GPU instances of 3, 2, 1 and 1 slices. Memory simplified: the A100 has eight memory slices, and a 3-slice instance gets four [63, §6.2].

An A100 GPU instance is built from "GPU slices" of one GPC with 14 SMs and one group of ten L2 slices; the GPU has seven [1]. Profiles run from 1g.5gb to 7g.40gb on an A100 40 GB [63, §6.2]. The H100's second-generation MIG keeps seven instances and adds confidential computing [2].

Caution Lecture vs document: MPS does not give each client its own SMs. A provisioning limit caps a client's share and reserves nothing; "kernels launched from different MPS client contexts may execute on the same SM". Only a non-uniform split (say 30 % and 70 %), or the static SM partitions of Ampere and later, confine the clients to disjoint SMs [60, §1.1.2.2.5, §1.1.2.2.11].

Caution Before Volta, MPS already let clients run concurrently, through a server that merged their work into one context; Volta added hardware support so that "MPS client processes have fully isolated GPU address spaces" and "limited execution resource provisioning", the per-client SM share (environment variable `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE`) [60].

Caution MPS isolation is limited in a second way. A fatal GPU fault of one Volta MPS client is reported to all clients that share the GPUs on which it occurred, "without indicating which client generated the error", and those clients must exit [60]. Under MIG a fault stays inside its instance [1].

9.2.4 Multi-Process Service

Definition 9.13 (Time slicing, spatial partitioning, MPS). Under *time slicing* the GPU runs the work of one process (one CUDA context) at a time and switches between processes; under *spatial partitioning* the work of several processes runs at the same time on disjoint parts of the GPU, such as different SMs. The *Multi-Process Service (MPS)* is NVIDIA's runtime service that lets the CUDA work of several processes run concurrently on one GPU, optionally with a per-client limit on the SMs it may use.

Kernels of different CUDA contexts cannot run concurrently without MPS: "The GPU may time slice to provide forward progress to each context" (CPG §3.2.8.2 Concurrent Kernel Execution). With MPS, which supports spatial partitioning from the V100 on, the clients' kernels share the GPU at once, but the resources are not strictly partitioned: the L2 and the DRAM bandwidth are shared, so a client's performance depends on what the others do and can be unpredictable. MPS also makes it easy to bring MPI programs to CUDA: the MPI ranks on one node are separate processes, and with MPS their kernels can fill one GPU together. When QoS is not critical, MPS can achieve better utilization than MIG, because a client's share is not rounded to whole slices and idle SMs can be given to whoever has work.

Example 9.14 (Fitting three jobs). Three inference jobs need 30 %, 25 % and 10 % of a GPU's SMs. Under MIG with seven equal slices, a slice is $1/7 \approx 14.3$ %, so the jobs need 3, 2 and 1 slices (42.9 %, 28.6 % and 14.3 %): six of the seven slices are taken although only 65 % of the SMs do useful work, and a fourth job that needs 20 % no longer fits. Under MPS each job can be limited to its share (`CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` of 30, 25 and 10, rounded down to a supported thread count). Nothing is reserved, so the remaining capacity stays available to further clients, at the cost of sharing SMs as well as the L2 and the DRAM bandwidth, and no job is guaranteed its performance [60, §1.1.2.2.5].

9.2.5 Stream-based programming

Inside one process, streams are the means of concurrency.

Definition 9.15 (Concurrent kernel execution). The execution of several kernels of one CUDA context on the GPU at the same time is called *concurrent kernel execution*. Kernels issued to different streams may run concurrently when resources allow; kernels in the same stream run

Up to 128 kernels can be resident at once on V100, A100 and H100 ("maximum number of resident grids per device", CPG Table 21 [5]).

in order (CPG §3.2.8.2 Concurrent Kernel Execution).

Streams thus allow concurrent kernels, the overlap of communication and computation across streams (Lesson 5), and the submission of work by several CPU threads, each with its own streams. The loop below splits a job into more chunks than there are streams and deals the chunks out to the streams in turn; each iteration issues one chunk's copy-in, kernel and copy-out before moving to the next chunk, a *depth-first* launch order.

```
// h_x, h_y pinned; d_x, d_y device buffers of n floats
const int nStreams = 4, nChunks = 16;
size_t chunk = n / nChunks;
size_t bytes = chunk * sizeof(float);
cudaStream_t st[nStreams];
for (int k = 0; k < nStreams; ++k)
  cudaStreamCreate(&st[k]);
for (int i = 0; i < nChunks; ++i) {    // depth-first
  cudaStream_t s = st[i % nStreams];
  size_t off = i * chunk;
  cudaMemcpyAsync(d_x + off, h_x + off, bytes,
                  cudaMemcpyHostToDevice, s);
  square<<<(chunk + 255) / 256, 256, 0, s>>>(
      d_x + off, d_y + off, chunk);
  cudaMemcpyAsync(h_y + off, d_y + off, bytes,
                  cudaMemcpyDeviceToHost, s);
}
cudaDeviceSynchronize();               // all streams
```

Chunks 0, 4, 8 and 12 share stream 0 and therefore run one after the other, while the four streams overlap. The scheduling has a cost: for every operation the driver prepares the launch and the hardware must track the dependencies between operations of the same stream. For short kernels this overhead can be a significant fraction of the run time.

Tip An OpenMP program ports naturally to this style: a parallel loop whose iterations are independent becomes a loop that issues each iteration's work to its own stream, and each OpenMP thread can also drive a stream of its own.

Definition 9.16 (CUDA Graph). A *CUDA Graph* is a set of operations (kernel launches, copies and others) connected by dependencies, defined separately from its execution: the graph is built once, instantiated into an executable graph that is validated and set up in advance, and then launched as often as needed with one call (CPG §3.2.8.7 CUDA Graphs).

```
cudaGraph_t g;  cudaGraphExec_t ge;
cudaStreamBeginCapture(s, cudaStreamCaptureModeGlobal);
for (int k = 0; k < 20; ++k)           // 20 short kernels
  step<<<blocks, 256, 0, s>>>(d_u, n);
cudaStreamEndCapture(s, &g);           // record, not run
cudaGraphInstantiate(&ge, g, 0);       // set up once
for (int it = 0; it < 1000; ++it)
  cudaGraphLaunch(ge, s);              // one call each
cudaStreamSynchronize(s);
```

Stream capture turns existing stream code into a graph: between `cudaStreamBeginCapture` and `cudaStreamEndCapture` the work issued to the stream is recorded instead of executed. Because the whole dependency chain is known before the first launch, the setup is paid once rather than for every kernel, and the runtime can optimise the workflow as a whole (CPG §3.2.8.7).

Caution Stream capture does not work on the legacy default stream (CPG §3.2.8.7.3 Creating a Graph Using Stream Capture); `s` must not be the legacy default stream: a created stream or `cudaStreamPerThread` will do.

9.2.6 Programming several GPUs

The host program drives several GPUs by switching the current device: the call `cudaSetDevice(d)` makes GPU `d` current for the calling host thread: allocations and kernel launches go to the current device, and streams and events belong to the device that was current when they were created (CPG §3.2.9.2 Device Selection). The program below divides an array among all GPUs, the data decomposition of Lesson 2, with a separate allocation, copy and kernel launch per GPU.

With a pageable `h_x`, each `cudaMemcpyAsync` first copies into a staging buffer on the CPU (Lesson 5), and the GPUs start one after the other; pinned host memory lets all copies overlap.

```
int nDev;
cudaGetDeviceCount(&nDev);
size_t part = n / nDev;                // n divisible
size_t bytes = part * sizeof(float);
float *d_x[16];                        // nDev <= 16
```

Table 9.2: Streams, MPS and MIG compared.

	Streams	MPS	MIG
Partition type	none	logical	physical
Maximum partitions	unlimited	48 (see caution)	7
SM isolation	no	by percentage (a cap)	yes
Memory bandwidth isolation	no	no	yes
Performance QoS	no	partial	yes
Reconfiguration	dynamic	at process launch	when idle

```
for (int d = 0; d < nDev; ++d) {
  cudaSetDevice(d);                    // select GPU d
  cudaMalloc(&d_x[d], bytes);          // on GPU d
  cudaMemcpyAsync(d_x[d], h_x + d * part, bytes,
                  cudaMemcpyHostToDevice);
  square<<<(part + 255) / 256, 256>>>(d_x[d], d_x[d],
                                      part);
}
for (int d = 0; d < nDev; ++d) {       // wait for all
  cudaSetDevice(d);
  cudaDeviceSynchronize();
}
```

Kernel launches and asynchronous copies return at once, so the first loop starts the work on every GPU before any of it has finished; the GPUs then run in parallel, each on its own part. Each device has its own default stream (CPG §3.2.9.3 Stream and Event Behavior), and streams can be combined with several GPUs: each GPU receives its own streams, created while it is current.

A DGX H100 node holds eight H100 GPUs joined through four NVSwitches, and eight 400 Gbit/s ConnectX-7 network adapters (HWP App. A [2]).

Caution `cudaDeviceSynchronize` waits only for the *current* device. One call after the launch loop waits for the last GPU alone; the host must select each device in turn and synchronise it, as above.

9.2.7 Comparison

Table 9.2 summarises the three mechanisms. They are not exclusive: a process inside one MIG instance can still use streams, and MPS can run inside a MIG instance, with the client limit lowered in proportion to the instance size [63, §7.2].

Streams cost nothing to set up and adapt at run time but give no isolation; MIG gives the strongest isolation but changes its partition only when the GPU is idle; MPS lies in between, with SM limits normally set when a client process starts (per-context limits can also be set later [60, §1.1.2.2.5]).

Caution Lecture vs document: 48 MPS clients is the Volta (CC 7.0) limit of earlier documentation, 16 before Volta [64]. The current overview allows up to 60 per device and server with the default `CUDA_DEVICE_MAX_CONNECTIONS` of 2, fewer if it is raised [60, §1.1.2.2.6].

Tip Use MIG when QoS matters, MPS when utilization matters, and streams for concurrency inside one process.

Key point — Lesson summary

A monolithic GPU is limited by die size and yield, so GPUs scale as multi-chip modules of GPU modules, multi-GPU nodes and clusters. Within a package or node the memory is one NUMA space of near and far memory: first-touch placement, locality-aware thread block scheduling and page migration keep accesses near, and NVLink and NVSwitch make far accesses cheaper than PCIe. Across nodes the memory is distributed and RDMA, with GPUDirect RDMA for GPU buffers, moves data without the CPUs. To raise the utilization of one GPU, streams run independent work of one process without partitioning; MPS runs several processes at once, with optional per-client SM limits, but shares the L2 and DRAM; MIG splits SMs, L2 and DRAM into up to seven isolated instances with QoS. CUDA Graphs remove the per-launch overhead of streams, and `cudaSetDevice` selects the GPU for each allocation, copy and launch.

Lesson 10

Compiler Background - I

Lecture	M10, lessons 1–5 (GPU Compiler Flow; PTX; IR and Basic Block; Introduction to Data Flow Analysis; Example of Reaching Definitions)
Sources	NVCC documentation [65]; PTX ISA 9.4 [19]; Compiling CUDA with clang [66]; NVVM IR specification [67]; Lattner and Adve [68] (LLVM); LLVM Language Reference [69]
Textbook	K12 [3] §1.3.7; CPG [5] §3.1 Compilation with NVCC; Aho et al. [70] §8.4, §9.2
Keywords	compiler, front end, back end, intermediate representation, abstract syntax tree, nvcc, LLVM, NVPTX, PTX, SASS, virtual architecture, fat binary, JIT compilation, guard predicate, three-address code, leader, control flow graph, data-flow analysis, transfer function, reaching definitions, gen and kill sets, fixed point

The previous lessons treated a CUDA kernel as something the GPU simply runs; between the source file and the SM stands a compiler, and several mechanisms of earlier lessons were in fact the compiler's work: the predicated code of Lesson 6, the registers each thread receives in Lesson 3, the PTX fragments of Lesson 4. This lesson and the next give the compiler background needed to reason about such code. The first half follows a CUDA program through the compilation flow, from source code over an intermediate representation and PTX to the machine code that the GPU executes, and looks at PTX itself. The second half introduces the structures on which optimisations work: three-address code, basic blocks and control-flow graphs, and data-flow analysis, worked out for the analysis called reaching definitions. Lesson 11 continues with liveness, SSA form, classic optimisations and the divergence analysis that connects all of this back to warps.

10.1 GPU compiler flow

Source M10, GPU compiler flow; CPG §3.1 Compilation with NVCC; K12 §1.3.7; [65].

A *compiler* translates a program written in one language, the source program, into an equivalent program in another language, usually the instruction set of a machine. It is organised as a pipeline of three parts (Figure 10.1).

Definition 10.1 (Front end, IR, back end). The *front end* reads the source program, checks its syntax and meaning, and translates it into an *intermediate representation (IR)*, a program form that belongs neither to the source language nor to the target machine. The *optimizer* transforms IR into IR that computes the same results faster or with fewer instructions. The *back end* performs *code generation*: it translates IR into instructions of the target machine and applies the optimisations that depend on that machine.

The optimizer is also called the *middle end*; in LLVM it is a sequence of passes, each an IR-to-IR transformation [68].

The IR is what makes the split useful. One front end per language and one back end per target share a single optimizer, so that adding a language or a machine adds one component rather than one complete compiler per pair. The IR is also a good level for optimisation: it is simpler than the source language and still independent of any machine, and it can carry metadata, such as types and source positions, from the front end to the later stages.

With L languages and T targets, the IR split needs $L + T$ front and back ends instead of $L \cdot T$ complete compilers.

10.1.1 The flow for a GPU program

CPG §3.1.1.1 Offline Compilation [5]: nvcc separates device code from host code, compiles the device code to PTX or binary form, and replaces the kernel launch syntax with calls into the CUDA runtime.

A CUDA source file mixes host code and device code (Lesson 3). A GPU compiler therefore first separates the two and sends each through a pipeline of its own: the device code is compiled for the GPU, the host code with an ordinary C++ compiler, and the result for the GPU is packed into the host program, which loads it when the program runs.

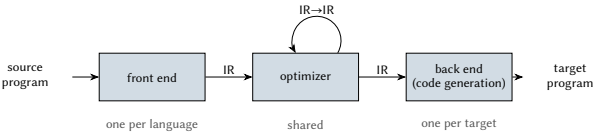


Figure 10.1: The classic compiler pipeline. The front end turns the source program into IR, the optimizer rewrites IR into better IR, and the back end generates code for the target machine.

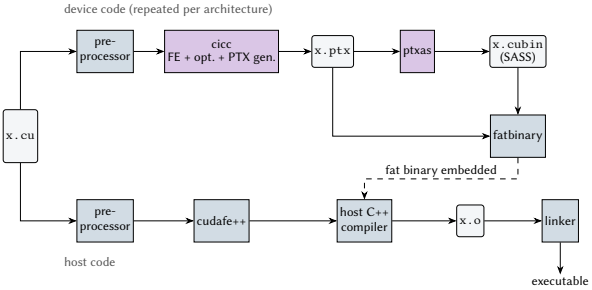


Figure 10.2: The nvcc compilation trajectory, simplified. Device code passes cicc and ptxas once per architecture; fatbinary packs the PTX and the machine code (cubin) into a fat binary that the host object embeds. The device linker nvlink is omitted.

NVIDIA's compiler driver *nvcc* implements this flow as the *CUDA compilation trajectory* of Figure 10.2 [65, §3]. The input file is preprocessed twice. On the device side the front end and optimizer `cicc` produces a `.ptx` file, and the assembler `ptxas` translates it into a `.cubin` file of machine code; this pair of steps is repeated for every architecture requested. The tool `fatbinary` packs the PTX and `.cubin` results into one image. On the host side `cudafe++` separates the host code and rewrites the CUDA extensions into standard C++, and the host C++ compiler compiles that code with the device image embedded. The linker finally produces the executable. With separate compilation of device code, a device linker `nvlink` first links the `.cubin` files of all translation units.

The tool names come from the trajectory figure of the NVCC documentation [65, §3]; `nvcc --dryrun` lists the commands it would run, and `--keep` keeps the intermediate `.ptx` and `.cubin` files (§4.2.5.9, §4.2.5.11).

Definition 10.2 (Virtual architecture). A *virtual architecture* is the GPU that PTX code is written for: a set of capabilities that a program may use, named `compute_XY` after a compute capability X.Y. An actual GPU, the target of `ptxas`, is a *real architecture*, named `sm_XY`.

Every nvcc command therefore names two architectures, a virtual one for the first stage and a real one for the second, and the real one must implement the virtual one [65, §5.4]. The virtual architecture states which features the program needs; it does not fix the GPU it will run on.

Tip Choose the virtual architecture as low as possible, so that the PTX runs on as many GPUs as possible, and the real architecture as high as the target GPUs allow (NVCC §5.4 [65]).

10.1.2 The open-source toolchain: Clang and LLVM

The same structure appears in the open-source CUDA compiler.

Definition 10.3 (LLVM, Clang, NVPTX). *LLVM* is a compiler infrastructure built around one typed IR, the LLVM IR, with a library of optimisations on it and back ends for many targets [68]. *Clang* is the LLVM front end for the C family of languages, including CUDA. *NVPTX* is the LLVM back end that generates PTX.

A splitter separates host and device code. The device code passes through Clang, the IR optimisations and the NVPTX code generator and becomes PTX assembly; the host compilation then embeds the device code into a fat binary.[1] NVIDIA's own device compiler is built on the same technology: its front end emits *NVVM IR*, an IR "based on the LLVM IR", and the NVVM compiler, itself based on LLVM, generates PTX from it [67].

The front end parses the source into a tree.

Definition 10.4 (Abstract syntax tree). An *abstract syntax tree (AST)* represents the structure of a program as a tree: each inner node is an operation or statement (a sequence, a loop, a branch, an assignment, an arithmetic operator) and its children are its operands or sub-statements; the leaves are variables and constants. Punctuation such as parentheses and semicolons does not appear: the tree shape already expresses it.

Example 10.5 (The AST of a loop). The loop below keeps the running maximum of an array.

```
while (i < n) {
  if (x[i] > m) m = x[i];
  i = i + 1;
}
```

[1] Clang does not split the file before parsing it: it uses "merged parsing", in which all host and device code is present in both compilation steps and must be semantically correct in both. Unlike nvcc, it always also runs `ptxas` and embeds machine code for each requested GPU (`--offload-arch=sm_80`). By default it does not embed PTX; `--cuda-include-ptx=sm_80` (or `all`) adds it for forward compatibility [66].

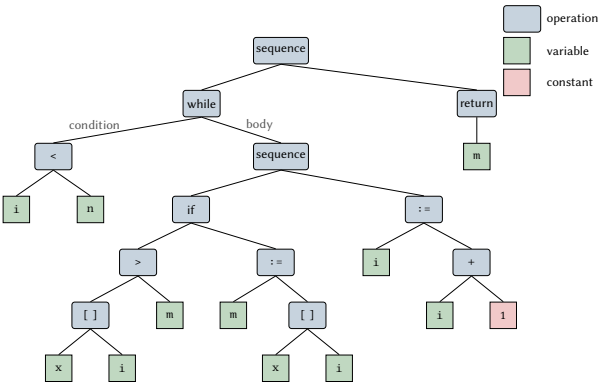


Figure 10.3: The abstract syntax tree of the loop in Example 10.5. Operations and statements are blue, variables green, constants red.

```
return m;
```

Its AST (Figure 10.3) has a sequence at the root with two children, the `while` loop and the `return`. The loop node has the condition `i < n` and the body, itself a sequence of the `if` statement and the increment. Each array access `x[i]` is an indexing node with the array and the index as children.

Tip `clang -fsyntax-only -Xclang -ast-dump file.c` prints the AST that Clang builds for a file.

Since the preprocessor works on text, a macro body such as `a + b` used in `2 * M` becomes `2 * a + b`; macro bodies are therefore written in parentheses.

Before the parser runs, the *preprocessor* of C and C++ performs pure text substitution: it expands `#include` files and macros and evaluates conditional compilation. With the definition

```
#define BLOCK_SIZE 256
int threads = BLOCK_SIZE;
```

the compiler proper sees only `int threads = 256;`.

10.1.3 PTX, SASS and the fat binary

The output of the device back end is not yet machine code.

Definition 10.6 (PTX and SASS). *PTX (Parallel Thread Execution)* is NVIDIA's virtual ISA (Lesson 1): an architecture-independent instruction set whose programs are translated into machine code later. Its registers are virtual, so PTX code has no register allocation. *SASS* (shader assembly) is the machine code of one GPU architecture, with physical registers allocated. SASS changes with every GPU generation; PTX stays compatible: code written for an older PTX version still compiles for newer GPUs, while new PTX versions add instructions.

PTX ISA 8.0, the `.version` of Example 10.12, came with CUDA 12.0; the release history of PTX ISA 9.4 runs from PTX ISA 1.0 with CUDA 1.0 [19, §13].

The PTX ISA states the purpose directly: "PTX defines a virtual machine and ISA for general purpose parallel thread execution", and its goals include "a stable ISA that spans multiple GPU generations" [19, §1.1–1.2].

Definition 10.7 (Fat binary). A *fat binary* is an executable that carries, next to the host code, several translations of the same device code: machine code for some real architectures and PTX for some virtual ones. At run time the device part is handed to the driver, which selects the most suitable translation for the GPU present [65, §5.6.2].

A PTX register is a variable of a type, and a kernel may declare any number; `ptxas` maps them onto at most 255 physical registers per thread (CPG Table 21 [5]) and spills the rest to local memory [19, §5.1.1]. Lesson 11 treats register allocation.

If the fat binary contains no machine code for the GPU present, the driver compiles the PTX at run time.

Definition 10.8 (JIT compilation). In *JIT compilation* (just-in-time compilation) the device driver compiles PTX into machine code when an application loads it. CPG §3.1.1.2 Just-in-Time Compilation states that it is "the only way for applications to run on devices that did not exist at the time the application was compiled" [5].

JIT compilation costs start-up time. The driver therefore keeps a copy of each result in the *compute cache*, which is invalidated when the driver is upgraded, so that a new JIT compiler

can improve old applications (CPG §3.1.1.2). A fat binary that also carries machine code for the expected GPUs avoids the delay altogether, and its PTX keeps the program running on later GPUs:

```
nvcc x.cu --gpu-architecture=compute_80 \
          --gpu-code=compute_80,sm_86,sm_89
```

builds machine code for compute capabilities 8.6 and 8.9 and keeps PTX for compute capability 8.0, which a later GPU can JIT-compile [65, §5.6.2]. PTX, in contrast, always compiles to an equal or higher compute capability, but the result cannot use features added after its virtual architecture: PTX for compute capability 6.0, JIT-compiled on a 7.0 GPU, uses no tensor cores (CPG §3.1.3 PTX Compatibility).

The compute cache lives in `~/.nv/ComputeCache` on Linux and holds 1 GiB by default (256 MiB on embedded platforms), at most 4 GiB, set by `CUDA_CACHE_MAXSIZE` (CPG §18 CUDA Environment Variables [5]).

Caution Machine code does not move across major compute capabilities: a cubin for compute capability X.y runs only on X.z with $z \geq y$ (CPG §3.1.2 Binary Compatibility [5]).

10.2 PTX

PTX is the lowest level at which CUDA code is normally read, so its instruction format is worth knowing in detail.

Source M10, PTX; PTX ISA §4.3, §9.2–9.3 [19]; K12 §1.3.7.

10.2.1 Instruction format

A PTX instruction consists of an *opcode*, the name of the operation, and a list of *operands*. The PTX ISA puts it as follows: "PTX instructions generally have from zero to four operands, plus an optional guard predicate appearing after an `@` symbol to the left of the opcode" [19, §9.2].

```
@p  opcode;
@p  opcode a;
@p  opcode d, a;
@p  opcode d, a, b;
@p  opcode d, a, b, c;
```

For an instruction that produces a value, `d` is the destination and `a`, `b`, `c` are sources; the destination comes first. Every arithmetic and memory instruction names the type it operates on, as in `add.s32` or `ld.global.f32`.

The opcode carries suffixes: a type (`.s32` signed, `.u32` unsigned, `.f32` float, `.b32` untyped bits, `.pred`) and, for memory instructions, a state space (`.global`, `.shared`, `.local`, `.param`).

Definition 10.9 (Guard predicate, predicate register). A *predicate register* is a one-bit PTX register of type `.pred`, declared like any other register, e.g. `.reg .pred p, q;`. A *guard predicate* `@p` placed before an instruction makes it conditional: the instruction takes effect only in threads where `p` is true; `@!p` uses the negation. An instruction without a guard is executed unconditionally.

Predicate registers are virtual like all PTX registers; the `.pred` type tells `ptxas` that one bit suffices [19, §9.3]. They are set most commonly by `setp`, which "compare[s] two numeric values with a relational operator" [19, §9.7.7.2]. `setp` can write two destinations, separated by `|` (a few other instructions, such as `shfl.sync`, use the same notation): the second receives the negation of the comparison.

Predicates are the mechanism behind the if-conversion and predicated execution of Lessons 5 and 6.

Example 10.10 (Removing a branch with predicates). The statement `if (a < b) c = c + 1; else c = c - 1;` with `a`, `b`, `c` in `%r1`, `%r2`, `%r3` needs no branch:

```
     setp.lt.s32  %p1|%p2, %r1, %r2; // p1 = a<b, p2 = !p1
@%p1 add.s32      %r3, %r3, 1;       // then: c = c + 1
@%p2 sub.s32      %r3, %r3, 1;       // else: c = c - 1
```

Every thread of a warp issues all three instructions, and each thread commits exactly one of the two updates. The same guard on a branch instruction gives a real conditional jump:

```
     setp.lt.s32  %p1, %r1, %r2;
@!%p1 bra         ELSE;             // a >= b: jump
```

The predicated form trades the risk of divergence (Lesson 6) for issuing both sides; for short bodies like these the compiler prefers it.

Tip The comparison operators are `eq`, `ne`, `lt`, `le`, `gt`, `ge` for signed integers and floats; unsigned integers use `eq`, `ne`, `lo`, `ls`, `hi`, `hs` (lower, lower or same, higher, higher or same) [19, §9.3.1].

10.2.2 Statements

A PTX program is a sequence of statements. In the words of the PTX ISA, "a PTX statement is either a directive or an instruction. Statements begin with an optional label and end with a semicolon" [19, §4.3].

Definition 10.11 (Directive, label). A *directive* is a statement whose keyword begins with a dot and which declares rather than computes: the PTX version and target (`.version`, `.target`, `.address_size`), functions and kernels (`.func`, `.entry`), and variables in a state space (`.reg`, `.global`, `.shared`). A *label* is a name followed by a colon in front of a statement; branch instructions use it as their target.

Example 10.12 (A complete PTX kernel). The kernel below doubles one element of a global array per thread.

```
.version 8.0
.target sm_80
.address_size 64
.global .f32 buf[1024];

.visible .entry twice()
{
    .reg .b32  %r<2>;
    .reg .b64  %rd<4>;
    .reg .f32  %f<3>;
start:
    mov.u32        %r1, %tid.x;
    mov.u64        %rd1, buf;      // address of buf
    mul.wide.u32   %rd2, %r1, 4;   // byte offset 4*tid
    add.u64        %rd3, %rd1, %rd2;
    ld.global.f32  %f1, [%rd3];
    add.f32        %f2, %f1, %f1;  // 2 * buf[tid]
    st.global.f32  [%rd3], %f2;
    ret;
}
```

The first four lines are directives; `.reg .b32 %r<2>` declares the registers `%r0` and `%r1`. The label `start` marks the first instruction, and every arithmetic and memory instruction names its type.

`%r<2>` is PTX shorthand for a numbered range of registers; `mul.wide.u32` multiplies two 32-bit values into a 64-bit result, as needed for a byte offset into a 64-bit address space.

10.2.3 Other instructions

Beyond arithmetic and memory access, PTX has the usual groups of instructions. Control flow uses `bra` (branch to a label), `call` (call a function) and `ret` (return). Synchronisation includes the barrier `bar.sync` that `__syncthreads()` compiles to and the *memory fence* instructions `membar` and `fence`, which "enforce an ordering of memory operations" [19, §9.7.15.4]: the memory accesses of a thread before the fence become visible to other threads before its accesses after the fence. Atomic read-modify-write operations use `atom`, always with an operation and a type, as in `atom.global.add.u32`.

Special registers (Lesson 4) give each thread its coordinates: `%tid` (thread index in the CTA), `%ntid` (number of threads per CTA dimension), `%ctaid` and `%nctaid` (block index and grid size) [19, §10]. The full list of instructions and their variants is in the PTX ISA itself.

Caution There is no stack-pointer special register in the PTX ISA, although such a register is sometimes listed with `%tid` and `%ntid`. Fixed-size local storage is a `.local` array, and compiler-generated PTX addresses it through ordinary registers it names `%SP` and `%SPL`. Stack storage whose size is known only at run time is allocated with `alloca`; `stacksave` and `stackrestore` read and reset the stack pointer [19, §7.3].

Not every combination exists: the half-precision atomic add is written `atom.add.noftz.f16` (no flush to zero) and requires PTX ISA 6.3 and `sm_70` [19, §9.7.15.5].

10.3 IR and basic blocks

An optimizer needs the program in a form whose instructions are simple and whose control flow is explicit. The first requirement leads to three-address code, the second to basic blocks and flow graphs.

Source M10, IR and basic block; Aho et al. §6.2, §8.4 [70].

10.3.1 Three-address code

Definition 10.13 (Three-address code). In *three-address code* every instruction has at most one operator and at most three addresses (names of variables, temporaries or constants), in the form `A = B op C`. A complex expression is broken into a sequence of such instructions with compiler-generated temporaries.

LLVM IR and PTX are close to this form: most of their instructions have one operation, one destination and one or two sources (PTX also has three-source instructions such as `mad`). In LLVM IR

```
%sum = add i32 %x, %y
```

`%sum` is the destination, `add` the operation, `i32` the type of the result (a 32-bit integer) and `%x`, `%y` the sources [69]. The PTX form of the same operation is `add.s32 %r3, %r1, %r2`. The expression `y = a * b + c` becomes two instructions, `t = a * b` and `y = t + c`.

LLVM IR has one `add` for signed and unsigned integers, since both produce the same bits; PTX writes the interpretation into the opcode (`add.s32`, `add.u32`).

10.3.2 Basic blocks

Lesson 6 introduced the *basic block* informally. The formal definition is the unit on which most optimisations work.

Definition 10.14 (Basic block). A basic block is a maximal sequence of consecutive instructions with one entry and one exit: control can enter only at its first instruction, and once it has entered, all instructions of the block execute in order, leaving the block only after the last one. The last instruction is usually a control-flow instruction (a branch, a jump or a return); a block also ends where the next instruction is a branch target.

Definition 10.15 (Control flow graph, predecessor, successor). A *control flow graph (CFG)* has the basic blocks of a function as its nodes and an edge $B \to C$ whenever control can pass from the end of B to the start of C: by a jump, by a taken conditional branch, or by falling through to the next block. B is then a *predecessor* of C, and C a *successor* of B. An *entry* node precedes the block of the first instruction, and an *exit* node follows every block that leaves the function.

An `if/else` gives the diamond of Lesson 6 (Figure 10.4): the block B1 with the condition has two successors, the two branches B2 and B3, which share one successor, the join block B4; the edges are B1→B2, B1→B3, B2→B4 and B3→B4. Optimisation inside one block is simple, because there is only one path through it: if a block computes `a + b` twice and neither operand is redefined in between, the second computation can reuse the result of the first.

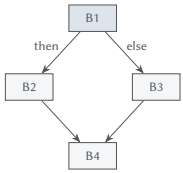


Figure 10.4: The diamond CFG of an `if/else`.

Definition 10.16 (Local optimization). A *local optimization* considers one basic block at a time, with no information about the paths that lead into the block or out of it.

10.3.3 Finding the basic blocks

The blocks of a list of instructions follow from its leaders.

Definition 10.17 (Leader). A *leader* is an instruction that starts a basic block. Three rules find all leaders: (1) the first instruction is a leader; (2) every target of a conditional or unconditional jump is a leader; (3) every instruction that immediately follows a jump is a leader. Each basic block then runs from one leader up to, and not including, the next leader.

The reconvergence point of a divergent branch (Lesson 6) is always a leader: it is the first instruction of the join block, a branch target.

Example 10.18 (Basic blocks of a PTX loop). The loop below sums $0 + 1 + \cdots + (n-1)$, with i in `%r1`, the sum s in `%r2` and n in `%r3`.

```
      mov.u32  %r1, 0;              // (1) i = 0
      mov.u32  %r2, 0;              //     s = 0
loop: setp.hs.u32 %p1, %r1, %r3;    // (2) i >= n (unsigned)
@%p1  bra      done;
      add.u32  %r2, %r2, %r1;       // (3) s = s + i
      add.u32  %r1, %r1, 1;         //     i = i + 1
      bra      loop;
done: st.global.u32 [%rd1], %r2;    // (2),(3)
```

The comments mark the rules that make an instruction a leader. The first `mov` is a leader by rule 1. The `setp` at `loop` is the target of `bra loop` (rule 2). The first `add` follows the conditional `@%p1 bra done` (rule 3). The store at `done` is both a target and the instruction after `bra loop`. The four leaders give four blocks: B1 (the two `mov`s), B2 (`setp` and the conditional branch), B3 (the two `add`s and `bra loop`) and B4 (the store). Figure 10.5 shows the CFG: B1→B2; B2→B3 by falling through and B2→B4 when the branch is taken; and B3→B2, the edge that closes the loop.

Caution A guarded branch `@%p1 bra` is a conditional jump: the instruction after it is a leader even though it is also reached by simply falling through.

An edge such as B3→B2, which leads back to a block that starts the loop, is called a *back edge*.

The edge from the end of a loop body to the loop test is a back edge; it is what makes the data-flow iteration of Section 10.5 need more than one pass.

10.4 Introduction to data-flow analysis

Many optimisations need facts that span several basic blocks.

Source M10, introduction to data-flow analysis; Aho et al. §9.1–9.2 [70].

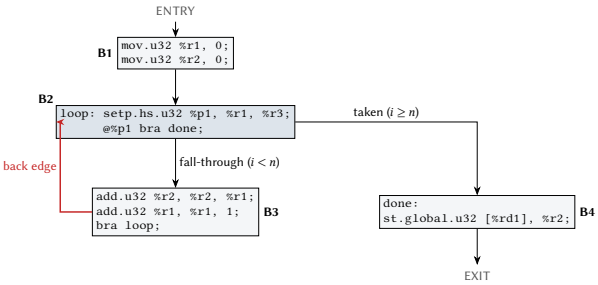


Figure 10.5: The basic blocks and the CFG of the loop in Example 10.18. The edge B3→B2 returns to an earlier block and closes the loop.

Definition 10.19 (Global optimization). A *global optimization* uses information collected across the basic blocks of a whole function, taking into account all paths of its CFG.

Every optimisation, local or global, must preserve the semantics of the original program: for every input the optimised program produces the same observable results. The common global optimisations, all treated in Lesson 11, are the removal of redundant instructions (an expression computed twice, on every path, with the same operands), constant and *copy propagation* (replacing a variable by the constant or the other variable it was assigned from, which often exposes further redundancy), dead code elimination (removing instructions whose results are never used), code motion (moving an instruction to where it executes less often, e.g. out of a loop), and the detection of induction variables with strength reduction. None of them can be decided inside one block: whether x in some block is still the constant assigned three blocks earlier depends on every path in between.

Caution Floating point bends this rule by default: nvcc contracts a multiply followed by an add into one fused multiply-add, which rounds once instead of twice, so results can differ in the last bit; `--fmad=false` turns the contraction off (NVCC §4.2.7.11 [65]).

Copy propagation in one line: after `y = x`, a later `z = y + 1` becomes `z = x + 1`; if no other use of y remains, `y = x` is dead.

Definition 10.20 (Data-flow analysis). The family of techniques called *data-flow analysis* computes, for every point of a program, a fact about all executions that can reach that point, by analysing the effect of each basic block and propagating the results along the edges of the CFG. The analyses differ in the property they examine.

10.4.1 The abstraction: points, states and transfer functions

Data-flow analysis describes a program by the states between its statements.

Definition 10.21 (Program point, transfer function). A *program point* lies before or after a statement. The analysis attaches to each point a state, the set of facts that hold there. For a block B, $\mathrm{IN}[B]$ is the state at the point before its first statement and $\mathrm{OUT}[B]$ the state after its last. The *transfer function* f_s of a statement s maps the state before s to the state after it.

Inside a block, the point after one statement is the point before the next (Figure 10.6), so the transfer function of a block of statements $s_1, \dots, s_n$ is the composition

$$\mathrm{OUT}[B] = f_B(\mathrm{IN}[B]), \qquad f_B = f_{s_n} \circ \cdots \circ f_{s_2} \circ f_{s_1}. \tag{10.1}$$

Between blocks, the state on entry to a block must combine the states of all its predecessors, since control can arrive from any of them.

Definition 10.22 (Meet operator, forward analysis). The *meet operator* combines the states that arrive at a join from several edges into one state. In a *forward analysis* the states flow in the direction of control: the equations are

$$\mathrm{IN}[B] = \bigcup_{P \in \mathrm{pred}(B)} \mathrm{OUT}[P], \qquad \mathrm{OUT}[B] = f_B(\mathrm{IN}[B]). \tag{10.2}$$

Liveness, the backward analysis behind register allocation, is the subject of Lesson 11. Both analyses of this course use union as the meet.

A backward problem runs the other way, from the uses of a value back to where it was computed: $\mathrm{IN}[B] = f_B(\mathrm{OUT}[B])$ and $\mathrm{OUT}[B] = \bigcup_{S \in \mathrm{succ}(B)} \mathrm{IN}[S]$. In the CFG of Figure 10.7 below, B5 has the predecessors B3 and B4, so in a forward analysis $\mathrm{IN[B5]} = \mathrm{OUT[B3]} \cup \mathrm{OUT[B4]}$; B5 has the successors B2 and EXIT, so in a backward analysis $\mathrm{OUT[B5]} = \mathrm{IN[B2]} \cup \mathrm{IN[EXIT]}$.

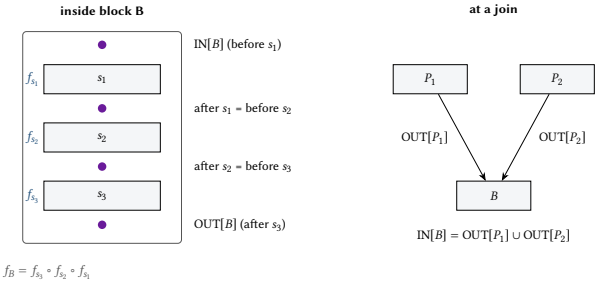


Figure 10.6: Left: the program points of a block of three statements; each statement's transfer function maps the state before it to the state after it. Right: at a join, the meet (here the union) combines the states of the predecessors.

10.4.2 Reaching definitions

The first analysis concerns where values come from.

Definition 10.23 (Definition, use). A *definition* of a variable is a statement that assigns it a value; a *use* is a statement that reads it. In `a = x + y` the statement defines `a` and uses `x` and `y`.

Definition 10.24 (Reaching definitions). A definition d *reaches* a point p if there is a path from the point immediately after d to p along which d is not killed, that is, along which no other definition of the same variable overwrites it. The analysis of *reaching definitions* computes, for every program point, the set of definitions that reach it.

Caution One path suffices: a definition reaches p if it survives on *some* path to p, not necessarily on all of them. The analysis is conservative in this direction, since it may list a definition whose path is never taken at run time.

If exactly one definition of `x`, say `x = 4`, reaches a use of `x`, the use can be replaced by the constant 4; if a second definition also reaches it, it cannot. This is how constant propagation uses the analysis.

Definition 10.25 (Gen and kill sets). For a definition d: `u = v + w`, the *gen set* is $\mathrm{gen}_d = \{d\}$, the definition the statement creates, and the *kill set* kill_d is the set of all other definitions of `u` in the program, which the statement overwrites. The transfer function of d is

$$f_d(x) = \mathrm{gen}_d \cup (x - \mathrm{kill}_d). \tag{10.3}$$

Notation: $x - y$ is set difference, the elements of x not in y. x stands for the set of definitions that reach the point before the statement.

A definition that arrives at d survives unless d kills it, and d itself is added. For a block, the functions compose as in Equation (10.1). Two statements give

$$\begin{aligned} f_2(f_1(x)) &= \mathrm{gen}_2 \cup \big((\mathrm{gen}_1 \cup (x - \mathrm{kill}_1)) - \mathrm{kill}_2\big) \\ &= \big(\mathrm{gen}_2 \cup (\mathrm{gen}_1 - \mathrm{kill}_2)\big) \cup \big(x - (\mathrm{kill}_1 \cup \mathrm{kill}_2)\big), \end{aligned}$$

which has the same form as Equation (10.3). By induction, a block of n statements has the transfer function $f_B(x) = \mathrm{gen}_B \cup (x - \mathrm{kill}_B)$ with

$$\begin{aligned} \mathrm{kill}_B &= \mathrm{kill}_1 \cup \mathrm{kill}_2 \cup \cdots \cup \mathrm{kill}_n, \\ \mathrm{gen}_B &= \mathrm{gen}_n \cup (\mathrm{gen}_{n-1} - \mathrm{kill}_n) \cup (\mathrm{gen}_{n-2} - \mathrm{kill}_{n-1} - \mathrm{kill}_n) \\ &\quad \cup \cdots \cup (\mathrm{gen}_1 - \mathrm{kill}_2 - \cdots - \mathrm{kill}_n). \end{aligned} \tag{10.4}$$

A block kills every definition that one of its statements kills, and generates every definition of its own that no later statement in the block overwrites.

Tip Read a block bottom-up to find gen_B: a definition belongs to it unless a later statement of the block defines the same variable again.

Example 10.26 (Gen and kill sets of a CFG). The CFG of Figure 10.7 sums the absolute values of `x[0]` to `x[n-1]` into `s` (for $n \geq 1$; the test is at the bottom of the loop). It has seven definitions: `i` is defined by d_1 and d_7, `s` by d_2, d_4 and d_6, `t` by d_3 and d_5. Block B4 holds two statements: d_5: `t = -t` with $\mathrm{gen} = \{d_5\}$, $\mathrm{kill} = \{d_3\}$, and d_6: `s = s + t` with $\mathrm{gen} = \{d_6\}$, $\mathrm{kill} = \{d_2, d_4\}$. By Equation (10.4), $\mathrm{kill}_{B4} = \{d_2, d_3, d_4\}$ and $\mathrm{gen}_{B4} = \{d_6\} \cup (\{d_5\} - \{d_2, d_4\}) = \{d_5, d_6\}$. Table 10.1 lists all five blocks.

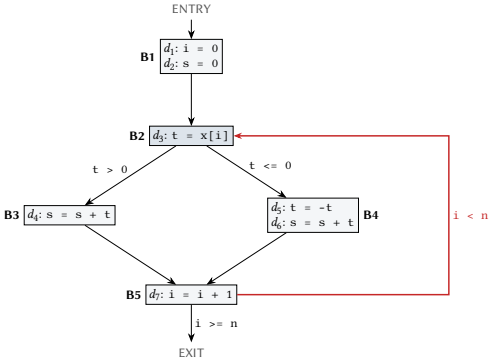


Figure 10.7: The CFG of the reaching-definitions example: a loop that adds `t` when it is positive and `−t` otherwise. The edge B5→B2 is the back edge.

Table 10.1: Gen and kill sets of the blocks of Figure 10.7.

Block	gen_B	kill_B
B1	$\{d_1, d_2\}$	$\{d_4, d_6, d_7\}$
B2	$\{d_3\}$	$\{d_5\}$
B3	$\{d_4\}$	$\{d_2, d_6\}$
B4	$\{d_5, d_6\}$	$\{d_2, d_3, d_4\}$
B5	$\{d_7\}$	$\{d_1\}$

10.5 An example of reaching definitions

Source M10, example of reaching definitions; Aho et al. §9.2.4 [70].

With the gen and kill sets of every block, the reaching definitions follow from a system of equations over the whole CFG.

10.5.1 Equations and the iterative algorithm

For every block B other than ENTRY, reaching definitions is the forward problem of Equation (10.2) with union as the meet:

$$\text{IN}[B] = \bigcup_{P \in \text{pred}(B)} \text{OUT}[P], \qquad \text{OUT}[B] = \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B), \tag{10.5}$$

The exit node has no statements, so OUT[EXIT] = IN[EXIT], the definitions that reach the end of the function.

together with the boundary condition $\text{OUT}[\text{ENTRY}] = \emptyset$: no definition reaches the start of the function. With a loop in the CFG the equations are circular: IN[B2] depends on OUT[B5], which depends on IN[B2]. They are solved by iteration.

Definition 10.27 (Iterative algorithm, fixed point). The *iterative algorithm* for a data-flow problem starts from an initial value for every state and evaluates the equations for all blocks in turn, pass after pass, until a whole pass changes no state. The states are then a solution of the equations, a *fixed point* of the passes: applying the equations once more leaves them unchanged.

For reaching definitions the algorithm reads:

```
OUT[ENTRY] = ∅
for each block B ≠ ENTRY:   OUT[B] = ∅
while some OUT changed in the last pass:
    for each block B ≠ ENTRY:
        IN[B] = ⋃_{P∈pred(B)} OUT[P]
        OUT[B] = gen_B ∪ (IN[B] − kill_B)
```

Caution The order in which a pass visits the blocks changes the number of passes, not the result. Visiting predecessors before successors, as in the example below, lets the information travel far in one pass; only back edges then require another.

The algorithm terminates. Every OUT set starts empty, and since Equation (10.5) only adds definitions when its inputs grow, no set ever shrinks; the sets are subsets of the finite set of definitions, so they can grow only finitely often. Starting from empty sets yields the smallest solution, which contains only definitions that really have a path to the point.

Table 10.2: Reaching definitions for Figure 10.7, pass by pass. Bit k stands for d_k; the superscript is the pass. A third pass reproduces pass 2.

Block	OUT^0	IN^1	OUT^1	IN^2	OUT^2
B1	0 0 0 0 0 0 0	0 0 0 0 0 0 0	1 1 0 0 0 0 0	0 0 0 0 0 0 0	1 1 0 0 0 0 0
B2	0 0 0 0 0 0 0	1 1 0 0 0 0 0	1 1 1 0 0 0 0	1 1 1 1 1 1 1	1 1 1 1 0 1 1
B3	0 0 0 0 0 0 0	1 1 1 0 0 0 0	1 0 1 1 0 0 0	1 1 1 1 0 1 1	1 0 1 1 0 0 1
B4	0 0 0 0 0 0 0	1 1 1 0 0 0 0	1 0 0 0 1 1 0	1 1 1 1 0 1 1	1 0 0 0 1 1 1
B5	0 0 0 0 0 0 0	1 0 1 1 1 1 0	0 0 1 1 1 1 1	1 0 1 1 1 1 1	0 0 1 1 1 1 1
EXIT	0 0 0 0 0 0 0	0 0 1 1 1 1 1	0 0 1 1 1 1 1	0 0 1 1 1 1 1	0 0 1 1 1 1 1

Definition 10.28 (Bit vector). A *bit vector* represents a subset of a fixed, numbered set of N elements as N bits, bit k being 1 when element k is in the subset. Union becomes bitwise OR, intersection bitwise AND and $x - y$ becomes x AND NOT y.

With the definitions numbered $d_1, \ldots, d_N$, each set of the algorithm is one bit vector, and a whole transfer function is two bitwise operations per machine word.

The seven definitions of the example fit into one machine word; a function with 1000 definitions needs 16 words of 64 bits per set.

10.5.2 The example worked out

Table 10.2 applies the algorithm to the CFG of Figure 10.7, visiting B1 to B5 and EXIT in this order. Each bit vector lists $d_1, \ldots, d_7$ grouped by the block that contains the definition (B1: $d_1 d_2$; B2: d_3; B3: d_4; B4: $d_5 d_6$; B5: d_7).

Example 10.29 (The first pass). B1 has only ENTRY as predecessor, so IN[B1] $= \varnothing$ and OUT[B1] $= \text{gen}_{B1} = \{d_1, d_2\}$. B2 has the predecessors B1 and B5, and OUT[B5] is still empty, so IN[B2] $= \{d_1, d_2\}$ and OUT[B2] $= \{d_3\} \cup (\{d_1, d_2\} - \{d_5\}) = \{d_1, d_2, d_3\}$. B3 kills d_2 and adds d_4: $\{d_1, d_3, d_4\}$. B4 kills d_2 and d_3 and adds d_5, d_6: $\{d_1, d_5, d_6\}$. At the join B5 the union of the two is $\{d_1, d_3, d_4, d_5, d_6\}$, and B5 replaces d_1 by d_7: OUT[B5] $= \{d_3, d_4, d_5, d_6, d_7\}$, which also reaches EXIT.

Example 10.30 (The second and third passes). In pass 2, OUT[B5] is no longer empty, and the back edge brings its definitions into B2: IN[B2] $= \{d_1, d_2\} \cup \{d_3, \ldots, d_7\}$, all seven. B2 kills only d_5, so OUT[B2] gains d_4, d_6 and d_7, and so do the IN sets of B3 and B4. Of these, B3 and B4 pass on only d_7, since each kills or regenerates the other two, and at B5 only d_7 is new, which B5 generates anyway: OUT[B5] is unchanged. Because B2, B3 and B4 changed, a third pass is run; its IN[B2] is the same as in pass 2, so no set changes, and the states of pass 2 are the fixed point.

Only d_3 among the definitions of `t` reaches B3 and B4: d_5 arrives at B2 through the back edge but is killed there. Each use of `t` in B3 and B4 thus reads the value loaded in the same iteration.

The solution answers questions an optimizer asks. Both definitions of `i`, d_1 and d_7, reach the use `x[i]` in B2, so `i` cannot be replaced there by the constant 0; without the back edge only d_1 would reach it. The initialisation d_2 (`s = 0`) reaches B2 but not B5: both paths through the loop body redefine `s`. The definitions that reach EXIT, d_3 to d_7, are the ones whose values the function can still observe at its end.

Key point — Lesson summary

A compiler consists of a front end that builds an AST and an IR, an optimizer that rewrites the IR, and a back end that generates code. For CUDA, nvcc splits host from device code; the device code becomes PTX through cicc (or Clang and the LLVM NVPTX back end), PTX becomes SASS through ptxas, and a fat binary built by nvcc carries both, so that the driver can JIT-compile the PTX for GPUs that did not exist at build time. PTX is a virtual ISA with typed instructions of zero to four operands, unlimited virtual registers and guard predicates set by `setp`. Optimisers work on three-address code divided into basic blocks, found with the three leader rules, and connected into a control-flow graph. Global optimisations rest on data-flow analysis: transfer functions per block, a meet at joins, and an iterative algorithm that reaches a fixed point. Reaching definitions is the forward analysis with $\mathrm{OUT}[B] = \mathrm{gen}_B \cup (\mathrm{IN}[B] - \mathrm{kill}_B)$ and $\mathrm{IN}[B]$ the union over the predecessors, computed efficiently with bit vectors; it is fixed by its domain, direction, transfer function, meet, boundary condition and initial value, the six items that Lesson 11 fills in for liveness.

Compiler Background - II

Lecture	M11, lessons 1–4 (Live-Variable Analysis; SSA; Examples of Compiler Optimizations; Divergence Analysis)
Sources	Cytron et al. [71] (SSA construction with dominance frontiers); Chaitin [72] (register allocation by graph colouring); LLVM Language Reference [69] (`phi`, `select`); LLVM passes [73] (`mem2reg`); PTX ISA [19]
Textbook	Aho et al. [70] §6.2 Three-Address Code, §8.8 Register Allocation and Assignment, §9.1 The Principal Sources of Optimization, §9.2 Introduction to Data-Flow Analysis, §9.6 Loops in Flow Graphs; CPG [5] §5.2.3 Multiprocessor Level, §5.3.2 Device Memory Accesses, §5.4.2 Control Flow Instructions, §7.1.5 __noinline__ and __forceinline__, §7.39 #pragma unroll; PMPP [6] ch. 10
Keywords	live variable, liveness analysis, backward analysis, def and use sets, register allocation, interference graph, register spill, static single assignment, phi function, path-convergence criterion, dominance frontier, loop unrolling, function inlining, dead code elimination, constant propagation, strength reduction, loop-invariant code motion, induction variable, divergence analysis, thread-dependent and uniform values

Lesson 10 followed a CUDA program through the compiler to PTX, cut the PTX into basic blocks and a control-flow graph, and set up data-flow analysis with reaching definitions, a forward problem. This lesson completes the toolkit with its backward counterpart, live-variable analysis, and with its most important client, register allocation, which on a GPU decides how many warps fit on an SM. It then presents static single assignment form, the representation in which LLVM, and with it NVIDIA's LLVM-based NVVM compiler, carries a program, and the classic optimisations that the analyses enable: loop unrolling, inlining, dead code elimination, constant propagation, strength reduction and code motion. The last section turns to a question that only parallel code poses: which branches can diverge. It connects the hardware of Lesson 6, the SIMT stack and its reconvergence points, to an analysis the compiler performs on the def-use chains of the thread index.

11.1 Live-variable analysis

Reaching definitions asks, for each program point, which assignments may have produced the values seen there. Liveness asks the opposite question: which of the values present at a point will still be needed later.

Source M11, liveness; Aho et al. §9.2 Introduction to Data-Flow Analysis [70].

Definition 11.1 (Live variable). A variable x is a *live variable* at a program point p if the value of x at p may be used along some path in the control-flow graph that starts at p, before x is redefined on that path; otherwise x is *dead* at p. The data-flow analysis that computes, for every point, the set of live variables is *liveness analysis*.

Lesson 6 used the same notion for the hardware: a live register holds a value that will still be read. Liveness analysis computes it at compile time for every variable and every point.

A dead value occupies a register for nothing. The main use of liveness is therefore register allocation (Section 11.1.2): only live variables need registers, and the register of a variable that has died can be handed to another. The same information tells which assignments are useless, the basis of the dead code elimination of Section 11.3.

11.1.1 Data-flow equations

The effect of one basic block B on liveness is summarised by two sets.

Definition 11.2 (Def and use sets). The *def set* def_B of a block B is the set of variables that B defines before any use of them in B. The *use set* use_B is the set of variables that B may use before any definition of them in B.

Caution An assignment such as `k = k - 1` reads `k` before writing it, so `k` belongs to use_B and, by the definition, not to def_B. Listing it in both sets, as is sometimes done, gives the same IN set, because a variable in use_B is live on entry anyway.

A variable in use_B is live on entry to B whatever happens after B, since B itself reads it. A variable in def_B is dead on entry to B unless B reads it first, since B overwrites it. Every other variable passes through B unchanged: it is live on entry exactly when it is live on exit. With

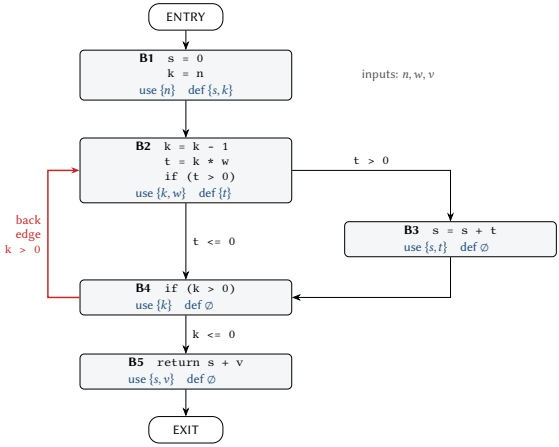


Figure 11.1: The running example of this section: a loop over k with a branch on t. Each block lists its code and its use and def sets; n, w and v are inputs.

IN[B] and OUT[B] the sets of live variables just before and just after B, this gives the equations

$$\text{IN}[\text{EXIT}] = \emptyset, \tag{11.1}$$

$$\text{IN}[B] = use_B \cup \big(\text{OUT}[B] - def_B\big), \tag{11.2}$$

$$\text{OUT}[B] = \bigcup_{S \in \text{succ}(B)} \text{IN}[S]. \tag{11.3}$$

Notation as in Lesson 10: succ(B) is the set of successors of B, IN and OUT the values at the program points before and after the block.

Equation (11.1) is the boundary condition: no variable is live when the program ends. Equation (11.2) is the transfer function of the block, Equation (11.3) the meet: a variable is live on exit from B if and only if it is live on entry to at least one successor, since some path continues through that successor.

Definition 11.3 (Backward analysis). A data-flow problem in which information flows against the edges of the control-flow graph, the IN set of a block being computed from its OUT set and the OUT set from the IN sets of the successors, is a *backward analysis*. Liveness is backward because whether a value is needed depends on what the program does *later*; reaching definitions is forward because whether a definition arrives depends on what happened *earlier*.

The equations are solved by the same iterative method as reaching definitions, run in the other direction. Every IN set starts empty (Equation (11.1) and the initial guess for every other block); then the blocks are visited repeatedly, each time computing OUT[B] from the current IN sets of the successors and IN[B] from OUT[B], until a whole pass changes no IN set. The sets only grow, and there are finitely many variables, so the method reaches a fixed point; it is the smallest solution of the equations, which is the precise one.

Tip Visit the blocks in reverse order (exit first) for a backward problem: most values are then already up to date when they are read, and fewer passes are needed. The result does not depend on the order, only the work does.

Example 11.4. Figure 11.1 shows a loop that counts k down from n and adds $t = k \cdot w$ to s whenever t is positive; the result is $s + v$. The blocks are visited in the order B5, B4, B3, B2, B1, and Table 11.1 gives the sets after the first and second passes.

- *Pass 1.* OUT[B5] = IN[EXIT] = $\emptyset$, so IN[B5] = $\{s, v\}$. B4 has the successors B2 and B5; IN[B2] is still empty, so OUT[B4] = $\{s, v\}$ and IN[B4] = $\{k\} \cup \{s, v\} = \{k, s, v\}$. Then IN[B3] = $\{s, t\} \cup \{k, s, v\} = \{k, s, t, v\}$. OUT[B2] is the union of IN[B3] and IN[B4], $\{k, s, t, v\}$; removing $def = \{t\}$ and adding $use = \{k, w\}$ gives IN[B2] = $\{k, s, v, w\}$. Finally IN[B1] = $\{n\} \cup (\{k, s, v, w\} - \{s, k\}) = \{n, v, w\}$.
- *Pass 2.* The back edge now carries information: IN[B2] is no longer empty, so OUT[B4] = $\{k, s, v, w\}$ and w enters IN[B4] and IN[B3]. OUT[B2] gains w, but w was already in IN[B2], so IN[B2] and everything above it are unchanged.
- *Pass 3* recomputes every set and changes nothing: the fixed point is reached.

The result is what the code suggests: the inputs w and v are live throughout the loop because they are read in every iteration and at the end; n is live only on entry to B1; and t is live only on the edge from B2 to B3, so its register is free again in B4.

Our numbers: five variables of the loop body live at once at most (k, s, t, v, w on the edge B2→B3), so the loop needs five registers before any spilling; n can share one of them, being dead once B1 has copied it into k.

Block	use_B	def_B	OUT^1	IN^1	OUT^2	IN^2
B5	$\{s, v\}$	$\emptyset$	$\emptyset$	$\{s, v\}$	$\emptyset$	$\{s, v\}$
B4	$\{k\}$	$\emptyset$	$\{s, v\}$	$\{k, s, v\}$	$\{k, s, v, w\}$	$\{k, s, v, w\}$
B3	$\{s, t\}$	$\emptyset$	$\{k, s, v\}$	$\{k, s, t, v\}$	$\{k, s, v, w\}$	$\{k, s, t, v, w\}$
B2	$\{k, w\}$	$\{t\}$	$\{k, s, t, v\}$	$\{k, s, v, w\}$	$\{k, s, t, v, w\}$	$\{k, s, v, w\}$
B1	$\{n\}$	$\{s, k\}$	$\{k, s, v, w\}$	$\{n, v, w\}$	$\{k, s, v, w\}$	$\{n, v, w\}$

Table 11.1: Liveness for Figure 11.1, blocks in the order visited. Superscripts number the passes; the third pass repeats the second.

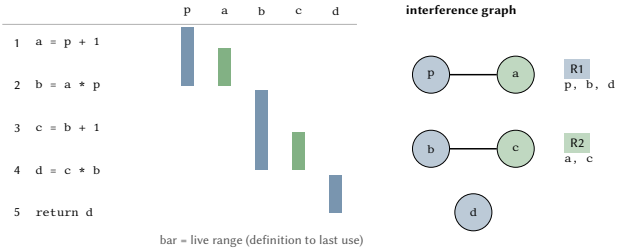


Figure 11.2: Liveness drives register allocation. Left: the live range of each value of a five-instruction sequence. Right: the interference graph; two colours suffice, so the five values fit into two registers.

11.1.2 Register allocation

Definition 11.5 (Register allocation). The compiler pass that maps the values of a program onto the finite set of machine registers is *register allocation*. Two values may share a register if they are never live at the same point. The *interference graph* has one node per value and an edge between two values that are live at the same time; an assignment of registers is a colouring of this graph in which adjacent nodes get different colours.

Graph-colouring register allocation, with spilling when the colouring fails, goes back to Chaitin [72]; Aho et al. §8.8 [70] presents it as the standard global method.

Figure 11.2 applies this to a straight-line sequence. After instruction 2 neither p nor a is read again, so the register of either can receive b; after instruction 4 neither b nor c is read again, so the register of either can receive d. Only two pairs of values are ever live together, p with a and b with c, so two registers serve five values.

The same reasoning on a branching program uses the IN and OUT sets of the blocks. In Figure 11.1, t is not in OUT[B4]: of the two successors of B4, B2 redefines t before reading it and B5 never reads it, so t is dead on the back edge; its register is free for other values from the last read of t until B2 writes it again. Conversely, B2 and B4 never read s, yet s is in their IN and OUT sets (Table 11.1): B3 and B5 read it later, so s stays live on every path to those reads and keeps its register throughout the loop.

When more values are live at some point than there are registers, the colouring fails and some values must leave the register file.

Definition 11.6 (Spill and fill). A *register spill* stores a live value from a register to memory, usually a stack slot, to free the register; the later load that brings the value back into a register before its next use is a *fill (reload)*.

PTX has no register allocation: its registers are virtual and unlimited in number, and the PTX that the front end emits contains no spills. Registers are allocated by `ptxas` when it translates PTX into SASS for a real architecture, and the spills it inserts go to the thread's local memory (Lesson 3). CPG §5.3.2 lists "any variable if the kernel uses more registers than available (this is also known as register spilling)" among the variables placed in local memory [5]. Local memory is private to a thread but resides in device memory, so a fill that misses in the L1 cache costs as much as a global memory access.

A PTX kernel declares as many virtual registers as it needs, e.g. `.reg .b32 %r<100>;` for `%r0` to `%r99` (PTX ISA, Parameterized Variable Names [19]).

On a GPU the allocator balances two costs. Fewer registers per thread mean more resident warps, and so more latency hiding (Lesson 3); more registers mean fewer spills. A single register can matter: on an SM with 65,536 registers, two blocks of 512 threads fit at 64 registers per thread, but only one at 65 (CPG §5.2.3 Multiprocessor Level [5]). The limit is 255 registers per thread (CPG Table 21 [5]), and the programmer can lower it with the `-maxrregcount` option or the `__launch_bounds__()` qualifier, accepting spills for occupancy.

Tip Compile with `--ptxas-options=-v`: the compiler then reports, per kernel, its registers and its local memory use (`1mem`, CPG §5.3.2 [5]); `ptxas` also prints the bytes of spill stores and loads, and nonzero values mean the allocator ran out of registers.

	Reaching definitions (Lesson 10)	Live variables
Domain	sets of definitions	sets of variables
Direction	forward	backward
Transfer function	$gen_B \cup (x - kill_B)$	$use_B \cup (x - def_B)$
Meet	$\cup$ over predecessors	$\cup$ over successors
Equations	$\mathrm{OUT}[B] = f_B(\mathrm{IN}[B])$, $\mathrm{IN}[B] = \bigcup_P \mathrm{OUT}[P]$	$\mathrm{IN}[B] = f_B(\mathrm{OUT}[B])$, $\mathrm{OUT}[B] = \bigcup_S \mathrm{IN}[S]$
Boundary	$\mathrm{OUT}[\mathrm{ENTRY}] = \varnothing$	$\mathrm{IN}[\mathrm{EXIT}] = \varnothing$
Initialisation	$\mathrm{OUT}[B] = \varnothing$	$\mathrm{IN}[B] = \varnothing$

Table 11.2: Reaching definitions and liveness side by side.

11.1.3 Liveness and reaching definitions compared

The two analyses are the standard pair of data-flow problems: the same iterative solver, different sets and opposite directions (Table 11.2).

> **Key point**
> Liveness runs backward from $\mathrm{IN}[\mathrm{EXIT}] = \varnothing$ with $\mathrm{IN}[B] = use_B \cup (\mathrm{OUT}[B] - def_B)$ and $\mathrm{OUT}[B]$ the union over the successors. Values that are never live together can share a register; when too many are live at once, `ptxas` spills some to local memory.

11.2 Static single assignment

In ordinary three-address code a variable may be assigned many times, so every analysis must first find out which assignment a use refers to, which is the question reaching definitions answers. The link from a definition to the uses it reaches is a *def-use chain*. Static single assignment form builds the chains into the names of the program.

Source M11, SSA; Aho et al. §6.2 Three-Address Code [70]; Cytron et al. [71].

Definition 11.7 (Static single assignment). A program is in *SSA form* when every variable is assigned at exactly one place in the program text. The property is called *static single assignment (SSA)*: *static* because the single assignment may still execute many times, in a loop. A program is converted by giving every definition of a variable a new version, `a1`, `a2`, ..., and renaming every use to the version that reaches it.

In SSA form a use names its definition, so a def-use chain is a direct link and does not need a separate analysis. Constant propagation, dead code elimination and most other optimisations become simpler, which is why SSA is the basis of most compiler intermediate representations.[1]

LLVM IR is in SSA form for all its virtual registers (LLVM Language Reference [69]); the NVPTX back end of Lesson 10 receives the program in that form.
[1] LLVM front ends do not build SSA themselves. Clang emits every local variable as a stack slot (`alloca`) with loads and stores, and the `mem2reg` pass promotes such slots to SSA registers, inserting ϕ functions with the standard algorithm based on the dominance frontiers introduced below to obtain pruned SSA (LLVM passes [73]).

11.2.1 Joins and phi functions

Renaming is straightforward in straight-line code. At a block with several predecessors it is not: in Figure 11.3 the use of `s` in BB4 is reached by `s1` along BB2 and by `s2` along BB3, and no single name is correct for both paths.

Definition 11.8 (Join point and phi function). A *join point* is a block with more than one predecessor. A *phi function* placed at the start of a join point, $x_3 = \phi(x_1, x_2)$, has one argument per predecessor and defines a new version x_3 whose value is the argument of the edge along which control arrived. After the ϕ, every later use refers to x_3.

In Figure 11.3, `s3 = φ(s1, s2)` is `s1` when control comes from BB2 and `s2` when it comes from BB3; `t3` merges `t2` (BB2) and `t1` (BB3). The use in `u1 = s3 + t3` then has exactly one reaching definition per name. A ϕ is not an instruction of any machine. When the program leaves SSA form, it becomes a move on each incoming edge, or, when the two paths are short, a select or a *conditional move* that picks one of two registers by a predicate.

The GPU counterpart of the conditional move is the `selp` instruction of PTX, which picks one of two source operands by a predicate operand (PTX ISA [19]); unlike the guarded `@p` execution of Lesson 6 it is not itself predicated.

LLVM writes the ϕ with the predecessor label beside each value. In SSA form as `mem2reg` leaves it, before any further optimisation, the function `min(a, b)` keeps its branch and merges the two results in the block `end`:

```
define i32 @min(i32 %a, i32 %b) {
entry:
  %lt = icmp slt i32 %a, %b
  br i1 %lt, label %ta, label %tb
ta:
  br label %end
tb:
  br label %end
```

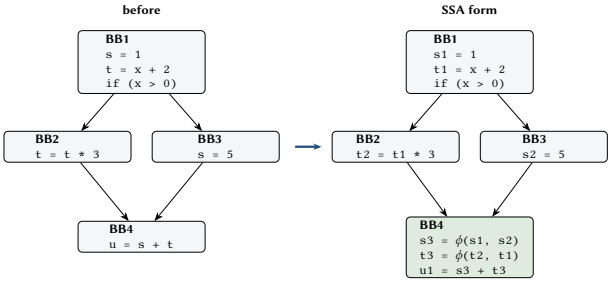


Figure 11.3: Conversion to SSA form. Each definition gets a new version; at the join BB4, ϕ functions merge the versions of `s` and `t` that arrive from BB2 (first argument) and BB3 (second argument).

```
end:
  %r = phi i32 [ %a, %ta ], [ %b, %tb ]
  ret i32 %r
}
```

SimplifyCFG replaces this diamond by one `select i1 %lt, i32 %a, i32 %b`, the conditional move of the IR (LLVM Language Reference [69]); InstCombine then canonicalises this min pattern to `@llvm.smin.i32`.

When the target is lowered, `%r` becomes either a register written on both incoming edges or a single conditional move.

11.2.2 Where to place phi functions

The simplest placement inserts a ϕ for every variable at every join point. It is correct but wasteful: in Figure 11.3 the variable `u` is defined only in BB4 and `x` is never redefined, so a ϕ for either would merge one value with itself. A ϕ is needed only where different definitions actually meet.

Definition 11.9 (Path-convergence criterion). The *path-convergence criterion* requires a ϕ for variable a at block z when all of the following hold:

1. a block x contains a definition of a;
2. a block $y \neq x$ contains a definition of a;
3. there is a nonempty path P_{xz} from x to z;
4. there is a nonempty path P_{yz} from y to z;
5. P_{xz} and P_{yz} have no node in common other than z;
6. z does not appear within both P_{xz} and P_{yz} before their ends, though it may appear in one of them.

The start node counts as containing a definition of every variable. Because each inserted ϕ is itself a new definition of a, the criterion is applied again until no further ϕ is required.

Caution Lecture vs paper: production compilers do not search paths. Cytron et al. place ϕ functions at the iterated dominance frontier of the defining blocks, in time that is linear in practice (quadratic in the worst case) [71]. *Pruned* SSA, which `mem2reg` builds, also drops ϕs whose result is never used (LLVM passes [73]).

Condition 5 is the one that saves ϕs. Two definitions whose paths already met at an earlier join were merged there; the ϕ at that earlier join is the single definition that flows on, and a second ϕ further down would be redundant. Condition 6 allows loops: a path may pass through z once before arriving there for the second time along a back edge. The start-node rule matters for a variable assigned on one branch only: if BB3 of Figure 11.3 also set `w = 7`, with no earlier assignment to `w`, the path from the start node through BB2 and the path from BB3 would first meet at BB4, so BB4 would receive `w2 = φ(w0, w1)`, whose first argument is the initial version `w0` of the start node.

Example 11.10. Figure 11.4 sums the integers below `n`. Before renaming, `i` and `s` are defined in B1 and in B3. The paths B1→B2 and B3→B2 share only B2, so both conditions 5 and 6 hold at B2 and each variable gets a ϕ there; its second argument comes from the back edge. At B4 every path from B1 and every path from B3 runs through B2, so condition 5 fails and B4 uses `s2` directly, although two definitions of `s` reach it. `n` is defined only at the start, so condition 2 fails and it needs no ϕ anywhere. The iteration adds nothing either: the new definitions in B2 meet the others only at B2 itself, which already has its ϕs.

The faster algorithm rests on dominance, the forward counterpart of the post-dominance of Lesson 6. A block d is a *dominator* of a block n if every path from the entry to n passes through d. The *dominance frontier* of d is the set of blocks z such that d dominates a predecessor of z

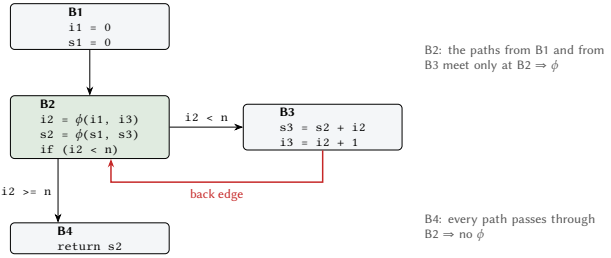


Figure 11.4: SSA form of a counting loop. The loop header B2 joins the entry path and the back edge, so `i` and `s` get ϕ functions there and nowhere else.

In Figure 11.4 B2 dominates B3 and B4; the dominance frontier of B3 is {B2}, exactly where its definitions need ϕs. Dominators: Aho et al. §9.6 [70].

but does not strictly dominate z itself: the first blocks at which paths from d meet paths that bypass d [71]. A definition in d needs ϕ functions exactly at the blocks of the frontier of d, then at the frontiers of those blocks, and so on, which is the result of the path-convergence criterion without enumerating paths.

> **Key point**
> SSA gives every assignment its own name, so each use has one definition. At a join point, ϕ functions merge the versions arriving along different edges; they are needed only where distinct definitions first meet, which the path-convergence criterion (or dominance frontiers) determines. A ϕ is lowered to moves or a conditional move.

11.3 Examples of compiler optimizations

Source M11, compiler optimizations; Aho et al. §9.1 The Principal Sources of Optimization [70]; CPG §7.39 #pragma unroll, §7.1.5 [5].

The analyses above exist to justify transformations. Each optimisation below must preserve the meaning of the program, and each needs some fact about it that a data-flow analysis or the SSA form provides. The examples are given in CUDA C++ or PTX; Table 11.3 summarises them.

11.3.1 Loop unrolling

Definition 11.11 (Loop unrolling). The transformation called *loop unrolling* replicates the body of a loop u times, the *unroll factor*, so that one iteration of the new loop does the work of u old iterations. When the trip count is unknown or not a multiple of u, an *epilogue loop* after the unrolled loop executes the remaining iterations one at a time.

```
// before
for (int i = 0; i < n; i++)
  y[i] = c * x[i];

// unrolled by 4, with an epilogue loop
int i = 0;
for (; i + 3 < n; i += 4) {
  y[i]     = c * x[i];
  y[i + 1] = c * x[i + 1];
  y[i + 2] = c * x[i + 2];
  y[i + 3] = c * x[i + 3];
}
for (; i < n; i++)          // 0 to 3 iterations
  y[i] = c * x[i];
```

"By default, the compiler unrolls small loops with a known trip count"; `#pragma unroll` with an optional factor controls any loop, and `#pragma unroll 1` forbids unrolling (CPG §7.39 [5]).

Caution An unroll factor that pushes a kernel over its register budget can cost more through lost occupancy or spills than it saves in branches.

Unrolling removes three of every four compare-and-branch sequences and increment instructions. More importantly, it puts four independent multiplications and memory accesses into one basic block, where the scheduler can interleave them to hide latency and a vectorising compiler can combine them. The cost is code size and, on a GPU, registers: the four loaded values are live at the same time.

11.3.2 Function inlining

Definition 11.12 (Function inlining). The transformation called *function inlining* replaces a call by a copy of the body of the called function, with the arguments substituted for the parameters. Deciding it needs facts about both caller and callee, so it is an *inter-procedural analysis* decision, as opposed to the intra-procedural analyses of the previous sections.

```
__device__ float sq(float v) { return v * v; }
__device__ float norm2(float a, float b) {
  return sq(a) + sq(b);
}
// after inlining both levels at a call norm2(x, y):
float r = x * x + y * y;
```

Inlining removes the call and return, the passing of arguments and the saving of registers on the stack. It also exposes the body to the optimisations of the caller: constants of the call site propagate into it, and code that is dead for these arguments disappears. The cost is code size, since every call site receives its own copy.

The CUDA compiler inlines a `__device__` function "when deemed appropriate"; `__noinline__` asks it not to and `__forceinline__` forces it (CPG §7.1.5 [5]).

11.3.3 Dead code elimination

Definition 11.13 (Dead code elimination). The optimisation called *dead code elimination* removes instructions whose results never influence the output of the program: an assignment to a variable that is dead immediately after it, and code that can never execute.

```
ld.param.f32  %f1, [x];
mul.f32       %f2, %f1, %f1;   // dead: %f2 is
add.f32       %f2, %f1, 1.0;   //   rewritten here
st.global.f32 [%rd1], %f2;
```

The product written by `mul.f32` is overwritten before any instruction reads it: `%f2` is dead after the `mul`, in the sense of Definition 11.1, so the instruction is removed. The argument is a liveness argument; in SSA form it becomes simpler still, because a definition with no uses is dead by inspection. Removing an instruction can make the definitions it read dead in turn, so the elimination is repeated.

Caution An instruction with a side effect (a store, an atomic, a barrier) is never dead merely because its result is unused; only pure computations are removed.

11.3.4 Constant propagation and folding

Definition 11.14 (Constant propagation and folding). The optimisation called *constant propagation* replaces a use of a variable by a constant when every definition reaching the use assigns that constant. The optimisation called *constant folding* evaluates at compile time an operation all of whose operands are constants.

```
// before
mov.f32  %f1, 2.0;
mov.f32  %f2, 4.0;
add.f32  %f3, %f1, %f2;
mul.f32  %f4, %f3, %f5;
// after propagation, folding, propagation
mul.f32  %f4, %f5, 6.0;
```

Propagating the two constants into the `add` makes it 2.0 + 4.0, which folding turns into the constant 6.0; propagating that constant into the `mul` leaves the two `mov` instructions and the `add` without uses, and dead code elimination removes them. One optimisation thus enables the next.

Constant propagation is a client of reaching definitions (Lesson 10): a use is constant if all definitions reaching it assign the same constant. Copy propagation, also from Lesson 10, works the same way for copies.

11.3.5 Strength reduction

Definition 11.15 (Strength reduction). The optimisation called *strength reduction* replaces an operation by an equivalent one that costs less: a division by a multiplication, a multiplication by a shift or an addition, a call to a power function by a multiplication.

The two classic examples are a division by two, which becomes a right shift, and a square, which becomes a multiplication of the value by itself. In PTX they are given as the following pairs, the square in the form of an `ex2.approx.f32`:

Optimisation	What it saves	What it needs to know
Loop unrolling	branches, counter updates; exposes parallelism	trip count, register budget
Function inlining	call overhead; enables other optimisations	callee body (inter-procedural)
Dead code elimination	instructions with unused results	liveness or SSA uses
Constant propagation and folding	run-time computation of constants	reaching definitions
Strength reduction	expensive operations	exact equivalence of the cheaper form
Loop-invariant code motion	repeated work in loops	reaching definitions; loop structure

Table 11.3: The optimisations of this section and the analyses behind them.

```
div.rn.f32      %f2, %f1, 2.0;   // becomes
mul.f32         %f2, %f1, 0.5;
ex2.approx.f32 %f3, %f1;         // becomes
mul.f32         %f3, %f1, %f1;
```

Caution Lecture vs ISA: both pairs are mislabelled. The first is a floating-point division turned into a multiplication, not a shift; a right shift replaces division only for unsigned or non-negative integers, since for a negative signed value it rounds toward $-\infty$ while division rounds toward zero. And `ex2` computes 2^x, not x^2 (PTX ISA [19]), so the second pair is not an equivalence; the correct square example is `pow(a, 2)` → `a * a`.

A floating-point division by two equals a multiplication by 0.5 exactly, because both scale by a power of two. The shift belongs to integers: an unsigned division by 2^k, such as `div.u32 %r2, %r1, 8`, becomes the right shift `shr.u32 %r2, %r1, 3`, while a signed one needs a correction for negative values. A square is reduced from a call: `pow(a, 2)` becomes `a * a`. Division by a constant that is not a power of two, $x/3$, cannot be replaced by $x \cdot (1/3)$ in floating point without changing some results, so the compiler does it only when the program allows approximate division, as `nvcc --prec-div=false` (implied by `-use_fast_math`) does for single precision. A divisor that is a power of two only at run time is beyond the compiler; Lesson 5 shows the rewrite by hand.

11.3.6 Loop-invariant code motion and induction variables

Definition 11.16 (Code motion). The transformation called *code motion* moves a computation to another place in the program where it executes less often with the same result. Its most common form, *loop-invariant code motion*, hoists a computation whose operands do not change inside a loop out of the loop, so that it executes once instead of once per iteration.

```
// before: scale is recomputed in every iteration
for (int i = 0; i < n; i++) {
  float scale = alpha / beta;
  y[i] += scale * x[i];
}
// after: computed once, before the loop
float scale = alpha / beta;
for (int i = 0; i < n; i++)
  y[i] += scale * x[i];
```

An instruction is loop invariant when each of its operands is a constant, is defined outside the loop, or is defined by another loop-invariant instruction; the test is applied repeatedly (Aho et al. §9.1 [70]).

The number of dynamic instructions falls by one division per iteration. To be allowed, the compiler must know that neither `alpha` nor `beta` is changed in the loop, a question for reaching definitions, and that executing the division when the loop runs zero times is harmless.

Definition 11.17 (Induction variable). An *induction variable* is a variable that changes by the same amount in every iteration of a loop, such as the counter `i` that controls it. In PTX its update is a single instruction such as `add.s32 %r4, %r4, 1;`.

Recognising induction variables enables further optimisations. The address of `x[i]` is $base + 4i$, itself an induction variable that grows by 4 per iteration, so strength reduction can replace the multiplication by an addition of 4 to a pointer; the trip count of the loop, needed for unrolling, follows from the initial value, step and bound of the counter.

11.4 Divergence analysis

Source M11, divergence analysis; CPG §5.4.2 Control Flow Instructions, §4.1 SIMT Architecture [5]; PMPP ch. 10 [6]. PTX can record the result: `bra.uni` marks a branch that is guaranteed to be non-divergent, all active threads of the warp having the same predicate and target (PTX ISA, `bra` [19]).

Lesson 6 showed how the hardware executes a divergent branch: the SIMT stack runs the paths one after the other under partial masks and joins them at the reconvergence point, normally the immediate post-dominator of the branch. The compiler supplies the reconvergence information, and it can omit the stack operations, or emit simpler code, for branches it proves warp-uniform. It therefore needs to know statically which branches may diverge.

Definition 11.18 (Divergence analysis). The static analysis called *divergence analysis* is one that determines, for each branch of a kernel, whether its condition may evaluate differently

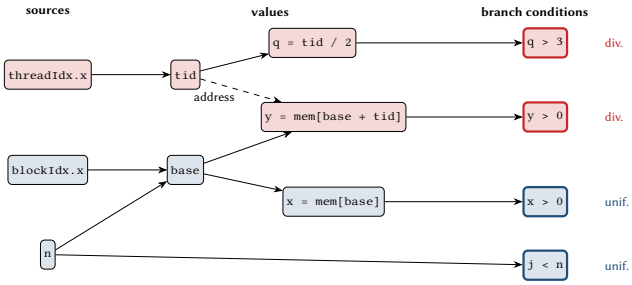


Figure 11.5: The def-use edges of the kernel above, from the sources to the branch conditions. Thread dependence (red) spreads from `threadIdx.x`; the condition of a branch decides whether it may diverge.

in different threads of a warp. Its output is the set of branches that may diverge and need reconvergence information; every other branch is uniform.

The question predates GPUs. A compiler that turns a loop into vector instructions, *vectorization*, must check that all iterations it packs together execute the same instructions in lockstep, including inside branches of the loop body; GPU compilers inherited these tests. For a simple if-else the test is easy; with complex control flow and synchronisation it becomes hard.

The two settings are the same problem: the iterations of a vectorised loop play the role of the threads of a warp, and a branch on the loop index plays the role of a branch on the thread index.

11.4.1 Thread-dependent and uniform values

Definition 11.19 (Thread-dependent and uniform values). An expression is a *thread-dependent value* at a program point p if it may evaluate to different values in different threads at p. It is a *uniform value* at p if it has the same value in all threads of a warp (or, more strongly, of the kernel). A branch is divergent only if its condition is thread dependent.

The source of thread dependence is the thread index: `threadIdx` differs in every lane. Kernel parameters, `blockIdx`, `blockDim` and constants are uniform within a block, and so are values computed only from them. Dependence then spreads along the def-use chains: a value computed from a thread-dependent operand is thread dependent. The kernel below has four branches.

```
__global__ void k(const int *mem, int *out, int n) {
  int tid  = threadIdx.x;
  int q    = tid / 2;
  int base = blockIdx.x * n;
  int x    = mem[base];          // same address in all
  int y    = mem[base + tid];    // address per thread
  int acc  = 0;
  if (q > 3)  acc += 1;          // divergent
  if (y > 0)  acc += 2;          // divergent
  if (x > 0)  acc += 4;          // uniform
  for (int j = 0; j < n; j++)    // uniform
    acc += j;
  out[base + tid] = acc;
}
```

Figure 11.5 draws the def-use edges. The condition `q > 3` depends, through `tid` and q, on `threadIdx.x`: indirect dependence is still dependence. The two loads differ in their addresses. A value loaded from memory is thread dependent if its address is, because different threads then read different words; `mem[base]` is read at one address by all threads, so x is uniform and so is the branch on it. The loop condition `j < n` depends only on the parameter n and the uniform counter `j`: all threads run the same number of iterations.

Checking a branch thus becomes a reachability question on the def-use chains: does a definition that depends on a thread index reach the condition? As with reaching definitions, the dependence facts are propagated through the control-flow graph until nothing changes; loops make the iteration necessary, because a value can become thread dependent through its own back edge.

Uniform within a warp is all that divergence needs: `blockIdx.x` and `base` differ between blocks, but never between the threads of one warp, since a warp never spans two blocks.

Caution The lecture tracks data dependence only, which is not enough for a sound analysis. After `b = 0; if (q > 3) b = 1;` the value of b at the join differs between threads although both assignments are constants, because the branch that chose between them diverged. A sound analysis also marks values defined under a divergent branch and merged at its join (a ϕ in SSA form) as thread dependent: *control dependence*.

11.4.2 Conservative analysis

Definition 11.20 (Conservative analysis). A static analysis is a *conservative analysis* when its approximation errs only on the safe side: here, it may report a uniform branch as divergent but never a divergent branch as uniform.

CPG §5.4.2 gives the standard uniform branch that a naive analysis calls divergent: a condition that depends only on `threadIdx / warpSize` cannot diverge, since warps consist of consecutive thread IDs (CPG §4.1 [5]), yet it is derived from `threadIdx`.

Treating a uniform branch as divergent costs only performance: an unnecessary reconvergence point and the stack operations for it.Treating a divergent branch as uniform would be wrong, so every practical analysis is conservative. The difficulty is precision: an analysis that marks every value derived from `threadIdx` as thread dependent soon marks all branches and all loops as possibly divergent, and the analysis is useless. Compilers therefore add rules that prune such false positives, for example by recognising values that are uniform within a warp although they differ between the warps of a block (such as `threadIdx.x / warpSize`), and the larger and more irregular the control-flow graph, the harder it is to stay both safe and precise.

Tip Writing a branch condition in terms of `threadIdx.x / 32` (in a one-dimensional block), `blockIdx` or kernel parameters keeps it warp-uniform; this is the programmer's side of the analysis (Lesson 5).

Key point — Lesson summary

Liveness is the backward data-flow problem: $\mathrm{IN}[B] = use_B \cup (\mathrm{OUT}[B] - def_B)$, $\mathrm{OUT}[B]$ the union of the successors' IN sets, $\mathrm{IN}[\mathrm{EXIT}] = \emptyset$, solved iteratively to a fixed point. Register allocation colours the interference graph of live ranges; when registers run out, `ptxas` spills to local memory, trading occupancy against memory traffic. SSA form gives every definition its own name and merges versions with ϕ functions at join points where distinct definitions first meet. Unrolling, inlining, dead code elimination, constant propagation and folding, strength reduction and loop-invariant code motion each rest on such an analysis. Divergence analysis propagates thread dependence from `threadIdx` along def-use chains (and, to be sound, control dependence) to the branch conditions; it must be conservative, and its value lies in pruning the uniform branches.

ML Accelerations On GPUs

Lecture	M12, lessons 1–3 (GPU and ML; Floating Point Formats; Tensor Core)
Sources	NVIDIA A100 [1] and H100 [2] whitepapers (required, as the tensor-core resource); NVIDIA GEMM guide [54] (optional); PTX ISA [19]; IEEE 754-2019 [74]; Cambier et al. [75] (S2FP8); Micikevicius et al. [76] (FP8); Kung [77]; Jouppi et al. [78]; Dao et al. [79, 80] (FlashAttention, background)
Textbook	K12 [3] §1.3.9; PMPP [6] ch. 14, ch. 16, App. A; CPG [5] §7.24 Warp Matrix Functions, §7.26 Asynchronous Barrier, §7.27 Asynchronous Data Copies; BPG [8] §7.3 Numerical Accuracy and Precision
Keywords	machine learning, GEMM, arithmetic intensity, ops:byte ratio, tensor core, matrix multiply-accumulate, systolic array, Transformer Engine, floating-point format, IEEE 754, subnormal number, bfloat16, TF32, FP8, quantization, mixed precision, structured sparsity, SpMV, asynchronous copy

Machine learning is the workload that has shaped GPUs most in the last decade: the tensor cores, FP16, FP8 and the Transformer Engine of Lesson 1 were all added for it. This last lesson asks why GPUs suit machine learning and what dedicated hardware they add for it. It first identifies the operations that dominate neural networks, above all matrix multiplication, and measures them with the arithmetic intensity and Roofline reasoning of Lesson 8. It then follows the number formats from IEEE single precision down to 8 bits, because the format decides how many bytes each operation moves. The last part turns to the design of the matrix units themselves: which operation to accelerate, how to build the unit, how to feed it from memory, and how to exploit the zeros of sparse matrices. The compiler view of the previous two lessons stays in the background: the new units appear to software as new instructions of the ISA.

12.1 GPU and ML

The field of *machine learning (ML)* builds models whose parameters are fitted to data, and its dominant model today is the *deep neural network (DNN)*, a stack of layers that each transform a large array of numbers. This section explains why GPUs suit this workload, which operations it consists of, how matrix multiplication is measured, and the two hardware routes to accelerating it: SIMD lanes and dedicated matrix units.

Source M12, GPU and ML; PMPP ch. 16; [54].

12.1.1 Why GPUs suit machine learning

Four properties of GPUs match the needs of ML training and inference.

Floating-point throughput. ML performs a very large number of floating-point operations with high data parallelism, and most of a GPU's transistors serve floating-point throughput (Lesson 1).

Memory bandwidth. The GPU and its memory are connected directly on the package, so stacked high bandwidth memory (HBM) is easy to adopt; a CPU socket must keep its standard memory interfaces.

HBM bandwidth per GPU: 900 GB/s (V100), 1555 GB/s (A100 40 GB), 3000 GB/s (H100 SXM5, preliminary) (AWP Table 4 [1]; HWP Table 3 [2]).

Flexible formats. GPUs add new number formats quickly: tensor formats, bfloat16, 8-bit floating point, INT8 and INT4 (Section 12.2).

Relaxed semantics. ML is statistical computing: a result that differs in the last bits, or an operation reordered between threads, does not change what the model learns (floating-point addition is not associative, so a reordering changes only the last bits: BPG §7.3.2 Floating Point Math Is Not Associative). This fits the GPU's weak memory consistency and its lack of precise exceptions, both of which would cost hardware to provide.

Lesson 1 contrasts the precise exceptions of a CPU with the GPU, which does not guarantee them.

12.1.2 Operations of a neural network

The layers of a DNN are built from three classes of operations, all highly data parallel.

Definition 12.1 (Classes of DNN operations). The operations of a DNN fall into three classes.

- An *element-wise operation* applies the same function independently to every element of an array. The typical example is an *activation function*, the non-linear function applied to every output of a layer, such as $\mathrm{ReLU}(x) = \max(0, x)$.

- A reduction (Lesson 1) combines many elements into one; the DNN example is *pooling*, which replaces each window of an array by one value, such as its maximum (max pooling).
- A *dot product* $\sum_k a_k b_k$ multiplies two vectors element by element and sums the products. Dot products make up matrix multiplications and convolutions.

The dot-product class dominates the arithmetic. A *fully connected layer* connects every input to every output with its own weight, so a batch of inputs times the weight matrix is one *matrix multiplication*. A *convolution layer* slides a small filter over its input and is likewise computed as a matrix multiplication (PMPP ch. 16).

Element-wise operations and reductions read and write every element once and are therefore memory-bound in the sense of the Roofline model (Lesson 8); only the dot-product class reuses its operands.

12.1.3 GEMM and its arithmetic intensity

Libraries express the matrix multiplications of ML in one standard form.

Definition 12.2 (GEMM). The operation *GEMM (general matrix multiplication)* computes

$$C \leftarrow \alpha AB + \beta C, \tag{12.1}$$

where A is an $M \times K$ matrix, B is $K \times N$, C is $M \times N$, and α, β are scalars. With $\alpha = 1$ and $\beta = 0$ it is the plain product $C = AB$.

The GEMM guide counts "a total of `2 * M * N * K` FLOPS" [54, §2].

Each of the MN elements of C is a dot product of length K, so a GEMM performs MNK fused multiply-adds (Lesson 8), that is $2MNK$ floating-point operations. With FP16 inputs (two bytes per element) it reads A and B and writes C, $2(MK + NK + MN)$ bytes if every matrix crosses the memory interface exactly once. The ratio is its arithmetic intensity (Lesson 8):

$$I_{\text{GEMM}} = \frac{2MNK}{2(MK + NK + MN)} = \frac{MNK}{MK + NK + MN} \text{ FLOP/B}. \tag{12.2}$$

Caution The intensity of Equation (12.2) is a first-order estimate: it assumes each matrix is read or written exactly once from memory and ignores caches and on-chip reuse. A real kernel moves more bytes (tiles of A and B are read several times) or fewer (operands that stay in L2).

The machine has an intensity of its own, the peak arithmetic rate divided by the memory bandwidth, and the comparison of the two decides which resource limits the kernel.

Definition 12.3 (Ops:byte ratio; math- and memory-limited). The *ops:byte ratio* of a processor is its peak arithmetic rate divided by its peak memory bandwidth, in FLOP per byte; it is the ridge point of the Roofline model of Lesson 8. The side of the ratio on which a kernel's intensity lies makes it *math-limited / memory-limited*: above the ratio the arithmetic units are the bottleneck (math-limited), below it the memory bandwidth (memory-limited).

Example 12.4. A square GEMM with $M = N = K = 1024$ in FP16 has $I = 1024^3/(3 \cdot 1024^2) = 1024/3 \approx 341$ FLOP/B, well above the 138.9 FLOP/B of a V100, so it is math-limited. A skinny GEMM with $M = K = 4096$ and $N = 16$, a weight matrix applied to a batch of 16 inputs, has

$$I = \frac{4096 \cdot 16 \cdot 4096}{4096 \cdot 4096 + 16 \cdot 4096 + 4096 \cdot 16} = \frac{16 \cdot 4096}{4096 + 32} \approx 15.9 \text{ FLOP/B},$$

far below the ratio: it is memory-limited, and a faster arithmetic unit would not speed it up. The small dimension N caps the intensity, since $I < N$ whenever $N \le M, K$.

V100 with FP16 tensor-core inputs and FP32 accumulation: "the FLOPS:B ratio is 138.9 if data is loaded from the GPU's memory" [54, §2].

Tip For a skinny GEMM with $N \ll M, K$, $I \approx N$ FLOP/B in FP16 (and $2N$ in FP8): read the intensity off the smallest dimension.

The GEMM guide adds two effects that Equation (12.2) does not capture, because a GPU computes C in tiles, each tile assigned to one thread block. The first, *tile quantization*, "occurs when matrix dimensions are not divisible by the thread block tile size" [54, §3.1]: the last tile of a row or column is only partly useful but costs as much as a full one. The second, *wave quantization*, arises because "the total number of tiles is quantized to the number of multiprocessors on the GPU" [54, §3.2]: tiles run in waves of one tile per SM, and a last wave with few tiles leaves most SMs idle. Tensor cores are also most efficient when the GEMM dimensions are multiples of 8 elements for FP16 (on A100, multiples of 64) [54, §2.2].

The guide's example: on an A100 with 108 SMs, $9 \times 12 = 108$ tiles make one full wave, while $9 \times 13 = 117$ tiles add a second wave of only 9 tiles [54, §3.2].

12.1.4 SIMD lanes versus a matrix unit

On ordinary SIMT lanes, a matrix product is a loop of scalar multiply-adds. The kernel of Listing 12.1 computes a 4×4 product with one thread per element of C; each thread runs the loop over k sequentially and keeps its running sum in a private register.

Listing 12.1: A 4×4 matrix product on SIMT lanes, one thread per output element.

```
__global__ void matmul4(const float *A,
```

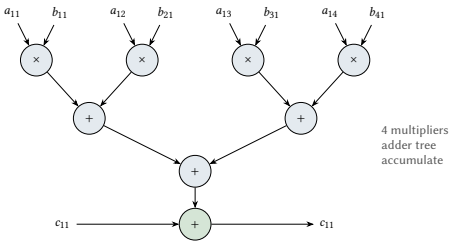


Figure 12.1: One SIMD dot-product step: four multipliers, an adder tree and the accumulation compute $c_{11} \mathrel{+}= \sum_k a_{1k} b_{k1}$. A 4×4 product needs 16 such steps.

```
                        const float *B, float *C)
{
  const int N = 4;
  int row = threadIdx.y, col = threadIdx.x;
  float sum = 0.0f;            // private register
  for (int i = 0; i < N; ++i)
    sum += A[row * N + i] * B[i * N + col];
  C[row * N + col] = sum;
}
// launch: matmul4<<<1, dim3(4, 4)>>>(dA, dB, dC);
```

Because a CUDA thread's registers are private, one thread must perform the whole accumulation of an output element. Fewer threads (four, say, one per row) simply loop longer. Spreading one dot product over several threads instead, for instance 16 threads that each compute one product $a_{ik}b_{kj}$, requires a reduction of the partial sums through shared memory or warp shuffles (Lesson 5), which adds instructions and synchronisation.

The reduction patterns of Lesson 5 (tree reduction in shared memory, `__shfl_xor_sync`) are exactly this overhead.

A SIMD datapath can at best compute one element per operation: four multipliers take a row of A and a column of B, an adder tree sums the four products, and the result is accumulated (Figure 12.1). Row 1 of C alone takes four of them,

$$c_{1j} = \sum_{k=1}^{4} a_{1k} b_{kj}, \qquad j = 1, \ldots, 4,$$

each the dot product of row 1 of A with one column of B, and the product of two 4×4 matrices needs 16 such operations, one per output element.

Definition 12.5 (MAC, MMA, tensor core instruction). A *multiply-accumulate (MAC)* adds a product to a running sum, $c \leftarrow c + ab$; done with a single rounding it is the FMA. A *matrix multiply-accumulate (MMA)* applies the same idea to whole matrices, $D \leftarrow AB + C$, as one operation. The tensor core (Lesson 1) is NVIDIA's MMA unit; the GPU has hundreds of them. In NVIDIA's machine code (SASS) the half-precision MMA instruction is *HMMA*, for instance `HMMA.1688.F16`: one instruction does the work that the SIMT loop does with many.

Caution Lecture vs ISA: `HMMA.1688` names the warp-level shape $m16n8k8$ (a 16×8 result, $K = 8$), the PTX `mma` shape `.m16n8k8` [19], not a 4×4 product. The $4 \times 4 \times 4$ figure is the per-clock datapath of one Volta tensor core [1].

The 32 threads of a warp issue the HMMA together, each supplying a fragment of the operands from its registers, so the tensor core sidesteps the private-register problem above. Tensor-core generations differ mainly in the size of the product they complete per clock and in the formats they accept (Table 12.1). The A100 tensor core "can execute 256 FP16 FMA operations per clock", an $8 \times 4 \times 8$ mixed-precision product, and accepts TF32, BF16 and FP64 inputs as well as INT8, INT4 and binary (1-bit) [1]; the integer and binary modes date from Turing, the generation between the two; sparsity support doubles the peak again (Section 12.3).

CUDA C++ reaches the tensor cores through the warp-level matrix functions (`wmma::load_matrix_sync`, `wmma::mma_sync`): CPG §7.24 Warp Matrix Functions. PTX and SASS are the subject of Lesson 10.

12.1.5 Systolic arrays

The traditional hardware for matrix multiplication is the systolic array.

Definition 12.6 (Systolic array). A *systolic array* is a grid of identical processing elements (PEs), each a MAC unit, connected only to their neighbours. Data enter at the edges and move one PE per cycle; every PE computes on the values that pass through it and forwards them.[1]

In the *output-stationary* arrangement of Figure 12.2, PE(i, j) owns the output element c_{ij}. The rows of A enter from the left and the columns of B from the top, each row and column

[1] Kung named the structure after the rhythmic, pulsed flow of data through the array. Each value fetched from memory is used by many PEs before it leaves, which is what makes the structure attractive when memory bandwidth is the limit [77]. Google's first TPU is built around a 256×256 array, "a 65,536 8-bit MAC matrix multiply unit" with a peak of 92 TOPS [78].

Table 12.1: Tensor cores of three NVIDIA generations (AWP [1]; HWP V1.02, SXM5, preliminary [2]). Peak rates are dense / with 2:4 sparsity.

	V100	A100	H100
Tensor cores per SM	8	4	4
FP16 FMA per clock per core	64	256	512
Product per core per clock	$4\times4\times4$	$8\times4\times8$	—
FP16 tensor TFLOP/s	125	312 / 624	1000 / 2000
TF32 tensor TFLOP/s	—	156 / 312	500 / 1000
FP8 tensor TFLOP/s	—	—	2000 / 4000
FP32 (non-tensor) TFLOP/s	15.7	19.5	60

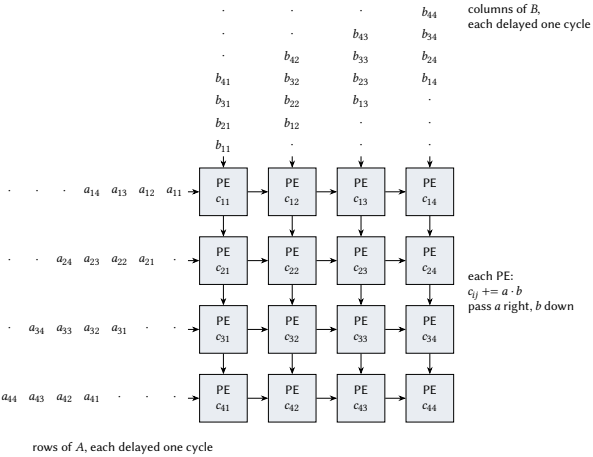


Figure 12.2: A 4×4 output-stationary systolic array at the start of $C = AB$. The skew makes a_{ik} and b_{kj} meet in PE(i, j) in cycle $i + j + k - 2$; dots mark empty slots.

delayed by one cycle relative to the previous one (the *skew*). In every cycle each PE multiplies the a and b that arrive, adds the product to its stationary partial sum, and passes a to the right and b downwards. In cycle 1 PE$(1, 1)$ computes $a_{11}b_{11}$; in cycle 2 a_{11} has moved right to meet b_{12} in PE$(1, 2)$, while PE$(1, 1)$ adds $a_{12}b_{21}$; and so on.

With this skew, PE(i, j) performs step k in cycle $t = i + j + k - 2$. For an $n\times n$ array the first PE starts in cycle 1, the last PE(n, n) starts in cycle $2n-1$ and finishes in cycle $3n-2$. Table 12.2 traces $n = 4$: the 64 MACs take 10 cycles, and no cycle has all 16 PEs busy.

Table 12.2: Activity of the 4×4 array of Figure 12.2 for one $4\times4\times4$ product (our model: PE(i, j) performs step k in cycle $i + j + k - 2$).

Cycle	1	2	3	4	5	6	7	8	9	10
Busy PEs	1	3	6	10	12	12	10	6	3	1
Outputs finished	–	–	–	1	2	3	4	3	2	1

Caution The lecture describes the array as fully busy around cycle 4 and finished in about seven cycles; it counts differently from the model above. Under our model, cycle 4 has 10 of 16 PEs busy, PE$(4, 4)$ starts in cycle $2n - 1 = 7$, and the product completes in cycle $3n - 2 = 10$.

Definition 12.7 (Fill and drain). The *fill and drain* of a systolic array are the cycles at the start of a computation, while the skewed inputs are still entering and only some PEs are busy, and at its end, while the last operands pass through and the PEs fall idle again.

In the trace, the array does 64 MACs in 10 cycles on 16 PEs, a utilisation of $64/160 = 40\,\%$; the rest is fill and drain. When many tiles are streamed through back to back, the next tile fills the array while the previous one drains, and the utilisation approaches 100 %.

12.1.6 Transformer Engine and data movement

Hopper's Transformer Engine (Lesson 1) accelerates the layers of transformer models by scaling tensor data dynamically into the representable range of a narrow format, which makes FP8 arithmetic usable [2]. Fast arithmetic is only half of the problem: the units must be fed. The Tensor Memory Accelerator (Lesson 1) moves blocks of a tensor between global and shared memory in chunks and generates the addresses itself.

Section 12.2 explains why a narrow format needs this scaling: FP8 E4M3 covers magnitudes only from 2^{-9} to 448.

Definition 12.8 (Copy descriptor). A *copy descriptor* is a data structure that describes a multidimensional tensor and the blocks to be copied from it (base address, tensor sizes and strides, and the shape of the block), so that the TMA generates all addresses of a transfer from it. The coordinates of a block are given with each copy instruction, so one descriptor serves all the tiles of a tensor. In Hopper, "a single thread creates a copy descriptor" before issuing the transfer [2], instead of every thread computing its own addresses.

> **Key point**
> SIMD/SIMT lanes can accelerate matrix multiplication, dedicated matrix units accelerate it much further, and in both cases keeping the memory bandwidth fully used is critical: a GEMM whose intensity is below the ops:byte ratio gains nothing from a faster unit.

12.2 Floating-point formats

Scientific computing relies on IEEE single (32-bit) and double (64-bit) precision; ML reduces the precision. This section explains why narrower formats pay, how a floating-point number is encoded, and how ML-specific formats trade precision for range.

Source M12, floating-point formats; PMPP App. A; BPG §7.3 Numerical Accuracy and Precision; [74, 1, 2].

12.2.1 Why lower precision pays

The number of floating-point operations of a GEMM depends on its shape, not on its format. The bytes moved depend on the format: halving the input width halves the bytes and therefore doubles the arithmetic intensity. With FP8 inputs (one byte per element), Equation (12.2) becomes

$$I_{\text{GEMM, FP8}} = \frac{2MNK}{MK + NK + MN}, \tag{12.3}$$

twice the FP16 value; the skinny GEMM of Example 12.4 rises from 15.9 to 31.8 FLOP/B.

Equation (12.3) counts C at the input width as well; an FP32 output would add $2MN$ bytes.

Definition 12.9 (Quantization, mixed precision, accumulator). The representation of values with fewer bits than the format they were computed in, for instance FP32 weights stored in FP8, is called *quantization*. In *mixed precision* arithmetic the inputs of an operation are in a narrow format while the *accumulator*, the register or variable that holds the running sum, stays in a wider one, typically FP16 or FP8 inputs with FP32 accumulation.

Quantization brings three benefits: smaller storage, higher arithmetic intensity, and, on recent hardware, higher arithmetic throughput, because a unit of the same area completes more narrow operations per cycle. On older units only the first two applied: the operands shrank, but the operations per cycle stayed the same.

Hopper: "FP8 halves data storage requirements and doubles throughput compared to FP16 or BF16" [2]; compare the FP16 and FP8 rows of Table 12.1.

Caution The accumulator stays wide for a reason: a long sum of narrow terms loses the small addends (Lesson 5, lower precision). Volta's tensor cores already took FP16 inputs with "FP32 accumulation" [1].

12.2.2 Sign, exponent and fraction

A floating-point number is the binary form of scientific notation. In decimal, $0.0012 = 1.2\times10^{-3}$ and $0.0000012 = 1.2 \times 10^{-6}$ share the fraction 1.2 and differ only in the exponent; a binary format uses base 2 in the same way.

Definition 12.10 (Floating-point format). A binary *floating-point format* stores a number in three fields: a *sign bit* s, an *exponent* field e of w bits, and a *mantissa (fraction)* field f of p bits. The total width is $1 + w + p$ bits. Because the exponent must represent both negative and positive powers (values below and above 1) while the sign bit belongs to the value, the exponent is stored with an offset, the *exponent bias* $b = 2^{w-1} - 1$. A *normal* number has $0 < e < 2^w - 1$ and the value

$$x = (-1)^s \times \left(1 + \frac{f}{2^p}\right) \times 2^{e-b}, \tag{12.4}$$

where the leading 1 is implicit and not stored.

"Mantissa", "fraction" and "significand" name the same field in practice; IEEE 754 calls the stored field the trailing significand [74].

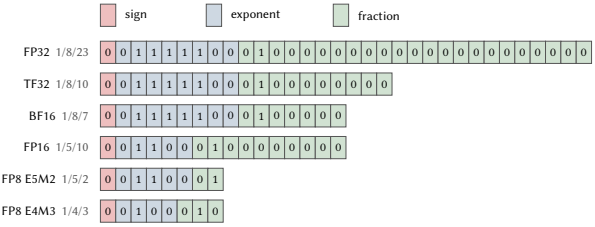


Figure 12.3: The value $0.15625 = 1.01_2 \times 2^{-3}$ in six formats (sign/exponent/fraction bits on the left). The biased exponent is $-3 + 127 = 124$ in FP32, TF32 and BF16, $-3 + 15 = 12$ in FP16 and E5M2, and $-3 + 7 = 4$ in E4M3; the fraction is 01 followed by zeros.

More exponent bits cover a wider range; more fraction bits give more precision. Figure 12.3 encodes the same value in the six formats of this chapter, and Table 12.3 gives their parameters. Formats need not have power-of-two widths: TF32 is $1 + 8 + 10 = 19$ bits wide.

Definition 12.11 (ML formats). The format *bfloat16 (BF16)* "includes an 8-bit exponent, 7-bit mantissa, and 1 sign-bit" [1]: the range of FP32 with less precision than FP16. The format *TensorFloat-32 (TF32)* has an "8-bit exponent (same as FP32), 10-bit mantissa (same precision as FP16) and 1 sign-bit" [1]; the A100 tensor cores use it for FP32 inputs. The 8-bit format *FP8* has two encodings: "E4M3 with 4 exponent bits, 3 mantissa bits, and 1 sign bit" and "E5M2, with 5 exponent bits, 2 mantissa bits, and 1 sign bit" [2].

In practice E4M3 tends to be used for weights and activations and E5M2 for gradients, which span a wider range [76].

The two FP8 encodings divide the work of training: "E4M3 supports computations requiring less dynamic range with more precision, while E5M2 provides a wider dynamic range and less precision" [2]. The *dynamic range* of a format is the ratio of its largest to its smallest representable positive magnitude, often quoted in powers of two (binades).

Table 12.3: Parameters of the formats of Figure 12.3. Maxima and minima are derived from Equation (12.4), except the E4M3 maximum, which uses the exponent field 1111 (no infinities; see the caution in the text); $\varepsilon = 2^{-p}$ is the spacing of values just above 1.

Format	$s/w/p$	Bias	Max normal	Min normal	ε
FP64	1/11/52	1023	$\approx 1.80 \times 10^{308}$	2^{-1022}	2^{-52}
FP32	1/8/23	127	$\approx 3.40 \times 10^{38}$	2^{-126}	2^{-23}
TF32	1/8/10	127	$\approx 3.40 \times 10^{38}$	2^{-126}	2^{-10}
BF16	1/8/7	127	$\approx 3.39 \times 10^{38}$	2^{-126}	2^{-7}
FP16	1/5/10	15	65 504	2^{-14}	2^{-10}
FP8 E5M2	1/5/2	15	57 344	2^{-14}	2^{-2}
FP8 E4M3	1/4/3	7	448	2^{-6}	2^{-3}

Caution E4M3 is not IEEE-like at the top: it has no infinity, and only the all-ones pattern `S.1111.111` is NaN [76]; the PTX ISA lists its NaNs as `0x7f` and `0xff` [19, §5.2.3 Alternate Floating-Point Data Formats]. The largest value is therefore $1.110_2 \times 2^{15-7} = 1.75 \times 256 = 448$, not the 240 the IEEE rules would give.

Tip BF16 is the upper half of FP32: converting FP32 to BF16 drops the low 16 fraction bits, by truncation or by rounding.

Example 12.12. Range against precision in FP16 and BF16. The value 70 000 exceeds the FP16 maximum of 65 504 and overflows; in BF16 it is in range, and since the spacing of BF16 values between 2^{16} and 2^{17} is $2^{16-7} = 512$, it rounds to the nearest multiple, $137 \times 512 = 70\,144$. Conversely, $1 + 2^{-8}$ is exact in FP16 ($\varepsilon = 2^{-10}$), but in BF16 it lies halfway between 1 and $1 + 2^{-7}$ and rounds to the neighbour with an even fraction, 1: the increment is lost.

12.2.3 Subnormals, special values and IEEE 754

Equation (12.4) cannot represent zero, and its smallest magnitude 2^{1-b} leaves a gap down to zero. The exponent field values 0 and $2^w - 1$ are reserved to fill these holes.

Definition 12.13 (Subnormal number, underflow, overflow, NaN). A *subnormal number* has exponent field $e = 0$ and the value $(-1)^s \times (f/2^p) \times 2^{1-b}$, without the implicit leading 1; with $f = 0$ it is ± 0. Subnormals fill the gap between zero and the smallest normal number with evenly spaced values. A nonzero result whose magnitude is below the smallest normal number 2^{1-b} suffers *underflow* (it is rounded to a subnormal or to zero); a result beyond the largest finite value suffers *overflow* (under the default rounding it becomes $\pm\infty$; a format without infinities, such as E4M3, saturates or gives NaN). The exponent field $e = 2^w - 1$ encodes $\pm\infty$

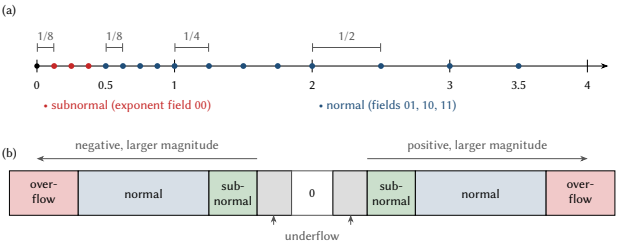


Figure 12.4: (a) The non-negative values of a toy format (1/2/2 bits, bias 2, no infinities): three subnormals, then normals whose spacing doubles at every power of two. (b) The regions of a format on a logarithmic magnitude axis.

($f = 0$) and *NaN (not a number)* ($f \neq 0$), the result of an invalid operation such as $0/0$ or $\infty - \infty$.

Figure 12.4(a) lists every non-negative value of a toy format with two exponent bits (bias 2) and two fraction bits, using all exponent codes for numbers. The normal values in $[2^k, 2^{k+1})$ are evenly spaced, and the spacing doubles at each power of two; the three subnormals continue the spacing of the lowest binade down to zero. Representable values are therefore dense near zero and sparse at large magnitudes: precision, the spacing relative to the value, is roughly constant, but the absolute error grows with the magnitude. Figure 12.4(b) shows the regions of a real format on a logarithmic scale.

Toy format: $s/w/p = 1/2/2, b = 2$. Normals: $1.f \times 2^{e-2}$ for $e = 1, 2, 3$, from 0.5 to 3.5; subnormals $0.f \times 2^{-1}$: 1/8, 2/8, 3/8.

Definition 12.14 (IEEE 754; rounding mode). The standard *IEEE 754* defines binary floating-point arithmetic [74]. It specifies the layout of sign, exponent and significand for its binary formats (binary16, binary32, binary64, ...), the special values (±0, ±∞, NaN), subnormal numbers, the exact results of the basic operations, and the *rounding modes* that choose a representable result when the exact one is not representable: round toward $+\infty$ (up), toward $-\infty$ (down), toward zero, and to nearest.

IEEE 754 binary16, binary32 and binary64 are the FP16, FP32 and FP64 of this chapter (Lesson 1 introduced the three precisions).

Caution The default rounding mode, which the list "round up, round down" omits, is round to nearest with ties to even [74], as in Example 12.12.

BF16, TF32 and FP8 are not IEEE 754 interchange formats; they borrow its encoding rules, and E4M3 departs from them at the top of its range.

12.2.4 Quantization in practice

Fewer bits mean fewer and cheaper operations, so ML typically quantizes the inputs of an operation to a low precision and keeps a high-precision accumulator. The values of a tensor are rarely spread evenly: most weights and gradients cluster near zero, with a long tail. A format whose values are spaced evenly in the log domain then wastes codes, and ML practice transforms the values before quantizing them, a non-uniform quantization. The resolution near zero matters most.

The Transformer Engine of Section 12.1.6 applies the simplest such transform: it scales a tensor, from statistics of its values, before casting it to FP8 [2].

Definition 12.15 (S2FP8). The format *shifted and squeezed FP8 (S2FP8)* [75] stores a tensor X in FP8 after a power transform with two per-tensor parameters α (squeeze) and β (shift):

$$X_{\text{truncated}} = \left[2^{-\beta} \cdot \text{truncate}_{\text{FP8}}\left(2^{\beta} \cdot |X|^{\alpha}\right) \right]^{1/\alpha}. \tag{12.5}$$

The transform acts on magnitudes; each element keeps the sign of X. In the log domain, $\log_2(2^{\beta}|X|^{\alpha}) = \alpha \log_2 |X| + \beta$: α scales the spread of the exponents to fit the range of FP8 and β shifts their centre, so the codes of the format fall where the values of the tensor are. Both are computed for each tensor from the mean μ and the maximum m of $\log_2 |X|$ over its non-zero elements: $\alpha = 15/(m - \mu)$ and $\beta = -\alpha\mu$, so that the transformed values have a mean exponent of 0 and a maximum of 15 [75, §3.2].

Key point
A floating-point format is a choice between range (exponent bits) and precision (fraction bits) at a given width. ML lowers the width of the inputs and keeps the accumulator wide; each halving of the input width doubles the arithmetic intensity, and on recent tensor cores also the peak rate.

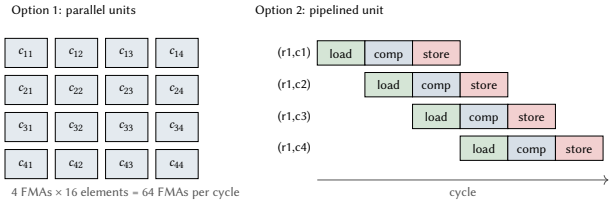


Figure 12.5: Two ways to build a $4 \times 4 \times 4$ MMA unit. Option 1: 16 dot-product units, one per element of C. Option 2: a pipelined unit through which the row–column pairs pass one cycle apart.

12.3 Tensor core

Source M12, tensor core design; CPG §7.24, §7.26, §7.27; PMPP ch. 14; [1, 2].

A tensor core is one instance of a general task: adding an accelerator for ML to a GPU. This section follows the design questions in order: which function to build, how to build the compute units, how to store and move their data, and how to exploit sparsity.

12.3.1 Design questions and steps

The design of an ML accelerator component raises four questions: what functionality to add; what benefit it brings over the existing GPU; how to build the compute units, and how many; and how to store and move the data they need. The answers follow a common sequence of steps.

1. Identify the frequently executed operations, from knowledge of the algorithms or from profiling.
2. Estimate the benefit over the existing data-parallel units: how much faster the operation would be in hardware than in software with the existing ISA.
3. Design the interface: the new instructions, how data are fed to the unit, the storage format, and whether operands live in registers or in memory (private or shared).
4. Decide whether to merge accelerator components into one unit, weighing programmability and usability against hardware cost and timing overhead.

For the V100, at equal precision, a factor of about 4: 125 TFLOP/s on the FP16 tensor cores against 31.4 TFLOP/s for FP16 on the SIMT lanes (AWP Table 4 [1]); against FP32's 15.7 TFLOP/s the factor of 8 includes the narrower format.

Step 3 is where the compiler of Lessons 10 and 11 enters: a new unit is reached only through new instructions, which the compiler or a library must emit.

12.3.2 The matrix multiply-accumulate unit

GPUs chose an MMA unit because GEMM dominates ML (Section 12.1), and SIMD lanes still work one row–column pair at a time. The design space is between many small MMA units and one large unit such as a systolic array, and every unit takes chip area from other GPU functions; programmability matters as much as the raw rate. NVIDIA started with a 4×4 unit (Volta's $4 \times 4 \times 4$ product per clock), and other vendors have similar units: Intel's *Advanced Matrix Extensions (AMX)* add 16×16 matrix tiles and tile instructions to x86 CPUs.

Within an MMA unit, three organisations are possible.

Systolic array. Efficient for large tiles, but the fill and drain time is wasted on small ones: a 4×4 array was 40 % busy on a single product (Table 12.2).

Option 1: parallel units. One dot-product unit (Figure 12.1) per output element: 16 units of 4 FMAs compute a whole $4 \times 4 \times 4$ product at once (Figure 12.5, left).

Option 2: pipelining. Fewer units, shared over time: row 1 times column 1, then row 1 times column 2, and so on, with the load, compute and store of successive pairs overlapped (Figure 12.5, right).

Parallel units give the highest throughput for the most area; a pipelined unit trades throughput for area.

Caution Lecture vs documentation: Intel documents eight AMX tile registers of at most 16 rows of 64 bytes each; its INT8 example multiplies 16×64 inputs into a 16×16 INT32 result [81]. So 16×16 is the output tile of that example, not a fixed matrix size.

Option 1 performs $16 \times 4 = 64$ FMAs per cycle, the rate of one Volta tensor core (Table 12.1).

12.3.3 Storing and moving the operands

The operands of an MMA unit come from memory, and their volume is large: a $4 \times 4 \times 4$ operation needs at least 3×16 values (A, B and C) in registers. NVIDIA's registers are private to a thread, which complicates feeding one unit from them; the warp-level interface of Section 12.1 spreads the operands over the registers of 32 threads. The alternative is a memory close to the unit, shared memory (Lesson 3), at the cost of moving the data there from global memory first.

For that move NVIDIA added an asynchronous copy inside the GPU. The A100's instruction

Register file size and banking, and the pressure that wide operands put on them, are the subject of Lessons 4 and 6.

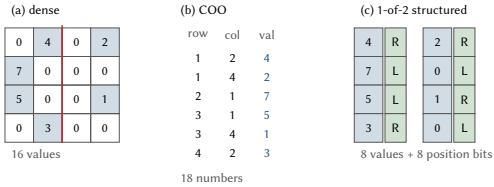


Figure 12.6: One sparse matrix in three forms: (a) dense, 16 values; (b) COO triples, 18 numbers; (c) a structured format keeping one value per pair of columns (left of the red line, right of it) with a bit for its position, left (L) or right (R).

"loads data directly from global memory into shared memory, optionally bypassing L1 cache, and eliminating the need for intermediate register file (RF) usage" [1]. Because the threads no longer wait for the data, they need a way to learn when it has arrived.

Definition 12.16 (Asynchronous barrier). An *asynchronous barrier* is a barrier split into an *arrive* and a *wait* operation: a thread signals its arrival, continues with independent work, and waits only when it needs the result (CPG §7.26 Asynchronous Barrier). Combined with the asynchronous copy, a block can start the copy of the next tile, compute on the current one, and wait for the copy just before it is used.

Caution Two different copies are called asynchronous: `cudaMemcpyAsync` of Lesson 5 is a host-issued transfer between host and device; the A100 copy is a device instruction (`cp.async` in PTX, `memcpy_async` in CUDA C++, CPG §7.27) between global and shared memory.

`__syncthreads()` (Lesson 3) is the synchronous case: arrival and wait are one operation. Hopper extends the barrier to count the bytes of TMA transfers (the asynchronous transaction barrier [2]).

12.3.4 Attention: data movement beyond GEMM

The same concern, keeping fast units fed, drives the main kernel of transformer models. The operation called *attention* computes, for query, key and value matrices Q, K, V of N rows, $\mathrm{softmax}(QK^{\mathsf{T}}/\sqrt{d})\,V$, where d is the row length; the intermediate $N \times N$ score matrix is large, and a straightforward implementation writes it to HBM and reads it back. *FlashAttention* is "an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM" [79]: it computes the same result without storing the full score matrix in HBM. Its argument is the one of Equation (12.2): attention is limited by memory traffic, not by arithmetic, so the gain comes from fewer bytes.

FlashAttention-2 partitions the work better across thread blocks and warps, "reaching 50-73 % of the theoretical maximum FLOPs/s on A100" [80].

12.3.5 Sparse operations

Many matrices of HPC and graph workloads are mostly zeros, and ML creates more of them on purpose.

Definition 12.17 (Sparsity, sparse matrix, compressed format). The *sparsity* of a matrix or tensor is the fraction of its elements that are zero; a *sparse matrix* is one with enough zeros that storing and computing only the non-zeros pays. A *compressed sparse format* stores only the non-zeros together with their positions. The simplest is the coordinate format (COO), one (row, column, value) triple per non-zero.

Compressed sparse row (CSR), the usual format for SpMV, replaces the row indices of COO by one offset per row (PMPP ch. 14).

Figure 12.6(a) and (b) show a 4×4 matrix with six non-zeros in dense and COO form. COO needs 18 numbers where the dense form needs 16: the index overhead outweighs the saved zeros unless the matrix is much sparser, and the irregular indices make hardware support hard. ML therefore restricts where the non-zeros may be.

Definition 12.18 (Pruning, structured sparsity). The operation called *pruning* sets weights of a trained network that contribute little to zero, which raises its sparsity. The constraint called *structured sparsity* restricts the pattern: only a fixed fraction of the elements in every small group may be non-zero, so every group compresses to the same size and the index is short and regular.

With 50 % structured sparsity and one non-zero per pair of elements, 16 elements are stored as 8 values plus 8 one-bit indices that say whether each kept value was the left or the right element of its pair (Figure 12.6(c)). The fixed layout simplifies the hardware and nearly halves the storage, and the unit skips the multiplications by zero, which raises its throughput.

Caution The one-bit left/right index here is a toy for one non-zero in two. In 2:4 sparsity each kept value can sit in any of four positions, and for FP16 and BF16 the non-zeros of a group "are indicated by two 2-bit indices in the metadata operand" [19] (`mma.sp`).

Definition 12.19 (2:4 structured sparsity). The scheme *2:4 structured sparsity*, introduced with the A100, is "a new 2:4 sparse matrix definition that allows two non-zero values in every

four-entry vector" [1]. A row of A is stored as half its values plus metadata giving the positions of the two kept values in each group of four.

Row 1 of the matrix of Figure 12.6, $[0, 4, 0, 2]$, is already 2:4: it keeps 4 and 2 with the indices 1 and 3 (counting from 0).

Tip 2:4 with FP16 stores a group of four as two values plus two 2-bit indices: $32 + 4 = 36$ of 64 bits, about 56 %.

The A100's sparse format "can be compressed efficiently and reduce memory storage and bandwidth by almost 2x" [1], and its sparse tensor-core rates are twice the dense ones (Table 12.1). Beyond matrix–matrix products, *sparse matrix-vector multiplication (SpMV)*, $y = Ax$ with a sparse A, is the core of many HPC and graph algorithms, and accelerating it is an active research topic: its intensity is low and its accesses irregular, so it profits little from a dense matrix unit.

Key point — Lesson summary

GPUs suit ML: it is floating-point heavy, data parallel and tolerant of relaxed semantics, and GPUs have HBM and adopt new formats quickly. GEMM dominates DNNs; its intensity $MNK/(MK + NK + MN)$ (FP16) against the ops:byte ratio tells whether it is math- or memory-limited. An MMA unit (the tensor core) replaces 16 SIMD dot products of a 4×4 product by one instruction; systolic arrays pay fill and drain. A floating-point format trades exponent bits (range) for fraction bits (precision); ML narrows the inputs (BF16, TF32, FP8), keeps the accumulator wide and transforms values (S2FP8). A tensor-core design chooses parallel or pipelined units, operand storage fed by asynchronous copies and barriers, and a sparse format: 2:4 structured sparsity nearly halves storage and doubles the peak tensor-core rate.

Bibliography

[1] NVIDIA Corporation. *NVIDIA A100 Tensor Core GPU Architecture*. Whitepaper. Optional reading, Module 12. NVIDIA, 2020. URL: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf.

[2] NVIDIA Corporation. *NVIDIA H100 Tensor Core GPU Architecture*. Whitepaper V1.02. H100 figures in this version are preliminary. Fetched from mirror https://www.ece.lsu.edu/koppel/gp/refs/gtc22-whitepaper-hopper.pdf. NVIDIA, 2022. URL: https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper.

[3] Hyesoon Kim et al. *Performance Analysis and Tuning for General Purpose Graphics Processing Units (GPGPU)*. Synthesis Lectures on Computer Architecture 20. Reissued by Springer, doi:10.1007/978-3-031-01737-7. Chapters: 1 GPU Design, Programming, and Trends (pp. 1–24); 2 Performance Principles (p. 25); 3 From Principles to Practice: Analysis and Tuning (p. 39); 4 Using Detailed Performance Analysis to Guide Optimization (p. 49). Full section TOC verified from the PDF on co-author Jee Choi's page (https://jeewhanchoi.github.io/publication/pdf/perf_analysis_book.pdf). Morgan & Claypool, 2012. ISBN: 978-1-60845-954-4. DOI: 10.2200/S00451ED1V01Y201209CAC020.

[4] Tor M. Aamodt, Wilson Wai Lun Fung, and Timothy G. Rogers. *General-Purpose Graphics Processor Architectures*. Synthesis Lectures on Computer Architecture 44. Reissued by Springer, doi:10.1007/978-3-031-01759-9. Chapters (start pages per Google Books): 1 Introduction (1); 2 Programming Model (9); 3 The SIMT Core: Instruction and Register Data Flow (21); 4 Memory System (67); 5 Crosscutting Research on GPU Computing Architectures (85). Section headings not verified. Morgan & Claypool, 2018. ISBN: 978-1-62705-923-7. DOI: 10.2200/S00848ED1V01Y201804CAC044.

[5] NVIDIA Corporation. *CUDA C++ Programming Guide*. Release 12.4. Assigned: sections 1–2 and 4 (Module 1), section 3 (Module 3). Numbering refers to the 12.x guide; from CUDA 13 the guide was reorganised into five parts at https://docs.nvidia.com/cuda/cuda-programming-guide/. 2024. URL: https://docs.nvidia.com/cuda/archive/12.4.0/cuda-c-programming-guide/index.html.

[6] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. 4th ed. Not on the course reading lists; used here as a secondary textbook. Cambridge, MA: Morgan Kaufmann (Elsevier), 2022. ISBN: 978-0-323-91231-0.

[7] Timothy G. Mattson, Beverly A. Sanders, and Berna L. Massingill. *Patterns for Parallel Programming*. Copyright 2005 printing. Optional reading: ch. 2 (Module 1); ch. 3, appendices A and B (Module 2). Boston: Addison-Wesley, 2004. ISBN: 0-321-22811-1.

[8] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Assigned: sections 9 (Performance Metrics), 10 (Memory Optimizations), 11 (Execution Configuration Optimizations), Module 5. Accessed 2026-10-29. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html.

[9] Intel Corporation. *Intel 64 and IA-32 Architectures Optimization Reference Manual: Volume 1*. 248966-051US. Section 2.4.1 Golden Cove Microarchitecture Overview: six decoders, up to six instructions decoded per cycle. Intel Corporation. Oct. 2026. URL: https://cdrdv2.intel.com/v1/dl/getContent/671488 (visited on 10/30/2026).

[10] Glenn Hinton et al. "The Microarchitecture of the Pentium 4 Processor". In: *Intel Technology Journal* (2001). Q1 2001 issue. Willamette: 20-stage branch-misprediction pipeline.

[11] Darrell Boggs et al. "The Microarchitecture of the Intel Pentium 4 Processor on 90nm Technology". In: *Intel Technology Journal* 8.1 (Feb. 2004). Prescott: 31-stage branch-misprediction pipeline, pp. 1–17.

[12] Advanced Micro Devices. *AMD EPYC 9654*. Product page: 96 cores, 192 threads; launched 2022. 2026. URL: https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9654.html (visited on 10/30/2026).

[13] Blaise Barney and Donald Frederick. *Introduction to Parallel Computing Tutorial*. Lawrence Livermore National Laboratory. Required reading, Module 2. Page version dated June 16, 2026; accessed 2026-10-29. 2026. URL: https://hpc.llnl.gov/documentation/tutorials/introduction-parallel-computing-tutorial.

[14] OpenMP Architecture Review Board. *OpenMP Application Programming Interface*. Version 5.2. OpenMP Architecture Review Board. Nov. 2021. URL: https://www.openmp.org/specifications/ (visited on 10/30/2026).

[15] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard*. Version 4.1. Message Passing Interface Forum. Nov. 2023. URL: https://www.mpi-forum.org/docs/ (visited on 10/30/2026).

[16] Linux man-pages project. *pthread_create(3)*. Feb. 2026. URL: https://man7.org/linux/man-pages/man3/pthread_create.3.html (visited on 10/30/2026).

[17] Free Software Foundation. *GNU Offloading and Multi-Processing Runtime Library (GNU libgomp): OMP_NUM_THREADS*. "If undefined one thread per CPU is used." 2026. URL: https://gcc.gnu.org/onlinedocs/libgomp/OMP_005fNUM_005fTHREADS.html (visited on 10/30/2026).

[18] Mark Harris. *Using Shared Memory in CUDA C/C++*. NVIDIA Technical Blog. Required reading, Module 5. 2013. URL: https://developer.nvidia.com/blog/using-shared-memory-cuda-cc/.

[19] NVIDIA Corporation. *Parallel Thread Execution ISA*. Version 9.4. Accessed 2026-10-29. 2026. URL: https://docs.nvidia.com/cuda/parallel-thread-execution/index.html.

[20] Peter Kipfer and Rüdiger Westermann. "Improved GPU Sorting". In: *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*. Ed. by Matt Pharr and Randima Fernando. Assigned in Module 5 as Project 2 preparation. Addison-Wesley, 2005. Chap. 46. URL: https://developer.nvidia.com/gpugems/gpugems2/part-vi-simulation-and-numerical-algorithms/chapter-46-improved-gpu-sorting.

[21] Mark Harris. *How to Optimize Data Transfers in CUDA C/C++*. NVIDIA Technical Blog. Dec. 2012. URL: https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/ (visited on 10/30/2026).

[22] Mark Harris. *GPU Pro Tip: CUDA 7 Streams Simplify Concurrency*. NVIDIA Technical Blog. Jan. 2015. URL: https://developer.nvidia.com/blog/gpu-pro-tip-cuda-7-streams-simplify-concurrency/ (visited on 10/30/2026).

[23] NVIDIA Corporation. *Nsight Systems User Guide*. 2026. URL: https://docs.nvidia.com/nsight-systems/UserGuide/index.html (visited on 10/30/2026).

[24] NVIDIA Corporation. *Nsight Compute Documentation*. 2026. URL: https://docs.nvidia.com/nsight-compute/NsightCompute/index.html (visited on 10/30/2026).

[25] NVIDIA Corporation. *CUDA Profiler User's Guide*. 2026. URL: https://docs.nvidia.com/cuda/profiler-users-guide/index.html (visited on 10/30/2026).

[26] NVIDIA Corporation. *Maxwell Tuning Guide*. Oct. 2026. URL: https://docs.nvidia.com/cuda/maxwell-tuning-guide/index.html (visited on 10/30/2026).

[27] Kenneth E. Batcher. "Sorting Networks and Their Applications". In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference (AFIPS '68 (Spring))*. ACM, 1968, pp. 307–314. DOI: 10.1145/1468075.1468121.

[28] Wilson W. L. Fung et al. "Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow". In: *Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2007, pp. 407–420. DOI: 10.1109/MICRO.2007.30.

[29] Veynu Narasiman et al. "Improving GPU Performance via Large Warps and Two-Level Warp Scheduling". In: *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44)*. ACM, 2011, pp. 308–317. DOI: 10.1145/2155620.2155656.

[30] Mark Gebhart et al. "Energy-efficient Mechanisms for Managing Thread Context in Throughput Processors". In: *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2011, pp. 235–246. DOI: 10.1145/2000064.2000093.

[31] Mark Gebhart, Stephen W. Keckler, and William J. Dally. "A Compile-Time Managed Multi-Level Register File Hierarchy". In: *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-44)*. ACM, 2011, pp. 465–476. DOI: 10.1145/2155620.2155675.

[32] Mark Gebhart et al. "Unifying Primary Cache, Scratch, and Register File Memories in a Throughput Processor". In: *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2012, pp. 96–106. DOI: 10.1109/MICRO.2012.18.

[33] Hyeran Jeon et al. "GPU Register File Virtualization". In: *Proceedings of the 48th International Symposium on Microarchitecture (MICRO-48)*. Full text not obtained (ACM DL blocked automated access); metadata verified. ACM, 2015, pp. 420–432. DOI: 10.1145/2830772.2830784.

[34] NVIDIA Corporation. *NVIDIA's Next Generation CUDA Compute Architecture: Fermi*. Whitepaper V1.1. NVIDIA, 2009. URL: https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf (visited on 11/03/2026).

[35] Jason Power, Mark D. Hill, and David A. Wood. "Supporting x86-64 Address Translation for 100s of GPU Lanes". In: *Proceedings of the 20th IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 568–578. DOI: 10.1109/HPCA.2014.6835965.

[36] Rachata Ausavarungnirun et al. "Mosaic: A GPU Memory Manager with Application-Transparent Support for Multiple Page Sizes". In: *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-50)*. ACM, 2017, pp. 136–150. DOI: 10.1145/3123939.3123975.

[37] Bingyao Li et al. "Improving Address Translation in Multi-GPUs via Sharing and Spilling Aware TLB Design". In: *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-54)*. ACM, 2021, pp. 1154–1168. DOI: 10.1145/3466752.3480083.

[38] Debashis Ganguly et al. "Interplay between Hardware Prefetcher and Page Eviction Policy in CPU-GPU Unified Virtual Memory". In: *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 224–235. DOI: 10.1145/3307650.3322224.

[39] Hyojong Kim et al. “Batch-Aware Unified Memory Management in GPUs for Irregular Workloads”. In: *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2020, pp. 1357–1370. DOI: 10.1145/3373376.3378529.

[40] Timothy G. Rogers, Mike O’Connor, and Tor M. Aamodt. “Cache-Conscious Wavefront Scheduling”. In: *Proceedings of the 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2012, pp. 72–83. DOI: 10.1109/MICRO.2012.16.

[41] Adwait Jog et al. “Orchestrated Scheduling and Prefetching for GPGPUs”. In: *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*. The course list cites the SIGARCH Computer Architecture News version, doi:10.1145/2508148.2485951. ACM, 2013, pp. 332–343. DOI: 10.1145/2485922.2485951.

[42] Adwait Jog et al. “OWL: Cooperative Thread Array Aware Scheduling Techniques for Improving GPGPU Performance”. In: *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2013, pp. 395–406. DOI: 10.1145/2451116.2451158.

[43] Jen-Cheng Huang et al. “GPUMech: GPU Performance Modeling Technique Based on Interval Analysis”. In: *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2014, pp. 268–279. DOI: 10.1109/MICRO.2014.59.

[44] Changxi Liu, Yifan Sun, and Trevor E. Carlson. “Photon: A Fine-grained Sampled Simulation Methodology for GPU Workloads”. In: *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. ACM, 2023, pp. 1227–1241. DOI: 10.1145/3613424.3623773.

[45] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785.

[46] Sunpyo Hong and Hyesoon Kim. “An Analytical Model for a GPU Architecture with Memory-level and Thread-level Parallelism Awareness”. In: *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2009, pp. 152–163. DOI: 10.1145/1555754.1555775.

[47] Tejas S. Karkhanis and James E. Smith. “A First-Order Superscalar Processor Model”. In: *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2004, pp. 338–349. DOI: 10.1109/ISCA.2004.1310786.

[48] Ali Bakhoda et al. “Analyzing CUDA Workloads Using a Detailed GPU Simulator”. In: *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2009, pp. 163–174. DOI: 10.1109/ISPASS.2009.4919648.

[49] Mahmoud Khairy et al. “Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling”. In: *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486. DOI: 10.1109/ISCA45697.2020.00047.

[50] Yifan Sun et al. “MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization”. In: *Proceedings of the 46th International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 197–209. DOI: 10.1145/3307650.3322230.

[51] HPArch, Georgia Institute of Technology. *MacSim: A Heterogeneous Architecture Timing Model Simulator*. URL: https://github.com/gthparch/macsim (visited on 11/03/2026).

[52] Jason Power et al. “gem5-gpu: A Heterogeneous CPU-GPU Simulator”. In: *IEEE Computer Architecture Letters* 14.1 (2015), pp. 34–36. DOI: 10.1109/LCA.2014.2299539.

[53] Timothy Sherwood et al. “Automatically Characterizing Large Scale Program Behavior”. In: *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2002, pp. 45–57. DOI: 10.1145/605397.605403.

[54] NVIDIA Corporation. *Matrix Multiplication Background User’s Guide*. Optional reading, Module 12. Accessed 2026-10-29. NVIDIA Deep Learning Performance Documentation. 2026. URL: https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html.

[55] Lieven Eeckhout. *Computer Architecture Performance Evaluation Methods*. Synthesis Lectures on Computer Architecture 10. Reissued by Springer, doi:10.1007/978-3-031-01727-8. Optional reading, Module 8. Morgan & Claypool, 2010. DOI: 10.2200/S00273ED1V01Y201006CAC010.

[56] Daniel Sanchez and Christos Kozyrakis. “ZSim: Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems”. In: *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2013, pp. 475–486. DOI: 10.1145/2485922.2485963.

[57] Akhil Arunkumar et al. “MCM-GPU: Multi-Chip-Module GPUs for Continued Performance Scalability”. In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2017, pp. 320–332. DOI: 10.1145/3079856.3080231.

[58] NVIDIA Corporation. *NVLink and NVLink Switch*. Generation table: 900 GB/s and 18 links per GPU (4th gen, Hopper), 1,800 GB/s and 18 links (5th gen, Blackwell), 3,000 GB/s and 36 links (6th gen, Vera Rubin, preliminary). 2026. URL: https://www.nvidia.com/en-us/data-center/nvlink/ (visited on 11/03/2026).

[59] NVIDIA Corporation. *GPUDirect RDMA*. 2026. URL: https://docs.nvidia.com/cuda/gpudirect-rdma/index.html (visited on 11/03/2026).

[60] NVIDIA Corporation. *Multi-Process Service*. Release 615. Up to 60 client CUDA contexts per device per MPS server (default CUDA_DEVICE_MAX_CONNECTIONS of 2). NVIDIA Corporation. Sept. 2026. URL: https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf (visited on 11/03/2026).

[61] Fuxun Yu et al. *A Survey of Multi-Tenant Deep Learning Inference on GPU*. 2022. arXiv: 2203.09040 [cs.DC].

[62] Zhisheng Ye et al. "Deep Learning Workload Scheduling in GPU Datacenters: A Survey". In: *ACM Computing Surveys* 56.6 (2024), 146:1–146:38. DOI: 10.1145/3638757.

[63] NVIDIA Corporation. *NVIDIA Multi-Instance GPU User Guide*. Release 615. §6.1 Terminology (eight memory slices), §6.2 Partitioning (Table 2, A100 profiles), §7.2 Application Considerations (CUDA MPS on MIG). NVIDIA Corporation. Oct. 2026. URL: https://docs.nvidia.com/datacenter/tesla/mig-user-guide/pdf/MIG_User_Guide.pdf (visited on 11/03/2026).

[64] NVIDIA Corporation. *nvidia-cuda-mps-control(1): NVIDIA CUDA Multi Process Service management program*. driver 580.65.06. Manual page, Ubuntu package nvidia-compute-utils-580-server; client limit 16 for SM 3.5 to 6.0, 48 for SM 7.0. NVIDIA Corporation. 2025. URL: https://manpages.ubuntu.com/manpages/noble/man1/nvidia-cuda-mps-control.1.html (visited on 11/03/2026).

[65] NVIDIA Corporation. *CUDA Compiler Driver NVCC*. Version 13.4. Sections 3 The CUDA Compilation Trajectory, 5 GPU Compilation. Accessed 2026-11-03. 2026. URL: https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html.

[66] LLVM Project. *Compiling CUDA with clang*. 2026. URL: https://llvm.org/docs/CompileCudaWithLLVM.html (visited on 11/03/2026).

[67] NVIDIA Corporation. *NVVM IR Specification*. Accessed 2026-11-03. 2026. URL: https://docs.nvidia.com/cuda/nvvm-ir-spec/index.html.

[68] Chris Lattner and Vikram Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation". In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86. DOI: 10.1109/CGO.2004.1281665.

[69] LLVM Project. *LLVM Language Reference Manual*. Sections "'phi' Instruction" and "'select' Instruction". 2026. URL: https://llvm.org/docs/LangRef.html (visited on 11/03/2026).

[70] Alfred V. Aho et al. *Compilers: Principles, Techniques, and Tools*. 2nd ed. The "Dragon book"; copyright 2007. Sections cited: 6.2 Three-Address Code (SSA), 8.8 Register Allocation and Assignment, 9.1 The Principal Sources of Optimization, 9.2 Introduction to Data-Flow Analysis, 9.6 Loops in Flow Graphs. Pearson Addison-Wesley, 2006. ISBN: 978-0-321-48681-3.

[71] Ron Cytron et al. "Efficiently Computing Static Single Assignment Form and the Control Dependence Graph". In: *ACM Transactions on Programming Languages and Systems* 13.4 (Oct. 1991), pp. 451–490. DOI: 10.1145/115372.115320.

[72] Gregory J. Chaitin. "Register Allocation & Spilling via Graph Coloring". In: *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction*. ACM, 1982, pp. 98–101. DOI: 10.1145/800230.806984.

[73] LLVM Project. *LLVM's Analysis and Transform Passes*. Section "mem2reg: Promote Memory to Register". 2026. URL: https://llvm.org/docs/Passes.html (visited on 11/03/2026).

[74] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Standard IEEE Std 754-2019. Revision of IEEE Std 754-2008. DOI verified to resolve 2026-10-30. IEEE, 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[75] Léopold Cambier et al. *Shifted and Squeezed 8-bit Floating Point format for Low-Precision Training of Deep Neural Networks*. S2FP8, named on the M12 quantization slide. Venue reported as ICLR 2020 (not verified in this pass). 2020. arXiv: 2001.05674 [cs.LG].

[76] Paulius Micikevicius et al. *FP8 Formats for Deep Learning*. E4M3 and E5M2 encodings; E4M3 has no infinities and a single NaN bit pattern (abstract, checked 2026-11-03). 2022. arXiv: 2209.05433 [cs.LG].

[77] H. T. Kung. "Why Systolic Architectures?" In: *Computer* 15.1 (1982), pp. 37–46. DOI: 10.1109/MC.1982.1653825.

[78] Norman P. Jouppi et al. "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. Abstract: a 65,536 8-bit MAC matrix multiply unit with a peak of 92 TOPS. 2017, pp. 1–12. DOI: 10.1145/3079856.3080246.

[79] Tri Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*. Not on the lowyx reading lists; background for Project 4 (from Spring 2026). 2022. DOI: 10.52202/068431-1189. arXiv: 2205.14135.

[80] Tri Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. Published at ICLR 2024 (venue not re-verified in this pass). Not on the lowyx reading lists. 2023. arXiv: 2307.08691 [cs.LG].

[81] Alberto Villarreal. *Code Sample: Intel Advanced Matrix Extensions (Intel AMX) – Intrinsics Functions*. Palette 1: 8 tile registers, each at most 16 rows of 64 bytes; INT8 example multiplies 16 x 64 inputs into a 16 x 16 INT32 result. Intel Corporation. 2024. URL: https://www.intel.com/content/www/us/en/developer/articles/code-sample/advanced-matrix-extensions-intrinsics-functions.html (visited on 11/03/2026).

Index

Symbols
2:4 structured sparsity, 119
3D-stacked memory, 37

A
abstract syntax tree (AST), 91
accelerator, 1
accumulator, 115
activation function, 111
active mask, 36
active warp, 23
active warp set / pending warp set, 67
address translation, 60
Advanced Matrix Extensions (AMX), 118
alignment, 38
Amdahl's law, 4
AMX, *see* Advanced Matrix Extensions (AMX)
analytical model, 75
architectural register, 57
arithmetic intensity, 76
AST, *see* abstract syntax tree (AST)
asynchronous barrier, 119
asynchronous copy, 42
atomic operation, 14
attention, 119

B
back end, 90
backward analysis, 102
bank, 32
bank conflict, 32
bank-level parallelism, 68
barrier, 14
barrier synchronization, 21
basic block, 51, 95
BF16, *see* bfloat16 (BF16)
bfloat16 (BF16), 116
bit vector, 99
bitonic sort, 49
block index, 19
boundary condition, 26
boundary exchange, 17
branch prediction, 3
broadcast, 16
BSP, *see* bulk synchronous parallel (BSP)
bulk synchronous parallel (BSP), 21

C
cache line, 38
cache miss, 29
cache thrashing, 68
cache-conscious wavefront scheduling (CCWS), 68
CCWS, *see* cache-conscious wavefront scheduling (CCWS)
CFG, *see* control flow graph (CFG)
checkpoint, 80
chiplet, 82
chunk size, 14
Clang, 91
clause, 13
clock domain, 71
coalesced access, 38
code generation, 90
code motion, 108
collective communication, 16
compiler, 90
compiler directive, 11
compressed sparse format, 119
compulsory miss, 68
computation warp parallelism (CWP), 79
compute capability, 24
compute-bound, 76, 112
concurrency, 9
concurrent copy and execute, 42
concurrent kernel execution, 87
conditional move, 104
conservative analysis, 110
constant folding, 107
constant memory, 22
constant propagation, 107
context switch, 30
control flow graph (CFG), 95
convolution, 27
convolution layer, 112
cooperative groups, 47
cooperative thread array (CTA), 19
copy descriptor, 115
copy propagation, 96
copy-then-execute model, 63
CPI, *see* cycles per instruction (CPI)
CPI stack, 77
CPU, 1
critical section, 12
CTA, *see* cooperative thread array (CTA)
CTA-aware scheduling, 69
cuBLAS, 48
CUDA core, *see* stream processor (CUDA core)
CUDA Graph, 88
CUDA stream, 42
cuDNN, 49
CWP, *see* computation warp parallelism (CWP)
cycle-level simulation, 70
cycles per instruction (CPI), 77

D
data decomposition, 4
data race, 12
data reuse, 26
data transfer, 41
data-driven model, 81
data-flow analysis, 96
datacenter, 86
dead code elimination, 107
dead register, 55

decode, 35
deep neural network (DNN), 111
def set, 101
def-use chain, 104
definition, 97
denormal number, *see* subnormal number
device, 18
device code, 18
device memory, 24
direct memory access, *see* DMA (direct memory access)
directive, 94
distributed-memory programming, 11
divergence analysis, 108
divergent branch, 51
divergent warp, 51
DMA (direct memory access), 64
DNN, *see* deep neural network (DNN)
dominance frontier, 105
dominator, 105
dot product, 112
double precision (FP64), 7
DRAM, 37
DWF, *see* dynamic warp formation (DWF)
dynamic, 33
dynamic instruction count, 33
dynamic range, 116
dynamic scheduling, 14
dynamic warp formation (DWF), 54

E
effective bandwidth, 44
element-wise operation, 111
environment variable, 11
epilogue loop, 106
event-driven simulation, 79
execution configuration, 18
execution-driven simulation, 71
exponent, 115
exponent bias, 115

F
far fault, 64
fat binary, 92
fetch group, 67
FIFO, 71
fill (reload), 103
fill and drain, 114
filter, 27
fine-grained multithreading, 29
first-order model, 75
first-touch page placement, 83
fixed point, 98
FlashAttention, 119
floating-point format, 115
Flynn's taxonomy, 4
FMA, *see* fused multiply-add (FMA)
fork/join pattern, 10
forward analysis, 96
FP16, *see* half precision (FP16)
FP32, *see* single precision (FP32)
FP64, *see* double precision (FP64)
FP8, 116
FPGA emulation, 70
fraction, *see* mantissa (fraction)
front end, 90
fully connected layer, 112
function inlining, 107
fused multiply-add (FMA), 75

G
GEMM (general matrix multiplication), 112
gen set, 97
general matrix multiplication, *see* GEMM (general matrix multiplication)
ghost cells, *see* halo (ghost cells)
global memory, 22
global optimization, 96
GMMU, *see* GPU memory management unit (GMMU)
GPGPU, 7
GPM, *see* GPU module (GPM)
GPU, 1
GPU instance, 86
GPU memory management unit (GMMU), 64
GPU module (GPM), 82
GPUDirect RDMA, 85
graphics pipeline, 7
greedy-then-oldest (GTO), 67
grid, 19
GTO, *see* greedy-then-oldest (GTO)
guard predicate, 93
guided scheduling, 14

H
half precision (FP16), 8
halo (ghost cells), 16
hardware-managed cache, 58
HBM, *see* high bandwidth memory (HBM)
hierarchical register file, 56
high bandwidth memory (HBM), 8
HIP, 49
HMMA, 113
host, 1
host bypass, 85
host code, 18
host memory, 24
hotspot, 40

I
IEEE 754, 117
if-conversion, 52
ILP, *see* instruction-level parallelism (ILP)
immediate post-dominator (IPDOM), 52
in-order execution, 2
independent thread scheduling, 48
induction variable, 108
instruction fetch, 35
instruction optimization, 45
instruction set architecture (ISA), 1
instruction-level parallelism (ILP), 2
instructions per cycle (IPC), 2
inter-procedural analysis, 107
inter-warp locality, 68

interconnect, 84
interference graph, 103
intermediate representation (IR), 90
internal fragmentation, 63
interval analysis, 78
intra-warp locality, 68
intrinsic function, 45
IOMMU, 64
IPC, *see* instructions per cycle (IPC)
IPDOM, *see* immediate post-dominator (IPDOM)
IR, *see* intermediate representation (IR)
ISA, *see* instruction set architecture (ISA)
issue, 35
iterative algorithm, 98

J
JIT compilation, 92
join point, 104

K
kernel, 18
kernel launch, 18
kernel-level sampling, 80
kill set, 97

L
L1 data cache, 59
label, 94
lane, *see* SIMT lane
large page, 63
large warp, 54
last result file (LRF), 56
latency, 1
latency hiding, 29
leader, 95
least recently used, *see* LRU (least recently used)
live register, 55
live variable, 101
liveness analysis, 101
LLD, *see* lost intra-wavefront locality detector (LLD)
LLVM, 91
load balancing, 14
local memory, 22
local optimization, 95
lock, 12
loop parallelism, 10
loop unrolling, 106
loop-invariant code motion, 108
loose round-robin (LRR), 67
lost intra-wavefront locality detector (LLD), 68
LRF, *see* last result file (LRF)
LRR, *see* loose round-robin (LRR)
LRU (least recently used), 58

M
MAC, *see* multiply-accumulate (MAC)
machine learning (ML), 111
main register file (MRF), 56
mantissa (fraction), 115
master/worker pattern, 9
math library, 48
math-limited, *see* compute-bound
matrix multiplication, 112
matrix multiply-accumulate (MMA), 113
MCM, *see* multi-chip module (MCM)
meet operator, 96
memory allocation, 25
memory bandwidth, 37
memory coalescing, 38
memory fence, 94
memory oversubscription, 66
memory transaction, 38
memory warp parallelism (MWP), 79
memory-bound, 76, 112
memory-level parallelism (MLP), 29
memory-limited, *see* memory-bound
merge sort, 49
message passing, 16
Message Passing Interface, *see* MPI (Message Passing Interface)
microbenchmark, 81
MIG, *see* Multi-Instance GPU (MIG)
MIMD, 4
miss event, 78
miss status holding register, *see* MSHR (miss status holding register)
mixed precision, 115
ML, *see* machine learning (ML)
MLP, *see* memory-level parallelism (MLP)
MMA, *see* matrix multiply-accumulate (MMA)
monolithic GPU, 82
MPI (Message Passing Interface), 16
MPS, *see* Multi-Process Service (MPS)
MRF, *see* main register file (MRF)
MSHR (miss status holding register), 78
multi-chip module (MCM), 82
multi-GPU, 82
Multi-Instance GPU (MIG), 86
multi-level page table, 61
Multi-Process Service (MPS), 87
multi-tenancy, 86
multiply-accumulate (MAC), 113
multiprocessor, 3
multithreading, 3
mutex, 12
mutual exclusion, 12
MWP, *see* memory warp parallelism (MWP)

N
NaN (not a number), 117
NCCL, 49
near memory / far memory, 83
nested branch, 53
network interface card (NIC), 85
NIC, *see* network interface card (NIC)
non-uniform memory access (NUMA), 83
non-volatile memory (NVM), 56
not a number, *see* NaN (not a number)
Nsight Compute, 40
Nsight Systems, 40
NUMA, *see* non-uniform memory access (NUMA)
nvcc, 91
NVLink, 84

NVM, *see* non-volatile memory (NVM)
NVPTX, 91
NVSwitch, 84

O
occupancy, 23
oldest-first, 66
on-demand paging, 64
opcode, 93
OpenCL, 49
OpenMP, 11
operand, 93
operand buffer, 33
operand collector, 33
operand register file (ORF), 56
operational intensity, 77
ops:byte ratio, 112
ORF, *see* operand register file (ORF)
out-of-order execution, 2
overflow, 116
overlap of computation and transfer, 42

P
padding, 34
page, 60
page eviction, 66
page fault, 60
page migration, 84
page offset, 60
page prefetching, 65
page table, 60
page table entry (PTE), 60
page table walk, 61
page table walker (PTW), 61
page walk cache, 62
page-locked memory, *see* pinned memory
parallel programming pattern, 9
parallel sections, 15
Parallel Thread Execution, *see* PTX (Parallel Thread Execution)
path-convergence criterion, 105
PC, *see* program counter (PC)
PCI Express (PCIe), 41
PCIe, *see* PCI Express (PCIe)
performance modeling, 70
PFN, *see* physical frame number (PFN)
phi function, 104
physical address, 60
physical frame number (PFN), 60
physical register, 57
pinned memory, 41
pipeline pattern, 10
pipeline width, 71
pixel shader, 7
point-to-point communication, 16
pooling, 112
port, 31
POSIX threads (pthreads), 12
post-dominator, 52
PPN, *see* physical frame number (PFN)
pre-eviction, 66
precise exception, 1
predecessor, 95
predicate, 52
predicate register, 93
predicated execution, 45
prefetching, 65
preprocessor, 92
private variable, 14
process, 16
profiler, 40
profiling, 40
program counter (PC), 30
program phase, 80
program point, 96
pruning, 119
PTE, *see* page table entry (PTE)
pthreads, *see* POSIX threads (pthreads)
PTW, *see* page table walker (PTW)
PTX (Parallel Thread Execution), 92

Q
QoS, *see* quality of service (QoS)
quality of service (QoS), 87
quantization, 115
queue-based model, 71

R
rank, 16
RDMA, *see* remote direct memory access (RDMA)
reaching definitions, 97
read port, 31
reconvergence point, 52
reduction, 4
register, 23
register allocation, 103
register bank conflict, 32
register file, 30
register file cache (RFC), 56
register file virtualization, 57
register pressure, 23
register renaming, 57
register spill, 103
reload, *see* fill (reload)
remote direct memory access (RDMA), 85
replacement policy, 58
representative workload, 81
resource constraint, 22
RFC, *see* register file cache (RFC)
ridge point, 76
Roofline model, 76
round-robin (RR), 67
rounding mode, 117
RR, *see* round-robin (RR)

S
S2FP8, *see* shifted and squeezed FP8 (S2FP8)
sampled simulation, 80
SASS, 92
scoreboard, 33
scratchpad memory, 20
sector, 74

sectored cache, 74
segment mapping, 63
serial fraction, 3
SFU, *see* special function unit (SFU)
shared memory, 20
shared memory bank conflict, 34
shared memory carve-out, 59
shared variable, 14
shared virtual memory (SVM), 63
shared-memory programming, 10
shifted and squeezed FP8 (S2FP8), 117
sign bit, 115
silicon interposer, 37
SIMD, 2
SIMD efficiency, 54
SimPoint, 80
SIMT, 5
SIMT lane, 5, 36
SIMT stack, 53
simulator, 70
single precision (FP32), 7
SISD, 2
SM, *see* streaming multiprocessor (SM)
software-managed cache, 58
sorting network, 49
sparse matrix, 119
sparse matrix-vector multiplication (SpMV), 120
sparsity, 119
spatial partitioning, 87
special function unit (SFU), 35
special register, 35
split arrive/wait barrier, 48
SPMD, 2
SpMV, *see* sparse matrix-vector multiplication (SpMV)
SRAM, 37
SSA, *see* static single assignment (SSA)
SSA form, 104
static, 33
static instruction count, 33
static scheduling, 14
static single assignment (SSA), 104
stencil, 16
stream processor (CUDA core), 5
streaming multiprocessor (SM), 5
strength reduction, 107
stride, 34
structured sparsity, 119
sub-core, 76
subnormal number, 116
successor, 95
superscalar, 2
SVM, *see* shared virtual memory (SVM)
SYCL, 49
systolic array, 113

T

tag, 58
task decomposition, 4
task queue, 9
TBNp, *see* tree-based neighborhood prefetching
tensor core, 8
Tensor Memory Accelerator (TMA), 8
TensorFloat-32 (TF32), 116
texture memory, 22
TF32, *see* TensorFloat-32 (TF32)
theoretical bandwidth, 44
thread, 11
thread block, 19
thread block cluster, 22
thread block scheduling, 83
thread hierarchy, 19
thread index, 19
thread-dependent value, 109
three-address code, 94
throughput, 1
Thrust, 49
tile, 26
tile quantization, 112
tiling, 26
time slicing, 87
TLB (translation lookaside buffer), 60
TLB miss, 60
TMA, *see* Tensor Memory Accelerator (TMA)
trace, 71
trace-driven simulation, 71
transfer function, 96
Transformer Engine, 8
translation lookaside buffer, *see* TLB (translation lookaside buffer)
tree-based neighborhood prefetching, 65
two-level warp scheduling, 67

U

uncoalesced access, 38
underflow, 116
unified local memory, 59
unified memory, 26
unified shader, 7
unified virtual addressing (UVA), 44
uniform value, 109
use, 97
use set, 101
utilization, 86
UVA, *see* unified virtual addressing (UVA)

V

value lifetime, 55
vectorization, 109
vertex shader, 7
virtual address, 60
virtual architecture, 91
virtual ISA, 1
virtual memory, 60
virtual page number (VPN), 60
VPN, *see* virtual page number (VPN)

W

warm-up, 80
warp, 6
warp divergence, 51
warp scheduler, 66
warp shuffle, 46
warp size, 6

warp vote function, 47
wave quantization, 112
wavefront, 6
working set, 68
write port, 31
write-back, 35

Z
zero copy, 43
zero padding, 26